开发者社区

DTK

DTK 26.10
HIP Runtime API 开发手册

DTK-261000-005-0915-920

目 录

1 Device Management 1
1.1 hipChooseDevice 1
1.2 hipDeviceFlushGPUDirectRDMAWrites* 1
1.3 hipDeviceGetAttribute 1
1.4 hipDeviceGetByPCIBusId 2
1.5 hipDeviceGetCacheConfig 3
1.6 hipDeviceGetDefaultMemPool* 3
1.7 hipDeviceGetLimit 4
1.8 hipDeviceGetMemPool* 5
1.9 hipDeviceGetNvSciSyncAttributes* 5
1.10 hipDeviceGetP2PAttribute 5
1.11 hipDeviceGetPCIBusId 6
1.12 hipDeviceGetSharedMemConfig 7
1.13 hipDeviceGetStreamPriorityRange 7
1.14 hipDeviceGetTexture1DLinearMaxWidth* 8
1.15 hipDeviceReset 9
1.16 hipDeviceSetCacheConfig 9
1.17 hipDeviceSetLimit* 10
1.18 hipDeviceSetMemPool* 10
1.19 hipDeviceSetSharedMemConfig 11
1.20 hipDeviceSynchronize 11
1.21 hipGetDevice 12
1.22 hipGetDeviceCount 13
1.23 hipGetDeviceFlags 13
1.24 hipGetDeviceProperties 14
1.25 hipIpcCloseMemHandle 14
1.26 hipIpcGetEventHandle 15
1.27 hipIpcGetMemHandle 15

开发者社区

1.28 hipIpcOpenEventHandle 16
1.29 hipIpcOpenMemHandle 16
1.30 hipSetDevice 17
1.31 hipSetDeviceFlags 18
1.32 hipSetValidDevices* 20
1.33 hipDeviceGetProperties 20
1.34 hipInitDevice【dtk-25.04.1 新增】 20
2 Tread Management 21
2.1 hipThreadExit* 21
2.2 hipThreadGetCacheConfig* 21
2.3 hipThreadGetLimit* 21
2.4 hipThreadSetCacheConfig* 22
2.5 hipThreadSetLimit* 22
2.6 hipThreadSynchronize* 22
3 Error Handing 22
3.1 hipGetErrorName 22
3.2 hipGetErrorString 23
3.3 hipGetLastError 24
3.4 hipPeekAtLastError 25
4 Stream Management 25
4.1 hipCtxResetPersistingL2Cache* 25
4.2 hipStreamAddCallback 25
4.3 hipStreamAttachMemAsync 26
4.4 hipStreamBeginCapture 27
4.5 hipStreamCopyAttributes* 27
4.6 hipStreamCreate 28
4.7 hipStreamCreateWithFlags 29
4.8 hipStreamCreateWithPriority 30
4.9 hipStreamDestroy 31
4.10 hipStreamEndCapture 32

开发者社区

4.11 hipStreamGetAttribute* 32
4.12 hipStreamGetCaptureInfo 33
4.13 hipStreamGetCaptureInfo_v2* 33
4.14 hipStreamGetFlags 33
4.15 hipStreamGetPriority 34
4.16 hipStreamIsCapturing* 35
4.17 hipStreamQuery 35
4.18 hipStreamSetAttribute* 36
4.19 hipStreamSynchronize 36
4.20 hipStreamUpdateCaptureDependencies* 37
4.21 hipStreamWaitEvent 38
4.22 hipThreadExchangeStreamCaptureMode* 39
4.23 hipStreamGetDevice 39
4.24 hipStreamBeginCaptureToGraph【dtk-25.04.1 新增】 39
4.25 hipStreamGetId【dtk-25.04.1 新增】 41
4.26 hipStreamBatchMemOp【dtk-26.10 新增】 41
4.27 hipGraphAddBatchMemOpNode【dtk-26.10 新增】 42
4.28 hipGraphBatchMemOpNodeGetParams【dtk-26.10 新增】 42
4.29 hipGraphBatchMemOpNodeSetParams【dtk-26.10 新增】 43
4.30 hipGraphExecBatchMemOpNodeSetParams【dtk-26.10 新增】 43
4.31 hipStreamSetAttribute【dtk-26.10 新增】 44
4.32 hipStreamGetAttribute【dtk-26.10 新增】 45
4.33 hipStreamGetId【dtk-26.10 新增】 45
5 Event Management 46
5.1 hipEventCreate 46
5.2 hipEventCreateWithFlags 46
5.3 hipEventDestroy 47
5.4 hipEventElapsedTime 48
5.5 hipEventQuery 49
5.6 hipEventRecord 50
开发者社区

5.7 hipEventRecordWithFlags* 51
5.8 hipEventSynchronize 52
5.9 hipEventRecordWithFlags【dtk-26.10 新增】 53
6 External Resource Interoperability 54
6.1 hipDestroyExternalMemory 54
6.2 hipDestroyExternalSemaphore 54
6.3 hipExternalMemoryGetMappedBuffer 54
6.4 hipExternalMemoryGetMappedMipmappedArray* 55
6.5 hipImportExternalMemory 55
6.6 hipImportExternalSemaphore 56
6.7 hipSignalExternalSemaphoresAsync 56
6.8 hipWaitExternalSemaphoresAsync 57
7 Execution Control 57
7.1 hipFuncGetAttributes 57
7.2 hipFuncSetAttribute 58
7.3 hipFuncSetCacheConfig 58
7.4 hipFuncSetSharedMemConfig 59
7.5 hipGetParameterBuffer* 59
7.6 hipGetParameterBufferV2* 59
7.7 hipLaunchCooperativeKernel 59
7.8 hipLaunchCooperativeKernelMultiDevice 60
7.9 hipLaunchHostFunc* 60
7.10 hipLaunchKernel 61
7.11 hipSetDoubleForDevice* 62
7.12 hipSetDoubleForHost* 62
7.13 hipLaunchKernelExC 62
7.14 hipFuncGetName【dtk-25.04.1 新增】 63
7.15 hipFuncGetParamInfo【dtk-25.04.1 新增】 64
7.16 hipLaunchKernelExC【dtk-26.10 新增】 64
开发者社区

7.17 hipDrvLaunchKernelEx【dtk-26.10 新增】 65
7.18 hipLaunchKernelEx【dtk-26.10 新增】 65
8 Occupancy 66
8.1 hipOccupancyAvailableDynamicSMemPerBlock* 66
8.2 hipOccupancyMaxActiveBlocksPerMultiprocessor 66
8.3 hipOccupancyMaxActiveBlocksPerMultiprocessorWithFlags 67
9 Execution Control [DEPRECATED since 7.0] 67
9.1 hipConfigureCall 67
9.2 hipLaunchByPtr 68
9.3 hipSetupArgument 68
9.4 hipLaunch* 69
10 Memory Management 69
10.1 hipArrayGetInfo 69
10.2 hipArrayGetMemoryRequirements* 69
10.3 hipArrayGetPlane* 70
10.4 hipArrayGetSparseProperties* 70
10.5 hipFree 71
10.6 hipFreeArray 71
10.7 hipFreeHost 72
10.8 hipFreeMipmappedArray 72
10.9 hipGetMipmappedArrayLevel 72
10.10 hipGetSymbolAddress 73
10.11 hipGetSymbolSize 74
10.12 hipHostAlloc 74
10.13 hipHostGetDevicePointer 75
10.14 hipHostGetFlags 76
10.15 hipHostRegister 77
10.16 hipHostUnregister 79
10.17 hipMalloc 80
10.18 hipMalloc3D 80
开发者社区

10.19 hipMalloc3DArray 81
10.20 hipMallocArray 81
10.21 hipHostMalloc 82
10.22 hipMallocManaged 83
10.23 hipMallocMipmappedArray 83
10.24 hipMallocPitch 84
10.25 hipMemAdvise 85
10.26 hipMemGetInfo 86
10.27 hipMemPrefetchAsync 86
10.28 hipMemRangeGetAttribute 87
10.29 hipMemRangeGetAttributes 87
10.30 hipMemcpy 88
10.31 hipMemcpy2D 90
10.32 hipMemcpy2DArrayToArray* 91
10.33 hipMemcpy2DAsync 92
10.34 hipMemcpy2DFromArray 93
10.35 hipMemcpy2DFromArrayAsync 94
10.36 hipMemcpy2DToArray 94
10.37 hipMemcpy2DToArrayAsync 96
10.38 hipMemcpy3D 97
10.39 hipMemcpy3DAsync 97
10.40 hipMemcpy3DPeer* 98
10.41 hipMemcpy3DPeerAsync* 99
10.42 hipMemcpyAsync 99
10.43 hipMemcpyFromSymbol 101
10.44 hipMemcpyFromSymbolAsync 101
10.45 hipMemcpyPeer 102
10.46 hipMemcpyPeerAsync 103
10.47 hipMemcpyToSymbol 104
10.48 hipMemcpyToSymbolAsync 105

10.49 hipMemset....107
10.50 hipMemset2D....108
10.51 hipMemset2DAsync....109
10.52 hipMemset3D....111
10.53 hipMemset3DAsync....113
10.54 hipMemsetAsync....114
10.55 hipMipmappedArrayGetMemoryRequirements*....116
10.56 hipMipmappedArrayGetSparseProperties*....117
10.57 make_hipExtent....117
10.58 make_hipPitchedPtr*....118
10.59 make_hipPos....118
10.60 hipMemAdvise_v2【dtk-25.04.1 新增】....118
10.61 hipMemPrefetchAsync_v2【dtk-25.04.1 新增】....119
10.62 hipDeviceRegisterAsyncNotification【dtk-25.04.1 新增】....120
10.63 hipDeviceUnregisterAsyncNotification【dtk-25.04.1 新增】....121
10.64 hipMemsetD2D8【dtk-26.10 新增】....121
10.65 hipMemsetD2D8Async【dtk-26.10 新增】....122
10.66 hipMemsetD2D16【dtk-26.10 新增】....123
10.67 hipMemsetD2D16Async【dtk-26.10 新增】....123
10.68 hipMemsetD2D32【dtk-26.10 新增】....124
10.69 hipMemsetD2D32Async【dtk-26.10 新增】....124
10.70 hipMemcpyBatchAsync【dtk-26.10 新增】....125
10.71 hipMemcpy3DBatchAsync【dtk-26.10 新增】....126
10.72 hipMemcpy3DPeer【dtk-26.10 新增】....126
10.73 hipMemcpy3DPeerAsync【dtk-26.10 新增】....127
10.74 hipMemPrefetchAsync_v2【dtk-26.10 新增】....127
10.75 hipMemAdvise_v2【dtk-26.10 新增】....128
11 Memory Management [DEPRECATED]....128
11.1 hipMemcpyArrayToArray*....128
11.2 hipMemcpyFromArray....129

11.3 hipMemcpyFromArrayAsync* 130
11.4 hipMemcpyToArray 131
11.5 hipMemcpyToArrayAsync* 132
12 Stream Ordered Memory Allocator 132
12.1 hipFreeAsync* 132
12.2 hipMallocAsync* 133
12.3 hipMallocFromPoolAsync* 133
12.4 hipMemPoolCreate* 134
12.5 hipMemPoolDestroy* 134
12.6 hipMemPoolExportPointer* 135
12.7 hipMemPoolExportToShareableHandle* 135
12.8 hipMemPoolGetAccess* 135
12.9 hipMemPoolGetAttribute* 136
12.10 hipMemPoolImportFromShareableHandle* 136
12.11 hipMemPoolImportPointer* 137
12.12 hipMemPoolSetAccess* 137
12.13 hipMemPoolSetAttribute* 138
12.14 hipMemPoolTrimTo* 138
13 Unified Addressing 139
13.1 hipPointerGetAttributes 139
14 Peer Device Memory Access 139
14.1 hipDeviceCanAccessPeer 139
14.2 hipDeviceDisablePeerAccess 141
14.3 hipDeviceEnablePeerAccess 142
15 Texture Reference Management [DEPRECATED] 143
15.1 hipBindTexture 143
15.2 hipBindTexture2D 144
15.3 hipBindTextureToArray 144
15.4 hipBindTextureToMipmappedArray 144
15.5 hipGetTextureAlignmentOffset 145

15.6 hipGetTextureReference 145
15.7 hipUnbindTexture 145
16 Surface Reference Management [DEPRECATED] 146
16.1 hipBindSurfaceToArray* 146
16.2 hipGetSurfaceReference* 147
17 Texture Object Management 148
17.1 hipCreateChannelDesc* 148
17.2 hipCreateTextureObject 148
17.3 hipDestroyTextureObject 148
17.4 hipGetChannelDesc 149
17.5 hipGetTextureObjectResourceDesc 149
17.6 hipGetTextureObjectResourceViewDesc 150
17.7 hipGetTextureObjectTextureDesc 150
17.8 hipDeviceGetTexture1DLinearMaxWidth【dtk-26.10 新增】 150
18 Surface Object Management 150
18.1 hipCreateSurfaceObject 150
18.2 hipDestroySurfaceObject 151
18.3 hipGetSurfaceObjectResourceDesc* 151
19 Version Management 153
19.1 hipDriverGetVersion 153
19.2 hipRuntimeGetVersion 153
20 Graph Management 154
20.1 hipDeviceGetGraphMemAttribute* 154
20.2 hipDeviceGraphMemTrim* 154
20.3 hipDeviceSetGraphMemAttribute* 155
20.4 hipGraphAddChildGraphNode* 155
20.5 hipGraphAddDependencies 156
20.6 hipGraphAddEmptyNode 156
20.7 hipGraphAddEventRecordNode 157
20.8 hipGraphAddEventWaitNode 158

20.9 hipGraphAddExternalSemaphoresSignalNode* 158
20.10 hipGraphAddExternalSemaphoresWaitNode* 159
20.11 hipGraphAddHostNode 160
20.12 hipGraphAddKernelNode 160
20.13 hipGraphAddMemAllocNode* 161
20.14 hipGraphAddMemFreeNode* 162
20.15 hipGraphAddMemcpyNode 163
20.16 hipGraphAddMemcpyNode1D 163
20.17 hipGraphAddMemcpyNodeFromSymbol 164
20.18 hipGraphAddMemcpyNodeToSymbol 165
20.19 hipGraphAddMemsetNode 166
20.20 hipGraphChildGraphNodeGetGraph* 166
20.21 hipGraphClone* 167
20.22 hipGraphCreate 167
20.23 hipGraphDebugDotPrint* 168
20.24 hipGraphDestroy 168
20.25 hipGraphDestroyNode* 168
20.26 hipGraphEventRecordNodeGetEvent 169
20.27 hipGraphEventRecordNodeSetEvent 169
20.28 hipGraphEventWaitNodeGetEvent 170
20.29 hipGraphEventWaitNodeSetEvent 170
20.30 hipGraphExecChildGraphNodeSetParams* 171
20.31 hipGraphExecDestroy 171
20.32 hipGraphExecEventRecordNodeSetEvent* 172
20.33 hipGraphExecEventWaitNodeSetEvent* 172
20.34 hipGraphExecExternalSemaphoresSignalNodeSetParams* 172
20.35 hipGraphExecExternalSemaphoresWaitNodeSetParams* 173
20.36 hipGraphExecHostNodeSetParams* 174
20.37 hipGraphExecKernelNodeSetParams 174
20.38 hipGraphExecMemcpyNodeSetParams* 175

20.39 hipGraphExecMemcpyNodeSetParams1D* 175
20.40 hipGraphExecMemcpyNodeSetParamsFromSymbol* 176
20.41 hipGraphExecMemcpyNodeSetParamsToSymbol* 176
20.42 hipGraphExecMemsetNodeSetParams* 177
20.43 hipGraphExecUpdate* 177
20.44 hipGraphExternalSemaphoresSignalNodeGetParams* 178
20.45 hipGraphExternalSemaphoresSignalNodeSetParams* 178
20.46 hipGraphExternalSemaphoresWaitNodeGetParams* 179
20.47 hipGraphExternalSemaphoresWaitNodeSetParams* 179
20.48 hipGraphGetEdges 180
20.49 hipGraphGetNodes 180
20.50 hipGraphGetRootNodes 181
20.51 hipGraphHostNodeGetParams 181
20.52 hipGraphHostNodeSetParams 182
20.53 hipGraphInstantiate 182
20.54 hipGraphInstantiateWithFlags* 183
20.55 hipGraphKernelNodeCopyAttributes* 183
20.56 hipGraphKernelNodeGetAttribute* 183
20.57 hipGraphKernelNodeGetParams 184
20.58 hipGraphKernelNodeSetAttribute* 184
20.59 hipGraphKernelNodeSetParams 185
20.60 hipGraphLaunch 185
20.61 hipGraphMemAllocNodeGetParams* 185
20.62 hipGraphMemFreeNodeGetParams* 186
20.63 hipGraphMemcpyNodeGetParams 186
20.64 hipGraphMemcpyNodeSetParams 187
20.65 hipGraphMemcpyNodeSetParams1D* 187
20.66 hipGraphMemcpyNodeSetParamsFromSymbol* 187
20.67 hipGraphMemcpyNodeSetParamsToSymbol* 188
20.68 hipGraphMemsetNodeGetParams 189

20.69 hipGraphMemsetNodeSetParams 189
20.70 hipGraphNodeFindInClone* 189
20.71 hipGraphNodeGetDependencies* 190
20.72 hipGraphNodeGetDependentNodes* 190
20.73 hipGraphNodeGetEnabled* 191
20.74 hipGraphNodeGetType* 191
20.75 hipGraphNodeSetEnabled* 191
20.76 hipGraphReleaseUserObject* 192
20.77 hipGraphRemoveDependencies* 192
20.78 hipGraphRetainUserObject* 193
20.79 hipGraphUpload* 193
20.80 hipUserObjectCreate* 194
20.81 hipUserObjectRelease* 194
20.82 hipUserObjectRetain* 194
20.83 hipGraphAddMemAllocNode 195
20.84 hipGraphAddMemFreeNode 195
20.85 hipGraphDebugDotPrint 196
20.86 hipGraphKernelNodeCopyAttributes 196
20.87 hipGraphMemAllocNodeGetParams 197
20.88 hipGraphMemFreeNodeGetParams 197
20.89 hipGraphNodeGetEnabled 198
20.90 hipGraphNodeSetEnabled 198
20.91 hipGraphAddNode【dtk-25.04.1 新增】 199
20.92 hipGraphExecGetFlags【dtk-25.04.1 新增】 200
20.93 hipGraphExecNodeSetParams【dtk-25.04.1 新增】 201
20.94 hipGraphInstantiateWithParams【dtk-25.04.1 新增】 202
20.95 hipGraphNodeSetParams【dtk-25.04.1 新增】 202
20.96 hipGraphConditionalHandleCreate【dtk-25.04.1 新增】 204
21 Driver Entry Point Access 204
21.1 hipGetProcAddress 204
开发者社区

22 C++ API Routines 205
22.1 hipBindSurfaceToArray* 205
22.2 hipBindTexture 205
22.3 hipBindTexture2D 206
22.4 hipBindTextureToArray 206
22.5 hipBindTextureToMipmappedArray 207
22.6 hipCreateChannelDesc* 207
22.7 hipEventCreate 207
22.8 hipFuncGetAttributes 208
22.9 hipFuncSetAttribute 208
22.10 hipFuncSetCacheConfig 209
22.11 hipGetSymbolAddress 209
22.12 hipGetSymbolSize 210
22.13 hipGetTextureAlignmentOffset 210
22.14 hipGraphAddMemcpyNodeFromSymbol* 212
22.15 hipGraphAddMemcpyNodeToSymbol* 213
22.16 hipGraphExecMemcpyNodeSetParamsFromSymbol* 213
22.17 hipGraphExecMemcpyNodeSetParamsToSymbol* 214
22.18 hipGraphMemcpyNodeSetParamsFromSymbol* 215
22.19 hipGraphMemcpyNodeSetParamsToSymbol* 215
22.20 hipLaunchCooperativeKernel 216
22.21 hipLaunchKernel 217
22.22 hipMallocAsync* 217
22.23 hipMallocHost 218
22.24 hipMallocManaged 218
22.25 hipMemcpyFromSymbol 219
22.26 hipMemcpyFromSymbolAsync 219
22.27 hipMemcpyToSymbol 220
22.28 hipMemcpyToSymbolAsync 221
22.29 hipOccupancyAvailableDynamicSMemPerBlock* 223

22.30 hipOccupancyMaxActiveBlocksPerMultiprocessor 223
22.31 hipOccupancyMaxActiveBlocksPerMultiprocessorWithFlags 224
22.32 hipOccupancyMaxPotentialBlockSize* 224
22.33 hipOccupancyMaxPotentialBlockSizeVariableSMem* 225
22.34 hipOccupancyMaxPotentialBlockSizeVariableSMemWithFlags* 226
22.35 hipOccupancyMaxPotentialBlockSizeWithFlags* 226
22.36 hipStreamAttachMemAsync 227
22.37 hipUnbindTexture 227
23 Interactions with the CUDA Driver API 228
23.1 hipGetKernel【dtk-25.04.1 新增】 228
24 Profiler Control [DEPRECATED] 229
24.1 hipProfilerInitialize* 229
25 Profiler Control 229
25.1 hipProfilerStart 229
25.2 hipProfilerStop 229
26 Extend API 230
26.1 hipExtGetNearstCPU 230
26.2 hipExtGetLinkTypeAndHopCount 230
26.3 hipExtStreamCreateWithCUMask 231
26.4 hipExtStreamGetCUMask 231
26.5 hipExtMallocWithFlags 232
26.6 hipExtHostMalloc 232
26.7 hipExtMallocManaged 233
26.8 hipExtLaunchMultiKernelMultiDevice 233
26.9 hipExtLaunchKernel 233
27 Virtual Memory Management 234
27.1 hipMemAddressFree 234
27.2 hipMemAddressReserve 235
27.3 hipMemCreate 235
27.4 hipMemExportToShareableHandle 237

开发者社区

27.5 hipMemGetAccess 237
27.6 hipMemGetAllocationGranularity 238
27.7 hipMemGetAllocationPropertiesFromHandle 238
27.8 hipMemImportFromShareableHandle 239
27.9 hipMemMap 241
27.10 hipMemMapArrayAsync 241
27.11 hipMemRelease 243
27.12 hipMemRetainAllocationHandle 245
27.13 hipMemSetAccess 246
27.14 hipMemUnmap 248
28 Module Management 250
28.1 hipModuleGetGlobal 250
28.2 hipModuleLoad 250
28.3 hipModuleLoadFatBinary【dtk-26.10 新增】 251
28.4 hipLibraryLoadFromFile【dtk-26.10 新增】 251
28.5 hipLibraryLoadData【dtk-26.10 新增】 252
28.6 hipLibraryGetModule 253
28.7 hipLibraryGetKernelCount【dtk-26.10 新增】 253
28.8 hipLibraryGetKernel【dtk-26.10 新增】 254
28.9 hipKernelGetName 254
28.10 hipKernelGetLibrary 255
28.11 hipKernelGetParamInfo 255
28.12 hipLibraryGetGlobal 256
28.13 hipLibraryEnumerateKernels 256
28.14 hipLibraryGetManaged 257
28.15 hipKernelGetAttribute 257
28.16 hipKernelGetFunction 258
28.17 hipKernelSetAttribute 258
28.18 hipLibraryUnload【dtk-26.10 新增】 259
28.19 hipModuleUnload 259

28.20 hipModuleGetFunction 260
28.21 hipModuleGetFunctionCount 【dtk-26.10 新增】 260
28.22 hipModuleEnumerateFunctions 260
28.23 hipFuncIsLoaded 261
28.24 hipFuncLoad 261
28.25 hipFuncGetAttributes 262
28.26 hipFuncGetAttribute 262
28.27 hipFuncGetModule 263
28.28 hipFuncGetName 263
28.29 hipDrvFuncGetName 263
28.30 hipFuncGetParamInfo 264
28.31 hipDrvFuncGetParamInfo 264
28.32 hipModuleGetTexRef 265
28.33 hipModuleLoadData 265
28.34 hipModuleLoadDataEx 266
28.35 hipLinkAddData【dtk-26.10 新增】 267
28.36 hipLinkAddFile【dtk-26.10 新增】 267
28.37 hipLinkComplete【dtk-26.10 新增】 268
28.38 hipLinkCreate【dtk-26.10 新增】 268
28.39 hipLinkDestroy【dtk-26.10 新增】 269
28.40 hipModuleLaunchKernel 269
28.41 hipModuleLaunchCooperativeKernel 271
28.42 hipModuleLaunchCooperativeKernelMultiDevice 272
28.43 hipLaunchCooperativeKernel 272
28.44 hipLaunchCooperativeKernelMultiDevice 273
28.45 hipParamSeti 274
28.46 hipParamSetf 275
28.47 hipParamSetv 275
28.48 hipFuncSetBlockShape 276
28.49 hipFuncSetSharedSize 276

28.50 hipLaunch 276
28.51 hipLaunchGrid 277
28.52 hipLaunchGridAsync 277
28.53 hipModuleGetLoadingMode 278
28.54 hipDrvFuncSetAttribute 278
28.55 hipGetFuncBySymbol 279
28.56 hipGetDriverEntryPoint【dtk-26.10 新增】 279
28.57 hipParamSetSize 280
29 定制化 API 280
29.1 duDeviceGetHwQueueNum 280
29.2 duStreamGetHwQueue 281
29.3 duListStreamQueueIds 281
29.4 duDeviceFlushCache 282
29.5 duLaunchKernel 282
29.6 duModuleLaunchKernel【dtk-26.10 新增】 283
29.7 duAqlPacketCreate 286
29.8 duAqlPacketDestroy 286
29.9 duKernelGetAql 286
29.10 duKernelAqlLaunch 287
29.11 duKernelAqlBatchLaunch 289
29.12 duDeviceFlushHDP 289
29.13 duGraphGetNodeByLabel 290
29.14 duGraphExecKernelNodeGetParamsByLabel 294
29.15 duGraphExecKernelNodeSetParamsByLabel 294
29.16 duStreamWaitEvent 294
29.17 duEventRecord 295
29.18 duMemcpyAsync 295
29.19 duModuleLaunchKernel 296

开发者社区

修订记录：

日期	修正内容
2023.02.01	创建文档
2023.04.20	更新部分 Graph 相关接口内容
2025.6.14	DTK-25.04.1 新增
2026.3.13	新增 du 定制化接口和 vmm 接口

开发者社区

1 Device Management

1.1 hipChooseDevice

__host__cudaError_t cudaChooseDevice (int* device, const cudaDeviceProp* prop)

---

hipError_t hipChooseDevice(int* device, const hipDeviceProp_t* prop)

Device which matches hipDeviceProp_t is returned.

Parameters

[out]device ID

[in] device properties pointer

Returns

#hipSuccess, #hipErrorInvalidValue

```
hipDeviceProp_t prop;
int dev;
hipChooseDevice(&dev, &prop);
```

1.2 hipDeviceFlushGPUDirectRDMAWrites*

__host__cudaError_t cudaDeviceFlushGPUDirectRDMAWrites (cudaFlushGPUDirectRDMAWritesTarget target, cudaFlushGPUDirectRDMAWritesScope scope)

No hip

1.3 hipDeviceGetAttribute

__host____device__cudaError_t cudaDeviceGetAttribute (int* value, cudaDeviceAttr attr, int device)

---

hipError_t hipDeviceGetAttribute(int* pi, hipDeviceAttribute_t attr, int deviceId)

Query for a specific device attribute.

Parameters

[out]pi Pointer to value to return

[in] attr Attribute to query

[in] deviceId which device to query for information

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

```
int deviceId;
hipGetDevice(&deviceId);
hipDeviceProp_t props;
hipGetDeviceProperties(&props, deviceId) printf("info: running on device #%d %s\n", deviceId, props.name);
hipDeviceGetAttribute(deviceId, hipDeviceAttributeMaxThreadsPerBlock,\
props.maxThreadsPerBlock)
```

1.4 hipDeviceGetByPCIBusId

__host__cudaError_t cudaDeviceGetByPCIBusId (int* device, const char* pciBusId)

---

hipError_t hipDeviceGetByPCIBusId(int* device, const char* pciBusId)

Returns a handle to a compute device.

Parameters

[out]device Handle

[in] PCI Bus ID

Returns

#hipSuccess, #hipErrorInavlidDevice, #hipErrorInvalidValue

```
int main(void) {
    char pciBusId[13];
    int deviceCount = 0;
    hipGetDeviceCount(&deviceCount);
    HIPASSERT(deviceCount != 0);
    for (int i = 0; i < deviceCount; i++) {
        int pciBusID = -1;
        int pciDeviceID = -1;
        int pciDomainID = -1;
        int tempPciBusId = -1;
        int tempDeviceId = -1;
        HIPCHECK(hipDeviceGetPCIBusId(&pciBusId[0], 13, i));
```

开发者社区

```
            sscanf(pciBusId, "%04x:%02x:%02x", &pciDomainID, &pciBusID,
&pciDeviceID);
            HIPCHECK(hipDeviceGetAttribute(&tempPciBusId,
hipDeviceAttributePciBusId, i));
            if (pciBusID != tempPciBusId) {
               exit(EXIT_FAILURE);
            }
            HIPCHECK(hipDeviceGetByPCIBusId(&tempDeviceId, pciBusId));
            if (tempDeviceId != i) {
               exit(EXIT_FAILURE);
            }
         }
         passed();
      }
```

1.5 hipDeviceGetCacheConfig

__host___device__cudaError_t cudaDeviceGetCacheConfig (cudaFuncCache* pCacheConfig)

---

hipError_t hipDeviceGetCacheConfig(hipFuncCache_t* cacheConfig)

Set Cache configuration for a specific function.

Parameters

[in] cacheConfig

Returns

#hipSuccess, #hipErrorInitializationError

1.6 hipDeviceGetDefaultMemPool*

__host__cudaError_t cudaDeviceGetDefaultMemPool (cudaMemPool_t* memPool, int device)

---

hipError_t hipDeviceGetDefaultMemPool(hipMemPool_t* mem_pool, int device)

Returns the default memory pool of the specified device

Parameters

[out] mem_pool Default memory pool to return

[in] device Device index for query the default memory pool

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue, #hipErrorNotSupported

```
int mem_pool_support = 0;
HIP_CHECK(hipDeviceGetAttribute(&mem_pool_support,
hipDeviceAttributeMemoryPoolsSupported, device));
if (!mem_pool_support) {
  HipTest::HIP_SKIP_TEST("Test only runs on devices with memory pool
  support");
  return;
}
hipMemPool_t mem_pool;
HIP_CHECK(hipDeviceGetDefaultMemPool(&mem_pool, device));
REQUIRE(mem_pool != nullptr);
```

1.7 hipDeviceGetLimit

__host____device__cudaError_t cudaDeviceGetLimit (size_t* pValue, cudaLimit limit)

---

hipError_t hipDeviceGetLimit(size_t* pValue, enum hipLimit_t limit)

Get Resource limits of current device.

Parameters

[out]pValue

[in] limit

Returns

#hipSuccess, #hipErrorUnsupportedLimit, #hipErrorInvalidValue

Note: Currently, only hipLimitMallocHeapSize is available

```
int main() {
    size_t heap;
    assert(hipSuccess == hipDeviceGetLimit(&heap, hipLimitMallocHeapSize));
    assert(heap == 4194304);
```

```
}
```

1.8 hipDeviceGetMemPool*

__host__cudaError_t cudaDeviceGetMemPool (cudaMemPool_t* memPool, int device)

---

hipError_t hipDeviceGetMemPool(hipMemPool_t* mem_pool, int device)

Gets the current memory pool for the specified device

Parameters

[out] mem_pool Current memory pool on the specified device

[in] device Device index to query the current memory pool

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorNotSupported

See also

hipDeviceGetDefaultMemPool

1.9 hipDeviceGetNvSciSyncAttributes*

__host__cudaError_t cudaDeviceGetNvSciSyncAttributes (void* nvSciSyncAttrList, int device, int flags)

No hip

1.10 hipDeviceGetP2PAttribute

__host__cudaError_t cudaDeviceGetP2PAttribute (int* value, cudaDeviceP2PAttr attr, int srcDevice, int dstDevice)

---

hipError_t hipDeviceGetP2PAttribute(int* value, hipDeviceP2PAttr attr, int srcDevice, int dstDevice);

Returns a value for attr of link between two devices.

Parameters

[out] value

[in] attr

[in] srcDevice

[in] dstDevice

Returns

#hipSuccess, #hipErrorInavlidDevice

开发者社区

1.11 hipDeviceGetPCIBusId

__host__cudaError_t cudaDeviceGetPCIBusId (char* pciBusId, int len, int device)

hipError_t hipDeviceGetPCIBusId(char* pciBusId, int len, int device)

Returns a PCI Bus Id string for the device, overloaded to take int device ID.

Parameters

[out]pciBusId

[in] len

[in] device

Returns

#hipSuccess, #hipErrorInavlidDevice

```
 int main(void) {
    char pciBusId[13];
    int deviceCount = 0;
    HIPCHECK(hipGetDeviceCount(&deviceCount));
    HIPASSERT(deviceCount != 0);
    for (int i = 0; i < deviceCount; i++) {
       int pciBusID = -1;
       int pciDeviceID = -1;
       int pciDomainID = -1;
       int tempPciBusId = -1;
       int tempDeviceId = -1;
       HIPCHECK(hipDeviceGetPCIBusId(&pciBusId[0], 13, i));
       sscanf(pciBusId, "%04x:%02x:%02x", &pciDomainID, &pciBusID,
&pciDeviceID);
       HIPCHECK(hipDeviceGetAttribute(&tempPciBusId,
hipDeviceAttributePciBusId, i));
       if (pciBusID != tempPciBusId) {
          exit(EXIT_FAILURE);
       }
       HIPCHECK(hipDeviceGetByPCIBusId(&tempDeviceId, pciBusId));
       if (tempDeviceId != i) {
```

开发者社区

```
            exit(EXIT_FAILURE);
          }
        }
        passed();
    }
```

1.12 hipDeviceGetSharedMemConfig

__host____device__cudaError_t cudaDeviceGetSharedMemConfig(cudaSharedMemConfig* pConfig)

---

hipError_t hipDeviceGetSharedMemConfig(hipSharedMemConfig* pConfig)

Returns bank width of shared memory for current device.

Parameters

[out] pConfig

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInitializationError

1.13 hipDeviceGetStreamPriorityRange

__host__cudaError_t cudaDeviceGetStreamPriorityRange (int* leastPriority, int* greatestPriority)

---

hipError_t hipDeviceGetStreamPriorityRange(int* leastPriority, int* greatestPriority)

Returns numerical values that correspond to the least and greatest stream priority.

Parameters

[in,out] leastPriority pointer in which value corresponding to least priority is returned.

[in,out] greatestPriority pointer in which value corresponding to greatest priority is returned.

Returns in *leastPriority and *greatestPriority the numerical values that correspond to the least and greatest stream priority respectively. Stream priorities follow a convention where lower numbers imply greater priorities. The range of meaningful stream priorities is given by [*greatestPriority, *leastPriority]. If the user attempts to

create a stream with a priority value that is outside the the meaningful range as specified by this API, the priority is automatically clamped to within the valid range.

```
    #define OP(x) \
        int priority_##x; \
        bool enable_priority_##x = false;
      OP(low)
      OP(normal)
      OP(high)
      #undef OP
      HIPCHECK(hipDeviceGetStreamPriorityRange(&priority_low,
&priority_high));
      printf("HIP stream priority range - low: %d to high: %d\n", priority_low,
priority_high);

      // Check if priorities are indeed supported
      if ((priority_low - priority_high) == 0) { passed(); }

      // Enable/disable priorities based on number of available priority levels
      enable_priority_low = true;
      enable_priority_high = true;
      if ((priority_low - priority_high) > 1) enable_priority_normal = true;
      if (enable_priority_normal) priority_normal = ((priority_low - priority_high)
/ 2);
```

1.14 hipDeviceGetTexture1DLinearMaxWidth*

__host__cudaError_t cudaDeviceGetTexture1DLinearMaxWidth (size_t* maxWidthInElements, const cudaChannelFormatDesc* fmtDesc, int device)

---

hipError_t hipDeviceGetTexture1DLinearMaxWidth(size_t *maxWidthInElements, hipArray_Format format, unsigned numChannels, hipDevice_t device)

Returns the maximum number of elements allocatable in a 1D linear texture for a given texture element size.

Parameters

[out] maxWidthInElements - Returned maximum number of texture elements allocatable for given format and numChannels.

[in] format - Texture format.

[in] numChannels - Number of channels per texture element.

[in] device - Device handle.

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

1.15 hipDeviceReset

__host__cudaError_t cudaDeviceReset (void)

---

hipError_t hipDeviceReset(void)

The state of current device is discarded and updated to a fresh state.

Calling this function deletes all streams created, memory allocated, kernels running, events created. Make sure that no other thread is using the device or streams, memory, kernels, events associated with the current device.

Returns

#hipSuccess

See also

hipDeviceSynchronize

The state of current device is discarded and updated to a fresh state. Calling this function deletes all streams created, memory allocated, kernels running, events created. Make sure that no other thread is using the device or streams, memory, kernels, events associated with the current device. Returns #hipSuccess

1.16 hipDeviceSetCacheConfig

__host__cudaError_t cudaDeviceSetCacheConfig (cudaFuncCache cacheConfig)

---

hipError_t hipDeviceSetCacheConfig(hipFuncCache_t cacheConfig)

Set L1/Shared cache partition.

Parameters

[in] cacheConfig

Returns

#hipSuccess, #hipErrorInitializationError

1.17 hipDeviceSetLimit*

__host__cudaError_t cudaDeviceSetLimit (cudaLimit limit, size_t value)

---

hipError_t hipDeviceSetLimit (enum hipLimit_t limit, size_t value)

Sets resource limits of current device

Parameters

[in] limit enum of hipLimit_t to set

[in] value the size of limit value in bytes

Returns

#hipSuccess, #hipErrorUnsupportedLimit, #hipErrorInvalidValue

1.18 hipDeviceSetMemPool*

__host__cudaError_t cudaDeviceSetMemPool(int device, cudaMemPool_t memPool)

---

hipError_t hipDeviceSetMemPool(int device, hipMemPool_t mem_pool)

Sets the current memory pool of a device

Parameters

[in] device Device index for the update

[in] mem_pool Memory pool for update as the current on the specified device

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDevice, #hipErrorNotSupported

```
hipMemPool_t mem_pool;
int device = 0;
hipMemPoolProps pool_props;
```

开发者社区

```
pool_props.allocType = hipMemAllocationTypePinned;
pool_props.handleTypes = hipMemHandleTypePosixFileDescriptor;
pool_props.location.type = hipMemLocationTypeDevice;
pool_props.location.id = device;
pool_props.win32SecurityAttributes = nullptr;
memset(pool_props.reserved, 0, sizeof(pool_props.reserved));
HIP_CHECK(hipMemPoolCreate(&mem_pool, &pool_props));
HIP_CHECK(hipDeviceSetMemPool(device, mem_pool));
```

1.19 hipDeviceSetSharedMemConfig

__host__cudaError_t cudaDeviceSetSharedMemConfig (cudaSharedMemConfig config)

---

hipError_t hipDeviceSetSharedMemConfig(hipSharedMemConfig config)

he bank width of shared memory on current device is set.

Parameters

[in] config

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInitializationError

1.20 hipDeviceSynchronize

__host____device__cudaError_t cudaDeviceSynchronize (void)

---

hipError_t hipDeviceSynchronize(void)

Waits on all active streams on current device.

When this command is invoked, the host thread gets blocked until all the commands associated with streams associated with the device. HIP does not support multiple blocking modes (yet!).

Returns

#hipSuccess

See also

hipSetDevice, hipDeviceReset

```
HIPCHECK(memcopy(A_d, A_h, Nbytes, hipMemcpyHostToDevice));
```

开发者社区

```
        HIPCHECK(memcopy(B_d, B_h, Nbytes, hipMemcpyHostToDevice));
        hipLaunchKernelGGL(HipTest::vectorADD,                dim3(blocks),
dim3(threadsPerBlock), 0, 0,\
        static_cast<const int*>(A_d), static_cast<const int*>(B_d), C_d, N);
        HIPCHECK(memcopy(C_h, C_d, Nbytes, hipMemcpyDeviceToHost));
        HIPCHECK(hipDeviceSynchronize());
```

1.21 hipGetDevice

__host____device__cudaError_t cudaGetDevice (int* device)

---

hipError_t hipGetDevice(int* deviceId)

Return the default device id for the calling host thread.

Parameters

[out] device *device is written with the default device

HIP maintains an default device for each thread using thread-local-storage. This device is used implicitly for HIP runtime APIs called by this thread. hipGetDevice returns in * device the default device for the calling host thread.

Returns

#hipSuccess

See also

hipSetDevice, hipGetDevicesizeByte.

```
 int main() {
    int numDevices = 0;
    int device;
    HIPCHECK(hipGetDeviceCount(&numDevices));
    for (int i = 0; i < numDevices; i++) {
       HIPCHECK(hipSetDevice(i));
       HIPCHECK(hipGetDevice(&device));
       HIPASSERT(device == i);
    }
    passed();
 }
```

开发者社区

1.22 hipGetDeviceCount

__host____device__cudaError_t cudaGetDeviceCount (int* count)

hipError_t hipGetDeviceCount(int* count)

Return number of compute-capable devices.

Parameters

[output] count Returns number of compute-capable devices.

Returns

#hipSuccess, #hipErrorNoDevice

Returns in *count the number of devices that have ability to run compute commands. If there are no such devices, then hipGetDeviceCount will return #hipErrorNoDevice. If 1 or more devices can be found, then hipGetDeviceCount returns #hipSuccess.

```
int main() {
    int numDevices = 0;
    int device;
    HIPCHECK(hipGetDeviceCount(&numDevices));
    for (int i = 0; i < numDevices; i++) {
        HIPCHECK(hipSetDevice(i));
        HIPCHECK(hipGetDevice(&device));
        HIPASSERT(device == i);
    }
    passed();
}
```

1.23 hipGetDeviceFlags

__host__cudaError_t cudaGetDeviceFlags (unsigned int* flags)

hipError_t hipGetDeviceFlags(unsigned int* flags)

Gets the flags set for current device.

Parameters

[out] flags

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

1.24 hipGetDeviceProperties

__host__cudaError_t cudaGetDeviceProperties (cudaDeviceProp* prop, int device)

---

hipError_t hipGetDeviceProperties(hipDeviceProp_t* prop, int deviceId)

Returns device properties.

Parameters

[out] prop Written with device properties

[in] deviceId Which device to query for information

Returns

#hipSuccess, #hipErrorInvalidDevice

Bug:

HCC always returns 0 for maxThreadsPerMultiProcessor

HCC always returns 0 for regsPerBlock

HCC always returns 0 for l2CacheSize

Populates hipGetDeviceProperties with information for the specified device.

```
int main(int argc, char* argv[]) {
   int deviceId;
CHECK(hipGetDevice(&deviceId));
   hipDeviceProp_t props;
   CHECK(hipGetDeviceProperties(&props, deviceId));
printf("info: running on device #%d %s\n", deviceId, props.name)
}
```

1.25 hipIpcCloseMemHandle

__host__cudaError_t cudaIpcCloseMemHandle (void* devPtr)

---

hipError_t hipIpcCloseMemHandle(void* devPtr)

Close memory mapped with hipIpcOpenMemHandle.

Unmaps memory returnd by hipIpcOpenMemHandle. The original allocation in the exporting process as well as imported mappings in other processes will be unaffected.

Any resources used to enable peer access will be freed if this is the last mapping using them.

Parameters

devPtr Device pointer returned by hipIpcOpenMemHandle

Returns

hipSuccess, hipErrorMapBufferObjectFailed, hipErrorInvalidResourceHandle

1.26 hipIpcGetEventHandle

__host__cudaError_t cudaIpcGetEventHandle (cudaIpcEventHandle_t* handle, cudaEvent_t event)

---

hipError_t hipIpcGetEventHandle(hipIpcEventHandle_t* handle, hipEvent_t event)

Opens an interprocess event handles.

Opens an interprocess event handle exported from another process with cudaIpcGetEventHandle. The returned hipEvent_t behaves like a locally created event with the hipEventDisableTiming flag specified. This event need be freed with hipEventDestroy. Operations on the imported event after the exported event has been freed with hipEventDestroy will result in undefined behavior. If the function is called within the same process where handle is returned by hipIpcGetEventHandle, it will return hipErrorInvalidContext.

Parameters

[out] event Pointer to hipEvent_t to return the event

[in] handle The opaque interprocess handle to open

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidContext

1.27 hipIpcGetMemHandle

__host__cudaError_t cudaIpcGetMemHandle (cudaIpcMemHandle_t* handle, void* devPtr)

---

hipError_t hipIpcGetMemHandle(hipIpcMemHandle_t* handle, void* devPtr)

Gets an interprocess memory handle for an existing device memory allocation.

Takes a pointer to the base of an existing device memory allocation created with hipMalloc and exports it for use in another process. This is a lightweight operation and may be called multiple times on an allocation without adverse effects.

If a region of memory is freed with hipFree and a subsequent call to hipMalloc returns memory with the same device address, hipIpcGetMemHandle will return a unique handle for the new memory.

Parameters

handle Pointer to user allocated hipIpcMemHandle to return the handle in.

devPtr Base pointer to previously allocated device memory

Returns

#hipSuccess, #hipErrorInvalidResourceHandle, #hipErrorMemoryAllocation, #hipErrorMapBufferObjectFailed

1.28 hipIpcOpenEventHandle

__host__cudaError_t cudaIpcOpenEventHandle (cudaEvent_t* event, cudaIpcEventHandle_t handle)

---

hipError_t hipIpcOpenEventHandle(hipEvent_t* event, hipIpcEventHandle_t handle)

1.29 hipIpcOpenMemHandle

__host__cudaError_t cudaIpcOpenMemHandle (void** devPtr, cudaIpcMemHandle_t handle, unsigned int flags)

---

hipError_t hipIpcOpenMemHandle (void** devPtr, hipIpcMemHandle_t handle, unsigned int flags)

Opens an interprocess memory handle exported from another process and returns a device pointer usable in the local process.

Maps memory exported from another process with hipIpcGetMemHandle into the current device address space. For contexts on different devices hipIpcOpenMemHandle

can attempt to enable peer access between the devices as if the user called hipDeviceEnablePeerAccess. This behavior is controlled by the hipIpcMemLazyEnablePeerAccess flag. hipDeviceCanAccessPeer can determine if a mapping is possible.

Contexts that may open hipIpcMemHandles are restricted in the following way. hipIpcMemHandles from each device in a given process may only be opened by one context per device per other process.

Memory returned from hipIpcOpenMemHandle must be freed with hipIpcCloseMemHandle.

Calling hipFree on an exported memory region before calling hipIpcCloseMemHandle in the importing context will result in undefined behavior.

Parameters

devPtr Returned device pointer

handle HipIpcMemHandle to open

flags Flags for this operation. Must be specified as hipIpcMemLazyEnablePeerAccess.

Returns

#hipSuccess, #hipErrorMapBufferObjectFailed, #hipErrorInvalidResourceHandle, #hipErrorTooManyPeers

Note

No guarantees are made about the address returned in *devPtr. In particular, multiple processes may not receive the same address for the same handle.

1.30 hipSetDevice

__host__cudaError_t cudaSetDevice (int device)

---

hipError_t hipSetDevice(int deviceId)

Set default device to be used for subsequent hip API calls from this thread.

Parameters

[in] deviceId Valid device in range 0...hipGetDeviceCount().

Sets device as the default device for the calling host thread. Valid device id's are

0... (hipGetDeviceCount()-1).

Many HIP APIs implicitly use the "default device" :

Any device memory subsequently allocated from this host thread (using hipMalloc) will be allocated on device.

Any streams or events created from this host thread will be associated with device.

Any kernels launched from this host thread (using hipLaunchKernel) will be executed on device (unless a specific stream is specified, in which case the device associated with that stream will be used).

This function may be called from any host thread. Multiple host threads may use the same device. This function does no synchronization with the previous or new device, and has very little runtime overhead. Applications can use hipSetDevice to quickly switch the default device before making a HIP runtime call which uses the default device.

The default device is stored in thread-local-storage for each thread. Thread-pool implementations may inherit the default device of the previous thread. A good practice is to always call hipSetDevice at the start of HIP coding sequency to establish a known standard device.

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorDeviceAlreadyInUse

See also

hipGetDevice, hipGetDeviceCount

```
int main() {
    HIP_CHECK(hipSetDevice(0), hipSetDevice);
    HIP_CHECK(hipSetDevice(1026), hipSetDevice);
    HIP_CHECK(hipSetDevice(-1), hipSetDevice);
}
```

1.31 hipSetDeviceFlags

__host__cudaError_t cudaSetDeviceFlags (unsigned int flags)

---

hipError_t hipSetDeviceFlags(unsigned flags)

The current device behavior is changed according the flags passed.

Parameters

[in] flags The schedule flags impact how HIP waits for the completion of a command running on a device. hipDeviceScheduleSpin : HIP runtime will actively spin in the thread which submitted the work until the command completes. This offers the lowest latency, but will consume a CPU core and may increase power. hipDeviceScheduleYield : The HIP runtime will yield the CPU to system so that other tasks can use it. This may increase latency to detect the completion but will consume less power and is friendlier to other tasks in the system. hipDeviceScheduleBlockingSync : On ROCm platform, this is a synonym for hipDeviceScheduleYield. hipDeviceScheduleAuto : Use a hueristic to select between Spin and Yield modes. If the number of HIP contexts is greater than the number of logical processors in the system, use Spin scheduling. Else use Yield scheduling.

hipDeviceMapHost : Allow mapping host memory. On ROCM, this is always allowed and the flag is ignored. hipDeviceLmemResizeToMax :

Warning

ROCm silently ignores this flag.

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorSetOnActiveProcess

```
 for (int j = 0; j < deviceCount; j++) {
        HIPCHECK(hipSetDevice(j));
        for (int i = 0; i < 4; i++) {
           flag = 1 << i;
           printf("Flag=%x\n", flag);
           HIPCHECK(hipSetDeviceFlags(flag));
           // HIPCHECK_API(hipSetDeviceFlags(flag), hipErrorInvalidValue);
        }
        flag = 0;
     }
```

1.32 hipSetValidDevices*

__host__cudaError_t cudaSetValidDevices (int* device_arr, int len)

---

hipError_t hipSetValidDevices(int* device_arr, int len)

Set a list of devices that can be used for dtk.

Parameters

[in] device_arr List of devices to try.

[in] len Number of devices in specified list.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDevice

```
int DeviceArray[2] = {0, 1};
HIP_CHECK(hipSetValidDevices(DeviceArray, 2));
```

1.33 hipDeviceGetProperties

__host__ CUresult cuDeviceGetProperties(CUdevprop* prop, CUdevice dev)

---

hipError_t hipDeviceGetProperties (hipDevprop* prop, int dev)

Returns properties for a selected device.

Parameters

[out] prop Returned properties of device

[in] dev Device to get properties for

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDevice

```
hipDevprop prop;
memset(&prop, 0, sizeof(hipDevprop));
HIP_CHECK(hipDeviceGetProperties(&prop, 0));
```

1.34 hipInitDevice【dtk-25.04.1 新增】

__host__cudaError_t cudaInitDevice (int device, unsigned int deviceFlags, unsigned int flags)

---

hipError_t hipInitDevice(int device, unsigned int deviceFlags, unsigned int flags)

Initialize device to be used for GPU executions.

Parameters

[in] device Device on which the runtime will initialize itself.

[in] deviceFlags Parameters for device operation.

[in] flags Flags for controlling the device initialization.

Returns

#hipSuccess, #hipErrorInvalidValue

```
int device = 0;
unsigned int flags = 0;
unsigned int deviceFlags = 0;
HIP_CHECK(hipSetDevice(device));
HIP_CHECK(hipInitDevice(device, deviceFlags, flags));
```

2 Tread Management

2.1 hipThreadExit*

__host__cudaError_t cudaThreadExit (void)

No hip

2.2 hipThreadGetCacheConfig*

__host__cudaError_t cudaThreadGetCacheConfig (cudaFuncCache *pCacheConfig)

No hip

2.3 hipThreadGetLimit*

__host__cudaError_t cudaThreadGetLimit (size_t *pValue, cudaLimit limit)

---

hipError_t hipThreadGetLimit (size_t* pValue, enum hipLimit_t limit)

Gets resource limits of current device

Parameters

[out] pValue returns the size of the limit in bytes

[in] limit the limit to query

Returns

#hipSuccess, #hipErrorUnsupportedLimit, #hipErrorInvalidValue

开发者社区

```
size_t value = 10;
hipLimit_t limit = hipLimitStackSize;
REQUIRE(hipSuccess == hipThreadSetLimit(limit, value));
REQUIRE(hipSuccess == hipThreadGetLimit(&value, limit));
```

2.4 hipThreadSetCacheConfig*

__host__cudaError_t cudaThreadSetCacheConfig (cudaFuncCache cacheConfig)

No hip

2.5 hipThreadSetLimit*

__host__cudaError_t cudaThreadSetLimit (cudaLimit limit, size_t value)

hipError_t hipDeviceSetLimit (enum hipLimit_t limit, size_t value)

Sets resource limits of current device.

As the input enum limit, hipLimitStackSize sets the limit value of the stack size on current GPU devie, hipLimitMallocHeapSize sets the limit value of the heap used by the malloc()/free() calls.

Parameters

[in] limit enum of hipLimit_t to set

[in] value the size of limit value in bytes

Returns

#hipSuccess, #hipErrorUnsupportedLimit, #hipErrorInvalidValue

See also

hipThreadGetLimit

2.6 hipThreadSynchronize*

__host__cudaError_t cudaThreadSynchronize (void)

No hip

3 Error Handing

3.1 hipGetErrorName

__host____device__const char *cudaGetErrorName (cudaError_t error)

const char* hipGetErrorName(hipError_t hip_error)

开发者社区

Return name of the specified error code in text form.

Parameters

hip_error Error code to convert to name.

Returns

const char pointer to the NULL-terminated error name

See also

hipGetErrorString, hipGetLastError, hipPeakAtLastError, hipError_t

```
 hipError_t e = hipEventElapsedTime(&t, start, start);
    if ((e != hipSuccess) && (e != hipErrorNotReady)) {
      failed("start event not in expected state, was %d=%s\n", e,
hipGetErrorName(e));
    }
```

3.2 hipGetErrorString

__host____device__const char *cudaGetErrorString (cudaError_t error)

---

const char* hipGetErrorString(hipError_t hipError)

Return handy text string message to explain the error which occurred.

Parameters

hipError Error code to convert to string.

Returns

const char pointer to the NULL-terminated error string

Warning

On HCC, this function returns the name of the error (same as hipGetErrorName)

See also

hipGetErrorName, hipGetLastError, hipPeakAtLastError, hipError_t

开发者社区

```
#define CHECK(cmd)
    {
        hipError_t error = cmd;
        if (error != hipSuccess) {
            fprintf(stderr, "error: '%s'(%d) at %s:%d\n", hipGetErrorString(error), error,        \
            __FILE__, __LINE__);
            return (EXIT_FAILURE);
        }
    }
```

3.3 hipGetLastError

__host____device__cudaError_t cudaGetLastError (void)

hipError_t hipGetLastError(void)

Return last error returned by any HIP runtime API call and resets the stored error code to #hipSuccess.

Returns

return code from last HIP called from the active host thread

Returns the last error that has been returned by any of the runtime calls in the same host thread, and then resets the saved error to #hipSuccess.

See also

hipGetErrorString, hipGetLastError, hipPeakAtLastError, hipError_t

```
#define CHECK_HIP_ERROR()
    {
        hipError_t err = hipGetLastError();
        if (err != hipSuccess) {
            printf("error=%d name=%s at " "ln: %d\n  ",  err, hipGetErrorString(err),
__LINE__);
                                                        exit(EXIT_FAILURE);
        }
    }
```

3.4 hipPeekAtLastError

__host____device__cudaError_t cudaPeekAtLastError (void)

---

hipError_t hipPeekAtLastError(void)

Return last error returned by any HIP runtime API call.

Returns

#hipSuccess

Returns the last error that has been returned by any of the runtime calls in the same host thread. Unlike hipGetLastError, this function does not reset the saved error code.

See also

hipGetErrorString, hipGetLastError, hipPeakAtLastError, hipError_t

```
int main() {
    hipSetDevice(-1);
    if (hipPeekAtLastError() != hipSuccess) passed();
}
```

4 Stream Management

4.1 hipCtxResetPersistingL2Cache*

__host__cudaError_t cudaCtxResetPersistingL2Cache (void)

No hip

4.2 hipStreamAddCallback

__host__cudaError_t cudaStreamAddCallback (cudaStream_t stream, cudaStreamCallback_t callback, void *userData, unsigned int flags)

---

hipError_t hipStreamAddCallback(hipStream_t stream, hipStreamCallback_t callback, void* userData, unsigned int flags)

Adds a callback to be called on the host after all currently enqueued items in the stream have completed. For each cudaStreamAddCallback call, a callback will be executed exactly once. The callback will block later work in the stream until it is finished.

Parameters

[in] stream Stream to add callback to

[in] callback The function to call once preceding stream operations are complete

[in] userData User specified data to be passed to the callback function

[in] flags Reserved for future use, must be 0

Returns

#hipSuccess, #hipErrorInvalidResourceHandle, #hipErrorNotSupported

See also

hipStreamCreate, hipStreamCreateWithFlags, hipStreamQuery, hipStreamSynchronize, hipStreamWaitEvent, hipStreamDestroy, hipStreamCreateWithPriority

```
hipStream_t mystream;
HIPCHECK(hipStreamCreateWithFlags(&mystream, hipStreamNonBlocking))
HIPCHECK(hipStreamAddCallback(mystream, Callback, NULL, 0));
onBlocking));
```

4.3 hipStreamAttachMemAsync

__host__cudaError_t cudaStreamAttachMemAsync (cudaStream_t stream, void *devPtr, size_t length, unsigned int flags)

---

hipError_t hipStreamAttachMemAsync(hipStream_t stream,void* dev_ptr,size_t length __dparm(0),unsigned int flags __dparm(hipMemAttachSingle));

Attach memory to a stream asynchronously.

Parameters

[out] ptr Pointer to the allocated host pinned memory

[in] size Requested memory size

[in] flags Type of host memory allocation

If size is 0, no memory is allocated, *ptr returns nullptr, and hipSuccess is returned.

Returns

#hipSuccess, #hipErrorOutOfMemory

Deprecated

Use hipHostMalloc() instead

4.4 hipStreamBeginCapture

__host__cudaError_t cudaStreamBeginCapture (cudaStream_t stream)

---

hipError_t hipStreamBeginCapture(hipStream_t stream, hipStreamCaptureMode mode);

4.5 hipStreamCopyAttributes*

__host__cudaError_t cudaStreamCopyAttributes (cudaStream_t dst, cudaStream_t src)

---

hipError_t hipStreamCopyAttributes(hipStream_t dst, hipStream_t src)

Copies attributes from source stream to destination stream.

Copies attributes from source stream src to destination stream dst.Both streams must have the same context.

Parameters

[out] dst Destination stream

[in] src Source stream

Returns

#hipSuccess, # hipErrorNotSupported

```
hipStream_t stream1 = nullptr;
hipStream_t stream2 = nullptr;
hipStreamAttrValue attrValue1;
hipStreamAttrValue attrValue2;
memset(&attrValue1, 0, sizeof(hipStreamAttrValue));
memset(&attrValue2, 0, sizeof(hipStreamAttrValue));
HIP_CHECK(hipStreamCreate(&stream1));
HIP_CHECK(hipStreamCreate(&stream2));
attrValue1.syncPolicy = hipSyncPolicyYield;
attrValue2.syncPolicy = hipSyncPolicySpin;
HIP_CHECK(hipStreamSetAttribute(stream1,
```

```
hipStreamAttributeSynchronizationPolicy, &attrValue1));
    HIP_CHECK(hipStreamSetAttribute(stream2,
hipStreamAttributeSynchronizationPolicy, &attrValue2));
    memset(&attrValue1, 0, sizeof(hipStreamAttrValue));
    memset(&attrValue2, 0, sizeof(hipStreamAttrValue));
    HIP_CHECK(hipStreamGetAttribute(stream1,
hipStreamAttributeSynchronizationPolicy, &attrValue1));
    HIP_CHECK(hipStreamGetAttribute(stream2,
hipStreamAttributeSynchronizationPolicy, &attrValue2));
    REQUIRE(attrValue1.syncPolicy == hipSyncPolicyYield);
    REQUIRE(attrValue2.syncPolicy == hipSyncPolicySpin);
    HIP_CHECK(hipStreamCopyAttributes(stream1, stream2));
    memset(&attrValue1, 0, sizeof(hipStreamAttrValue));
    memset(&attrValue2, 0, sizeof(hipStreamAttrValue));
    HIP_CHECK(hipStreamGetAttribute(stream1,
hipStreamAttributeSynchronizationPolicy, &attrValue1));
    HIP_CHECK(hipStreamGetAttribute(stream2,
hipStreamAttributeSynchronizationPolicy, &attrValue2));
    REQUIRE(attrValue1.syncPolicy == hipSyncPolicySpin);
    REQUIRE(attrValue2.syncPolicy == hipSyncPolicySpin);
```

4.6 hipStreamCreate

__host__cudaError_t cudaStreamCreate (cudaStream_t *pStream)

---

hipError_t hipStreamCreate(hipStream_t* stream)

Create an asynchronous stream.

Parameters

[in,out] stream Valid pointer to hipStream_t. This function writes the memory with the newly created stream.

Returns

#hipSuccess, #hipErrorInvalidValue

Create a new asynchronous stream. stream returns an opaque handle that can be used to reference the newly created stream in subsequent hipStream* commands. The

stream is allocated on the heap and will remain allocated even if the handle goes out-of-scope. To release the memory used by the stream, applicaiton must call hipStreamDestroy.

Returns

#hipSuccess, #hipErrorInvalidValue

See also

hipStreamCreateWithFlags, hipStreamCreateWithPriority, hipStreamSynchronize, hipStreamWaitEvent, hipStreamDestroy

```
hipStream_t stream;
HIPCHECK(hipStreamCreate(&stream));
hipLaunchKernelGGL(HIP_KERNEL_NAME(Inc), dim3(N / 512), dim3(512), 0, stream, Ad);
```

4.7 hipStreamCreateWithFlags

__host____device__cudaError_t cudaStreamCreateWithFlags (cudaStream_t *pStream, unsigned int flags)

---

hipError_t hipStreamCreateWithFlags(hipStream_t* stream, unsigned int flags)

Create an asynchronous stream.

Parameters

[in,out] stream	Pointer to new stream.
[in] flags	To control stream creation.

Returns

#hipSuccess, #hipErrorInvalidValue

Create a new asynchronous stream. stream returns an opaque handle that can be used to reference the newly created stream in subsequent hipStream* commands. The stream is allocated on the heap and will remain allocated even if the handle goes out-of-scope. To release the memory used by the stream, applicaiton must call hipStreamDestroy. Flags controls behavior of the stream. See hipStreamDefault, hipStreamNonBlocking.

See also

hipStreamCreate, hipStreamCreateWithPriority, hipStreamSynchronize, hipStreamWaitEvent, hipStreamDestroy

```
hipStream_t stream;
HIPCHECK(hipStreamCreateWithFlags(&stream, hipStreamDefault));
```

4.8 hipStreamCreateWithPriority

__host__cudaError_t cudaStreamCreateWithPriority (cudaStream_t *pStream, unsigned int flags, int priority)

---

hipError_t hipStreamCreateWithPriority(hipStream_t* stream, unsigned int flags, int priority)

Create an asynchronous stream with the specified priority.

Parameters

[in,out] stream Pointer to new stream

[in] flags to Control stream creation.

[in] priority Of the stream. Lower numbers represent higher priorities.

Returns

#hipSuccess, #hipErrorInvalidValue

Create a new asynchronous stream with the specified priority. stream returns an opaque handle that can be used to reference the newly created stream in subsequent hipStream* commands. The stream is allocated on the heap and will remain allocated even if the handle goes out-of-scope. To release the memory used by the stream, applicaiton must call hipStreamDestroy. Flags controls behavior of the stream. See hipStreamDefault, hipStreamNonBlocking.

See also

hipStreamCreate, hipStreamSynchronize, hipStreamWaitEvent, hipStreamDestroy

```
#define OP(x) \
    hipStream_t stream_##x; \
    if (enable_priority_##x) { \
        HIPCHECK(hipStreamCreateWithPriority(&stream_##x,
```

开发者社区

```
hipStreamDefault, priority_##x)); \
        }
    OP(low)
    OP(normal)
    OP(high)
    #undef OP
```

4.9 hipStreamDestroy

__host____device__cudaError_t cudaStreamDestroy (cudaStream_t stream)

---

hipError_t hipStreamDestroy(hipStream_t stream)

Destroys the specified stream.

Parameters

[in,out] stream Valid pointer to hipStream_t. This function writes the memory with the newly created stream.

Returns

#hipSuccess #hipErrorInvalidResourceHandle

Destroys the specified stream.

If commands are still executing on the specified stream, some may complete execution before the queue is deleted.

The queue may be destroyed while some commands are still inflight, or may wait for all commands queued to the stream before destroying it.

See also

hipStreamCreate, hipStreamCreateWithFlags, hipStreamCreateWithPriority, hipStreamQuery, hipStreamWaitEvent, hipStreamSynchronize

Returns

#hipSuccess, #hipErrorInvalidResourceHandle

```
  hipStream_t stream;
  unsigned int flags;
  HIPCHECK(hipStreamCreateWithFlags(&stream, hipStreamDefault));
  HIPCHECK(hipStreamGetFlags(stream, &flags));
  HIPASSERT(flags == 0);
```

开发者社区

```
HIPCHECK(hipStreamDestroy(stream));
```

4.10 hipStreamEndCapture

__host__cudaError_t cudaStreamEndCapture (cudaStream_t stream, cudaGraph_t *pGraph)

---

hipError_t hipStreamEndCapture(hipStream_t stream, hipGraph_t* pGraph)

Ends capture on a stream, returning the captured graph.

Parameters

[out] pGraph Pointer to graph to create.

[in] flags Flags for graph creation, must be 0.

Returns

#hipSuccess

4.11 hipStreamGetAttribute*

_host__cudaError_t cudaStreamGetAttribute (cudaStream_t hStream, cudaStreamAttrID attr, cudaStreamAttrValue* value_out)

---

hipError_t hipStreamGetAttribute(hipStream_t hStream, hipStreamAttrID attr, hipStreamAttrValue *value_out)

Queries stream attribute.

Queries attribute attr from hStream and stores it in corresponding member of value_out.

Parameters

[in] hStream Stream to query

[in] attr attributes to query

[out] value_out value of attributes

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidResourceHandle

See also

hipStreamCopyAttributes

开发者社区

4.12 hipStreamGetCaptureInfo

__host__cudaError_t cudaStreamGetCaptureInfo(cudaStream_t stream, cudaStreamCaptureStatus ** pCaptureStatus, unsigned long long* pId)

---

hipError_t hipStreamGetCaptureInfo(hipStream_t stream, hipStreamCaptureStatus* pCaptureStatus, unsigned long long* pId)

Get capture status of a stream.

Parameters

[in] stream - Stream under capture.

[out] pCaptureStatus - returns current status of the capture.

[out] pId - unique ID of the capture.

Returns

#hipSuccess, #hipErrorStreamCaptureImplicit

4.13 hipStreamGetCaptureInfo_v2*

__host__cudaError_t cudaStreamGetCaptureInfo_v2(cudaStream_t stream, cudaStreamCaptureStatus ** captureStatus_out, unsigned long long* id_out = 0, cudaGraph_t* graph_out = 0, const cudaGraphNode_t** dependencies_out = 0, size_t* numDependencies_out = 0)

No hip

4.14 hipStreamGetFlags

__host__cudaError_t cudaStreamGetFlags (cudaStream_t hStream, unsigned int *flags)

---

hipError_t hipStreamGetFlags(hipStream_t stream, unsigned int* flags)

Return flags associated with this stream.

Parameters

[in] stream Stream to be queried

[in,out]flags Pointer to an unsigned integer in which the stream's flags are returned

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidResourceHandle

#hipSuccess #hipErrorInvalidValue #hipErrorInvalidResourceHandle

Return flags associated with this stream in *flags.

See also

hipStreamCreateWithFlags

```
 hipStream_t stream;
    unsigned int flags;
    HIPCHECK(hipStreamCreateWithFlags(&stream, hipStreamDefault));
    HIPCHECK(hipStreamGetFlags(stream, &flags));
    HIPASSERT(flags == 0);
    HIPCHECK(hipStreamDestroy(stream));

    HIPCHECK(hipStreamCreateWithFlags(&stream, hipStreamNonBlocking));
    HIPCHECK(hipStreamGetFlags(stream, &flags));
    HIPASSERT(flags == 1);
    HIPCHECK(hipStreamDestroy(stream));
    passed();
```

4.15 hipStreamGetPriority

__host__cudaError_t cudaStreamGetPriority (cudaStream_t hStream, int *priority)

---

hipError_t hipStreamGetPriority(hipStream_t stream, int* priority)

Query the priority of a stream.

Parameters

[in] stream Stream to be queried

[in,out] priority Pointer to an unsigned integer in which the stream's priority is returned

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidResourceHandle

#hipSuccess #hipErrorInvalidValue #hipErrorInvalidResourceHandle

Query the priority of a stream. The priority is returned in in priority.

See also

hipStreamCreateWithFlags

4.16 hipStreamIsCapturing*

__host__cudaError_t cudaStreamIsCapturing (cudaStream_t stream, cudaStreamCaptureStatus *pCaptureStatus)

---

hipError_t hipStreamIsCapturing(hipStream_t stream, hipStreamCaptureStatus* pCaptureStatus)

Get stream's capture state

Parameters

[in] stream - Stream under capture.

[out] pCaptureStatus - returns current status of the capture.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorStreamCaptureImplicit

4.17 hipStreamQuery

__host__cudaError_t cudaStreamQuery (cudaStream_t stream)

---

hipError_t hipStreamQuery(hipStream_t stream)

Return #hipSuccess if all of the operations in the specified stream have completed, or #hipErrorNotReady if not.

Parameters

[in]stream Stream to query

Returns

#hipSuccess, #hipErrorNotReady, #hipErrorInvalidResourceHandle

This is thread-safe and returns a snapshot of the current state of the queue. However, if other host threads are sending work to the stream, the status may change immediately after the function is called. It is typically used for debug.

See also

hipStreamCreate, hipStreamCreateWithFlags, hipStreamCreateWithPriority, hipStreamWaitEvent, hipStreamSynchronize, hipStreamDestroy

```
void Streamer<T>::queryUntilComplete() {
```

```
    HIPCHECK(hipSetDevice(_deviceId));
    int numQueries = 0;
    hipError_t e = hipSuccess;
    do {
      numQueries++;
      e = hipStreamQuery(_stream);
    } while (e != hipSuccess);

    printf("info: hipStreamQuery completed after %d queries\n", numQueries);
  };
```

4.18 hipStreamSetAttribute*

__host__cudaError_t cudaStreamSetAttribute (cudaStream_t hStream, cudaStreamAttrID attr, const cudaStreamAttrValue* value)

---

hipError_t hipStreamSetAttribute(hipStream_t hStream, hipStreamAttrID attr, const hipStreamAttrValue *value)

Sets attribute attr on hStream from corresponding attribute of value. The updated attribute will be applied to subsequent work submitted to the stream. It will not affect previously submitted work.

Parameters

[out] hStream Stream to set

[in] attr attributes to set

[in] value value to set

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidResourceHandle

See also

hipStreamCopyAttributes

4.19 hipStreamSynchronize

__host__cudaError_t cudaStreamSynchronize (cudaStream_t stream)

---

hipError_t hipStreamSynchronize(hipStream_t stream)

Wait for all commands in stream to complete.

Parameters

[in] stream Stream identifier.

Returns

#hipSuccess, #hipErrorInvalidResourceHandle

This command is host-synchronous : the host will block until the specified stream is empty.

This command follows standard null-stream semantics. Specifically, specifying the null stream will cause the command to wait for other streams on the same device to complete all pending operations.

This command honors the hipDeviceLaunchBlocking flag, which controls whether the wait is active or blocking.

See also

hipStreamCreate, hipStreamCreateWithFlags, hipStreamCreateWithPriority, hipStreamWaitEvent, hipStreamDestroy

```
hipStream_t stream;
    hipStreamCreate(&stream);
    hipMemcpyHtoDAsync(Ad, Ah, SIZE, stream);
    hipStreamSynchronize(stream);
```

4.20 hipStreamUpdateCaptureDependencies*

__host__cudaError_t cudaStreamUpdateCaptureDependencies (cudaStream_t stream, cudaGraphNode_t* dependencies, size_t numDependencies, unsigned int flags = 0)

---

hipError_t hipStreamUpdateCaptureDependencies(hipStream_t stream, hipGraphNode_t* dependencies, size_t numDependencies, unsigned int flags __dparm(0))

Update the set of dependencies in a capturing stream.

Parameters

[in] stream - Stream under capture.

[in] dependencies - pointer to an array of nodes to Add/Replace.

[in] numDependencies - size of the array in dependencies.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorIllegalState

4.21 hipStreamWaitEvent

__host____device__cudaError_t cudaStreamWaitEvent (cudaStream_t stream, cudaEvent_t event, unsigned int flags)

---

hipError_t hipStreamWaitEvent(hipStream_t stream, hipEvent_t event, unsigned int flags)

Make the specified compute stream wait for an event.

Parameters

[in] stream Stream to make wait.

[in] event Event to wait on

[in] flags Control operation [must be 0]

Returns

#hipSuccess, #hipErrorInvalidResourceHandle

This function inserts a wait operation into the specified stream. All future work submitted to stream will wait until event reports completion before beginning execution.

This function only waits for commands in the current stream to complete. Notably,, this function does not impliciy wait for commands in the default stream to complete, even if the specified stream is created with hipStreamNonBlocking = 0.

See also

hipStreamCreate, hipStreamCreateWithFlags, hipStreamCreateWithPriority, hipStreamSynchronize, hipStreamDestroy

```
hipStream_t _stream;
hipEvent_t _event;
int _deviceId;
HIPCHECK(hipEventCreate(&_event));
HIPCHECK(hipStreamWaitEvent(_stream, depStreamer->event(), 0));
```

4.22 hipThreadExchangeStreamCaptureMode*

__host__cudaError_t cudaThreadExchangeStreamCaptureMode (cudaStreamCaptureMode ** mode)

---

hipError_t hipThreadExchangeStreamCaptureMode(hipStreamCaptureMode* mode)

Swaps the stream capture mode of a thread.

Parameters

[in] mode - Pointer to mode value to swap with the current mode

Returns

#hipSuccess, #hipErrorInvalidValue

4.23 hipStreamGetDevice

No cuda

---

hipError_t hipStreamGetDevice(hipStream_t stream, hipDevice_t* device)

Get the device assocaited with the stream

Parameters

[in]stream Stream to be queried.

[out]device device associated with the stream

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorContextIsDestroyed, #hipErrorInvalidHandle, #hipErrorNotInitialized, #hipErrorDeinitialized, #hipErrorInvalidContext

See also

hipStreamCreate, hipStreamDestroy, hipDeviceGetStreamPriorityRange

4.24 hipStreamBeginCaptureToGraph【dtk-25.04.1 新增】

__host__cudaError_t cudaStreamBeginCaptureToGraph (cudaStream_t stream, cudaGraph_t graph, const cudaGraphNode_t* dependencies, const cudaGraphEdgeData* dependencyData, size_t numDependencies, cudaStreamCaptureMode mode)

---

hipError_t hipStreamBeginCaptureToGraph(hipStream_t stream, hipGraph_t graph, const hipGraphNode_t* dependencies, const hipGraphEdgeData* dependencyData, size_t numDependencies, hipStreamCaptureMode mode);

Begins graph capture on a stream to an existing graph.

Parameters

[in] stream - Stream to initiate capture.

[in] graph - Graph to capture into.

[in] dependencies - Dependencies of the first node captured in the stream. Can be NULL if numDependencies is 0.

[in] dependencyData - Optional array of data associated with each dependency.

[in] numDependencies - Number of dependencies.

[in] mode - Controls the interaction of this capture sequence with other API calls that are not safe.

Returns

#hipSuccess, #hipErrorInvalidValue

See also

hipStreamCreate, hipStreamIsCapturing, hipStreamEndCapture, hipThreadExchangeStreamCaptureMode

```
hipStream_t stream;
hipGraph_t graph;
hipGraphNode_t dummyNode;
hipStreamCaptureMode mode = hipStreamCaptureModeGlobal;
hipGraphNode_t deps[] = {dummyNode};
hipGraphEdgeData dependencyData{};
hipStreamCreate(&stream);
hipGraphCreate(&graph, 0);
hipGraphAddEmptyNode(&dummyNode, graph, nullptr, 0);
hipStreamBeginCaptureToGraph(stream, graph, deps, nullptr, 1, hipStreamCaptureModeGlobal)
```

开发者社区

4.25 hipStreamGetId【dtk-25.04.1 新增】

__host__cudaError_t cudaStreamGetId (cudaStream_t hStream, unsigned long long* streamId)

---

hipError_t hipStreamGetId(hipStream_t hStream, unsigned long long* streamId);

Query the Id of a stream.

Parameters

[in] Handle to the stream to be queried.

[out] Pointer to an unsigned long long in which the stream Id is returned.

Returns

#hipSuccess, #hipErrorInvalidValue

See also

hipStreamCreateWithPriority, hipStreamCreateWithFlags, hipStreamGetPriority, hipStreamGetFlags

```
hipStream_t stream;
unsigned long long id = 0;
(hipStreamCreate(&stream);
hipStreamGetId(stream, &id);
hipStreamDestroy(stream);
```

4.26 hipStreamBatchMemOp【dtk-26.10 新增】

CUresult cuStreamBatchMemOp(CUstream stream, unsigned int count, CUstreamBatchMemOpParams *paramArray, unsigned int flags)

---

hipError_t hipStreamBatchMemOp(hipStream_t stream, unsigned int count, hipStreamBatchMemOpParams *paramArray, unsigned int flags)

Batch operations to synchronize the stream via memory operations.

Parameters:

stream – [in] - The stream to enqueue the operations in.

count – [in] - The number of operations in the array. Must be less than 256.

开发者社区

paramArray – [in] - The types and parameters of the individual operations.

flags – [in] - Reserved for future expansion; must be 0.

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorContextIsDestroyed, hipErrorOutOf Memory, hipErrorNotInitialized, hipErrorInvalidDevice, hipErrorNoDevice

4.27 hipGraphAddBatchMemOpNode【dtk-26.10 新增】

CUresult cuGraphAddBatchMemOpNode (CUgraphNode* phGraphNode, CUgr aph hGraph, const CUgraphNode* dependencies, size_t numDependencies, const CUDA_BATCH_MEM_OP_NODE_PARAMS* nodeParams)

---

hipError_t hipGraphAddBatchMemOpNode(hipGraphNode_t *pGraphNode, hip Graph_t graph, const hipGraphNode_t *pDependencies, size_t numDependencies, co nst hipBatchMemOpNodeParams *pNodeParams)

Creates a batch memory operation node and adds it to a graph.

Parameters:

pGraphNode – [out] - pointer to graph node to create.

graph – [in] - instance of graph to add the created node.

pDependencies – [in] - const pointer to the dependencies on the memcpy execution node.

numDependencies – [in] - the number of the dependencies.

pNodeParams – [in] - pointer to memory address to the destination.

Returns:

hipErrorNoDevice, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInva lidDevice, hipErrorInvalidValue, hipSuccess

4.28 hipGraphBatchMemOpNodeGetParams【dtk-26.10 新增】

CUresult cuGraphBatchMemOpNodeGetParams(CUgraphNode hNode, CUDA_ BATCH_MEM_OP_NODE_PARAMS *nodeParams_out)

---

hipError_t hipGraphBatchMemOpNodeGetParams(hipGraphNode_t node, hipBatchMemOpNodeParams *pNodeParams)

Returns a batch mem op node’s parameters.

Parameters:

node – [in] - Node to get the parameters for.

pNodeParams – [out] - Pointer to return the parameters.

Returns:

hipErrorNoDevice, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorInvalidValue, hipSuccess

4.29 hipGraphBatchMemOpNodeSetParams【dtk-26.10 新增】

CUresult cuGraphBatchMemOpNodeSetParams(CUgraphNode hNode, const CUDA_BATCH_MEM_OP_NODE_PARAMS *nodeParams)

---

hipError_t hipGraphBatchMemOpNodeSetParams(hipGraphNode_t node, const hipBatchMemOpNodeParams *pNodeParams)

Sets a batch mem op node’s parameters.

Parameters:

node – [in] - Node to set the parameters for.

pNodeParams – [in] - Parameters to copy.

Returns:

hipErrorNoDevice, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorInvalidValue, hipSuccess

4.30 hipGraphExecBatchMemOpNodeSetParams【dtk-26.10 新增】

CUresult cuGraphExecBatchMemOpNodeSetParams(CUgraphExec hGraphExec, CUgraphNode hNode, const CUDA_BATCH_MEM_OP_NODE_PARAMS *nodeParams)

hipError_t hipGraphExecBatchMemOpNodeSetParams(hipGraphExec_t hGraphExec, hipGraphNode_t node, const hipBatchMemOpNodeParams *pNodeParams)

Sets the parameters for a batch mem op node in the given graphExec.

Parameters:

hGraphExec – [in] - The executable graph in which to set the specified node

node – [in] - Batch mem op node from the graph from which graphExec was instantiated.

pNodeParams – [in] - Updated Parameters to set.

Returns:

hipErrorNoDevice, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorInvalidValue, hipSuccess

4.31 hipStreamSetAttribute【dtk-26.10 新增】

CUresult cuStreamSetAttribute (CUstream hStream, CUstreamAttrID attr, const CUstreamAttrValue* value)

---

hipError_t hipStreamSetAttribute(hipStream_t hStream, hipStreamAttrID attr, const hipStreamAttrValue *value)

Sets stream attribute.

Sets attribute attr on hStream from corresponding attribute of value. The updated attribute will be applied to subsequent work submitted to the stream. It will not affect previously submitted work.

Parameters:

hStream – [out] Stream to update

attr – [in] Attribute to set

value – [in] Attribute value to set

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidResourceHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

4.32 hipStreamGetAttribute【dtk-26.10 新增】

CUresult cuStreamGetAttribute (CUstream hStream, CUstreamAttrID attr, CUstreamAttrValue* value_out)

---

hipError_t hipStreamGetAttribute(hipStream_t hStream, hipStreamAttrID attr, hipStreamAttrValue *value_out)

Queries stream attribute.

Queries attribute attr from hStream and stores it in corresponding member of value_out.

Parameters:

hStream – [in] Stream to query

attr – [in] Attribute to query

value_out – [out] Returned attribute value

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidResourceHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

4.33 hipStreamGetId【dtk-26.10 新增】

cudaError_t CUDARTAPI cudaStreamGetId(cudaStream_t hStream, unsigned long long *streamId)

---

hipError_t hipStreamGetId(hipStream_t hStream, unsigned long long *streamId)

Query the Id of a stream.

Parameters:

hStream – [in] Handle to the stream to be queried

streamId – [out] Pointer to an unsigned long long in which the stream Id is returned

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

开发者社区

5 Event Management

5.1 hipEventCreate

__host__cudaError_t cudaEventCreate (cudaEvent_t *event)

---

hipError_t hipEventCreate(hipEvent_t* event)

Create an event

Parameters

[in,out]event Returns the newly created event.

Returns

#hipSuccess, #hipErrorInitializationError, #hipErrorInvalidValue, #hipErrorLaunchFailure, #hipErrorMemoryAllocation

See also

hipEventCreateWithFlags, hipEventRecord, hipEventQuery, hipEventSynchronize, hipEventDestroy, hipEventElapsedTime

```
hipEvent_t start, stop;
HIPCHECK(hipEventCreate(&start));
HIPCHECK(hipEventSynchronize(start));
```

5.2 hipEventCreateWithFlags

__host____device__cudaError_t cudaEventCreateWithFlags (cudaEvent_t *event, unsigned int flags)

---

hipError_t hipEventCreateWithFlags(hipEvent_t* event, unsigned flags)

Create an event with the specified flags.

Parameters

[in,out] event Returns the newly created event.

[in] flags Flags to control event behavior. Valid values are hipEventDefault, hipEventBlockingSync, hipEventDisableTiming, hipEventInterprocess

hipEventDefault : Default flag. The event will use active synchronization and will support timing. Blocking synchronization provides lowest possible latency at the

expense of dedicating a CPU to poll on the eevent. hipEventBlockingSync : The event will use blocking synchronization : if hipEventSynchronize is called on this event, the thread will block until the event completes. This can increase latency for the synchroniation but can result in lower power and more resources for other CPU threads. hipEventDisableTiming : Disable recording of timing information. On ROCM platform, timing information is always recorded and this flag has no performance benefit.

Warning

On HCC platform, hipEventInterprocess support is under development. Use of this flag will return an error.

Returns

#hipSuccess, #hipErrorInitializationError, #hipErrorInvalidValue, #hipErrorLaunchFailure, #hipErrorMemoryAllocation

See also

hipEventCreate, hipEventSynchronize, hipEventDestroy, hipEventElapsedTime

```
hipEvent_t start, sync, stop;
check(hipEventCreate(&start));
check(hipEventCreateWithFlags(&sync, hipEventBlockingSync));
check(hipEventCreate(&stop));
```

5.3 hipEventDestroy

__host____device__cudaError_t cudaEventDestroy (cudaEvent_t event)

---

hipError_t hipEventDestroy(hipEvent_t event)

Destroy the specified event.

Parameters

[in]event Event to destroy.

Returns

#hipSuccess, #hipErrorInitializationError, #hipErrorInvalidValue, #hipErrorLaunchFailure

Releases memory associated with the event. If the event is recording but has not completed recording when hipEventDestroy() is called, the function will return

immediately and the completion_future resources will be released later, when the hipDevice is synchronized.

See also

hipEventCreate, hipEventCreateWithFlags, hipEventQuery, hipEventSynchronize, hipEventRecord, hipEventElapsedTime

Returns

#hipSuccess

```
hipEvent_t start, sync, stop;
check(hipEventCreate(&start));
check(hipEventCreateWithFlags(&sync, hipEventBlockingSync));
check(hipEventCreate(&stop));
check(hipEventDestroy(start));
check(hipEventDestroy(sync));
check(hipEventDestroy(stop));
```

5.4 hipEventElapsedTime

__host__cudaError_t cudaEventElapsedTime (float *ms, cudaEvent_t start, cudaEvent_t end)

---

hipError_t hipEventElapsedTime(float* ms, hipEvent_t start, hipEvent_t stop)

Return the elapsed time between two events.

Parameters

[out] ms Return time between start and stop in ms.

[in] start Start event.

[in] stop Stop event.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorNotReady, #hipErrorInvalidResourceHandle, #hipErrorInitializationError, #hipErrorLaunchFailure

Computes the elapsed time between two events. Time is computed in ms, with a resolution of approximately 1 us.

Events which are recorded in a NULL stream will block until all commands on all other streams complete execution, and then record the timestamp.

Events which are recorded in a non-NULL stream will record their timestamp when they reach the head of the specified stream, after all previous commands in that stream have completed executing. Thus the time that the event recorded may be significantly after the host calls hipEventRecord().

If hipEventRecord() has not been called on either event, then #hipErrorInvalidResourceHandle is returned. If hipEventRecord() has been called on both events, but the timestamp has not yet been recorded on one or both events (that is, hipEventQuery() would return #hipErrorNotReady on at least one of the events), then #hipErrorNotReady is returned.

See also

hipEventCreate, hipEventCreateWithFlags, hipEventQuery, hipEventDestroy, hipEventRecord, hipEventSynchronize

```
hipEvent_t start, stop;
HIPCHECK(hipEventCreate(&start));
HIPCHECK(hipEventCreate(&stop));
float eventMs = 1.0f;
HIPCHECK(hipEventElapsedTime(&eventMs, start, stop));
printf("kernel_time (hipEventElapsedTime) =%6.3fms\n", eventMs);
printf("\n");
```

5.5 hipEventQuery

__host__cudaError_t cudaEventQuery (cudaEvent_t event)

---

hipError_t hipEventQuery(hipEvent_t event)

Query event status.

Parameters

[in] event Event to query.

Returns

#hipSuccess, #hipErrorNotReady, #hipErrorInvalidResourceHandle,

#hipErrorInvalidValue, #hipErrorInitializationError, #hipErrorLaunchFailure

Query the status of the specified event. This function will return #hipErrorNotReady if all commands in the appropriate stream (specified to hipEventRecord()) have completed. If that work has not completed, or if hipEventRecord() was not called on the event, then #hipSuccess is returned.

See also

hipEventCreate, hipEventCreateWithFlags, hipEventRecord, hipEventDestroy, hipEventSynchronize, hipEventElapsedTime

```
IPCHECK(hipEventCreate(&stop));
HIPCHECK(hipEventRecord(stop, 0 /*default*/));
hipError_t done = hipEventQuery(stop);
if (expectMismatch)
{
    assert(done == hipErrorNotReady);
  }
else {
    assert(done == hipSuccess);
  }
```

5.6 hipEventRecord

__host__ __device__ cudaError_t cudaEventRecord (cudaEvent_t event, cudaStream_t stream)

---

hipError_t hipEventRecord(hipEvent_t event, hipStream_t stream = NULL)

Record an event in the specified stream.

Parameters

[in] event Event to record.

[in] stream Stream in which to record event.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInitializationError, #hipErrorInvalidResourceHandle, #hipErrorLaunchFailure

hipEventQuery() or hipEventSynchronize() must be used to determine when the

event transitions from "recording" (after hipEventRecord() is called) to "recorded" (when timestamps are set, if requested).

Events which are recorded in a non-NULL stream will transition to from recording to "recorded" state when they reach the head of the specified stream, after all previous commands in that stream have completed executing.

If hipEventRecord() has been previously called on this event, then this call will overwrite any existing state in event.

If this function is called on a an event that is currently being recorded, results are undefined

either outstanding recording may save state into the event, and the order is not guaranteed.

See also

hipEventCreate, hipEventCreateWithFlags, hipEventQuery, hipEventSynchronize, hipEventDestroy, hipEventElapsedTime

```
hipEvent_t start, stop;
// NULL stream check:
HIPCHECK(hipEventCreate(&start));
HIPCHECK(hipEventCreate(&stop));
HIPCHECK(hipEventRecord(start, NULL));
hipLaunchKernelGGL(HipTest::vectorADD, dim3(blocks), dim3(threadsPerBlock), 0, 0,
static_cast<const float*>(A_d), static_cast<const float*>(B_d), C_d, N);
HIPCHECK(hipEventRecord(stop, NULL));
HIPCHECK(hipEventSynchronize(stop));
```

5.7 hipEventRecordWithFlags*

__host__cudaError_t cudaEventRecordWithFlags (cudaEvent_t event, cudaStream_t stream = 0, unsigned int flags = 0)

---

hipError_t hipEventRecordWithFlags(hipEvent_t event, hipStream_t stream = NULL, unsigned int flags = 0)

Record an event in the specified stream with the specified flags.

hipEventRecordDefault : Default flag. Default event creation flag, it will perform seem with hipEventRecord.

hipEventRecordExternal : Event is captured in the graph as an EventRecord node when performing stream capture.

Parameters

[in] event event to record.

[in] stream stream in which to record event.

[in] flags Flags to control event behavior.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidValue, #hipErrorLaunchFailure, #hipErrorOutOfMemory

5.8 hipEventSynchronize

__host__cudaError_t cudaEventSynchronize (cudaEvent_t event)

---

hipError_t hipEventSynchronize(hipEvent_t event)

Wait for an event to complete.

This function will block until the event is ready, waiting for all previous work in the stream specified when event was recorded with hipEventRecord().

If hipEventRecord() has not been called on event, this function returns immediately.

TODO-hcc - This function needs to support hipEventBlockingSync parameter.

Parameters

[in] event Event on which to wait.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInitializationError, #hipErrorInvalidResourceHandle, #hipErrorLaunchFailure

See also

hipEventCreate, hipEventCreateWithFlags, hipEventQuery, hipEventDestroy, hipEventRecord, hipEventElapsedTime

```
hipEvent_t start, stop;
HIPCHECK(hipEventCreate(&start));
```

```
HIPCHECK(hipEventCreate(&stop));
int waitStart;
HIPCHECK(hipEventRecord(start, stream));
if (waitStart)
{
HIPCHECK(hipEventSynchronize(start));
}
```

5.9 hipEventRecordWithFlags【dtk-26.10 新增】

cudaError_t CUDARTAPI cudaEventRecordWithFlags(cudaEvent_t event, cudaStream_t stream, unsigned int flags)

---

hipError_t hipEventRecordWithFlags(hipEvent_t event, hipStream_t stream = NULL, unsigned int flags = 0)

Record an event in the specified stream with the specified flags.

hipEventRecordDefault : Default flag. Default event creation flag, it will perform seem with hipEventRecord. hipEventRecordExternal : When performing stream capture on a hipGraph, the event will be captured as an EventRecord node in the hipGraph.

Parameters:

event – [in] event to record.

stream – [in] stream in which to record event.

flags – [in] Flags to control event behavior. Valid values are hipEventRecordDefault, hipEventRecordExternal

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidHandle, hipErrorContextIsDestroyed, hipErrorIllegalState, hipErrorStreamCaptureInvalidated, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

See also

hipEventCreate, hipEventCreateWithFlags, hipEventQuery, hipEventSynchronize

, hipEventDestroy, hipEventElapsedTime, hipStreamWaitEvent, hipEventRecord

6 External Resource Interoperability

6.1 hipDestroyExternalMemory

__host__cudaError_t cudaDestroyExternalMemory (cudaExternalMemory_t extMem)

---

hipError_t hipDestroyExternalMemory(hipExternalMemory_t extMem);

Destroys an external memory object.

Parameters

[in] extMem External memory object to be destroyed

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

6.2 hipDestroyExternalSemaphore

__host__cudaError_t cudaDestroyExternalSemaphore (cudaExternalSemaphore_t extSem)

---

hipError_t hipDestroyExternalSemaphore(hipExternalSemaphore_t extSem)

Destroys an external semaphore object and releases any references to the underlying resource. Any outstanding signals or waits must have completed before the semaphore is destroyed.

Parameters

[in] extSem Handle to an external memory object

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

6.3 hipExternalMemoryGetMappedBuffer

__host__cudaError_t cudaExternalMemoryGetMappedBuffer (void www.nvidia.com CUDA Runtime API vRelease Version | 76 **devPtr, cudaExternalMemory_t extMem, const cudaExternalMemoryBufferDesc *bufferDesc)

---

hipError_t hipExternalMemoryGetMappedBuffer(void **devPtr, hipExternalMemory_t extMem, const hipExternalMemoryBufferDesc *bufferDesc)

developer.sourcefind.cn

Maps a buffer onto an imported memory object.

Parameters

[out] devPtr Returned device pointer to buffer

[in] extMem Handle to external memory object

[in] bufferDesc Buffer descriptor

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

6.4 hipExternalMemoryGetMappedMipmappedArray*

__host__cudaError_t cudaExternalMemoryGetMappedMipmappedArray (cudaMipmappedArray_t *mipmap, cudaExternalMemory_t extMem, const cudaExternalMemoryMipmappedArrayDesc *mipmapDesc)

---

hipError_t hipExternalMemoryGetMappedMipmappedArray(hipMipmappedArray_t* mipmap, hipExternalMemory_t extMem, const hipExternalMemoryMipmappedArrayDesc* mipmapDesc)

Maps a mipmapped array onto an external memory object.

Parameters

[out] mipmap mipmapped array to return

[in] extMem external memory object handle

[in] mipmapDesc external mipmapped array descriptor

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidResourceHandle

6.5 hipImportExternalMemory

__host__cudaError_t cudaImportExternalMemory (cudaExternalMemory_t *extMem_out, const cudaExternalMemoryHandleDesc *memHandleDesc)

---

hipError_t hipImportExternalMemory(hipExternalMemory_t* extMem_out, const hipExternalMemoryHandleDesc* memHandleDesc)

Imports an external memory object.

Parameters

[out] extMem_out Returned handle to an external memory object

[in] memHandleDesc Memory import handle descriptor

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

6.6 hipImportExternalSemaphore

__host__cudaError_t cudaImportExternalSemaphore (cudaExternalSemaphore_t *extSem_out, const cudaExternalSemaphoreHandleDesc *semHandleDesc)

---

hipError_t hipImportExternalSemaphore(hipExternalSemaphore_t* extSem_out,const hipExternalSemaphoreHandleDesc* semHandleDesc)

Imports an external semaphore.

Parameters

[out] extSem_out External semaphores to be waited on

[in] semHandleDesc Semaphore import handle descriptor

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

6.7 hipSignalExternalSemaphoresAsync

__host__cudaError_t cudaSignalExternalSemaphoresAsync (const cudaExternalSemaphore_t *extSemArray, const cudaExternalSemaphoreSignalParams* paramsArray, unsigned int numExtSems, cudaStream_t stream)

---

hipError_t hipSignalExternalSemaphoresAsync(const hipExternalSemaphore_t* extSemArray, const hipExternalSemaphoreSignalParams* paramsArray, unsigned int numExtSems, hipStream_t stream)

Signals a set of external semaphore objects.

Parameters

[in] extSem_out External semaphores to be waited on

[in] paramsArray Array of semaphore parameters

[in] numExtSems Number of semaphores to wait on

[in] stream Stream to enqueue the wait operations in

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

6.8 hipWaitExternalSemaphoresAsync

__host__cudaError_t cudaWaitExternalSemaphoresAsync (const cudaExternalSemaphore_t *extSemArray, const www.nvidia.com CUDA Runtime API vRelease Version | 85 cudaExternalSemaphoreWaitParams* paramsArray, unsigned int numExtSems, cudaStream_t stream)

---

hipError_t hipWaitExternalSemaphoresAsync(const hipExternalSemaphore_t* extSemArray, const hipExternalSemaphoreWaitParams* paramsArray, unsigned int numExtSems, hipStream_t stream)

Waits on a set of external semaphore objects.

Parameters

[in] extSem_out External semaphores to be waited on

[in] paramsArray Array of semaphore parameters

[in] numExtSems Number of semaphores to wait on

[in] stream Stream to enqueue the wait operations in

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

7 Execution Control

7.1 hipFuncGetAttributes

__host____device__cudaError_t cudaFuncGetAttributes (cudaFuncAttributes* attr, const void *func)

---

hipError_t hipFuncGetAttributes(struct hipFuncAttributes* attr, const void* func)

Find out attributes for a given function.

Parameters

[out] attr

[in] func

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDeviceFunction

7.2 hipFuncSetAttribute

__host__cudaError_t cudaFuncSetAttribute (const void* func, cudaFuncAttribute attr, int value)

---

hipError_t hipFuncSetAttribute(const void* func, hipFuncAttribute attr, int value)

Set attribute for a specific function.

Parameters

[in] func

[in] attr

[in] value

Returns

#hipSuccess, #hipErrorInvalidDeviceFunction, #hipErrorInvalidValue

7.3 hipFuncSetCacheConfig

__host__cudaError_t cudaFuncSetCacheConfig (const void *func, cudaFuncCache cacheConfig)

---

hipError_t hipFuncSetCacheConfig(const void* func, hipFuncCache_t config)

Set Cache configuration for a specific function.

Parameters

[in] config;

Returns

#hipSuccess, #hipErrorInitializationError

```
int main() {
    hipFuncCache_t cacheConfig;
    void* func;
    hipFuncSetCacheConfig(func, cacheConfig);
    passed();
}
```

开发者社区

7.4 hipFuncSetSharedMemConfig

__host__cudaError_t cudaFuncSetSharedMemConfig (constvoid* func, cudaSharedMemConfig config)

---

hipError_t hipFuncSetSharedMemConfig(const void* func, hipSharedMemConfig config)

Set shared memory configuation for a specific function.

Parameters

[in] func

[in] config

Returns

#hipSuccess, #hipErrorInvalidDeviceFunction, #hipErrorInvalidValue

7.5 hipGetParameterBuffer*

__device__ void *cudaGetParameterBuffer (size_t alignment, size_t size)

No hip

7.6 hipGetParameterBufferV2*

__device__ void *cudaGetParameterBufferV2 (void *func, dim3 gridDimension, dim3 blockDimension, unsigned int sharedMemSize)

No hip

7.7 hipLaunchCooperativeKernel

__host__cudaError_t cudaLaunchCooperativeKernel (const void *func, dim3 gridDim, dim3 blockDim, void **args, size_t sharedMem, cudaStream_t stream)

---

hipError_t hipLaunchCooperativeKernel(const void* f, dim3 gridDim, dim3 blockDimX, void** kernelParams, unsigned int sharedMemBytes, hipStream_t stream)

Launches kernel f with launch parameters and shared memory on stream with arguments passed to kernelparams or extra, where thread blocks can cooperate and synchronize as they execute.

Parameters

[in] f Kernel to launch.

[in] gridDim Grid dimensions specified as multiple of blockDim.

[in] blockDim Block dimensions specified in work-items.

[in] kernelParams A list of kernel arguments.

[in] sharedMemBytes Amount of dynamic shared memory to allocate for this kernel. The HIP-Clang compiler provides support for extern shared declarations.

[in] stream Stream where the kernel should be dispatched. May be 0, in which case the default stream is used with associated synchronization rules.

Returns

#hipSuccess, #hipInvalidDevice, #hipErrorNotInitialized, #hipErrorInvalidValue, #hipErrorCooperativeLaunchTooLarge

7.8 hipLaunchCooperativeKernelMultiDevice

__host__cudaError_t cudaLaunchCooperativeKernelMultiDevice (cudaLaunchParams *launchParamsList, unsigned int numDevices, unsigned int flags)

---

hipError_t hipLaunchCooperativeKernelMultiDevice(hipLaunchParams* launchParamsList, int numDevices, unsigned int flags)

Launches kernels on multiple devices and guarantees all specified kernels are dispatched on respective streams before enqueuing any other work on the specified streams from any other threads.

Parameters

[in] hipLaunchParams List of launch parameters, one per device.

[in] numDevices Size of the launchParamsList array.

[in] flags Flags to control launch behavior.

Returns

#hipSuccess, #hipInvalidDevice, #hipErrorNotInitialized, #hipErrorInvalidValue

7.9 hipLaunchHostFunc*

__host__cudaError_t cudaLaunchHostFunc (cudaStream_t stream, cudaHostFn_t fn, void *userData)

---

hipError_t hipLaunchHostFunc(hipStream_t stream, hipHostFn_t fn, void* userData)

Enqueues a host function call in a stream.

Parameters

[in] stream - stream to enqueue work to.

[in] fn - function to call once operations enqueued preceeding are complete.

[in] userData - User-specified data to be passed to the function.

Returns

#hipSuccess, #hipErrorInvalidResourceHandle, #hipErrorInvalidValue,

#hipErrorNotSupported

7.10 hipLaunchKernel

__host__cudaError_t cudaLaunchKernel (const void *func, dim3 gridDim, dim3 blockDim, void **args, size_t sharedMem, cudaStream_t stream)

---

hipLaunchKernel(F kernel, const dim3& numBlocks, const dim3& dimBlocks,std::uint32_t groupMemBytes, hipStream_t stream, Args... args)

hipLaunchKernel is a variadic macro which accepts as parameters the launch configurations (grid dims, group dims, stream, dynamic shared size) followed by a variable number of kernel arguments. This sequence is then expanded into the appropriate kernel launch syntax depending on the platform.

While this can be a convenient single-line kernel launch syntax, the macro implementation can cause issues when nested inside other macros.

```
    int blocks = N / p_blockSize;
    printf("running with N=%zu p_blockSize=%d blocks=%d\n", N, p_blockSize, blocks);
    HIPCHECK(hipMemcpy(A_d, A_h, Nbytes, hipMemcpyHostToDevice));
    HIPCHECK(hipGetLastError());
    hipLaunchKernel(myKern, dim3(blocks), dim3(p_blockSize), 0, 0, C_d, A_d, N, 0);
```

7.11 hipSetDoubleForDevice*

__host__cudaError_t cudaSetDoubleForDevice (double *d)

No hip

7.12 hipSetDoubleForHost*

__host__cudaError_t cudaSetDoubleForHost (double *d)

No hip

7.13 hipLaunchKernelExC

__host__ __host__cudaError_t cudaLaunchKernelExC (const cudaLaunchConfig_t* config, const void* func, void** args)

hipError_t hipLaunchKernelExC(const hipLaunchConfig_t *config, const void *func, void **args)

Launches a CUDA function with launch-time configuration.

Parameters

[in] config Launch configuration

[in] func Kernel to launch

[in] args Array of pointers to kernel parameters

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDeviceFunction, #hipErrorInvalidConfiguration, #hipErrorNoBinaryForGpu, #hipErrorInvalidResourceHandle, #hipErrorOutOfMemory, #hipErrorInvalidDevice, #hipErrorLaunchFailure

```
hipLaunchAttribute attr;
attr.id = hipLaunchAttributeCooperative;
attr.val.cooperative = 1;
hipStream_t stream;
HIP_CHECK(hipStreamCreate(&stream));

dim3 dimBlock = dim3(1);
dim3 dimGrid  = dim3(1);
```

开发者社区

```
hipLaunchConfig_t config;
config.gridDim = dimGrid;
config.blockDim = dimBlock;
config.dynamicSmemBytes = dimBlock.x * sizeof(int);
config.stream = stream;
config.numAttrs = 1;
config.attrs = &attr;

void *params[3];
params[0] = (void*)&d_A;
params[1] = (void*)&d_B;
params[2] = (void*)&d_C;
HIP_CHECK(hipLaunchKernelExC(&config, reinterpret_cast<const void*>(add),
(void**)params)));
```

7.14 hipFuncGetName【dtk-25.04.1 新增】

```
__host__cudaError_t cudaFuncGetName ( const char** name, const void* func )
```

hipError_t hipFuncGetName(const char** name, const void* func)

Returns the function name for a device entry function pointer.

Parameters

[out] name The returned name of the function

[in] func The function pointer to retrieve name for

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDeviceFunction

```
extern "C"
__global__ void myKernel(){
  return;
}

const char* funcName = nullptr;
HIP_CHECK(hipFuncGetName(&funcName, (void*)myKernel));
REQUIRE(strcmp(funcName, "myKernel") == 0);
```

7.15 hipFuncGetParamInfo【dtk-25.04.1 新增】

__host__cudaError_t cudaFuncGetParamInfo (const void* func, size_t paramIndex, size_t* paramOffset, size_t* paramSize)

---

hipError_t hipFuncGetParamInfo(const void* func, size_t paramIndex, size_t* paramOffset, size_t* paramSize)

Returns the offset and size of a kernel parameter in the device-side parameter layout.

Parameters

[in] func The function to query

[in] paramIndex The parameter index to query

[out] paramOffset The offset into the device-side parameter layout at which the parameter resides

[out] paramSize The size of the parameter in the device-side parameter layout

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDeviceFunction

```
size_t paramIndex = 1;
size_t paramOffset, paramSize;
HIP_CHECK(hipFuncGetParamInfo((void*)myKernel1, paramIndex, &paramOffset, &paramSize));
```

7.16 hipLaunchKernelExC【dtk-26.10 新增】

cudaError_t CUDARTAPI cudaLaunchKernelExC(const cudaLaunchConfig_t *config, const void *func, void **args)

---

hipError_t hipLaunchKernelExC(const hipLaunchConfig_t *config, const void *func, void **args)

Launch a HIP kernel using a generic function pointer and the specified configuration. This function is functionally equivalent to hipLaunchKernelEx, but accepts the kernel as a generic function pointer.

Parameters:

config – [in] - Pointer to the kernel launch configuration structure.

func – [in] - Pointer to the device kernel function.

args – [in] - Array of pointers to the kernel arguments.

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidDeviceFunction, hipErrorInvalidConfiguration, hipErrorInvalidImage, hipErrorInvalidKernelFile, hipErrorInvalidResourceHandle, hipErrorLaunchFailure, hipErrorCooperativeLaunchTooLarge, hipErrorNoBinaryForGpu, hipErrorUnknown, hipErrorIllegalState, hipErrorContextIsDestroyed, hipErrorOutOfMemory, hipErrorStreamCaptureInvalidated, hipErrorNotInitialized, hipErrorInvalidDevice, hipErrorNoDevice

7.17 hipDrvLaunchKernelEx【dtk-26.10 新增】

CUresult cuLaunchKernelEx(const CUlaunchConfig *config, CUfunction f, void **kernelParams, void **extra)

---

hipError_t hipDrvLaunchKernelEx(const HIP_LAUNCH_CONFIG *config, hipFunction_t func, void **kernelParams, void **extra)

Launch a HIP kernel using the driver API with the specified configuration. This function dispatches the device kernel represented by the HIP function object. It forwards the kernel arguments along with all additional configuration parameters to the kernel launch process.

Parameters:

config – [in] - Pointer to the kernel launch configuration structure.

func – [in] - HIP function object representing the device kernel to be launched.

kernelParams – [in] - Array of pointers to the kernel parameters.

extra – [in] - Array of pointers for additional launch parameters or extra configuration data.

Returns:

hipSuccess, hipErrorInvalidResourceHandle, hipErrorInvalidValue, hipErrorInvalidConfiguration

7.18 hipLaunchKernelEx【dtk-26.10 新增】

cudaError_t cudaLaunchKernelExC(const cudaLaunchConfig_t *config, const

void *func, void **args)

---

hipError_t hipLaunchKernelEx(const hipDrvLaunchConfig_t *config, hipFunction_t func, void **kernelParams, void **extra)

Launch a HIP kernel using the specified configuration. This function dispatches the provided kernel with the given launch configuration and forwards the kernel arguments.

Parameters:

config – [in] launch config

func – [in] kernel to launch

kernelParams – [in] Array of pointers to kernel parameters

extra – [in] Extra options

Returns:

hipSuccess, hipErrorInvalidResourceHandle, hipErrorInvalidValue, hipErrorInvalidConfiguration

8 Occupancy

8.1 hipOccupancyAvailableDynamicSMemPerBlock*

__host__cudaError_t cudaOccupancyAvailableDynamicSMemPerBlock(size_t* dynamicSmemSize, const void* func, int numBlocks, int blockSize)

No hip

8.2 hipOccupancyMaxActiveBlocksPerMultiprocessor

__host____device__cudaError_t cudaOccupancyMaxActiveBlocksPerMultiprocessor (int

*numBlocks, const void *func, int blockSize, size_t dynamicSMemSize)

---

hipError_t hipOccupancyMaxActiveBlocksPerMultiprocessor(int* numBlocks, const void* f, int blockSize, size_t dynSharedMemPerBlk)

Returns occupancy for a device function.

Parameters

[out] numBlocks	Returned occupancy
[in] func	Kernel function for which occupancy is calculated

[in] blockSize Block size the kernel is intended to be launched with

[in] dynSharedMemPerBlk Dynamic shared memory usage (in bytes) intended for each block

8.3 hipOccupancyMaxActiveBlocksPerMultiprocessorWithFlags

__host__cudaError_t cudaOccupancyMaxActiveBlocksPerMultiprocessorWithFlags (int *numBlocks, const void *func, int blockSize, size_t dynamicSMemSize, unsigned int flags)

---

hipError_t hipOccupancyMaxActiveBlocksPerMultiprocessorWithFlags(int* numBlocks, const void* f, int blockSize, size_t dynSharedMemPerBlk, unsigned int flags __dparm(hipOccupancyDefault))

Returns occupancy for a device function.

Parameters

[out] numBlocks Returned occupancy

[in] f Kernel function for which occupancy is calulated

[in] blockSize Block size the kernel is intended to be launched with

[in] dynSharedMemPerBlk Dynamic shared memory usage (in bytes) intended for each block

[in] flags Extra flags for occupancy calculation (currently ignored)

9 Execution Control [DEPRECATED since 7.0]

9.1 hipConfigureCall

__host__cudaError_t cudaConfigureCall (dim3 gridDim, dim3 blockDim, size_t sharedMem, cudaStream_t stream)

---

hipError_t hipConfigureCall(dim3 gridDim, dim3 blockDim, size_t sharedMem __dparm(0), hipStream_t stream __dparm(0))

Configure a kernel launch.

Parameters

[in] gridDim Grid dimension specified as multiple of blockDim.

[in] blockDim block Dimensions specified in work-items

[in] sharedMem Amount of dynamic shared memory to allocate for this kernel. The kernel can access this with HIP_DYNAMIC_SHARED.

[in] stream Stream where the kernel should be dispatched. May be 0, in which case the default stream is used with associated synchronization rules.

Returns

#hipSuccess, #hipInvalidDevice, #hipErrorNotInitialized, #hipErrorInvalidValue

9.2 hipLaunchByPtr

__host__cudaError_t cudaLaunch(const void *func)

---

hipError_t hipLaunchByPtr(const void* func)

Launch a kernel.

Parameters

[in] func Kernel to launch.

Returns

#hipSuccess, #hipInvalidDevice, #hipErrorNotInitialized, #hipErrorInvalidValue

9.3 hipSetupArgument

__host__cudaError_t cudaSetupArgument (const void *arg, size_t size, size_t offset)

---

hipError_t hipSetupArgument(const void* arg, size_t size, size_t offset)

Set a kernel argument.

Returns

#hipSuccess, #hipInvalidDevice, #hipErrorNotInitialized, #hipErrorInvalidValue

Parameters

[in] arg Pointer the argument in host memory.

[in] size Size of the argument.

[in] offset Offset of the argument on the argument stack.

9.4 hipLaunch*

__host__cudaError_t cudaLaunch (const void *func)

No hip

10 Memory Management

10.1 hipArrayGetInfo

__host__cudaError_t cudaArrayGetInfo (cudaChannelFormatDesc *desc, cudaExtent *extent, unsigned int *flags, cudaArray_t array)

---

hipError_t hipArrayGetInfo(hipChannelFormatDesc* desc, hipExtent* extent, unsigned int* flags, hipArray* array)

Gets info about the specified array

Parameters

[out] desc Returned array type

[out] extent Returned array shape. 2D arrays will have depth of zero

[out] flags Returned array flags

[in] array The HIP array to get info for

Returns

#hipSuccess, #hipErrorInvalidValue,#hipErrorInvalidHandle

See also

hipArrayGetDescriptor, hipArray3DGetDescriptor

10.2 hipArrayGetMemoryRequirements*

__host__cudaError_t cudaArrayGetMemoryRequirements (cudaArrayMemoryRequirements* memoryRequirements, cudaArray_t array, int device)

---

hipError_t hipArrayGetMemoryRequirements(hipArrayMemoryRequirements* memoryRequirements, hipArray_t array, int device)

Returns the memory requirements of a HIP array.

Returns the memory requirements of a HIP array in memoryRequirements. If the

开发者社区

HIP array is not allocated with flag hipArrayDeferredMapping hipErrorInvalidValue will be returned.

The returned value in hipArrayMemoryRequirements::size represents the total size of the HIP array.The returned value in hipArrayMemoryRequirements::alignment represents the alignment necessary for mapping the hip array.

Parameters

[out] memoryRequirements - Pointer to ::hipArrayMemoryRequirements

[in] array - HIP array to get the memory requirements of

[in] device - Device to get the memory requirements for

Returns

#hipSuccess, #hipErrorInvalidValue

```
hipArray *array = nullptr;
hipChannelFormatDesc desc;
desc.x = 32;
desc.y = 0;
desc.z = 0;
desc.w = 0;
desc.f = hipChannelFormatKindFloat;
HIP_CHECK(hipMallocArray(&array, &desc, 128, 128, hipArrayDeferredMapping));
hipArrayMemoryRequirements memReq;
HIP_CHECK(hipArrayGetMemoryRequirements(&memReq, array, 0));
```

10.3 hipArrayGetPlane*

__host__cudaError_t cudaArrayGetPlane(cudaArray_t* pPlaneArray, cudaArray_t hArray, unsigned int planeIdx)

No hip

10.4 hipArrayGetSparseProperties*

__host__cudaError_t cudaArrayGetSparseProperties(cudaArraySparseProperties* sparseProperties, cudaArray_t array)

No hip

10.5 hipFree

__host____device__cudaError_t cudaFree (void *devPtr)

---

hipError_t hipFree(void* ptr)

Free memory allocated by the hcc hip memory allocation API. This API performs an implicit hipDeviceSynchronize() call. If pointer is NULL, the hip runtime is initialized and hipSuccess is returned.

Parameters

[in] ptr Pointer to memory to be freed

Returns

#hipSuccess

#hipErrorInvalidDevicePointer (if pointer is invalid, including host pointers allocated with hipHostMalloc)

See also

hipMalloc, hipMallocPitch, hipMallocArray, hipFreeArray, hipHostFree, hipMalloc3D, hipMalloc3DArray, hipHostMalloc

```
hipMalloc((void**)&Ad, SIZE);
A = new float[N];
hipMemcpy(Ad, A, SIZE, hipMemcpyHostToDevice);
delete[] A;
hipFree(Ad);
```

10.6 hipFreeArray

__host__cudaError_t cudaFreeArray (cudaArray_t array)

---

hipError_t hipFreeArray(hipArray* array)

Frees an array on the device.

Parameters

[in] array Pointer to array to free

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInitializationError

See also

hipMalloc, hipMallocPitch, hipFree, hipMallocArray, hipHostMalloc, hipHostFree

```
hipArray *A_d;
hipFreeArray(A_d);
```

10.7 hipFreeHost

__host__cudaError_t cudaFreeHost (void *ptr)

---

hipError_t hipHostFree(void* ptr)

Free memory allocated by the hcc hip host memory allocation API. [Deprecated].

Parameters

[in] ptr Pointer to memory to be freed

Returns

#hipSuccess, #hipErrorInvalidValue (if pointer is invalid, including device pointers allocated with hipMalloc)

Deprecated:

use hipHostFree() instead

10.8 hipFreeMipmappedArray

__host__cudaError_t cudaFreeMipmappedArray (cudaMipmappedArray_t mipmappedArray)

---

hipError_t hipFreeMipmappedArray(hipMipmappedArray_t mipmappedArray)

Frees a mipmapped array on the device.

Parameters

[in] mipmappedArray Pointer to mipmapped array to free

Returns

#hipSuccess, #hipErrorInvalidValue

10.9 hipGetMipmappedArrayLevel

__host__cudaError_t cudaGetMipmappedArrayLevel (cudaArray_t *levelArray, cudaMipmappedArray_const_t mipmappedArray, unsigned int level)

---

hipError_t hipGetMipmappedArrayLevel(hipArray_t *levelArray,

hipMipmappedArray_const_t mipmappedArray, unsigned int level)

Gets a mipmap level of a HIP mipmapped array.

Parameters

[in] out levelArray Returned mipmap level HIP array

[in] mipmappedArray HIP mipmapped array

[in] level Mipmap level

Returns

#hipSuccess, #hipErrorInvalidValue

10.10 hipGetSymbolAddress

__host__cudaError_t cudaGetSymbolAddress (void **devPtr, const void *symbol)

---

hipError_t hipGetSymbolAddress(void** devPtr, const void* symbolName)

Copies the memory address of symbol symbolName to devPtr.

Parameters

[in] symbolName Symbol on device

[out]devPtr Pointer to a pointer to the memory referred to by the symbol

Returns

#hipSuccess, #hipErrorNotInitialized, #hipErrorNotFound

See also

hipGetSymbolSize, hipMemcpyToSymbol, hipMemcpyFromSymbol, hipMemcpyToSymbolAsync, hipMemcpyFromSymbolAsync

```
size_t symbolSize = 0;
int *symbolAddress;
hipGetSymbolSize(&symbolSize, HIP_SYMBOL(globalConst));
hipGetSymbolAddress((void**) &symbolAddress, HIP_SYMBOL(globalConst));
hipMalloc((void**)&checkOkD, sizeof(bool));
hipLaunchKernelGGL(checkAddress, dim3(1, 1, 1), dim3(1, 1, 1), 0, 0, symbolAddress, checkOkD);
```

```
    hipMemcpy(&checkOk, checkOkD, sizeof(bool), hipMemcpyDeviceToHost);
    hipFree(checkOkD);
```

10.11 hipGetSymbolSize

__host__cudaError_t cudaGetSymbolSize (size_t *size, const void *symbol)

---

hipError_t hipGetSymbolSize(size_t* size, const void* symbolName)

Copies the size of symbol symbolName to size.

Parameters

[in]	symbolName	Symbol on device
[out]	size	Pointer to the size of the symbol

Returns

#hipSuccess, #hipErrorNotInitialized, #hipErrorNotFound

See also

hipGetSymbolSize, hipMemcpyToSymbol, hipMemcpyFromSymbol, hipMemcpyToSymbolAsync, hipMemcpyFromSymbolAsync

```
    size_t symbolSize = 0;
      int *symbolAddress;
      hipGetSymbolSize(&symbolSize, HIP_SYMBOL(globalConst));
      hipGetSymbolAddress((void**) &symbolAddress, HIP_SYMBOL(globalConst));
      hipMalloc((void**)&checkOkD, sizeof(bool));
      hipLaunchKernelGGL(checkAddress, dim3(1, 1, 1), dim3(1, 1, 1), 0, 0, symbolAddress, checkOkD);
      hipMemcpy(&checkOk, checkOkD, sizeof(bool), hipMemcpyDeviceToHost);
    hipFree(checkOkD);
```

10.12 hipHostAlloc

__host__cudaError_t cudaHostAlloc (void **pHost, size_t size, unsigned int flags)

---

hipError_t hipHostMalloc(void** ptr, size_t size, unsigned int flags)

Allocate device accessible page locked host memory [Deprecated].

Parameters

[out]ptr Pointer to the allocated host pinned memory

[in] size Requested memory size

[in] flags Type of host memory allocation

If size is 0, no memory is allocated, *ptr returns nullptr, and hipSuccess is returned.

Returns

#hipSuccess, #hipErrorMemoryAllocation

10.13 hipHostGetDevicePointer

__host__cudaError_t cudaHostGetDevicePointer (void **pDevice, void *pHost, unsigned int flags)

---

hipError_t hipHostGetDevicePointer(void** devPtr, void* hstPtr, unsigned int flags)

Get Device pointer from Host Pointer allocated through hipHostMalloc.

Parameters

[out]dstPtr Device Pointer mapped to passed host pointer

[in] hstPtr Host Pointer allocated through hipHostMalloc

[in] flags Flags to be passed for extension

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryAllocation

See also

hipSetDeviceFlags, hipHostMalloc

```
int main() {
    float *A, *B, *C, *D;
    float *Ad, *Bd, *Cd, *Dd;
    unsigned int FlagA, FlagB, FlagC;
    FlagA = hipHostMallocWriteCombined | hipHostMallocMapped;
    FlagB = hipHostMallocWriteCombined | hipHostMallocMapped;
    FlagC = hipHostMallocMapped;
    hipDeviceProp_t prop;
    int device;
    HIPCHECK(hipGetDevice(&device));
```

```
        HIPCHECK(hipGetDeviceProperties(&prop, device));
        if (prop.canMapHostMemory != 1) {
           std::cout << "Exiting..." << std::endl;
        }
        HIPCHECK(hipHostMalloc((void**)&A,                                SIZE,
hipHostMallocWriteCombined | hipHostMallocMapped));
        HIPCHECK(hipHostMalloc((void**)&B,                                SIZE,
hipHostMallocWriteCombined | hipHostMallocMapped));
        HIPCHECK(hipHostMalloc((void**)&C, SIZE, hipHostMallocMapped));
        HIPCHECK(hipHostMalloc((void**)&D, SIZE, hipHostMallocDefault));
        unsigned int flagA, flagB, flagC;
        HIPCHECK(hipHostGetDevicePointer((void**)&Ad, A, 0));
        HIPCHECK(hipHostGetDevicePointer((void**)&Bd, B, 0));
        HIPCHECK(hipHostGetDevicePointer((void**)&Cd, C, 0));
        HIPCHECK(hipHostGetDevicePointer((void**)&Dd, D, 0));
        HIPCHECK(hipHostGetFlags(&flagA, A));
        HIPCHECK(hipHostGetFlags(&flagB, B));
        HIPCHECK(hipHostGetFlags(&flagC, C));}
```

10.14 hipHostGetFlags

__host__cudaError_t cudaHostGetFlags (unsigned int *pFlags, void *pHost)

---

hipError_t hipHostGetFlags(unsigned int* flagsPtr, void* hostPtr)

Return flags associated with host pointer.

Parameters

[out]flagsPtr　　Memory location to store flags

[in] hostPtr　　Host Pointer allocated through hipHostMalloc

Returns

#hipSuccess, #hipErrorInvalidValue

See also

hipHostMalloc

```
 int main() {
    float *A, *B, *C, *D;
```

```
        float *Ad, *Bd, *Cd, *Dd;
        unsigned int FlagA, FlagB, FlagC;
        FlagA = hipHostMallocWriteCombined | hipHostMallocMapped;
        FlagB = hipHostMallocWriteCombined | hipHostMallocMapped;
        FlagC = hipHostMallocMapped;
        hipDeviceProp_t prop;
        int device;
        HIPCHECK(hipGetDevice(&device));
        HIPCHECK(hipGetDeviceProperties(&prop, device));
        if (prop.canMapHostMemory != 1) {
           std::cout << "Exiting..." << std::endl;
        }
        HIPCHECK(hipHostMalloc((void**)&A,                                SIZE,
hipHostMallocWriteCombined | hipHostMallocMapped));
        HIPCHECK(hipHostMalloc((void**)&B,                                SIZE,
hipHostMallocWriteCombined | hipHostMallocMapped));
        HIPCHECK(hipHostMalloc((void**)&C, SIZE, hipHostMallocMapped));
        HIPCHECK(hipHostMalloc((void**)&D, SIZE, hipHostMallocDefault));
        unsigned int flagA, flagB, flagC;
        HIPCHECK(hipHostGetDevicePointer((void**)&Ad, A, 0));
        HIPCHECK(hipHostGetDevicePointer((void**)&Bd, B, 0));
        HIPCHECK(hipHostGetDevicePointer((void**)&Cd, C, 0));
        HIPCHECK(hipHostGetDevicePointer((void**)&Dd, D, 0));
        HIPCHECK(hipHostGetFlags(&flagA, A));
        HIPCHECK(hipHostGetFlags(&flagB, B));
      HIPCHECK(hipHostGetFlags(&flagC, C));
```

10.15 hipHostRegister

__host__cudaError_t cudaHostRegister (void *ptr, size_t size, unsigned int flags)

---

hipError_t hipHostRegister(void* hostPtr, size_t sizeBytes, unsigned int flags)

Register host memory so it can be accessed from the current device.

Parameters

[out] hostPtr Pointer to host memory to be registered.

[in] sizeBytes Size of the host memory

[in] flags. See below.

Flags:

hipHostRegisterDefault Memory is Mapped and Portable

hipHostRegisterPortable Memory is considered registered by all contexts. HIP only supports one context so this is always assumed true.

hipHostRegisterMapped Map the allocation into the address space for the current device. The device pointer can be obtained with hipHostGetDevicePointer.

After registering the memory, use hipHostGetDevicePointer to obtain the mapped device pointer. On many systems, the mapped device pointer will have a different value than the mapped host pointer. Applications must use the device pointer in device code, and the host pointer in device code.

On some systems, registered memory is pinned. On some systems, registered memory may not be actually be pinned but uses OS or hardware facilities to all GPU access to the host memory.

Developers are strongly encouraged to register memory blocks which are aligned to the host cache-line size. (typically 64-bytes but can be obtains from the CPUID instruction).

If registering non-aligned pointers, the application must take care when register pointers from the same cache line on different devices. HIP's coarse-grained synchronization model does not guarantee correct results if different devices write to different parts of the same cache block - typically one of the writes will "win" and overwrite data from the other registered memory region.

Returns

#hipSuccess, #hipErrorMemoryAllocation

See also

hipHostUnregister, hipHostGetFlags, hipHostGetDevicePointer

```
  const size_t size = N * sizeof(float);
  if (p_tests & 0x1) {
```

```
float *A, **Ad;
int num_devices;
HIPCHECK(hipGetDeviceCount(&num_devices));
Ad = new float*[num_devices];
A = (float*)malloc(size);
HIPCHECK(hipHostRegister(A, size, 0));
```

10.16 hipHostUnregister

__host__cudaError_t cudaHostUnregister (void *ptr)

---

hipError_t hipHostUnregister(void* hostPtr)

Un-register host pointer.

Parameters

[in] hostPtr Host pointer previously registered with hipHostRegister

Returns

Error code

See also

hipHostRegister

```
float *A, **Ad;
Ad = new float*[num_devices];
A = (float*)malloc(size);
// Reference the registered device pointer Ad from inside the kernel:
for (int i = 0; i < num_devices; i++) {
HIPCHECK(hipSetDevice(i));
hipLaunchKernelGGL(Inc, dim3(N / 512), dim3(512), 0, 0, Ad[i]);
HIPCHECK(hipDeviceSynchronize());
}
HIPASSERT(A[10] == 1.0f + float(num_devices));
HIPCHECK(hipHostUnregister(A));
free(A);
delete [] Ad;
```

开发者社区

10.17 hipMalloc

__host____device__cudaError_t cudaMalloc (void **devPtr, size_t size)

---

hipError_t hipMalloc(void** ptr, size_t size)

Allocate memory on the default accelerator.

Parameters

[out] ptr Pointer to the allocated memory

[in] size Requested memory size

If size is 0, no memory is allocated, *ptr returns nullptr, and hipSuccess is returned.

Returns

#hipSuccess, #hipErrorMemoryAllocation, #hipErrorInvalidValue (bad context, null *ptr)

See also

hipMallocPitch, hipFree, hipMallocArray, hipFreeArray, hipMalloc3D, hipMalloc3DArray, hipHostFree, hipHostMalloc

```
float *A, *Ad, *B, *C, *Bd, *Cd;
A = new float[N];
B = new float[N];
C = new float[N];
for (int i = 0; i < N; i++) {
A[i] = 1.0f;
}
hipMalloc((void**)&Ad, SIZE);
hipMalloc((void**)&Bd, SIZE);
hipMalloc((void**)&Cd, SIZE);
```

10.18 hipMalloc3D

__host__cudaError_t cudaMalloc3D (cudaPitchedPtr *pitchedDevPtr, cudaExtent extent)

---

hipError_t hipMalloc3D(hipPitchedPtr* pitchedDevPtr, hipExtent extent)

```
printf ("testhipMemset3D memsetval=%2x device=%d\n", memsetval,
```

```
p_gpuDevice);
    char *A_h;
    bool testResult = true;
    hipExtent extent = make_hipExtent(width, numH, depth);
    hipPitchedPtr devPitchedPtr;
    HIPCHECK(hipMalloc3D(&devPitchedPtr, extent));
    A_h = (char*)malloc(sizeElements);
    HIPASSERT(A_h != NULL);
    for (size_t i=0; i<elements; i++) {
    A_h[i] = 1;
```

10.19 hipMalloc3DArray

__host__cudaError_t cudaMalloc3DArray (cudaArray_t *array, const cudaChannelFormatDesc *desc, cudaExtent extent, unsigned int flags)

---

hipError_t hipMalloc3DArray(hipArray** array, const struct hipChannelFormatDesc* desc, struct hipExtent extent, unsigned int flags)

Allocate an array on the device.

Parameters

[out]array	Pointer to allocated array in device memory
[in] desc	Requested channel format
[in] extent	Requested array allocation width, height and depth
[in] flags	Requested properties of allocated array

Returns

#hipSuccess, #hipErrorMemoryAllocation

See also

hipMalloc, hipMallocPitch, hipFree, hipFreeArray, hipHostMalloc, hipHostFree

10.20 hipMallocArray

__host__cudaError_t cudaMallocArray (cudaArray_t *array, const cudaChannelFormatDesc *desc, size_t width, size_t height, unsigned int flags)

---

hipError_t hipMallocArray(hipArray** array, const struct

hipChannelFormatDesc* desc, size_t width, size_t height, unsigned int flags __dparm(hipArrayDefault))

Allocate an array on the device.

Parameters

[out]array	Pointer to allocated array in device memory
[in] desc	Requested channel format
[in] width	Requested array allocation width
[in] height	Requested array allocation height
[in] flags	Requested properties of allocated array

Returns

#hipSuccess, #hipErrorMemoryAllocation

See also

hipMalloc, hipMallocPitch, hipFree, hipFreeArray, hipHostMalloc, hipHostFree

```
hipArray *hipArray, *hipOutArray;
hipMallocArray(&hipArray, &channelDesc, width, height);
hipMemcpyToArray(hipArray, 0, 0, hData, size, hipMemcpyHostToDevice);
```

10.21 hipHostMalloc

__host__cudaError_t cudaMallocHost (void **ptr, size_t size)

---

hipError_t hipHostMalloc(void** ptr, size_t size, unsigned int flags)

Allocate device accessible page locked host memory.

Parameters

[out]ptr	Pointer to the allocated host pinned memory
[in] size	Requested memory size
[in] flags	Type of host memory allocation

If size is 0, no memory is allocated, *ptr returns nullptr, and hipSuccess is returned.

Returns

#hipSuccess, #hipErrorMemoryAllocation

See:

hipSetDeviceFlags, hipHostFree

10.22 hipMallocManaged

__host__cudaError_t cudaMallocManaged (void **devPtr, size_t size, unsigned int flags)

---

hipError_t hipMallocManaged(void** dev_ptr,size_t size,unsigned int flags __dparm(hipMemAttachGlobal))

Allocates memory that will be automatically managed by DTK HMM.

Parameters

[out] dev_ptr Pointer to allocated device memory

[in] size Requested allocation size in bytes

[in] flags Must be either hipMemAttachGlobal or hipMemAttachHost

Returns

#hipSuccess, #hipErrorMemoryAllocation, #hipErrorNotSupported, #hipErrorInvalidValue

10.23 hipMallocMipmappedArray

__host__cudaError_t cudaMallocMipmappedArray (cudaMipmappedArray_t *mipmappedArray, const cudaChannelFormatDesc *desc, cudaExtent extent, unsigned int numLevels, unsigned int flags)

---

hipError_t hipMallocMipmappedArray(hipMipmappedArray_t *mipmappedArray,const struct hipChannelFormatDesc* desc,struct hipExtent extent,unsigned int numLevels,unsigned int flags __dparm(0))

Allocate a mipmapped array on the device.

Parameters

[out] mipmappedArray Pointer to allocated mipmapped array in device memory

[in] desc Requested channel format

[in] extent Requested allocation size (width field in elements)

[in] numLevels Number of mipmap levels to allocate

[in] flags Flags for extensions

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryAllocation

10.24 hipMallocPitch

__host__cudaError_t cudaMallocPitch (void **devPtr, size_t *pitch, size_t width, size_t height)

---

hipError_t hipMallocPitch（void ** ptr，size_t * pitch，size_t width，size_t height）;

Allocates at least width (in bytes) * height bytes of linear memory Padding may occur to ensure alighnment requirements are met for the given row The change in width size due to padding will be returned in *pitch. Currently the alignment is set to 128 bytes

Parameters

[out]ptr	Pointer to the allocated device memory
[out]pitch	Pitch for allocation (in bytes)
[in] width	Requested pitched allocation width (in bytes)
[in] height	Requested pitched allocation height

If size is 0, no memory is allocated, *ptr returns nullptr, and hipSuccess is returned.

Returns

Error code

See also

hipMalloc, hipFree, hipMallocArray, hipFreeArray, hipHostFree, hipMalloc3D, hipMalloc3DArray, hipHostMalloc

```
    printf ("testhipMemset2D memsetval=%2x device=%d\n", memsetval,
p_gpuDevice);
    char *A_d;
    char *A_h;
    bool testResult = true;
    HIPCHECK ( hipMallocPitch((void**)&A_d, &pitch_A, width , numH) );
    A_h = (char*)malloc(sizeElements);
    HIPASSERT(A_h != NULL);
```

开发者社区

```
    for (size_t i=0; i<elements; i++) {
    A_h[i] = 1;
    }
    HIPCHECK ( hipMemset2D(A_d, pitch_A, memsetval, numW, numH) );
    HIPCHECK ( hipMemcpy2D(A_h, width, A_d, pitch_A, numW, numH,
hipMemcpyDeviceToHost));
    for (int i=0; i<elements; i++) {
    if (A_h[i] != memsetval) {
    testResult = false;
    printf("testhipMemset2D mismatch at index:%d computed:%02x,
memsetval:%02x\n", i, (int)A_h[i], (int)memsetval);
    break;
    }
    }
    hipFree(A_d);
    free(A_h);
    return testResult;
```

10.25 hipMemAdvise

__host__cudaError_t cudaMemAdvise(const void* devPtr, size_t count, cudaMemoryAdvise advice, int device)

---

hipError_t hipMemAdvise(const void* dev_ptr,size_t count,hipMemoryAdvise advice,int device)

Advise about the usage of a given memory range to DTK HMM.

Parameters

[in] dev_ptr	Pointer to memory to set the advice for
[in] count	Size in bytes of the memory range
[in] advice	Advice to be applied for the specified memory range
[in] device	Device to apply the advice for

Returns

#hipSuccess, #hipErrorInvalidValue

10.26 hipMemGetInfo

__host__cudaError_t cudaMemGetInfo (size_t *free, size_t *total)

---

hipError_t hipMemGetInfo(size_t* free, size_t* total)

Query memory info. Return snapshot of free memory, and total allocatable memory on the device.

Returns in *free a snapshot of the current free memory.

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

Warning

On HCC, the free memory only accounts for memory allocated by this process and may be optimistic.

```
char* A_d;
char* A_Pinned_h;
char* A_OSAlloc_h;
hipError_t e;
HIPCHECK(hipMalloc(&A_d, Nbytes));
HIPCHECK(hipHostMalloc((void**)&A_Pinned_h, Nbytes,
hipHostMallocDefault));
A_OSAlloc_h = (char*)malloc(Nbytes);
size_t free, total;
HIPCHECK(hipMemGetInfo(&free, &total));
printf("hipMemGetInfo: free=%zu (%4.2f) Nbytes=%lu total=%zu (%4.2f)\n",
free,
(float)(free / 1024.0 / 1024.0), Nbytes, total, (float)(total / 1024.0 / 1024.0));
HIPASSERT(free + Nbytes <= total);
```

10.27 hipMemPrefetchAsync

__host__cudaError_t cudaMemPrefetchAsync (const void *devPtr, size_t count, int dstDevice, cudaStream_t stream)

---

hipError_t hipMemPrefetchAsync(const void* dev_ptr,size_t count,int device,hipStream_t stream __dparm(0))

Prefetches memory to the specified destination device using DTK HMM..

Parameters

[in] dev_ptr Pointer to be prefetched

[in] count Size in bytes for prefetching

[in] device Destination device to prefetch to

[in] stream Stream to enqueue prefetch operation

Returns

#hipSuccess, #hipErrorInvalidValue

10.28 hipMemRangeGetAttribute

__host__cudaError_t cudaMemRangeGetAttribute(void* data, size_t dataSize, cudaMemRangeAttribute attribute, const void* devPtr, size_t count)

---

hipError_t hipMemRangeGetAttribute(void* data,size_t data_size,hipMemRangeAttribute attribute,const void* dev_ptr,size_t count)

Query an attribute of a given memory range in DTK HMM.

Parameters

[in,out] data A pointer to a memory location where the result of each attribute query will be written to.

[in] data_sizes The size of data.

[in] attributes The attribute to query.

[in] dev_ptr Start of the range to query.

[in] count Size of the range to query.

Returns

#hipSuccess, #hipErrorInvalidValue

10.29 hipMemRangeGetAttributes

__host__cudaError_t cudaMemRangeGetAttributes(void** data, size_t* dataSizes, cudaMemRangeAttribute ** attributes, size_t numAttributes, const void* devPtr, size_t count)

---

hipError_t hipMemRangeGetAttributes(void** data,size_t*

data_sizes,hipMemRangeAttribute* attributes,size_t num_attributes,const void* dev_ptr,size_t count)

Query attributes of a given memory range in DTK HMM.

Parameters

[in/out] data A two-dimensional array containing pointers to memory locations where the result of each attribute query will be written to

[in] data_sizes An array, containing the sizes of each result

[in] attributes The attribute to query

[in] num_attributes An array of attributes to query (numAttributes and the number of attributes in this array should match)

[in] dev_ptr Start of the range to query

[in] count Size of the range to query

Returns

#hipSuccess, #hipErrorInvalidValue

10.30 hipMemcpy

__host__cudaError_t cudaMemcpy (void *dst, const void *src, size_t count, cudaMemcpyKind kind)

---

hipError_t hipMemcpy(void* dst, const void* src, size_t sizeBytes, hipMemcpyKind kind)

Copy data from src to dst.

It supports memory from host to device, device to host, device to device and host to host The src and dst must not overlap.

For hipMemcpy, the copy is always performed by the current device (set by hipSetDevice). For multi-gpu or peer-to-peer configurations, it is recommended to set the current device to the device where the src data is physically located. For optimal peer-to-peer copies, the copy device must be able to access the src and dst pointers (by calling hipDeviceEnablePeerAccess with copy agent as the current device and src/dest as the peerDevice argument. if this is not done, the hipMemcpy will still work, but will perform the copy using a staging buffer on the host.

Parameters

[out] dst Data being copy to

[in] src Data being copy from

[in] sizeBytes Data size in bytes

[in] copyType Memory copy type

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryFree, #hipErrorUnknowni

See also

hipArrayCreate, hipArrayDestroy, hipArrayGetDescriptor, hipMemAlloc, hipMemAllocHost, hipMemAllocPitch, hipMemcpy2D, hipMemcpy2DAsync, hipMemcpy2DUnaligned, hipMemcpyAtoA, hipMemcpyAtoD, hipMemcpyAtoH, hipMemcpyAtoHAsync, hipMemcpyDtoA, hipMemcpyDtoD, hipMemcpyDtoDAsync, hipMemcpyDtoH, hipMemcpyDtoHAsync, hipMemcpyHtoA, hipMemcpyHtoAAsync, hipMemcpyHtoDAsync, hipMemFree, hipMemFreeHost, hipMemGetAddressRange, hipMemGetInfo, hipMemHostAlloc, hipMemHostGetDevicePointer

```
loat *h0, *h1;
float *ph0, *ph1;
float *d0, *d1;
h0 = new float[len];
h1 = new float[len];
hmemset(h0, 1.0f);
int gpu0 = 0, gpu1 = 1;
hipHostMalloc((void**)&ph0, size);
hipMalloc(&d0, size);
hipHostMalloc((void**)&ph1, size);
hipMalloc(&d1, size);
hipMemcpy(h1, h0, size, hipMemcpyDefault);
hipMemcpy(ph0, h1, size, hipMemcpyDefault);
hipMemcpy(ph1, ph0, size, hipMemcpyDefault);
assert(h0[0] == ph1[0]);
```

开发者社区

```
    hmemset(ph1, 0.0f);
    hipMemcpy(h0, ph1, size, hipMemcpyDefault);
    assert(h0[0] == 0.0f);
```

10.31 hipMemcpy2D

__host__ cudaError_t cudaMemcpy2D (void* dst, size_t dpitch, const void* src, size_t spitch, size_t width, size_t height, cudaMemcpyKind kind)

---

hipError_t hipMemcpy2D(void* dst, size_t dpitch, const void* src, size_t spitch, size_t width, size_t height, hipMemcpyKind kind)

Copies data between host and device.

Parameters

[in] dst	Destination memory address
[in] dpitch	Pitch of destination memory
[in] src	Source memory address
[in] spitch	Pitch of source memory
[in] width	Width of matrix transfer (columns in bytes)
[in] height	Height of matrix transfer (rows)
[in] kind	Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpy, hipMemcpyToArray, hipMemcpy2DToArray, hipMemcpyFromArray, hipMemcpyToSymbol, hipMemcpyAsync

```
    void memcpy2Dtest(size_t numW, size_t numH, bool usePinnedHost) {
    size_t width = numW * sizeof(T);
    size_t sizeElements = width * numH;
    printf("memcpy2Dtest: %s<%s> size=%lu (%6.2fMB) W: %d, H:%d,
usePinnedHost: %d\n", __func__,
    TYPENAME(T), sizeElements, sizeElements / 1024.0 / 1024.0, (int)numW,
(int)numH,
```

```
    usePinnedHost);
    T *A_d, *B_d, *C_d;
    T *A_h, *B_h, *C_h;
    size_t pitch_A, pitch_B, pitch_C;
    hipChannelFormatDesc desc = hipCreateChannelDesc<T>();
    HipTest::initArrays2DPitch(&A_d, &B_d, &C_d, &pitch_A, &pitch_B,
&pitch_C, numW, numH);
    HipTest::initArraysForHost(&A_h, &B_h, &C_h, numW * numH,
usePinnedHost);
    unsigned blocks = HipTest::setNumBlocks(blocksPerCU, threadsPerBlock,
numW * numH);
    HIPCHECK(hipMemcpy2D(A_d, pitch_A, A_h, width, width, numH,
hipMemcpyHostToDevice));
    HIPCHECK(hipMemcpy2D(B_d, pitch_B, B_h, width, width, numH,
hipMemcpyHostToDevice));
    hipLaunchKernelGGL(HipTest::vectorADD, dim3(blocks),
dim3(threadsPerBlock), 0, 0, A_d, B_d, C_d,
    (pitch_C / sizeof(T)) * numH);
    HIPCHECK(hipMemcpy2D(C_h, width, C_d, pitch_C, width, numH,
hipMemcpyDeviceToHost));
    HIPCHECK(hipDeviceSynchronize());
    HipTest::checkVectorADD(A_h, B_h, C_h, numW * numH);
    HipTest::freeArrays(A_d, B_d, C_d, A_h, B_h, C_h, usePinnedHost);
    printf("  %s success\n", __func__);
    }
```

10.32 hipMemcpy2DArrayToArray*

__host__ cudaError_t cudaMemcpy2DArrayToArray (cudaArray_t dst, size_t wOffsetDst, size_t hOffsetDst, cudaArray_const_t src, size_t wOffsetSrc, size_t hOffsetSrc, size_t width, size_t height, cudaMemcpyKind kind = cudaMemcpyDeviceToDevice)

---

hipError_t hipMemcpy2DArrayToArray(hipArray_t dst, size_t wOffsetDst, size_t hOffsetDst, hipArray_const_t src, size_t wOffsetSrc, size_t hOffsetSrc, size_t width,

size_t height, hipMemcpyKind kind)

Copies data between host and device.

Parameters

[in] dst Destination memory address

[in] wOffsetDst Destination starting X offset

[in] hOffsetDst Destination starting Y offset

[in] src Source memory address

[in] wOffsetSrc Source starting X offset

[in] hOffsetSrc Source starting Y offset (columns in bytes)

[in] width Width of matrix transfer (columns in bytes)

[in] height Height of matrix transfer (rows)

[in] kind Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidMemcpyDirection

10.33 hipMemcpy2DAsync

__host____device__cudaError_t cudaMemcpy2DAsync (void *dst, size_t dpitch, const void *src, size_t spitch, www.nvidia.com CUDA Runtime API vRelease Version | 143 size_t width, size_t height, cudaMemcpyKind kind, cudaStream_t stream)

---

hipError_t hipMemcpy2DAsync(void* dst, size_t dpitch, const void* src, size_t spitch, size_t width, size_t height, hipMemcpyKind kind, hipStream_t stream __dparm(0))

Copies data between host and device.

Parameters

[in] dst Destination memory address

[in] dpitch Pitch of destination memory

[in] src Source memory address

[in] spitch Pitch of source memory

[in] width Width of matrix transfer (columns in bytes)

[in] height Height of matrix transfer (rows)

[in] kind Type of transfer

[in] stream Stream to use

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpy, hipMemcpyToArray, hipMemcpy2DToArray, hipMemcpyFromArray, hipMemcpyToSymbol, hipMemcpyAsync

10.34 hipMemcpy2DFromArray

__host__cudaError_t cudaMemcpy2DFromArray (void *dst, size_t dpitch, cudaArray_const_t src, size_t wOffset, size_t hOffset, size_t width, size_t height, cudaMemcpyKind kind)

---

hipError_t hipMemcpy2DFromArray(void* dst, size_t dpitch, hipArray_const_t src, size_t wOffset, size_t hOffset, size_t width, size_t height, hipMemcpyKind kind)

Copies data between host and device.

Parameters

[in] dst Destination memory address

[in] dpitch Pitch of destination memory

[in] src Source memory address

[in] wOffset Source starting X offset

[in] hOffset Source starting Y offset

[in] width Width of matrix transfer (columns in bytes)

[in] height Height of matrix transfer (rows)

[in] kind Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpy, hipMemcpy2DToArray, hipMemcpy2D, hipMemcpyFromArray,

hipMemcpyToSymbol, hipMemcpyAsync

10.35 hipMemcpy2DFromArrayAsync

__host__cudaError_t cudaMemcpy2DFromArrayAsync (void *dst, size_t dpitch, cudaArray_const_t src, size_t wOffset, size_t hOffset, size_t width, size_t height, cudaMemcpyKind kind, cudaStream_t stream)

---

hipError_t hipMemcpy2DFromArrayAsync(void* dst, size_t dpitch, hipArray_const_t src, size_t wOffset, size_t hOffset, size_t width, size_t height, hipMemcpyKind kind, hipStream_t stream __dparm(0))

Copies data between host and device asynchronously.

Parameters

[in] dst Destination memory address

[in] dpitch Pitch of destination memory

[in] src Source memory address

[in] wOffset Source starting X offset

[in] hOffset Source starting Y offset

[in] width Width of matrix transfer (columns in bytes)

[in] height Height of matrix transfer (rows)

[in] kind Type of transfer

[in] stream Accelerator view which the copy is being enqueued

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpy, hipMemcpy2DToArray, hipMemcpy2D, hipMemcpyFromArray, hipMemcpyToSymbol, hipMemcpyAsync

10.36 hipMemcpy2DToArray

__host__cudaError_t cudaMemcpy2DToArray (cudaArray_t dst, size_t wOffset, size_t hOffset, const www.nvidia.com CUDA Runtime API vRelease Version | 148 void

*src, size_t spitch, size_t width, size_t height, cudaMemcpyKind kind)

hipError_t hipMemcpy2DToArray(hipArray* dst, size_t wOffset, size_t hOffset, const void* src, size_t spitch, size_t width, size_t height, hipMemcpyKind kind)

Copies data between host and device.

Parameters

[in] dst	Destination memory address
[in] wOffset	Destination starting X offset
[in] hOffset	Destination starting Y offset
[in] src	Source memory address
[in] spitch	Pitch of source memory
[in] width	Width of matrix transfer (columns in bytes)
[in] height	Height of matrix transfer (rows)
[in] kind	Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpy, hipMemcpyToArray, hipMemcpy2D, hipMemcpyFromArray, hipMemcpyToSymbol, hipMemcpyAsync

```
    if (usePitch) {
    T *A_p, *B_p, *C_p;
    size_t pitch_A, pitch_B, pitch_C;
    HipTest::initArrays2DPitch(&A_p, &B_p, &C_p, &pitch_A, &pitch_B,
&pitch_C, numW, numH);
    HIPCHECK(hipMemcpy2D(A_p, pitch_A, A_h, width, width, numH,
hipMemcpyHostToDevice));
    HIPCHECK(hipMemcpy2D(B_p, pitch_B, B_h, width, width, numH,
hipMemcpyHostToDevice));
    HIPCHECK(hipMemcpy2DToArray(A_d, 0, 0, (void*)A_p, pitch_A, width,
numH,
```

```
    hipMemcpyDeviceToDevice));
    HIPCHECK(hipMemcpy2DToArray(B_d, 0, 0, (void*)B_p, pitch_B, width,
numH,
    hipMemcpyDeviceToDevice));
    hipFree(A_p);
    hipFree(B_p);
    hipFree(C_p);
    }
```

10.37 hipMemcpy2DToArrayAsync

__host__cudaError_t cudaMemcpy2DToArrayAsync (cudaArray_t dst, size_t wOffset, size_t hOffset, const void *src, size_t spitch, size_t width, size_t height, cudaMemcpyKind kind, cudaStream_t stream)

---

hipError_t hipMemcpy2DToArrayAsync(hipArray* dst, size_t wOffset, size_t hOffset, const void* src,size_t spitch, size_t width, size_t height, hipMemcpyKind kind,hipStream_t stream __dparm(0))

Copies data between host and device.

Parameters

[in] dst	Destination memory address
[in] wOffset	Destination starting X offset
[in] hOffset	Destination starting Y offset
[in] src	Source memory address
[in] count	Size in bytes to copy
[in] kind	Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpy, hipMemcpy2DToArray, hipMemcpy2D, hipMemcpyFromArray, hipMemcpyToSymbol, hipMemcpyAsync

10.38 hipMemcpy3D

__host__cudaError_t cudaMemcpy3D (const cudaMemcpy3DParms *p)

---

hipError_t hipMemcpy(void* dst, const void* src, size_t sizeBytes, hipMemcpyKind kind)

Copies data between host and device.

Parameters

[in] p 3D memory copy parameters

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpy, hipMemcpy2DToArray, hipMemcpy2D, hipMemcpyFromArray, hipMemcpyToSymbol, hipMemcpyAsync

```
HIPCHECK ( hipMemset3D( devPitchedPtr, memsetval, extent) );
hipMemcpy3DParms myparms = {0};
myparms.srcPos = make_hipPos(0,0,0);
myparms.dstPos = make_hipPos(0,0,0);
myparms.dstPtr = make_hipPitchedPtr(A_h, width , numW, numH);
myparms.srcPtr = devPitchedPtr;
myparms.extent = extent;
#ifdef __HIP_PLATFORM_NVCC__
myparms.kind =
hipMemcpyKindToCudaMemcpyKind(hipMemcpyDeviceToHost);
#else
myparms.kind = hipMemcpyDeviceToHost;
#endif
HIPCHECK(hipMemcpy3D(&myparms));
```

10.39 hipMemcpy3DAsync

__host____device__cudaError_t cudaMemcpy3DAsync (const cudaMemcpy3DParms *p, cudaStream_t stream)

---

hipError_t hipMemcpy3DAsync(const struct hipMemcpy3DParms* p, hipStream_t stream __dparm(0))

Copies data between host and device asynchronously.

[in] p 3D memory copy parameters

[in] stream Stream to use

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpy, hipMemcpy2DToArray, hipMemcpy2D, hipMemcpyFromArray, hipMemcpyToSymbol, hipMemcpyAsync

10.40 hipMemcpy3DPeer*

__host__cudaError_t cudaMemcpy3DPeer (const cudaMemcpy3DPeerParms *p)

---

hipError_t hipMemcpy3DPeer(const struct hipMemcpy3DPeerParms* p)

Copies memory between devices.

Parameters

[in] p - Parameters for the memory copy

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDevice

```
hipChannelFormatDesc channelDesc = hipCreateChannelDesc(32, 0, 0, 0,
  hipChannelFormatKindFloat);
hipMemcpy3DPeerParms myParams;
memset(&myParams, 0, sizeof(myParams));
myParams.srcPos = make_hipPos(0, 0, 0);
myParams.srcDevice = 0;
myParams.dstPos = make_hipPos(0, 0, 0);
myParams.dstDevice =1;
myParams.extent = make_hipExtent(4, 4, 4);


hipArray_t a;
```

```
HIP_CHECK(hipMalloc3DArray(&a, &channelDesc, make_hipExtent(4, 4, 4), hipArrayDefault));
myParams.srcArray = a;
hipArray_t b;
HIP_CHECK(hipMalloc3DArray(&b, &channelDesc, make_hipExtent(4, 4, 4), hipArrayDefault));
myParams.dstArray = b;
HIP_CHECK(hipMemcpy3DPeer(&myParams)));
```

10.41 hipMemcpy3DPeerAsync*

__host__cudaError_t cudaMemcpy3DPeerAsync (const cudaMemcpy3DPeerParms *p, cudaStream_t stream)

---

hipError_t hipMemcpy3DPeerAsync(const struct hipMemcpy3DPeerParms* p, hipStream_t stream __dparm(0))

Copies memory between devices asynchronously.

Parameters

[in] p - Parameters for the memory copy

[in] stream - Stream identifier

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDevice

See also

hipMemcpy3DPeer

10.42 hipMemcpyAsync

__host____device__cudaError_t cudaMemcpyAsync (void *dst, const void *src, size_t count, cudaMemcpyKind kind, cudaStream_t stream)

---

hipError_t hipMemcpyAsync(void* dst, const void* src, size_t sizeBytes, hipMemcpyKind copyKind, hipStream_t stream __dparm(0))

Copy data from src to dst asynchronously.

Warning

If host or dest are not pinned, the memory copy will be performed synchronously.

For best performance, use hipHostMalloc to allocate host memory that is transferred asynchronously.

on HCC hipMemcpyAsync does not support overlapped H2D and D2H copies. For hipMemcpy, the copy is always performed by the device associated with the specified stream.

For multi-gpu or peer-to-peer configurations, it is recommended to use a stream which is a attached to the device where the src data is physically located. For optimal peer-to-peer copies, the copy device must be able to access the src and dst pointers (by calling hipDeviceEnablePeerAccess with copy agent as the current device and src/dest as the peerDevice argument. if this is not done, the hipMemcpy will still work, but will perform the copy using a staging buffer on the host.

Parameters

[out] dst Data being copy to

[in] src Data being copy from

[in] sizeBytes Data size in bytes

[in] accelerator_view Accelerator view which the copy is being enqueued

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryFree, #hipErrorUnknown

See also

hipMemcpy, hipMemcpy2D, hipMemcpyToArray, hipMemcpy2DToArray, hipMemcpyFromArray, hipMemcpy2DFromArray, hipMemcpyArrayToArray, hipMemcpy2DArrayToArray, hipMemcpyToSymbol, hipMemcpyFromSymbol, hipMemcpy2DAsync, hipMemcpyToArrayAsync, hipMemcpy2DToArrayAsync, hipMemcpyFromArrayAsync, hipMemcpy2DFromArrayAsync, hipMemcpyToSymbolAsync, hipMemcpyFromSymbolAsync

```
    printf("testing: %s\n", __func__);
    hipError_t e;
    float *A_malloc, *A_pinned, *A_d;
    size_t Nbytes = N * sizeof(float);
```

```
    A_malloc = (float*)malloc(Nbytes);
    HIPCHECK(hipHostMalloc((void**)&A_pinned, Nbytes,
hipHostMallocDefault));
    A_d = NULL;
    HIPCHECK(hipMalloc(&A_d, Nbytes));
    HIPASSERT(A_d != NULL);
    // Can't use default with async copy
    e = hipMemcpyAsync(A_pinned, A_d, Nbytes, hipMemcpyDefault, NULL);
    //    HIPASSERT (e == hipSuccess);
    // Not sure what happens here, the memory must be pinned.
    e = hipMemcpyAsync(A_malloc, A_d, Nbytes, hipMemcpyDeviceToHost,
NULL);
    printf("  async memcpy of A_malloc to A_d. Result=%d\n", e);
    // HIPASSERT (e==hipErrorInvalidValue);
```

10.43 hipMemcpyFromSymbol

__host__cudaError_t cudaMemcpyFromSymbol (void *dst, const void *symbol, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipMemcpyFromSymbol(void* dst, const void* symbolName, size_t sizeBytes, size_t offset __dparm(0), hipMemcpyKind kind __dparm(hipMemcpyDeviceToHost))

```
    hipMemcpyToSymbol(HIP_SYMBOL(globalIn), A, SIZE, 0,
hipMemcpyHostToDevice);
    hipLaunchKernelGGL(Assign, dim3(1, 1, 1), dim3(NUM, 1, 1), 0, 0, Ad);
    hipMemcpy(B, Ad, SIZE, hipMemcpyDeviceToHost);
    hipMemcpyFromSymbol(C, HIP_SYMBOL(globalOut), SIZE, 0,
hipMemcpyDeviceToHost);
```

10.44 hipMemcpyFromSymbolAsync

__host__cudaError_t cudaMemcpyFromSymbolAsync (void *dst, const void *symbol, size_t count, size_t offset, cudaMemcpyKind kind, cudaStream_t stream)

---

hipError_t hipMemcpyFromSymbolAsync(void* dst, const void* symbolName,

size_t sizeBytes, size_t offset, hipMemcpyKind kind, hipStream_t stream __dparm(0))

```
    hipStreamCreate(&stream);
    hipMemcpyToSymbolAsync(HIP_SYMBOL(globalIn), Am, SIZE, 0, hipMemcpyHostToDevice, stream);
    hipStreamSynchronize(stream);
    hipLaunchKernelGGL(Assign, dim3(1, 1, 1), dim3(NUM, 1, 1), 0, 0, Ad);
    hipMemcpy(B, Ad, SIZE, hipMemcpyDeviceToHost);
    hipMemcpyFromSymbolAsync(Cm, HIP_SYMBOL(globalOut), SIZE, 0, hipMemcpyDeviceToHost, stream);
    hipStreamSynchronize(stream);
```

10.45 hipMemcpyPeer

__host__cudaError_t cudaMemcpyPeer (void *dst, int dstDevice, const void *src, int srcDevice, size_t count)

---

hipError_t hipMemcpyPeer(void* dst, int dstDevice, const void* src, int srcDevice, size_t count)

Copies memory from one device to memory on another device.

Parameters

[out] dst Destination device pointer.

[in] dstDeviceId Destination device

[in] src Source device pointer

[in] srcDeviceId Source device

[in] sizeBytes Size of memory copy in bytes

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDevice

```
    size_t Nbytes = N * sizeof(int);
    int numDevices = 0;
    int *A_d, *B_d, *C_d, *X_d, *Y_d, *Z_d;
    int *A_h, *B_h, *C_h;
    HIPCHECK(hipSetDevice(1));
    hipMemcpyPeer(
```

```
    X_d, 1, A_d, 0,
    Nbytes);  // this call is eqv to hipMemcpy(hipMemcpyD2D) which goes via stg bufs.
    hipMemcpyPeer(Y_d, 1, B_d, 0, Nbytes);
    hipLaunchKernelGGL(HipTest::vectorADD, dim3(blocks), dim3(threadsPerBlock), 0, 0,
    static_cast<const int*>(X_d), static_cast<const int*>(Y_d), Z_d, N);
    HIPCHECK(hipMemcpy(C_h, Z_d, Nbytes, hipMemcpyDeviceToHost));
    HIPCHECK(hipDeviceSynchronize());
    HipTest::checkVectorADD(A_h, B_h, C_h, N);
    HipTest::freeArrays(A_d, B_d, C_d, A_h, B_h, C_h, false);
    HIPCHECK(hipFree(X_d));
    HIPCHECK(hipFree(Y_d));
    HIPCHECK(hipFree(Z_d));
```

10.46 hipMemcpyPeerAsync

__host__cudaError_t cudaMemcpyPeerAsync (void *dst, int dstDevice, const void *src, int srcDevice, size_t count, cudaStream_t stream)

---

hipError_t hipMemcpyPeerAsync(void* dst, int dstDevice, const void* src, nt srcDevice, size_t count, hipStream_t stream __dparm(0))

Copies memory from one device to memory on another device.

Parameters

[out] dst	Destination device pointer.
[in] dstDevice	Destination device
[in] src	Source device pointer
[in] srcDevice	Source device
[in] sizeBytes	Size of memory copy in bytes
[in] stream	Stream identifier

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDevice

```
    HIPCHECK(hipStreamCreate(&s));
    HIPCHECK(hipSetDevice(1));
    HIPCHECK(hipMemcpyPeerAsync(X_d, 1, A_d, 0, Nbytes, s));
    HIPCHECK(hipMemcpyPeerAsync(Y_d, 1, B_d, 0, Nbytes, s));
    hipLaunchKernelGGL(HipTest::vectorADD, dim3(blocks),
dim3(threadsPerBlock), 0, 0,
    static_cast<const int*>(X_d), static_cast<const int*>(Y_d), Z_d, N);
    HIPCHECK(hipMemcpy(C_h, Z_d, Nbytes, hipMemcpyDeviceToHost));
    HIPCHECK(hipDeviceSynchronize());
    HIPCHECK(hipStreamSynchronize(s));
    HipTest::checkVectorADD(A_h, B_h, C_h, N);
    HIPCHECK(hipStreamDestroy(s));
    HipTest::freeArrays(A_d, B_d, C_d, A_h, B_h, C_h, false);
    HIPCHECK(hipFree(X_d));
    HIPCHECK(hipFree(Y_d));
    HIPCHECK(hipFree(Z_d));
```

10.47 hipMemcpyToSymbol

__host__cudaError_t cudaMemcpyToSymbol (const void *symbol, const void *src, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipMemcpyToSymbol(const void* symbol, const void* src, size_t sizeBytes, size_t offset __dparm(0), hipMemcpyKind copyType __dparm(hipMemcpyHostToDevice))

Copies data between host and device.

Parameters

[in] dst	Destination memory address
[in] dpitch	Pitch of destination memory
[in] src	Source memory address
[in] spitch	Pitch of source memory
[in] width	Width of matrix transfer (columns in bytes)
[in] height	Height of matrix transfer (rows)

开发者社区

[in] kind Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpy, hipMemcpy2DToArray, hipMemcpy2D, hipMemcpyFromArray, hipMemcpyToSymbol, hipMemcpyAsync

```
int *A, *Am, *B, *Ad, *C, *Cm;
A = new int[NUM];
B = new int[NUM];
C = new int[NUM];
for (int i = 0; i < NUM; i++) {
A[i] = -1 * i;
B[i] = 0;
C[i] = 0;
}
hipMalloc((void**)&Ad, SIZE);
hipHostMalloc((void**)&Am, SIZE);
hipHostMalloc((void**)&Cm, SIZE);
hipMemcpyToSymbol(HIP_SYMBOL(globalIn), A, SIZE, 0,
hipMemcpyHostToDevice);
hipLaunchKernelGGL(Assign, dim3(1, 1, 1), dim3(NUM, 1, 1), 0, 0, Ad);
hipMemcpy(B, Ad, SIZE, hipMemcpyDeviceToHost);
hipMemcpyFromSymbol(C, HIP_SYMBOL(globalOut), SIZE, 0,
hipMemcpyDeviceToHost);
```

10.48 hipMemcpyToSymbolAsync

__host__cudaError_t cudaMemcpyToSymbolAsync (const void *symbol, const void *src, size_t count, size_t offset, cudaMemcpyKind kind, cudaStream_t stream)

---

hipError_t hipMemcpyToSymbolAsync(const void* symbol, const void* src, size_t sizeBytes, size_t offset, hipMemcpyKind copyType, hipStream_t stream __dparm(0))

Copies sizeBytes bytes from the memory area pointed to by src to the memory area pointed to by offset bytes from the start of symbol symbol.

The memory areas may not overlap. Symbol can either be a variable that resides in global or constant memory space, or it can be a character string, naming a variable that resides in global or constant memory space. Kind can be either hipMemcpyHostToDevice or hipMemcpyDeviceToDevice hipMemcpyToSymbolAsync() is asynchronous with respect to the host, so the call may return before copy is complete. TODO: cudaErrorInvalidSymbol and cudaErrorInvalidMemcpyDirection is not supported, use hipErrorUnknown for now.

Parameters

[in] symbolName Symbol destination on device

[in] src Data being copy from

[in] sizeBytes Data size in bytes

[in] offset Offset from start of symbol in bytes

[in] kind Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryFree, #hipErrorUnknown

See also

hipMemcpy, hipMemcpy2D, hipMemcpyToArray, hipMemcpy2DToArray, hipMemcpyFromArray, hipMemcpy2DFromArray, hipMemcpyArrayToArray, hipMemcpy2DArrayToArray, hipMemcpyFromSymbol, hipMemcpyAsync, hipMemcpy2DAsync, hipMemcpyToArrayAsync, hipMemcpy2DToArrayAsync, hipMemcpyFromArrayAsync, hipMemcpy2DFromArrayAsync, hipMemcpyToSymbolAsync

```
int *A, *Am, *B, *Ad, *C, *Cm;
A = new int[NUM];
B = new int[NUM];
C = new int[NUM];
for (int i = 0; i < NUM; i++) {
A[i] = -1 * i;
```

开发者社区

```
    B[i] = 0;
    C[i] = 0;
    }
    hipMalloc((void**)&Ad, SIZE);
    hipHostMalloc((void**)&Am, SIZE);
    hipHostMalloc((void**)&Cm, SIZE);
    hipStream_t stream;
    hipStreamCreate(&stream);
    hipMemcpyToSymbolAsync(HIP_SYMBOL(globalIn), Am, SIZE, 0,
hipMemcpyHostToDevice, stream);
    hipStreamSynchronize(stream);
    hipLaunchKernelGGL(Assign, dim3(1, 1, 1), dim3(NUM, 1, 1), 0, 0, Ad);
    hipMemcpy(B, Ad, SIZE, hipMemcpyDeviceToHost);
    hipMemcpyFromSymbolAsync(Cm, HIP_SYMBOL(globalOut), SIZE, 0,
hipMemcpyDeviceToHost, stream);
    hipStreamSynchronize(stream);
```

10.49 hipMemset

__host__cudaError_t cudaMemset (void *devPtr, int value, size_t count)

---

hipError_t hipMemset(void* dst, int value, size_t sizeBytes)

Fills the first sizeBytes bytes of the memory area pointed to by dest with the constant byte value value.

Parameters

[out] dst Data being filled

[in] constant Value to be set

[in] sizeBytes Data size in bytes

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorNotInitialized

```
    size_t Nbytes = N * sizeof(char);
    char *A_d0, *A_d1;
    char* A_h;
    A_h = (char*)malloc(Nbytes);
```

```
// allocate and initialize memory on device0
HIPCHECK(hipSetDevice(g_currentDevice));
HIPCHECK(hipMalloc(&A_d0, Nbytes));
HIPCHECK(hipMemset(A_d0, memsetval, Nbytes));
// allocate and initialize memory on peer device
HIPCHECK(hipSetDevice(g_peerDevice));
HIPCHECK(hipMalloc(&A_d1, Nbytes));
HIPCHECK(hipMemset(A_d1, 0x13, Nbytes));
```

10.50 hipMemset2D

__host__cudaError_t cudaMemset2D (void *devPtr, size_t pitch, int value, size_t width, size_t height)

---

hipError_t hipMemset2D(void* dst, size_t pitch, int value, size_t width, size_t height)

Fills the memory area pointed to by dst with the constant value.

Parameters

[out] dst Pointer to device memory

[in] pitch Data size in bytes

[in] value Constant value to be set

[in] width

[in] height

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryFree

```
bool testhipMemset2D(int memsetval,int p_gpuDevice)
{
size_t numH = 256;
size_t numW = 256;
size_t pitch_A;
size_t width = numW * sizeof(char);
size_t sizeElements = width * numH;
size_t elements = numW* numH;
```

开发者社区

```
    printf ("testhipMemset2D memsetval=%2x device=%d\n", memsetval, p_gpuDevice);
    char *A_d;
    char *A_h;
    bool testResult = true;


    HIPCHECK ( hipMallocPitch((void**)&A_d, &pitch_A, width , numH) );
    A_h = (char*)malloc(sizeElements);
    HIPASSERT(A_h != NULL);
    for (size_t i=0; i<elements; i++) {
    A_h[i] = 1;
    }
    HIPCHECK ( hipMemset2D(A_d, pitch_A, memsetval, numW, numH) );
    HIPCHECK ( hipMemcpy2D(A_h, width, A_d, pitch_A, numW, numH, hipMemcpyDeviceToHost));
    for (int i=0; i<elements; i++) {
    if (A_h[i] != memsetval) {
    testResult = false;
    printf("testhipMemset2D mismatch at index:%d computed:%02x, memsetval:%02x\n", i, (int)A_h[i], (int)memsetval);
    break;
    }
    }
    hipFree(A_d);
    free(A_h);
    return testResult;
    }
```

10.51 hipMemset2DAsync

__host____device__cudaError_t cudaMemset2DAsync (void *devPtr, size_t pitch, int value, size_t width, size_t height, cudaStream_t stream)

---

hipError_t hipMemset2DAsync(void* dst, size_t pitch, int value, size_t width, size_t height,hipStream_t stream __dparm(0))

Fills asynchronously the memory area pointed to by dst with the constant value.

Parameters

[in] dst Pointer to device memory

[in] pitch Data size in bytes

[in] value Constant value to be set

[in] width

[in] height

[in] stream

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryFree

```
bool testhipMemset2DAsync(int memsetval,int p_gpuDevice)
{
size_t numH = 256;
size_t numW = 256;
size_t pitch_A;
size_t width = numW * sizeof(char);
size_t sizeElements = width * numH;
size_t elements = numW* numH;



printf ("testhipMemset2DAsync memsetval=%2x device=%d\n", memsetval,
p_gpuDevice);
char *A_d;
char *A_h;
bool testResult = true;
HIPCHECK ( hipMallocPitch((void**)&A_d, &pitch_A, width , numH) );
A_h = (char*)malloc(sizeElements);
HIPASSERT(A_h != NULL);
for (size_t i=0; i<elements; i++) {
A_h[i] = 1;
}
```

开发者社区

```
    hipStream_t stream;
    HIPCHECK(hipStreamCreate(&stream));
    HIPCHECK ( hipMemset2DAsync(A_d, pitch_A, memsetval, numW, numH,
stream) );
    HIPCHECK ( hipMemcpy2D(A_h, width, A_d, pitch_A, numW, numH,
hipMemcpyDeviceToHost));
    for (int i=0; i<elements; i++) {
    if (A_h[i] != memsetval) {
    testResult = false;
    printf("testhipMemset2DAsync mismatch at index:%d computed:%02x,
memsetval:%02x\n", i, (int)A_h[i], (int)memsetval);
    break;
    }
    }
    hipFree(A_d);
    HIPCHECK(hipStreamDestroy(stream));
    free(A_h);
    return testResult;
    }
```

10.52 hipMemset3D

__host__cudaError_t cudaMemset3D (cudaPitchedPtr pitchedDevPtr, int value, cudaExtent extent)

---

hipError_t hipMemset3D(hipPitchedPtr pitchedDevPtr, int value, hipExtent extent);

Fills synchronously the memory area pointed to by pitchedDevPtr with the constant value.

Parameters

[in] pitchedDevPtr

[in] value Constant value to be set

[in] extent

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryFree

```
bool testhipMemset3D(int memsetval,int p_gpuDevice)
{
size_t numH = 256;
size_t numW = 256;
size_t depth = 10;
size_t width = numW * sizeof(char);
size_t sizeElements = width * numH * depth;
size_t elements = numW* numH* depth;
printf ("testhipMemset3D memsetval=%2x device=%d\n", memsetval,
p_gpuDevice);
char *A_h;
bool testResult = true;
hipExtent extent = make_hipExtent(width, numH, depth);
hipPitchedPtr devPitchedPtr;
HIPCHECK(hipMalloc3D(&devPitchedPtr, extent));
A_h = (char*)malloc(sizeElements);
HIPASSERT(A_h != NULL);
for (size_t i=0; i<elements; i++) {
A_h[i] = 1;
}
HIPCHECK ( hipMemset3D( devPitchedPtr, memsetval, extent) );
hipMemcpy3DParms myparms = {0};
myparms.srcPos = make_hipPos(0,0,0);
myparms.dstPos = make_hipPos(0,0,0);
myparms.dstPtr = make_hipPitchedPtr(A_h, width , numW, numH);
myparms.srcPtr = devPitchedPtr;
myparms.extent = extent;
#ifdef __HIP_PLATFORM_NVCC__
myparms.kind =
hipMemcpyKindToCudaMemcpyKind(hipMemcpyDeviceToHost);
#else
myparms.kind = hipMemcpyDeviceToHost;
```

开发者社区

```
#endif
HIPCHECK(hipMemcpy3D(&myparms));
for (int i=0; i<elements; i++) {
if (A_h[i] != memsetval) {
testResult = false;
printf("mismatch at index:%d computed:%02x, memsetval:%02x\n", i,
(int)A_h[i], (int)memsetval);
break;
}
}
HIPCHECK(hipFree(devPitchedPtr.ptr));
free(A_h);
return testResult;
}
```

10.53 hipMemset3DAsync

__host____device__cudaError_t cudaMemset3DAsync (cudaPitchedPtr pitchedDevPtr, int value, cudaExtent extent, cudaStream_t stream)

---

hipError_t hipMemset3DAsync(hipPitchedPtr pitchedDevPtr, int value, hipExtent extent ,hipStream_t stream __dparm(0));

Fills asynchronously the memory area pointed to by pitchedDevPtr with the constant value.

Parameters

[in] pitchedDevPtr

[in] value Constant value to be set

[in] extent

[in] stream

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryFree

```
constexpr int memsetval = 0x22;
constexpr size_t numH = 256;
```

```
constexpr size_t numW = 256;
constexpr size_t depth = 10;
size_t width = numW * sizeof(char);
size_t sizeElements = width * numH * depth;
size_t elements = numW * numH * depth;
hipExtent extent = make_hipExtent(width, numH, depth);
hipPitchedPtr devPitchedPtr;
char *A_h;

HIP_CHECK(hipMalloc3D(&devPitchedPtr, extent));
A_h = reinterpret_cast<char *>(malloc(sizeElements));
REQUIRE(A_h != nullptr);

for (size_t i = 0; i < elements; i++) {
   A_h[i] = 1;
}

hipStream_t stream;
HIP_CHECK(hipStreamCreate(&stream));
HIP_CHECK(hipMemset3DAsync(devPitchedPtr, memsetval, extent, stream));
HIP_CHECK(hipStreamSynchronize(stream));
```

10.54 hipMemsetAsync

__host____device__cudaError_t cudaMemsetAsync (void *devPtr, int value, size_t count, cudaStream_t stream)

---

hipError_t hipMemsetAsync(void* dst, int value, size_t sizeBytes, hipStream_t stream __dparm(0))

Fills the first sizeBytes bytes of the memory area pointed to by dev with the constant byte value value.

hipMemsetAsync() is asynchronous with respect to the host, so the call may return before the memset is complete. The operation can optionally be associated to a stream by passing a non-zero stream argument. If stream is non-zero, the operation may

overlap with operations in other streams.

Parameters

[out]	dst	Pointer to device memory
[in]	value	Value to set for each byte of specified memory
[in]	sizeBytes	Size in bytes to set
[in]	stream	Stream identifier

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryFree

```
bool testhipMemsetAsync(int memsetval,int p_gpuDevice)
{
size_t Nbytes = N*sizeof(int);
printf ("testhipMemsetAsync N=%zu  memsetval=%2x device=%d\n", N,
memsetval, p_gpuDevice);
char *A_d;
char *A_h;
bool testResult = true;
HIPCHECK ( hipMalloc((void**)&A_d, Nbytes) );
A_h = (char*)malloc(Nbytes);
hipStream_t stream;
HIPCHECK(hipStreamCreate(&stream));
HIPCHECK ( hipMemsetAsync(A_d, memsetval, Nbytes, stream ));
HIPCHECK ( hipStreamSynchronize(stream));
HIPCHECK ( hipMemcpy(A_h, (void*)A_d, Nbytes,
hipMemcpyDeviceToHost));
for (int i=0; i<N; i++) {
if (A_h[i] != memsetval) {
testResult = false;
printf("mismatch at index:%d computed:%02x, memsetval:%02x\n", i,
(int)A_h[i], (int)memsetval);
break;
}
}
```

开发者社区

```
HIPCHECK(hipFree((void*)A_d));
HIPCHECK(hipStreamDestroy(stream));
free(A_h);
return testResult;
}
```

10.55 hipMipmappedArrayGetMemoryRequirements*

__host__cudaError_t cudaMipmappedArrayGetMemoryRequirements (cudaArrayMemoryRequirements* memoryRequirements, cudaMipmappedArray_t mipmap, int device)

---

hipError_t hipMipmappedArrayGetMemoryRequirements(struct hipArrayMemoryRequirements *memoryRequirements, hipMipmappedArray_t mipmap, int device)

Returns the memory requirements of a HIP mipmapped array. If the HIP mipmapped array is not allocated with flag HIPArrayDeferredMapping HIPErrorInvalidValue will be returned.

The returned value in ::hipArrayMemoryRequirements::size represents the total size of the HIP mipmapped array. The returned value in hipArrayMemoryRequirements::alignment represents the alignment necessary for mapping the HIP mipmapped array.

Parameters

[out] memoryRequirements - Pointer to ::hipArrayMemoryRequirements

[in] mipmap - HIP mipmapped array to get the memory requirements of

[in] device - Device to get the memory requirements for

Returns

#hipSuccess, #hipErrorInvalidValue

```
size_t elementCount = 128;
size_t memSizeByte = elementCount * sizeof(int);
hipMipmappedArray_t mipmappedArray = nullptr;
hipChannelFormatDesc desc;
desc.x = 32;
```

```
    desc.y = 0;
    desc.z = 0;
    desc.w = 0;
    desc.f = hipChannelFormatKindSigned;
    hipExtent extent;
    extent.width = 128;
    extent.height = 1;
    extent.depth = 1;
    HIP_CHECK(hipMallocMipmappedArray(&mipmappedArray, &desc, extent, 2,
hipArrayDeferredMapping));
    hipArrayMemoryRequirements memReq;
    memset(&memReq, 0, sizeof(memReq));
    HIP_CHECK(hipMipmappedArrayGetMemoryRequirements(&memReq,
mipmappedArray, 0));
    printf("mem:%zu\n", memReq.size);
```

10.56 hipMipmappedArrayGetSparseProperties*

__host__cudaError_t cudaMipmappedArrayGetSparseProperties (cudaArraySparseProperties* sparseProperties, cudaMipmappedArray_t mipmap)

No hip

10.57 make_hipExtent

__host__make_cudaExtent (size_t w, size_t h, size_t d)

make_hipExtent(size_t w, size_t h, size_t d)

```
    size_t numH = 256;
    size_t numW = 256;
    size_t depth = 10;
    size_t width = numW * sizeof(char);
    size_t sizeElements = width * numH * depth;
    size_t elements = numW* numH* depth;
    printf ("testhipMemset3D memsetval=%2x device=%d\n", memsetval,
p_gpuDevice);
    char *A_h;
```

```
bool testResult = true;
hipExtent extent = make_hipExtent(width, numH, depth);
hipPitchedPtr devPitchedPtr;
```

10.58 make_hipPitchedPtr*

__host__make_cudaPitchedPtr (void *d, size_t p, size_t xsz, size_t ysz)

No hip

10.59 make_hipPos

__host__make_cudaPos (size_t x, size_t y, size_t z)

make_hipPos(size_t x, size_t y, size_t z)

```
HIPCHECK ( hipMemset3D( devPitchedPtr, memsetval, extent) );
hipMemcpy3DParms myparms = {0};
myparms.srcPos = make_hipPos(0,0,0);
```

10.60 hipMemAdvise_v2【dtk-25.04.1 新增】

__host__cudaError_t cudaMemAdvise_v2 (const void* devPtr, size_t count, cudaMemoryAdvise advice, cudaMemLocation location)

hipError_t hipMemAdvise_v2(const void* dev_ptr, size_t count, hipMemoryAdvise advice, hipMemLocation location)

Advise about the usage of a given memory range to HIP.

Parameters

[in] dev_ptr pointer to memory to set the advice for

[in] count size in bytes of the memory range, it should be CPU page size alligned.

[in] advice advice to be applied for the specified memory range

[in] location location to apply the advice for

Returns

#hipSuccess, #hipErrorInvalidValue

```
int gpuDevice = 0;
int concurrentManagedAccess = 0;
HIP_CHECK(hipDeviceGetAttribute(&concurrentManagedAccess,
hipDeviceAttributeConcurrentManagedAccess, gpuDevice));
```

```
  if(!concurrentManagedAccess) {
    printf("info: concurrent managed access not supported on device %d\n Skipped\n",
gpuDevice);
    return;
  }

  HIP_CHECK(hipSetDevice(gpuDevice));
  int numElements = 1024 * 1024;
  float *A, *B;
  HIP_CHECK(hipMallocManaged(&A, numElements*sizeof(float)));
  for (int i = 0; i < numElements; i++) {
    A[i] = 1.0f;
  }
  hipMemLocation location;
  location.type = hipMemLocationTypeDevice;
  location.id = 0;
    HIP_CHECK(hipMemAdvise_v2(A,                numElements*sizeof(float),
hipMemAdviseSetReadMostly, location));
    HIP_CHECK(hipMemPrefetchAsync_v2(A,             numElements*sizeof(float),
location, 0));
```

10.61 hipMemPrefetchAsync_v2【dtk-25.04.1 新增】

__host__cudaError_t cudaMemPrefetchAsync_v2 (const void* devPtr, size_t count, cudaMemLocation location, unsigned int flags, cudaStream_t stream = 0)

---

hipError_t hipMemPrefetchAsync_v2(const void* dev_ptr, size_t count, hipMemLocation location, unsigned int flags, hipStream_t stream __dparm(0))

Prefetches memory to the specified destination location using HIP.

Parameters

[in] dev_ptr pointer to be prefetched

[in] count size in bytes for prefetching

[in] location location to prefetch to

[in] flags flags for future use, must be zero now.

[in] stream　stream to enqueue prefetch operation

Returns

#hipSuccess, #hipErrorInvalidValue

See also

hipMemAdvise_v2

10.62 hipDeviceRegisterAsyncNotification【dtk-25.04.1 新增】

__host__cudaError_t cudaDeviceRegisterAsyncNotification (int device, cudaAsyncCallback callbackFunc, void* userData, cudaAsyncCallbackHandle_t* callback)

---

hipError_t hipDeviceRegisterAsyncNotification(int device, hipAsyncCallback callbackFunc, void* userData, hipAsyncCallbackHandle_t* callback)

Registers a callback function to receive async notifications

Parameters

[in] device　The device on which to register the callback.

[in] callbackFunc　The function to register as a callback.

[in] userData　A generic pointer to user data. This is passed into the callback function.

[out] callback　A handle representing the registered callback instance.

Returns

#hipSuccess #hipErrorInvalidValue

```
static void myCallback(hipAsyncNotificationInfo_t* info, void* userData, hipAsyncCallbackHandle_t handle) {
printf("myCallback\n");
return;
}
size_t free = 0, total =0;
HIP_CHECK(hipMemGetInfo(&free, &total));
std::cout << "total: " << total << ", free: " << free << std::endl;

int device = 0;
```

```
hipAsyncCallback fn = myCallback;
hipAsyncCallbackHandle_t handle;
HIP_CHECK(hipDeviceRegisterAsyncNotification(device, fn, nullptr, &handle));

float* gpu;
size_t num = (free/4) + 4;
HIP_CHECK(hipMalloc(&gpu, num * sizeof(float)));
REQUIRE(hipMalloc(&gpu, num * sizeof(float)) == hipErrorOutOfMemory);
HIP_CHECK(hipDeviceUnregisterAsyncNotification(device, handle));
```

10.63 hipDeviceUnregisterAsyncNotification【dtk-25.04.1 新增】

__host__cudaError_t cudaDeviceUnregisterAsyncNotification (int device, cudaAsyncCallbackHandle_t callback)

---

hipError_t hipDeviceUnregisterAsyncNotification(int device, hipAsyncCallbackHandle_t callback)

Unregisters an async notification callback

Parameters

[in] The device from which to remove.

[in] The callback instance to unregister from receiving async notifications.

Returns

#hipSuccess #hipErrorInvalidValue

See also

hipDeviceRegisterAsyncNotification

10.64 hipMemsetD2D8【dtk-26.10 新增】

CUresult CUDAAPI cuMemsetD2D8(CUdeviceptr dstDevice, size_t dstPitch, unsigned char uc, size_t Width, size_t Height)

---

hipError_t hipMemsetD2D8(hipDeviceptr_t dstDevice, size_t dstPitch, unsigned char uc, size_t Width, size_t Height)

Fills a two-dimensional area of device memory, setting each element in the

specified range to a given unsigned char value.

Parameters:

dstDevice – [in] Pointer to device memory

dstPitch – [in] pitch size in bytes

uc – [in] value to be set

Width – [in] Width of row

Height – [in] number of rows

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorOutOfMemory, hipErrorStreamCapturelmplicit, hipErrorNotInitialized, hipErrorInvalidDevice, hipErrorNoDevice

10.65 hipMemsetD2D8Async【dtk-26.10 新增】

CUresult cuMemsetD2D8Async(CUdeviceptr dstDevice,size_t dstPitch,unsigned char uc,size_t Width,size_t Height,CUstream hStream)

---

hipError_t hipMemsetD2D8Async(hipDeviceptr_t dstDevice, size_t dstPitch, unsigned char uc, size_t Width, size_t Height, hipStream_t hStream)

Fills asynchronously a two-dimensional area of device memory, setting each element in the specified range to a given unsigned char value.

Parameters:

dstDevice – [in] Pointer to device memory

dstPitch – [in] pitch size in bytes

uc – [in] value to be set

Width – [in] Width of row

Height – [in] number of rows

hStream – [in] Stream identifie

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorContextIsDestroyed, hipErrorOutOfMemory, hipErrorStreamCaptureInvalidated, hipErrorNotInitialized, hipErrorInvalidDevice, hipErrorNoDevice

10.66 hipMemsetD2D16【dtk-26.10 新增】

CUresult CUDAAPI cuMemsetD2D16(CUdeviceptr dstDevice, size_t dstPitch, unsigned short us, size_t Width, size_t Height)

---

hipError_t hipMemsetD2D16(hipDeviceptr_t dstDevice, size_t dstPitch, unsigned short us, size_t Width, size_t Height)

Fills a two-dimensional area of device memory, setting each element in the specified range to a given unsigned short value.

Parameters:

dstDevice – [in] Pointer to device memory

dstPitch – [in] pitch size in bytes

us – [in] value to be set

Width – [in] Width of row

Height – [in] number of rows

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorOutOfMemory, hipErrorStreamCaptureImplicit, hipErrorNotInitialized, hipErrorInvalidDevice, hipErrorNoDevice

10.67 hipMemsetD2D16Async【dtk-26.10 新增】

CUresult cuMemsetD2D16Async(CUdeviceptr dstDevice,size_t dstPitch,unsigned short us,size_t Width,size_t Height,CUstream hStream)

---

hipError_t hipMemsetD2D16Async(hipDeviceptr_t dstDevice, size_t dstPitch, unsigned short us, size_t Width, size_t Height, hipStream_t hStream)

Fills asynchronously a two-dimensional area of device memory, setting each element in the specified range to a given unsigned short value.

Parameters:

dstDevice – [in] Pointer to device memory

dstPitch – [in] pitch size in bytes

us – [in] value to be set

开发者社区

Width – [in] Width of row

Height – [in] number of rows

hStream – [in] Stream identifier

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorContextIsDestroyed, hipErrorOutOfMemory, hipErrorStreamCaptureInvalidated, hipErrorNotInitialized, hipErrorInvalidDevice, hipErrorNoDevice

10.68 hipMemsetD2D32【dtk-26.10 新增】

CUresult cuMemsetD2D32(CUdeviceptr dstDevice, size_t dstPitch, unsigned int ui, size_t Width, size_t Height)

---

hipError_t hipMemsetD2D32(hipDeviceptr_t dstDevice, size_t dstPitch, unsigned int ui, size_t Width, size_t Height)

Fills a two-dimensional area of device memory, setting each element in the specified range to a given unsigned int value.

Parameters:

dstDevice – [in] Pointer to device memory

dstPitch – [in] pitch size in bytes

ui – [in] value to be set

Width – [in] Width of row

Height – [in] number of rows

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorOutOfMemory, hipErrorStreamCaptureImplicit, hipErrorNotInitialized, hipErrorInvalidDevice, hipErrorNoDevice

10.69 hipMemsetD2D32Async【dtk-26.10 新增】

CUresult cuMemsetD2D32Async(CUdeviceptr dstDevice,size_t dstPitch,unsigned int ui,size_t Width,size_t Height,CUstream hStream)

---

hipError_t hipMemsetD2D32Async(hipDeviceptr_t dstDevice, size_t dstPitch, unsigned int ui, size_t Width, size_t Height, hipStream_t hStream)

Fills asynchronously a two-dimensional area of device memory, setting each element in the specified range to a given unsigned int value.

Parameters:

dstDevice – [in] Pointer to device memory

dstPitch – [in] pitch size in bytes

ui – [in] value to be set

Width – [in] Width of row

Height – [in] number of rows

hStream – [in] Stream identifier

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorContextIsDestroyed, hipErrorOutOf Memory, hipErrorStreamCaptureInvalidated, hipErrorNotInitialized, hipErrorInvalid Device, hipErrorNoDevice

10.70 hipMemcpyBatchAsync【dtk-26.10 新增】

CUresult cuMemcpyBatchAsync(CUdeviceptr *dsts, CUdeviceptr *srcs, size_t *sizes, size_t count, CUmemcpyAttributes *attrs, size_t *attrsIdxs, size_t numAttrs, size_t *failIdx, CUstream hStream)

---

hipError_t hipMemcpyBatchAsync(void **dsts, void **srcs, size_t *sizes, size_t count, hipMemcpyAttributes *attrs, size_t *attrsIdxs, size_t numAttrs, size_t *failIdx , hipStream_t stream)

Perform Batch of 1D copies.

Parameters:

dsts – [in] - Array of destination pointers

srcs – [in] - Array of source pointers.

sizes – [in] - Array of sizes for memcpy operations

count – [in] - Size of dsts, srcs and sizes arrays

attrs – [in] - Array of memcpy attributes (not supported)

attrsIdxs – [in] - Array of indices to map attrs to copies (not supported)

numAttrs – [in] - Size of attrs and attrsIdxs arrays (not supported)

failIdx – [in] - Pointer to a location to return failure index inside the batch

stream – [in] - stream used to enqueue operations in.

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidResourceHandle, hipErrorNotSupported, hipErrorUnknown, hipErrorOutOfMemory, hipErrorNotInitialized, hipErrorInvalidDevice, hipErrorNoDevice

10.71 hipMemcpy3DBatchAsync【dtk-26.10 新增】

CUresult cuMemcpy3DBatchAsync(size_t numOps, CUDA_MEMCPY3D_BATCH_OP *opList, size_t *failIdx, unsigned long long flags, CUstream hStream)

---

hipError_t hipMemcpy3DBatchAsync(size_t numOps, struct hipMemcpy3DBatchOp *opList, size_t *failIdx, unsigned long long flags, hipStream_t stream)

Perform Batch of 3D copies.

Parameters:

numOps – [in] - Total number of memcpy operations.

opList – [in] - Array of size numOps containing the actual memcpy operations.

failIdx – [in] - Pointer to a location to return the index of the copy where a failure was encountered.

flags – [in] - Flags for future use, must be zero now.

stream – [in] - The stream to enqueue the operations in.

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidPitchValue, hipErrorInvalidResourceHandle, hipErrorOutOfMemory, hipErrorNotInitialized, hipErrorInvalidDevice, hipErrorNoDevice

10.72 hipMemcpy3DPeer【dtk-26.10 新增】

CUresult CUDAAPI cuMemcpy3DPeer(const CUDA_MEMCPY3D_PEER *pCopy)

---

hipError_t hipMemcpy3DPeer(const struct hipMemcpy3DPeerParms *p)

Copies memory between devices.

Parameters:

p – [in] - Parameters for the memory copy

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidPitchValue, hipErrorInvalidDevice, hipErrorContextIsDestroyed, hipErrorStreamCaptureImplicit, hipErrorOutOfMemory, hipErrorNotInitialized, hipErrorNoDevice

10.73 hipMemcpy3DPeerAsync【dtk-26.10 新增】

CUresult cuMemcpy3DPeerAsync(const CUDA_MEMCPY3D_PEER * pCopy,CUstream hStream)

---

hipError_t hipMemcpy3DPeerAsync(const struct hipMemcpy3DPeerParms *p, hipStream_t stream)

Copies memory between devices asynchronously.

Parameters:

p – [in] - Parameters for the memory copy

stream – [in] - Stream identifier

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidPitchValue, hipErrorInvalidDevice, hipErrorContextIsDestroyed, hipErrorStreamCaptureInvalidated, hipErrorOutOfMemory, hipErrorNotInitialized, hipErrorNoDevice

10.74 hipMemPrefetchAsync_v2【dtk-26.10 新增】

CUresult cuMemPrefetchAsync(CUdeviceptr devPtr,size_t count,CUdevice dstDevice,CUstream hStream)

---

hipError_t hipMemPrefetchAsync_v2(const void *dev_ptr, size_t count, hipMemLocation location, unsigned int flags, hipStream_t stream)

Prefetches memory to the specified destination location using HIP.

Parameters:

dev_ptr – [in] pointer to be prefetched

count – [in] size in bytes for prefetching

location – [in] location to prefetch to

flags – [in] flags for future use, must be zero now.

stream – [in] stream to enqueue prefetch operation

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidDevice, hipErrorNotSupported, hipErrorContextIsDestroyed, hipErrorStreamCaptureUnsupported, hipErrorOutOfMemory, hipErrorNotInitialized, hipErrorNoDevice

10.75 hipMemAdvise_v2【dtk-26.10 新增】

CUresult cuMemAdvise(CUdeviceptr devPtr,size_t count,CUmem_advise advice,CUdevice device)

---

hipError_t hipMemAdvise_v2(const void *dev_ptr, size_t count, hipMemoryAdvise advice, hipMemLocation location)

Advise about the usage of a given memory range to HIP.

Parameters:

dev_ptr – [in] pointer to memory to set the advice for

count – [in] size in bytes of the memory range, it should be CPU page size alligned.

advice – [in] advice to be applied for the specified memory range

location – [in] location to apply the advice for

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidDevice, hipErrorStreamCaptureUnsupported, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorNoDevice

11 Memory Management [DEPRECATED]

11.1 hipMemcpyArrayToArray*

__host__cudaError_t cudaMemcpyArrayToArray (cudaArray_t dst, size_t wOffsetDst, size_t hOffsetDst, cudaArray_const_t src, size_t wOffsetSrc, size_t hOffsetSrc, size_t count, cudaMemcpyKind kind)

---

hipError_t hipMemcpyArrayToArray(hipArray_t dst, size_t wOffsetDst, size_t hOffsetDst, hipArray_const_t src, size_t wOffsetSrc, size_t hOffsetSrc, size_t width, size_t height, hipMemcpyKind kind)

Copies data between host and device.

Parameters

[in] dst Destination memory address

[in] wOffsetDst Destination starting X offset

[in] hOffsetDst Destination starting Y offset

[in] src Source memory address

[in] wOffsetSrc Source starting X offset

[in] hOffsetSrc Source starting Y offset

[in] width width of array

[in] height height of array

[in] kind Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue

11.2 hipMemcpyFromArray

__host__cudaError_t cudaMemcpyFromArray (void *dst, cudaArray_const_t src, size_t wOffset, size_t hOffset, size_t count, cudaMemcpyKind kind)

---

hipError_t hipMemcpyFromArray(void* dst, hipArray_const_t srcArray, size_t wOffset, ize_t hOffset, size_t count, hipMemcpyKind kind)

Copies data between host and device.

Parameters

[in] dst Destination memory address

[in] srcArray Source memory address

[in] woffset Source starting X offset

[in] hOffset Source starting Y offset

[in] count Size in bytes to copy

[in] kind Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpy, hipMemcpy2DToArray, hipMemcpy2D, hipMemcpyFromArray, hipMemcpyToSymbol, hipMemcpyAsync

```
hipMallocArray(&hipOutArray, &channelDesc, width, height);
float* hOutputData = (float*)malloc(size);
memset(hOutputData, 0, size);
hipMemcpyFromArray(hOutputData, hipOutArray, 0, 0, size, hipMemcpyDeviceToHost);
hipFreeArray(hipArray);
hipFreeArray(hipOutArray);
```

11.3 hipMemcpyFromArrayAsync*

__host__cudaError_t cudaMemcpyFromArrayAsync (void *dst, cudaArray_const_t src, size_t wOffset, size_t hOffset, size_t count, cudaMemcpyKind kind, cudaStream_t stream)

---

hipError_t hipMemcpyFromArrayAsync(void* dst, hipArray_const_t src, size_t wOffsetSrc, size_t hOffsetSrc, size_t count, hipMemcpyKind kind, hipStream_t stream)

Copies data between host and device.

Parameters

[in] dst	Destination memory address
[in] src	Source memory address
[in] wOffsetSrc	Source starting X offset
[in] hOffsetSrc	Source starting Y offset
[in] count	Size in bytes to copy
[in] kind	Type of transfer
[in] stream	Stream identifier

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpyFromArray

11.4 hipMemcpyToArray

__host__cudaError_t cudaMemcpyToArray (cudaArray_t dst, size_t wOffset, size_t hOffset, const void *src, size_t count, cudaMemcpyKind kind)

---

hipError_t hipMemcpyToArray(hipArray* dst, size_t wOffset, size_t hOffset, const void* src, size_t count, hipMemcpyKind kind)

Copies data between host and device.

Parameters

[in] dst	Destination memory address
[in] wOffset	Destination starting X offset (columns in bytes)
[in] hOffset	Destination starting Y offset (rows)
[in] src	Source memory address
[in] count	Size in bytes to copy
[in] kind	Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpy, hipMemcpy2DToArray, hipMemcpy2D, hipMemcpyFromArray, hipMemcpyToSymbol, hipMemcpyAsync

```
    float* hData = (float*)malloc(size);
    memset(hData, 0, size);
    hipChannelFormatDesc channelDesc = hipCreateChannelDesc(32, 0, 0, 0,
hipChannelFormatKindFloat);
    hipArray *hipArray, *hipOutArray;
    hipMallocArray(&hipArray, &channelDesc, width, height);
```

```
hipMemcpyToArray(hipArray, 0, 0, hData, size, hipMemcpyHostToDevice);
```

11.5 hipMemcpyToArrayAsync*

__host__cudaError_t cudaMemcpyToArrayAsync (cudaArray_t dst, size_t wOffset, size_t hOffset, const void *src, size_t count, cudaMemcpyKind kind, cudaStream_t stream)

---

hipError_t hipMemcpyToArrayAsync(hipArray_t dst, size_t wOffset, size_t hOffset, const void* src, size_t count, hipMemcpyKind kind, hipStream_t stream)

Copies data between host and device.

Parameters

[in] dst	Destination memory address
[in] wOffset	Destination starting X offset (columns in bytes)
[in] hOffset	Destination starting Y offset (rows)
[in] src	Source memory address
[in] count	Size in bytes to copy
[in] kind	Type of transfer
[in] stream	Stream identifier

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidPitchValue, #hipErrorInvalidDevicePointer, #hipErrorInvalidMemcpyDirection

See also

hipMemcpyToArray

12 Stream Ordered Memory Allocator

12.1 hipFreeAsync*

__host__cudaError_t cudaFreeAsync (void* devPtr, cudaStream_t hStream)

---

hipError_t hipFreeAsync(void* dev_ptr, hipStream_t stream)

Frees memory with stream ordered semantics.

Inserts a free operation into stream.The allocation must not be used after stream execution reaches the free. After this API returns, accessing the memory from any

subsequent work launched on the GPU or querying its pointer attributes results in undefined behavior.

During stream capture, this function results in the creation of a free node and must therefore be passed the address of a graph allocation.

Parameters

[in] dev_ptr Pointer to device memory to free

[in] stream The stream, where the destruciton will occur according to the execution order

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorNotSupported

12.2 hipMallocAsync*

__host__cudaError_t cudaMallocAsync (void** devPtr, size_t size, cudaStream_t hStream)

---

hipError_t hipMallocAsync(void** dev_ptr, size_t size, hipStream_t stream)

Allocates memory with stream ordered semantics

Parameters

[out] dev_ptr Returned device pointer of memory allocation

[in] size Number of bytes to allocate

[in] stream The stream establishing the stream ordering contract and the memory pool to allocate from

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorNotSupported, #hipErrorOutOfMemory

12.3 hipMallocFromPoolAsync*

__host__cudaError_t cudaMallocFromPoolAsync (void** ptr, size_t size, cudaMemPool_t memPool, cudaStream_t stream)

---

hipError_t hipMallocFromPoolAsync(void** dev_ptr, size_t size, hipMemPool_t mem_pool, hipStream_t stream)

Allocates memory from a specified pool with stream ordered semantics.

Parameters

[out] dev_ptr Returned device pointer

[in] size Number of bytes to allocate

[in] mem_pool The pool to allocate from

[in] stream The stream establishing the stream ordering semantic

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorNotSupported, #hipErrorOutOfMemory

12.4 hipMemPoolCreate*

__host__cudaError_t cudaMemPoolCreate (cudaMemPool_t* memPool, const cudaMemPoolProps* poolProps)

---

hipError_t hipMemPoolCreate(hipMemPool_t* mem_pool, const hipMemPoolProps* pool_props)

Creates a memory pool

Parameters

[out] mem_pool Contains createed memory pool

[in] pool_props Memory pool properties

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorNotSupported

12.5 hipMemPoolDestroy*

__host__cudaError_t cudaMemPoolDestroy (cudaMemPool_t memPool)

---

hipError_t hipMemPoolDestroy(hipMemPool_t mem_pool)

Destroys the specified memory pool

Parameters

[in] mem_pool Memory pool for destruction

Returns

#hipSuccess, #hipErrorInvalidValue

12.6 hipMemPoolExportPointer*

__host__cudaError_t cudaMemPoolExportPointer (cudaMemPoolPtrExportData* exportData, void* ptr)

---

hipError_t hipMemPoolExportPointer(hipMemPoolPtrExportData* export_data, void* dev_ptr)

Export data to share a memory pool allocation between processes.

Parameters

[out] export_data Returned export data

[in] dev_ptr Pointer to memory being exported

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorOutOfMemory

12.7 hipMemPoolExportToShareableHandle*

__host__cudaError_t cudaMemPoolExportToShareableHandle (void* shareableHandle, cudaMemPool_t memPool, cudaMemAllocationHandleType handleType, unsigned int flags)

---

hipError_t hipMemPoolExportToShareableHandle(void* shared_handle, hipMemPool_t mem_pool, hipMemAllocationHandleType handle_type, unsigned int flags)

Exports a memory pool to the requested handle type.

Parameters

[out] shared_handle Pointer to the location in which to store the requested handle

[in] mem_pool Pool to export

[in] handle_type The type of handle to create

[in] flags Must be 0

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorOutOfMemory

12.8 hipMemPoolGetAccess*

__host__cudaError_t cudaMemPoolGetAccess (cudaMemAccessFlags ** flags,

cudaMemPool_t memPool, cudaMemLocation* location)

---

hipError_t hipMemPoolGetAccess(hipMemAccessFlags* flags, hipMemPool_t mem_pool, hipMemLocation* location)

Parameters

[out] flags Accessibility of the memory pool from the specified location/device

[in] mem_pool Memory pool being queried

[in] location Location/device for memory pool access

Returns

#hipSuccess, #hipErrorInvalidValue

12.9 hipMemPoolGetAttribute*

__host__cudaError_t cudaMemPoolGetAttribute (cudaMemPool_t memPool, cudaMemPoolAttr attr, void* value)

---

hipError_t hipMemPoolGetAttribute(hipMemPool_t mem_pool, hipMemPoolAttr attr, void* value)

Gets attributes of a memory pool

Parameters

[in] mem_pool The memory pool to get attributes of

[in] attr The attribute to get

[in] value Retrieved value

Returns

#hipSuccess, #hipErrorInvalidValue

12.10 hipMemPoolImportFromShareableHandle*

__host__cudaError_t cudaMemPoolImportFromShareableHandle (cudaMemPool_t* memPool, void* shareableHandle, cudaMemAllocationHandleType handleType, unsigned int flags)

---

hipError_t hipMemPoolImportFromShareableHandle(hipMemPool_t* mem_pool, void* shared_handle, hipMemAllocationHandleType handle_type, unsigned int flags)

Imports a memory pool from a shared handle.

Parameters

[out] mem_pool Returned memory pool

[in] shared_handle OS handle of the pool to open

[in] handle_type The type of handle being imported

[in] flags Must be 0

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorOutOfMemory

12.11 hipMemPoolImportPointer*

__host__cudaError_t cudaMemPoolImportPointer (void** ptr, cudaMemPool_t memPool, cudaMemPoolPtrExportData* exportData)

---

hipError_t hipMemPoolImportPointer(void** dev_ptr, hipMemPool_t mem_pool, hipMemPoolPtrExportData* export_data)

Import a memory pool allocation from another process.

Parameters

[out] dev_ptr Pointer to imported memory

[in] mem_pool Memory pool from which to import a pointer

[in] export_data Data specifying the memory to import

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorNotInitialized, #hipErrorOutOfMemory

12.12 hipMemPoolSetAccess*

__host__cudaError_t cudaMemPoolSetAccess (cudaMemPool_t memPool, const cudaMemAccessDesc* descList, size_t count)

---

hipError_t hipMemPoolSetAccess(hipMemPool_t mem_pool, const hipMemAccessDesc* desc_list, size_t count)

Controls visibility of the specified pool between devices

Parameters

[in] mem_pool Memory pool for acccess change

[in] desc_list Array of access descriptors. Each descriptor instructs the access to enable for a single gpu

[in] count Number of descriptors in the map array.

Returns

#hipSuccess, #hipErrorInvalidValue

12.13 hipMemPoolSetAttribute*

__host__cudaError_t cudaMemPoolSetAttribute (cudaMemPool_t memPool, cudaMemPoolAttr attr, void* value)

---

hipError_t hipMemPoolSetAttribute(hipMemPool_t mem_pool, hipMemPoolAttr attr, void* value)

Sets attributes of a memory pool

Parameters

[in] mem_pool The memory pool to modify

[in] attr The attribute to modify

[in] value Pointer to the value to assign

Returns

#hipSuccess, #hipErrorInvalidValue

12.14 hipMemPoolTrimTo*

__host__cudaError_t cudaMemPoolTrimTo (cudaMemPool_t memPool, size_t minBytesToKeep)

---

hipError_t hipMemPoolTrimTo(hipMemPool_t mem_pool, size_t min_bytes_to_hold)

Releases freed memory back to the OS

Parameters

[in] mem_pool The memory pool to trim allocations

[in] min_bytes_to_hold If the pool has less than min_bytes_to_hold reserved, then the TrimTo operation is a no-op. Otherwise the memory pool will contain at least

min_bytes_to_hold bytes reserved after the operation.

Returns

#hipSuccess, #hipErrorInvalidValue

13 Unified Addressing

13.1 hipPointerGetAttributes

__host__cudaError_t cudaPointerGetAttributes (cudaPointerAttributes *attributes, const void *ptr)

---

hipError_t hipPointerGetAttributes(hipPointerAttribute_t* attributes, const void* ptr)

Return attributes for the specified pointer.

Parameters

[out] attributes for the specified pointer

[in] pointer to get attributes for

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue

See also

hipGetDeviceCount, hipGetDevice, hipSetDevice, hipChooseDevice

```
char* A_d;
hipPointerAttribute_t attribs;
hipPointerAttribute_t attribs2;
// Device memory
printf("\nDevice memory (hipMalloc)\n");
HIPCHECK(hipPointerGetAttributes(&attribs, A_d));
```

14 Peer Device Memory Access

14.1 hipDeviceCanAccessPeer

__host__cudaError_t cudaDeviceCanAccessPeer (int *canAccessPeer, int device, int peerDevice)

---

hipError_t hipDeviceCanAccessPeer(int* canAccessPeer, int deviceId, int

peerDeviceId)

Determine if a device can access a peer's memory.

Parameters

[out] canAccessPeer Returns the peer access capability (0 or 1)

[in] device Device from where memory may be accessed.

[in] peerDevice Device where memory is physically located

Returns "1" in canAccessPeer if the specified device is capable of directly accessing memory physically located on peerDevice , or "0" if not.

Returns "0" in canAccessPeer if deviceId == peerDeviceId, and both are valid devices : a device is not a peer of itself.

Returns

#hipSuccess,

#hipErrorInvalidDevice if deviceId or peerDeviceId are not valid devices

```
void setupPeerTests() {
int deviceCnt;
HIPCHECK(hipGetDeviceCount(&deviceCnt));
g_currentDevice = p_gpuDevice;
g_peerDevice = (p_peerDevice == -1) ? ((g_currentDevice + 1) % deviceCnt) : p_peerDevice;
printf("N=%zu  device=%d peerDevice=%d (%d devices total)\n", N, g_currentDevice, g_peerDevice,
deviceCnt);
// Must be on a multi-gpu system:
assert(g_currentDevice != g_peerDevice);
int canAccessPeer;
HIPCHECK(hipDeviceCanAccessPeer(&canAccessPeer, g_currentDevice, g_peerDevice));
printf("dev#%d canAccessPeer:#%d=%d\n", g_currentDevice, g_peerDevice, canAccessPeer);
assert(canAccessPeer);
```

```
HIPCHECK(hipSetDevice(g_currentDevice));
HIPCHECK(hipDeviceReset());
HIPCHECK(hipSetDevice(g_peerDevice));
HIPCHECK(hipDeviceReset());
}
```

14.2 hipDeviceDisablePeerAccess

__host__cudaError_t cudaDeviceDisablePeerAccess (int peerDevice)

---

hipError_t hipDeviceDisablePeerAccess(int peerDeviceId)

Disable direct access from current device's virtual address space to memory allocations physically located on a peer device.

Returns hipErrorPeerAccessNotEnabled if direct access to memory on peerDevice has not yet been enabled from the current device.

Parameters

[in] peerDeviceId

Returns

#hipSuccess, #hipErrorPeerAccessNotEnabled

```
int g_peerDevi e = hipSuccess;
//---
// Enable same device twice in a row:
HIPCHECK(hipSetDevice(g_currentDevice));
HIPCHECK(hipDeviceEnablePeerAccess(g_peerDevice, 0));
e = (hipDeviceEnablePeerAccess(g_peerDevice, 0));
HIPASSERT(e == hipErrorPeerAccessAlreadyEnabled);
//---
// try disabling twice in a row
HIPCHECK(hipDeviceDisablePeerAccess(g_peerDevice));
e = (hipDeviceDisablePeerAccess(g_peerDevice));
HIPASSERT(e == hipErrorPeerAccessNotEnabled);
// More tests here:
printf("==done: %s\n\n", __func__);
```

开发者社区

14.3 hipDeviceEnablePeerAccess

__host__cudaError_t cudaDeviceEnablePeerAccess (int peerDevice, unsigned int flags)

---

hipError_t hipDeviceEnablePeerAccess(int peerDeviceId, unsigned int flags)

Enable direct access from current device's virtual address space to memory allocations physically located on a peer device.

Memory which already allocated on peer device will be mapped into the address space of the current device. In addition, all future memory allocations on peerDeviceId will be mapped into the address space of the current device when the memory is allocated. The peer memory remains accessible from the current device until a call to hipDeviceDisablePeerAccess or hipDeviceReset.

Parameters

[in] peerDeviceId

[in] flags Returns #hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidValue,

Returns

#hipErrorPeerAccessAlreadyEnabled if peer access is already enabled for this device.

```
void allocMemoryFirst(bool useAsyncCopy) {
printf("\n==testing: %s useAsyncCopy=%d\n", __func__, useAsyncCopy);
setupPeerTests();
size_t Nbytes = N * sizeof(char);
char *A_d0, *A_d1;
char* A_h;
A_h = (char*)malloc(Nbytes);
//---
// allocate and initialize memory on device0
HIPCHECK(hipSetDevice(g_currentDevice));
HIPCHECK(hipMalloc(&A_d0, Nbytes));
HIPCHECK(hipMemset(A_d0, memsetval, Nbytes));
```

开发者社区

```
    // allocate and initialize memory on peer device
    HIPCHECK(hipSetDevice(g_peerDevice));
    HIPCHECK(hipMalloc(&A_d1, Nbytes));
    HIPCHECK(hipMemset(A_d1, 0x13, Nbytes));
    //---
    // Enable peer access, for memory already allocated:
    HIPCHECK(hipSetDevice(g_currentDevice));
    HIPCHECK(hipDeviceEnablePeerAccess(g_peerDevice, 0));
    if (p_mirrorPeers) {
    int canAccessPeer;
    HIPCHECK(hipDeviceCanAccessPeer(&canAccessPeer, g_peerDevice,
g_currentDevice));
    assert(canAccessPeer);
    HIPCHECK(hipSetDevice(g_peerDevice));
    HIPCHECK(hipDeviceEnablePeerAccess(g_currentDevice, 0));
    }
```

15 Texture Reference Management [DEPRECATED]

15.1 hipBindTexture

__host__cudaError_t cudaBindTexture (size_t *offset, const textureReference *texref, const void *devPtr, const cudaChannelFormatDesc *desc, size_t size)

---

hipError_t hipBindTexture(size_t* offset, textureReference* tex, const void* devPtr,const hipChannelFormatDesc* desc, size_t size = UINT_MAX);

```
    texture<float, 1, hipReadModeElementType> tex;
    float *texBuf;
    size_t offset = 0;
    hipChannelFormatDesc chanDesc =
    hipCreateChannelDesc(32, 0, 0, 0, hipChannelFormatKindFloat);
    HIPCHECK(hipMalloc(&texBuf, N * sizeof(float)));
    HIPCHECK(hipMalloc(&devBuf, N * sizeof(float)));
```

```
    HIPCHECK(hipMemcpy(texBuf, val, N * sizeof(float), hipMemcpyHostToDevice));
    HIPCHECK(hipBindTexture(&offset, tex, (void *)texBuf, chanDesc, N * sizeof(float)));
```

15.2 hipBindTexture2D

__host__cudaError_t cudaBindTexture2D (size_t* offset, const textureReference* texref, const void* devPtr, const cudaChannelFormatDesc* desc, size_t width, size_t height, size_t pitch)

---

hipError_t hipBindTexture2D(size_t* offset,const textureReference* tex,const void* devPtr,const hipChannelFormatDesc* desc,size_t width,size_t height,size_t pitch)

15.3 hipBindTextureToArray

__host__cudaError_t cudaBindTextureToArray (const textureReference *texref, cudaArray_const_t array, const cudaChannelFormatDesc *desc)

---

hipError_t hipBindTextureToArray(const textureReference* tex,hipArray_const_t array,const hipChannelFormatDesc* desc)

```
    texture<float, 2, hipReadModeElementType> tex;
    hipChannelFormatDesc channelDesc = hipCreateChannelDesc(32, 0, 0, 0, hipChannelFormatKindFloat);
    hipArray* hipArray;
    hipMallocArray(&hipArray, &channelDesc, width, height);
    hipMemcpyToArray(hipArray, 0, 0, hData, size, hipMemcpyHostToDevice);
    tex.addressMode[0] = hipAddressModeWrap;
    tex.addressMode[1] = hipAddressModeWrap;
    tex.filterMode = hipFilterModePoint;
    tex.normalized = 0;
    hipBindTextureToArray(tex, hipArray, channelDesc);
```

15.4 hipBindTextureToMipmappedArray

__host__cudaError_t cudaBindTextureToMipmappedArray (const

textureReference *texref, cudaMipmappedArray_const_t mipmappedArray, const cudaChannelFormatDesc *desc)

---

hipError_t hipBindTextureToMipmappedArray(const textureReference* tex, hipMipmappedArray_const_t mipmappedArray,const hipChannelFormatDesc* desc);

15.5 hipGetTextureAlignmentOffset

__host__cudaError_t cudaGetTextureAlignmentOffset (size_t *offset, const textureReference *texref)

---

hipError_t hipGetTextureAlignmentOffset(size_t* offset, const textureReference* texref)

```
texture<float, 1, hipReadModeElementType> tex;
float *texBuf;
size_t offset = 0;
hipChannelFormatDesc chanDesc =
hipCreateChannelDesc(32, 0, 0, 0, hipChannelFormatKindFloat);
HIPCHECK(hipMalloc(&texBuf, N * sizeof(float)));
HIPCHECK(hipMalloc(&devBuf, N * sizeof(float)));
HIPCHECK(hipMemcpy(texBuf, val, N * sizeof(float), hipMemcpyHostToDevice));
HIPCHECK(hipBindTexture(&offset, tex, (void *)texBuf, chanDesc, N * sizeof(float)));
HIPCHECK(hipGetTextureAlignmentOffset(&offset,&tex));
```

15.6 hipGetTextureReference

__host__cudaError_t cudaGetTextureReference (const textureReference **texref, const void *symbol)

---

hipError_t hipGetTextureReference(const textureReference** texref, const void* symbol)

15.7 hipUnbindTexture

__host__cudaError_t cudaUnbindTexture (const textureReference *texref)

---

hipError_t hipUnbindTexture(const textureReference* tex)

```
texture<float, 1, hipReadModeElementType> tex;
float *texBuf;
size_t offset = 0;
hipChannelFormatDesc chanDesc =
hipCreateChannelDesc(32, 0, 0, 0, hipChannelFormatKindFloat);
HIPCHECK(hipMalloc(&texBuf, N * sizeof(float)));
HIPCHECK(hipMalloc(&devBuf, N * sizeof(float)));
HIPCHECK(hipMemcpy(texBuf, val, N * sizeof(float), hipMemcpyHostToDevice));
HIPCHECK(hipBindTexture(&offset, tex, (void *)texBuf, chanDesc, N * sizeof(float)));
hipLaunchKernelGGL(kernel, dim3(dimGrid), dim3(dimBlock), 0, 0, devBuf);
HIPCHECK(hipDeviceSynchronize());
HIPCHECK(hipUnbindTexture(&tex));
```

16 Surface Reference Management [DEPRECATED]

16.1 hipBindSurfaceToArray*

__host__cudaError_t cudaBindSurfaceToArray (const surfaceReference *surfref, cudaArray_const_t array, const cudaChannelFormatDesc *desc)

---

hipError_t hipBindSurfaceToArray(const surfaceReference *surf, hipArray_const_t array, const hipChannelFormatDesc *desc)

Binds a memory area to a surface.

Parameters

[in] surf Pointer of surface reference.

[in] array Array to bind.

[in] desc Pointer of channel format descriptor.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidValue

```
int width = 64;
unsigned int size = width * sizeof(int);
```

```
int *hData = (int*) malloc(size);
memset(hData, 0, size);
for (int j = 0; j < width; j++)
{
 hData[j]=1;
}
hipChannelFormatDesc channelDesc = hipCreateChannelDesc<int>();
hipArray *hipArray = nullptr;
HIP_CHECK(hipMallocArray(&hipArray, &channelDesc, width, 0, hipArraySurfaceLoadStore));
HIP_CHECK(hipMemcpyToArray(hipArray, 0, 0, hData, size, hipMemcpyHostToDevice));
REQUIRE(hipSuccess == hipBindSurfaceToArray(&SurfaceR<int, 1>::s_surfr, hipArray, &channelDesc));
HIP_CHECK(hipDeviceSynchronize());
HIP_CHECK(hipFreeArray(hipArray));
free(hData);
```

16.2 hipGetSurfaceReference*

__host__cudaError_t cudaGetSurfaceReference (const surfaceReference **surfref, const void *symbol)

---

hipError_t hipGetSurfaceReference(const surfaceReference** surfref, const void* symbol)

Gets the surface reference related with the symbol.

Parameters

[out] surfref surface reference

[in] symbol pointer to the symbol related with the surface for the reference

Returns

#hipSuccess, #hipErrorInvalidValue

17 Texture Object Management

17.1 hipCreateChannelDesc*

__host__cudaChannelFormatDesc cudaCreateChannelDesc (int x, int y, int z, int w, cudaChannelFormatKind f)

---

hipChannelFormatDesc hipCreateChannelDesc(int x, int y, int z, int w, hipChannelFormatKind f)

Returns a channel descriptor using the specified format.

Parameters

[in] x X component

[in] y Y component

[in] z Z component

[in] w W component

[in] f Channel format

Returns Channel descriptor with format f

17.2 hipCreateTextureObject

__host__cudaError_t cudaCreateTextureObject (cudaTextureObject_t* pTexObject, const cudaResourceDesc* pResDesc, const cudaTextureDesc* pTexDesc, const cudaResourceViewDesc* pResViewDesc)

---

hipError_t hipCreateTextureObject(hipTextureObject_t* pTexObject,const hipResourceDesc* pResDesc,const hipTextureDesc* pTexDesc,const struct hipResourceViewDesc* pResViewDesc)

17.3 hipDestroyTextureObject

__host__cudaError_t cudaDestroyTextureObject (cudaTextureObject_t texObject)

---

hipError_t hipDestroyTextureObject(hipTextureObject_t textureObject)

```
struct hipResourceDesc resDesc;
memset(&resDesc, 0, sizeof(resDesc));
struct hipTextureDesc texDesc;
memset(&texDesc, 0, sizeof(texDesc));
```

开发者社区

```
// Create texture object
hipTextureObject_t textureObject = 0;
hipCreateTextureObject(&textureObject, &resDesc, &texDesc, NULL);
hipDestroyTextureObject(textureObject);
```

17.4 hipGetChannelDesc

__host__cudaError_t cudaGetChannelDesc (cudaChannelFormatDesc *desc, cudaArray_const_t array)

---

hipError_t hipGetChannelDesc(hipChannelFormatDesc* desc, hipArray_const_t array)

```
bool runTest()
{
bool testResult=true;
hipChannelFormatDesc chan_test,chan_desc=hipCreateChannelDesc(32,0,0,0,hipChannelFormatKindSigned);
hipArray *hipArray;
HIPCHECK(hipMallocArray(&hipArray, &chan_desc,C,R,0));
HIPCHECK(hipGetChannelDesc(&chan_test,hipArray));
if((chan_test.x == 32)&&(chan_test.y == 0)&&(chan_test.z == 0)&&(chan_test.f == 0))
testResult=true;
else
testResult=false;
HIPCHECK(hipFreeArray(hipArray));
return testResult;
}
```

17.5 hipGetTextureObjectResourceDesc

__host__cudaError_t cudaGetTextureObjectResourceDesc (cudaResourceDesc* pResDesc, cudaTextureObject_t texObject)

---

hipError_t hipGetTextureObjectResourceDesc(hipResourceDesc* pResDesc,hipTextureObject_t textureObject)

17.6 hipGetTextureObjectResourceViewDesc

__host__cudaError_t cudaGetTextureObjectResourceViewDesc (cudaResourceViewDesc* pResViewDesc, cudaTextureObject_t texObject)

---

hipError_t hipGetTextureObjectResourceViewDesc(struct hipResourceViewDesc* pResViewDesc,hipTextureObject_t textureObject)

17.7 hipGetTextureObjectTextureDesc

__host__cudaError_t cudaGetTextureObjectTextureDesc (cudaTextureDesc* pTexDesc, cudaTextureObject_t texObject)

---

hipError_t hipGetTextureObjectTextureDesc(hipTextureDesc* pTexDesc,hipTextureObject_t textureObject)

17.8 hipDeviceGetTexture1DLinearMaxWidth【dtk-26.10 新增】

__host__ cudaError_t cudaDeviceGetTexture1DLinearMaxWidth(size_t *maxWidthInElements, const struct cudaChannelFormatDesc *fmtDesc, int device)

---

hipError_t hipDeviceGetTexture1DLinearMaxWidth(size_t* max_width, const hipChannelFormatDesc* desc,int device);

18 Surface Object Management

18.1 hipCreateSurfaceObject

__host__cudaError_t cudaCreateSurfaceObject (cudaSurfaceObject_t *pSurfObject, const cudaResourceDesc *pResDesc)

---

hipError_t hipCreateSurfaceObject(hipSurfaceObject_t* pSurfObject, const hipResourceDesc* pResDesc)

```
struct hipResourceDesc resOutDesc;
memset(&resOutDesc, 0, sizeof(resOutDesc));
resOutDesc.resType = hipResourceTypeArray;
resOutDesc.res.array.array = hipOutArray;
hipSurfaceObject_t outSurfaceObject = 0;
```

```
hipCreateSurfaceObject(&outSurfaceObject, &resOutDesc);
```

18.2 hipDestroySurfaceObject

__host__cudaError_t cudaDestroySurfaceObject (cudaSurfaceObject_t surfObject)

hipError_t hipDestroySurfaceObject(hipSurfaceObject_t surfaceObject)

```
struct hipResourceDesc resOutDesc;
memset(&resOutDesc, 0, sizeof(resOutDesc));
resOutDesc.resType = hipResourceTypeArray;
resOutDesc.res.array.array = hipOutArray;
hipSurfaceObject_t outSurfaceObject = 0;
hipCreateSurfaceObject(&outSurfaceObject, &resOutDesc);
float* dData = NULL;
hipMalloc((void**)&dData, size);
dim3 dimBlock(16, 16, 1);
dim3 dimGrid(width / dimBlock.x, height / dimBlock.y, 1);
hipLaunchKernelGGL(tex2DKernel, dim3(dimGrid), dim3(dimBlock), 0, 0,
surfaceObject,
outSurfaceObject, width, height);
hipDeviceSynchronize();
float* hOutputData = (float*)malloc(size);
memset(hOutputData, 0, size);
hipMemcpyFromArray(hOutputData, hipOutArray, 0, 0, size,
hipMemcpyDeviceToHost);
hipDestroySurfaceObject(outSurfaceObject);
```

18.3 hipGetSurfaceObjectResourceDesc*

__host__cudaError_t cudaGetSurfaceObjectResourceDesc (cudaResourceDesc *pResDesc, cudaSurfaceObject_t surfObject)

hipError_t hipGetSurfaceObjectResourceDesc(hipResourceDesc* pResDesc, hipSurfaceObject_t surfObject)

Gets resource descriptor for the surface object.

Parameters

[out] pResDesc pointer to resource descriptor

[in] surfObject surface object

Returns

#hipSuccess, #hipErrorInvalidValue

```
hipResourceDesc pResDesc;
memset(&pResDesc, 0, sizeof(hipResourceDesc));
int width = 100;
int height = 50;
float surfaceData[width][height];
for(int i = 0; i < width; i++){
  for(int j = 0; j < height; j++){
    surfaceData[i][j] = 1;
  }
}

hipArray *d_surfaceArray = nullptr;
hipChannelFormatDesc channelDesc = hipCreateChannelDesc<float>();
HIP_CHECK(hipMallocArray(&d_surfaceArray, &channelDesc, width, height));
HIP_CHECK(hipMemcpy2DToArray(d_surfaceArray, 0, 0, surfaceData, width *
sizeof(float), width * sizeof(float), height, hipMemcpyHostToDevice));

hipResourceDesc resDesc;
memset(&resDesc, 0, sizeof(hipResourceDesc));
resDesc.resType = hipResourceTypeArray;
resDesc.res.array.array = d_surfaceArray;
hipSurfaceObject_t surfObject =  nullptr;
HIP_CHECK(hipCreateSurfaceObject(&surfObject, &resDesc));
REQUIRE(hipSuccess == hipGetSurfaceObjectResourceDesc(&pResDesc,
surfObject));
```

开发者社区

19 Version Management

19.1 hipDriverGetVersion

__host__cudaError_t cudaDriverGetVersion (int *driverVersion)

---

hipError_t hipDriverGetVersion(int* driverVersion)

Returns the approximate HIP driver version.

Parameters

[out] driverVersion

Returns

#hipSuccess, #hipErrorInavlidValue

Warning

The HIP feature set does not correspond to an exact CUDA SDK driver revision. This function always set *driverVersion to 4 as an approximation though HIP supports some features which were introduced in later CUDA SDK revisions. HIP apps code should not rely on the driver revision number here and should use arch feature flags to test device capabilities or conditional compilation.

See also

hipRuntimeGetVersion

```
int v;
hipDriverGetVersion(&v);
passed();
```

19.2 hipRuntimeGetVersion

__host____device__cudaError_t cudaRuntimeGetVersion (int *runtimeVersion)

---

hipError_t hipRuntimeGetVersion(int* runtimeVersion)

Returns the approximate HIP Runtime version.

Parameters

[out] runtimeVersion

Returns

#hipSuccess, #hipErrorInavlidValue

Warning

On HIP/HCC path this function returns HIP runtime patch version however on HIP/NVCC path this function return CUDA runtime version.

See also

hipDriverGetVersion

```
int main()
{
int runtimeVersion;
HIPCHECK(hipRuntimeGetVersion(&runtimeVersion));
passed();
}
```

20 Graph Management

20.1 hipDeviceGetGraphMemAttribute*

__host__cudaError_t cudaDeviceGetGraphMemAttribute(int evice, cudaGraphMemAttributeType attr, void* value)

---

hipError_t hipDeviceGetGraphMemAttribute(int device, hipGraphMemAttributeType attr, void* value)

Get the mem attribute for graphs.

Parameters

[in] device - device the attr is get for.

[in] attr - attr to get.

[out] value - value for specific attr.

Returns

#hipSuccess, #hipErrorInvalidDevice

20.2 hipDeviceGraphMemTrim*

__host__cudaError_t cudaDeviceGraphMemTrim (int device)

---

hipError_t hipDeviceGraphMemTrim(int device)

Free unused memory on specific device used for graph back to OS

Parameters

[in] device - device the memory is used for graphs

Returns

#hipSuccess, #hipErrorInvalidDevice

20.3 hipDeviceSetGraphMemAttribute*

__host__cudaError_t cudaDeviceSetGraphMemAttribute (int device, cudaGraphMemAttributeType attr, void* value)

---

hipError_t hipDeviceSetGraphMemAttribute(int device, hipGraphMemAttributeType attr, void* value)

Set the mem attribute for graphs.

Parameters

[in] device - device the attr is set for.

[in] attr - attr to set.

[in] value - value for specific attr.

Returns

#hipSuccess, #hipErrorInvalidDevice

20.4 hipGraphAddChildGraphNode*

__host__cudaError_t cudaGraphAddChildGraphNode (cudaGraphNode_t *pGraphNode, cudaGraph_t graph, cudaGraphNode_t *pDependencies, size_t numDependencies, cudaGraph_t childGraph)

---

hipError_t hipGraphAddChildGraphNode(hipGraphNode_t* pGraphNode, hipGraph_t graph, const hipGraphNode_t* pDependencies, size_t numDependencies, hipGraph_t childGraph)

Creates a child graph node and adds it to a graph.

Parameters

[out] pGraphNode - pointer to the graph node to create.

[in] graph - instance of the graph to add the created node.

[in] pDependencies - const pointer to the dependencies on the memset execution node.

开发者社区

[in] numDependencies - the number of the dependencies.

[in] childGraph - the graph to clone into this node

Returns

#hipSuccess, #hipErrorInvalidValue

20.5 hipGraphAddDependencies

__host__cudaError_t cudaGraphAddDependencies (cudaGraph_t graph, cudaGraphNode_t *from, cudaGraphNode_t *to, size_t numDependencies)

---

hipError_t hipGraphAddDependencies(hipGraph_t graph, const hipGraphNode_t* from,const hipGraphNode_t* to, size_t numDependencies)

Adds dependency edges to a graph.

Parameters

[in] graph Instance of the graph to add dependencies.

[in] from Pointer to the graph nodes with dependenties to add from.

[in] to Pointer to the graph nodes to add dependenties to.

[in] numDependencies The number of dependencies to add.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.6 hipGraphAddEmptyNode

__host__cudaError_t cudaGraphAddEmptyNode (cudaGraphNode_t *pGraphNode, cudaGraph_t graph, cudaGraphNode_t *pDependencies, size_t numDependencies)

---

hipError_t hipGraphAddEmptyNode(hipGraphNode_t* pGraphNode, hipGraph_t graph,const hipGraphNode_t* pDependencies, size_t numDependencies)

Creates an empty node and adds it to a graph.

[out] pGraphNode Pointer to the graph node to create and add to

the graph.

[in,out] graph Instane of the graph the node is add to.

[in] pDependencies Const pointer to the node dependenties.

[in] numDependencies The number of dependencies.ies.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.7 hipGraphAddEventRecordNode

__host__cudaError_t cudaGraphAddEventRecordNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, cudaEvent_t event)

---

hipError_t hipGraphAddEventRecordNode(hipGraphNode_t* pGraphNode, hipGraph_t graph,const hipGraphNode_t* pDependencies, size_t numDependencies,hipEvent_t event)

Creates an event record node and adds it to a graph.

Parameters

[out] pGraphNode Pointer to the graph node to create and add to the graph.

[in] graph Instane of the graph the node to be added.

[in] pDependencies Const pointer to the node dependenties.

[in] numDependencies The number of dependencies.

[in] event Event for the node.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.8 hipGraphAddEventWaitNode

__host__cudaError_t cudaGraphAddEventWaitNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, cudaEvent_t event)

---

hipError_t hipGraphAddEventWaitNode(hipGraphNode_t* pGraphNode, hipGraph_t graph,const hipGraphNode_t* pDependencies, size_t numDependencies,hipEvent_t event)

Creates an event wait node and adds it to a graph.

Parameters

[out] pGraphNode Pointer to the graph node to create and add to the graph.

[in] graph Instane of the graph the node to be added.

[in] pDependencies Const pointer to the node dependenties.

[in] numDependencies The number of dependencies.

[in] event Event for the node.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.9 hipGraphAddExternalSemaphoresSignalNode*

__host__cudaError_t cudaGraphAddExternalSemaphoresSignalNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, const cudaExternalSemaphoreSignalNodeParams* nodeParams)

---

hipError_t hipGraphAddExternalSemaphoresSignalNode(hipGraphNode_t* pGraphNode, hipGraph_t graph, const hipGraphNode_t* pDependencies, size_t numDependencies, const hipExternalSemaphoreSignalNodeParams* nodeParams)

Creates a external semaphor signal node and adds it to a graph.

Parameters

[out] pGraphNode - pointer to the graph node to create.

[in] graph - instance of the graph to add the created node.

[in] pDependencies - const pointer to the dependencies on the memset execution node.

[in] numDependencies - the number of the dependencies.

[in] nodeParams -pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.10 hipGraphAddExternalSemaphoresWaitNode*

__host__cudaError_t cudaGraphAddExternalSemaphoresWaitNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, const cudaExternalSemaphoreWaitNodeParams* nodeParams)

---

hipError_t hipGraphAddExternalSemaphoresWaitNode(hipGraphNode_t* pGraphNode, hipGraph_t graph, const hipGraphNode_t* pDependencies, size_t numDependencies, const hipExternalSemaphoreWaitNodeParams* nodeParams)

Creates a external semaphor wait node and adds it to a graph.

Parameters

[out] pGraphNode - pointer to the graph node to create.

[in] graph - instance of the graph to add the created node.

[in] pDependencies - const pointer to the dependencies on the memset execution node.

[in] numDependencies - the number of the dependencies.

[in] nodeParams -pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.11 hipGraphAddHostNode

__host__cudaError_t cudaGraphAddHostNode (cudaGraphNode_t *pGraphNode, cudaGraph_t graph, cudaGraphNode_t *pDependencies, size_t numDependencies, const cudaHostNodeParams *pNodeParams)

---

hipError_t hipGraphAddHostNode(hipGraphNode_t* pGraphNode, hipGraph_t graph,const hipGraphNode_t* pDependencies, size_t numDependencies,const hipHostNodeParams* pNodeParams)

Creates a host execution node and adds it to a graph.

[out] pGraphNode Pointer to the graph node to create.

[in] graph Instance of the graph to add the created node.

[in] pDependencies Const pointer to the dependencies on the memset execution node.

[in] numDependencies The number of the dependencies.

[in] pNodeParams Pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.12 hipGraphAddKernelNode

__host__cudaError_t cudaGraphAddKernelNode (cudaGraphNode_t *pGraphNode, cudaGraph_t graph, cudaGraphNode_t *pDependencies, size_t www.nvidia.com CUDA Runtime API vRelease Version | 298 numDependencies, const cudaKernelNodeParams *pNodeParams)

---

hipError_t hipGraphAddKernelNode(hipGraphNode_t* pGraphNode, hipGraph_t graph,const hipGraphNode_t* pDependencies, size_t numDependencies,const hipKernelNodeParams* pNodeParams)

Creates a kernel execution node and adds it to a graph.

Parameters

[out] pGraphNode Pointer to graph node to create.

[in,out] graph Instance of graph to add the created node.

[in] pDependencies Pointer to the dependencies on the kernel execution node.

[in] numDependencies The number of the dependencies.

[in] pNodeParams Pointer to the parameters to the kernel execution node on the GPU.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDeviceFunction

20.13 hipGraphAddMemAllocNode*

__host__cudaError_t cudaGraphAddMemAllocNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, cudaMemAllocNodeParams* nodeParams)

---

hipError_t hipGraphAddMemAllocNode(hipGraphNode_t* pGraphNode, hipGraph_t graph, const hipGraphNode_t* pDependencies, size_t numDependencies, hipMemAllocNodeParams* pNodeParams)

Creates a memory allocation node and adds it to a graph.

Parameters

[out] pGraphNode - Pointer to the graph node to create and add to the graph

[in] graph - Instane of the graph the node to be added

[in] pDependencies - Const pointer to the node dependenties

[in] numDependencies - The number of dependencies

[in] pNodeParams - Node parameters for memory allocation

Returns

#hipSuccess, #hipErrorInvalidValue

```
constexpr size_t size = 512 * 1024 * 1024;
constexpr size_t Nbytes = size * sizeof(int);
hipGraph_t graph;
```

```
hipGraphExec_t graphExec;
hipStream_t stream;
hipGraphNode_t allocNodeA;
hipMemAllocNodeParams allocParam;
HIP_CHECK(hipGraphCreate(&graph, 0));
HIP_CHECK(hipStreamCreate(&stream));
memset(&allocParam, 0, sizeof(allocParam));
allocParam.bytesize = Nbytes;
allocParam.poolProps.allocType = hipMemAllocationTypePinned;
allocParam.poolProps.location.id = 0;
allocParam.poolProps.location.type = hipMemLocationTypeDevice;
HIP_CHECK(hipGraphAddMemAllocNode(&allocNodeA, graph, nullptr,
                        0, &allocParam));
```

20.14 hipGraphAddMemFreeNode*

__host__cudaError_t cudaGraphAddMemFreeNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, void* dptr)

---

hipError_t hipGraphAddMemFreeNode(hipGraphNode_t* pGraphNode, hipGraph_t graph, const hipGraphNode_t* pDependencies, size_t numDependencies, void* dev_ptr)

Creates a memory free node and adds it to a graph.

Parameters

[out] pGraphNode - Pointer to the graph node to create and add to the graph

[in] graph - Instane of the graph the node to be added

[in] pDependencies - Const pointer to the node dependenties

[in] numDependencies - The number of dependencies

[in] dev_ptr - Pointer to the memory to be freed

Returns

#hipSuccess, #hipErrorInvalidValue

See also

hipGraphAddMemAllocNode

20.15 hipGraphAddMemcpyNode

__host__cudaError_t cudaGraphAddMemcpyNode (cudaGraphNode_t *pGraphNode, cudaGraph_t graph, cudaGraphNode_t *pDependencies, size_t numDependencies, const cudaMemcpy3DParms *pCopyParams)

---

hipError_t hipGraphAddMemcpyNode(hipGraphNode_t* pGraphNode, hipGraph_t graph,const hipGraphNode_t* pDependencies, size_t numDependencies,const hipMemcpy3DParms* pCopyParams)

Creates a memcpy node and adds it to a graph.

Parameters

[out] pGraphNode Pointer to graph node to create.

[in,out] graph Instance of the graph to add the created node.

[in] pDependencies Const pointer to the dependencies on the kernel execution node.

[in] numDependencies The number of the dependencies.

[in] pCopyParams Const pointer to the parameters for the memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.16 hipGraphAddMemcpyNode1D

__host__cudaError_t cudaGraphAddMemcpyNode1D (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, void* dst, const void* src, size_t count, cudaMemcpyKind kind)

---

hipError_t hipGraphAddMemcpyNode1D(hipGraphNode_t* pGraphNode, hipGraph_t graph,const hipGraphNode_t* pDependencies, size_t

numDependencies,void* dst, const void* src, size_t count, hipMemcpyKind kind)

Creates a 1D memcpy node and adds it to a graph.

Parameters

[out] pGraphNode Pointer to the graph node to create.

[in,out] graph Instance of the graph to add the created node.

[in] pDependencies Const pointer to the dependencies on the kernel execution node.

[in] dst Pointer to memory address to the destination..

[in] src Pointer to memory address to the source.

[in] count The size of the memory to copy.

[in] kind The type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.17 hipGraphAddMemcpyNodeFromSymbol

__host__cudaError_t cudaGraphAddMemcpyNodeFromSymbol (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, void* dst, const void* symbol, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipGraphAddMemcpyNodeFromSymbol(hipGraphNode_t* pGraphNode, hipGraph_t graph,const hipGraphNode_t* pDependencies,size_t numDependencies, void* dst, const void* symbol,size_t count, size_t offset, hipMemcpyKind kind)

Creates a memcpy node to copy from a symbol on the device and adds it to a graph.

Parameters

[out] pGraphNode Pointer to graph node to create.

[in] graph Instance of graph to add the created node.

[in] pDependencies Const pointer to the dependencies on the memcpy execution node.

[in] numDependencies The size of the memory to copy.

[in] dst Pointer to memory address to the destination

[in] symbol Device symbol address.

[in] count The size of the memory to copy.

[in] offset Offset from start of symbol in bytes.

[in] kind The type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.18 hipGraphAddMemcpyNodeToSymbol

__host__cudaError_t cudaGraphAddMemcpyNodeToSymbol (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, const void* symbol, const void* src, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipGraphAddMemcpyNodeToSymbol(hipGraphNode_t* pGraphNode, hipGraph_t graph,

const hipGraphNode_t* pDependencies,size_t numDependencies, const void* symbol,const void* src, size_t count, size_t offset,hipMemcpyKind kind)

Creates a memcpy node to copy to a symbol on the device and adds it to a graph.

[in] node Instance of the node to set parameters to.

[in] symbol Device symbol address.

[in] src Pointer to memory address of the src.

[in] count The size of the memory to copy.

[in] offset Offset from start of symbol in bytes.

[in] kind The type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.19 hipGraphAddMemsetNode

__host__cudaError_t cudaGraphAddMemsetNode (cudaGraphNode_t *pGraphNode, cudaGraph_t graph, cudaGraphNode_t *pDependencies, size_t numDependencies, const cudaMemsetParams *pMemsetParams)

---

hipError_t hipGraphAddMemsetNode(hipGraphNode_t* pGraphNode, hipGraph_t graph,const hipGraphNode_t* pDependencies, size_t numDependencies,const hipMemsetParams* pMemsetParams)

Creates a memset node and adds it to a graph.

Parameters

[out] pGraphNode Pointer to the graph node to create.

[in,out] graph Instance of the graph to add the created node.

[in] pDependencies Const pointer to the dependencies on the kernel execution node.

[in] numDependencies The number of the dependencies

[in] pMemsetParams Const pointer to the parameters for the memory set

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.20 hipGraphChildGraphNodeGetGraph*

__host__cudaError_t cudaGraphChildGraphNodeGetGraph (cudaGraphNode_t node, cudaGraph_t *pGraph)

---

hipError_t hipGraphChildGraphNodeGetGraph(hipGraphNode_t node, hipGraph_t* pGraph)

Gets a handle to the embedded graph of a child graph node.

Parameters

[in] node - instane of the node to get child graph.

[out] pGraph - pointer to get the graph.

Returns

#hipSuccess, #hipErrorInvalidValue

20.21 hipGraphClone*

__host__cudaError_t cudaGraphClone (cudaGraph_t *pGraphClone, cudaGraph_t originalGraph)

---

hipError_t hipGraphClone(hipGraph_t* pGraphClone, hipGraph_t originalGraph)

Clones a graph

Parameters

[out] pGraphClone - Returns newly created cloned graph.

[in] originalGraph - original graph to clone from.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryAllocation

20.22 hipGraphCreate

__host__cudaError_t cudaGraphCreate (cudaGraph_t *pGraph, unsigned int flags)

---

hipError_t hipGraphCreate(hipGraph_t* pGraph, unsigned int flags)

Creates a graph.

Parameters

[out] pGraph	Pointer to graph to create.
[in] flags	Flags for graph creation, must be 0.

Returns

#hipSuccess

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.23 hipGraphDebugDotPrint*

__host__cudaError_t cudaGraphDebugDotPrint (cudaGraph_t graph, const char* path, unsigned int flags)

---

hipError_t hipGraphDebugDotPrint(hipGraph_t graph, const char* path, unsigned int flags)

Write a DOT file describing graph structure.

Parameters

[in] graph - graph object for which DOT file has to be generated.

[in] path - path to write the DOT file.

[in] flags - Flags from hipGraphDebugDotFlags to get additional node information.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorOperatingSystem

20.24 hipGraphDestroy

__host__cudaError_t cudaGraphDestroy (cudaGraph_t graph)

---

hipError_t hipGraphDestroy(hipGraph_t graph)

Destroys a graph.

Parameters

[in] graph Instance of graph to destroy.

Returns

#hipSuccess

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.25 hipGraphDestroyNode*

__host__cudaError_t cudaGraphDestroyNode (cudaGraphNode_t node)

---

hipError_t hipGraphDestroyNode(hipGraphNode_t node)

Remove a node from the graph.

Parameters

[in] node - graph node to remove

Returns

#hipSuccess, #hipErrorInvalidValue

20.26 hipGraphEventRecordNodeGetEvent

__host__cudaError_t cudaGraphEventRecordNodeGetEvent (cudaGraphNode_t node, cudaEvent_t* event_out)

---

hipError_t hipGraphEventRecordNodeGetEvent(hipGraphNode_t node, hipEvent_t* event_out)

Returns the event associated with an event record node.

Parameters

[in] node Instane of the node to get event from.

[out] event_out Pointer to return the event.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.27 hipGraphEventRecordNodeSetEvent

__host__cudaError_t cudaGraphEventRecordNodeSetEvent (cudaGraphNode_t node, cudaEvent_t event)

---

hipError_t hipGraphEventRecordNodeSetEvent(hipGraphNode_t node, hipEvent_t event)

Sets an event record node's event.

Parameters

[in] node Instane of the node to set event to.

[in] event Pointer to the event

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.28 hipGraphEventWaitNodeGetEvent

__host__cudaError_t cudaGraphEventWaitNodeGetEvent (cudaGraphNode_t node, cudaEvent_t* event_out)

---

hipError_t hipGraphEventWaitNodeGetEvent(hipGraphNode_t node, hipEvent_t* event_out)

Returns the event associated with an event wait node.

Parameters

[in] node Instane of the node to get event from.

[in] event_out Pointer to return the event.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.29 hipGraphEventWaitNodeSetEvent

__host__cudaError_t cudaGraphEventWaitNodeSetEvent (cudaGraphNode_t node, cudaEvent_t event)

---

hipError_t hipGraphEventWaitNodeSetEvent(hipGraphNode_t node, hipEvent_t event)

Sets an event wait node's event.

Parameters

[in] node Instane of the node to set event to.

[out] event Pointer to the event.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.30 hipGraphExecChildGraphNodeSetParams*

__host__cudaError_t cudaGraphExecChildGraphNodeSetParams (cudaGraphExec_t hGraphExec, cudaGraphNode_t node, cudaGraph_t childGraph)

---

hipError_t hipGraphExecChildGraphNodeSetParams(hipGraphExec_t hGraphExec, hipGraphNode_t node, hipGraph_t childGraph)

Updates node parameters in the child graph node in the given graphExec.

Parameters

[in] hGraphExec - instance of the executable graph with the node.

[in] node - node from the graph which was used to instantiate graphExec.

[in] childGraph - child graph with updated parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.31 hipGraphExecDestroy

__host__cudaError_t cudaGraphExecDestroy (cudaGraphExec_t graphExec)

---

hipError_t hipGraphExecDestroy(hipGraphExec_t pGraphExec)

Destroys an executable graph.

Parameters

[in] pGraphExec Instance of executable graph to destry.

Returns

#hipSuccess

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.32 hipGraphExecEventRecordNodeSetEvent*

__host__cudaError_t cudaGraphExecEventRecordNodeSetEvent (cudaGraphExec_t hGraphExec, cudaGraphNode_t hNode, cudaEvent_t event)

---

hipError_t hipGraphExecEventRecordNodeSetEvent(hipGraphExec_t hGraphExec, hipGraphNode_t hNode, hipEvent_t event)

Sets the event for an event record node in the given graphExec.

Parameters

[in] hGraphExec - instance of the executable graph with the node.

[in] hNode - node from the graph which was used to instantiate graphExec.

[in] event - pointer to the event.

Returns

#hipSuccess, #hipErrorInvalidValue

20.33 hipGraphExecEventWaitNodeSetEvent*

__host__cudaError_t cudaGraphExecEventWaitNodeSetEvent (cudaGraphExec_t hGraphExec, cudaGraphNode_t hNode, cudaEvent_t event)

---

hipError_t hipGraphExecEventWaitNodeSetEvent(hipGraphExec_t hGraphExec, hipGraphNode_t hNode, hipEvent_t event)

Sets the event for an event wait node in the given graphExec.

Parameters

[in] hGraphExec - instance of the executable graph with the node.

[in] hNode - node from the graph which was used to instantiate graphExec.

[in] event - pointer to the event.

Returns

#hipSuccess, #hipErrorInvalidValue

20.34 hipGraphExecExternalSemaphoresSignalNodeSetParams*

__host__cudaError_t cudaGraphExecExternalSemaphoresSignalNodeSetParams (cudaGraphExec_t hGraphExec, cudaGraphNode_t hNode, const

cudaExternalSemaphoreSignalNodeParams* nodeParams)

---

hipError_t hipGraphExecExternalSemaphoresSignalNodeSetParams (hipGraphExec_t hGraphExec, hipGraphNode_t hNode, const hipExternalSemaphoreSignalNodeParams* nodeParams)

Updates node parameters in the external semaphore signal node in the given graphExec.

Parameters

[in] hGraphExec - The executable graph in which to set the specified node.

[in] hNode - Node from the graph from which graphExec was instantiated.

[in] nodeParams - Pointer to the params to be set.

Returns

#hipSuccess, #hipErrorInvalidValue

20.35 hipGraphExecExternalSemaphoresWaitNodeSetParams*

__host__cudaError_t cudaGraphExecExternalSemaphoresWaitNodeSetParams (cudaGraphExec_t hGraphExec, cudaGraphNode_t hNode, const cudaExternalSemaphoreWaitNodeParams* nodeParams)

---

hipError_t hipGraphExecExternalSemaphoresWaitNodeSetParams (hipGraphExec_t hGraphExec, hipGraphNode_t hNode, const hipExternalSemaphoreWaitNodeParams* nodeParams)

Updates node parameters in the external semaphore wait node in the given graphExec.

Parameters

[in] hGraphExec - The executable graph in which to set the specified node.

[in] hNode - Node from the graph from which graphExec was instantiated.

[in] nodeParams - Pointer to the params to be set.

Returns

#hipSuccess, #hipErrorInvalidValue

20.36 hipGraphExecHostNodeSetParams*

__host__cudaError_t cudaGraphExecHostNodeSetParams (cudaGraphExec_t hGraphExec, cudaGraphNode_t node, const cudaHostNodeParams* pNodeParams)

---

hipError_t hipGraphExecHostNodeSetParams(hipGraphExec_t hGraphExec, hipGraphNode_t node, const hipHostNodeParams* pNodeParams)

Sets the parameters for a host node in the given graphExec.

Parameters

[in] hGraphExec - instance of the executable graph with the node.

[in] node - instance of the node to set parameters to.

[in] pNodeParams - pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.37 hipGraphExecKernelNodeSetParams

_host__cudaError_t cudaGraphExecKernelNodeSetParams (cudaGraphExec_t hGraphExec, cudaGraphNode_t node, const cudaKernelNodeParams* pNodeParams)

---

hipError_t hipGraphExecKernelNodeSetParams(hipGraphExec_t hGraphExec, hipGraphNode_t node,const hipKernelNodeParams* pNodeParams)

Sets the parameters for a kernel node in the given graphExec.

Parameters

[in] hGraphExec Instance of the executable graph with the node.

[in] node Instance of the node to set parameters to.

[in] pNodeParams Const pointer to the kernel node parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

开发者社区

20.38 hipGraphExecMemcpyNodeSetParams*

__host__cudaError_t cudaGraphExecMemcpyNodeSetParams (cudaGraphExec_t hGraphExec, cudaGraphNode_t node, const cudaMemcpy3DParms* pNodeParams)

---

hipError_t hipGraphExecMemcpyNodeSetParams(hipGraphExec_t hGraphExec, hipGraphNode_t node, hipMemcpy3DParms* pNodeParams)

Sets the parameters for a memcpy node in the given graphExec.

Parameters

[in] hGraphExec - instance of the executable graph with the node.

[in] node - instance of the node to set parameters to.

[in] pNodeParams - const pointer to the kernel node parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.39 hipGraphExecMemcpyNodeSetParams1D*

__host__cudaError_t cudaGraphExecMemcpyNodeSetParams1D (cudaGraphExec_t hGraphExec, cudaGraphNode_t node, void* dst, const void* src, size_t count, cudaMemcpyKind kind)

---

hipError_t hipGraphExecMemcpyNodeSetParams1D(hipGraphExec_t hGraphExec, hipGraphNode_t node, void* dst, const void* src, size_t count, hipMemcpyKind kind)

Sets the parameters for a memcpy node in the given graphExec to perform a 1-dimensional copy.

Parameters

[in] hGraphExec - instance of the executable graph with the node.

[in] node - instance of the node to set parameters to.

[in] dst - pointer to memory address to the destination.

[in] src - pointer to memory address to the source.

[in] count - the size of the memory to copy.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

20.40 hipGraphExecMemcpyNodeSetParamsFromSymbol*

__host__cudaError_t cudaGraphExecMemcpyNodeSetParamsFromSymbol (cudaGraphExec_t hGraphExec, cudaGraphNode_t node, void* dst, const void* symbol, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipGraphExecMemcpyNodeSetParamsFromSymbol(hipGraphExec_t hGraphExec, hipGraphNode_t node, void* dst, const void* symbol, size_t count, size_t offset, hipMemcpyKind kind)

Sets the parameters for a memcpy node in the given graphExec to copy from a symbol on the device.

Parameters

[in] hGraphExec - instance of the executable graph with the node.

[in] node - instance of the node to set parameters to.

[in] dst - pointer to memory address to the destination.

[in] symbol - Device symbol address.

[in] count - the size of the memory to copy.

[in] offset - Offset from start of symbol in bytes.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

20.41 hipGraphExecMemcpyNodeSetParamsToSymbol*

__host__cudaError_t cudaGraphExecMemcpyNodeSetParamsToSymbol (cudaGraphExec_t hGraphExec, cudaGraphNode_t node, const void* symbol, const void* src, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipGraphExecMemcpyNodeSetParamsToSymbol(hipGraphExec_t hGraphExec, hipGraphNode_t node, const void* symbol, const void* src, size_t count, size_t offset, hipMemcpyKind kind)

Sets the parameters for a memcpy node in the given graphExec to copy to a symbol on the device.

Parameters

[in] hGraphExec - instance of the executable graph with the node.

[in] node - instance of the node to set parameters to.

[in] symbol - Device symbol address.

[in] src - pointer to memory address of the src.

[in] count - the size of the memory to copy.

[in] offset - Offset from start of symbol in bytes.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

20.42 hipGraphExecMemsetNodeSetParams*

__host__cudaError_t cudaGraphExecMemsetNodeSetParams (cudaGraphExec_t hGraphExec, cudaGraphNode_t node, const cudaMemsetParams* pNodeParams)

---

hipError_t hipGraphExecMemsetNodeSetParams(hipGraphExec_t hGraphExec, hipGraphNode_t node, const hipMemsetParams* pNodeParams)

Sets the parameters for a memset node in the given graphExec.

Parameters

[in] hGraphExec - instance of the executable graph with the node.

[in] node - instance of the node to set parameters to.

[in] pNodeParams - pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.43 hipGraphExecUpdate*

__host__cudaError_t cudaGraphExecUpdate (cudaGraphExec_t hGraphExec, cudaGraph_t hGraph, cudaGraphNode_t* hErrorNode_out, cudaGraphExecUpdateResult ** updateResult_out)

---

hipError_t hipGraphExecUpdate(hipGraphExec_t hGraphExec, hipGraph_t hGraph, hipGraphNode_t* hErrorNode_out, hipGraphExecUpdateResult* updateResult_out)

Check whether an executable graph can be updated with a graph and perform the update if possible.

Parameters

[in] hGraphExec - instance of executable graph to update.

[in] hGraph - graph that contains the updated parameters.

[in] hErrorNode_out - node which caused the permissibility check to forbid the update.

[in] updateResult_out - Whether the graph update was permitted.

Returns

#hipSuccess, #hipErrorGraphExecUpdateFailure

20.44 hipGraphExternalSemaphoresSignalNodeGetParams*

__host__cudaError_t cudaGraphExternalSemaphoresSignalNodeGetParams (cudaGraphNode_t hNode, cudaExternalSemaphoreSignalNodeParams* params_out)

---

hipError_t hipGraphExternalSemaphoresSignalNodeGetParams(hipGraphNode_t hNode, hipExternalSemaphoreSignalNodeParams* params_out)

Returns external semaphore signal node params.

Parameters

[in] hNode - Node from the graph from which graphExec was instantiated.

[out] params_out - Pointer to params.

Returns

#hipSuccess, #hipErrorInvalidValue

20.45 hipGraphExternalSemaphoresSignalNodeSetParams*

__host__cudaError_t cudaGraphExternalSemaphoresSignalNodeSetParams (cudaGraphNode_t hNode, const cudaExternalSemaphoreSignalNodeParams* nodeParams)

---

hipError_t hipGraphExternalSemaphoresSignalNodeSetParams(hipGraphNode_t hNode, const hipExternalSemaphoreSignalNodeParams* nodeParams)

Updates node parameters in the external semaphore signal node.

Parameters

[in] hNode - Node from the graph from which graphExec was instantiated.

[in] nodeParams - Pointer to the params to be set.

Returns

#hipSuccess, #hipErrorInvalidValue

20.46 hipGraphExternalSemaphoresWaitNodeGetParams*

__host__cudaError_t cudaGraphExternalSemaphoresWaitNodeGetParams (cudaGraphNode_t hNode, cudaExternalSemaphoreWaitNodeParams* params_out)

---

hipError_t hipGraphExternalSemaphoresWaitNodeGetParams(hipGraphNode_t hNode, hipExternalSemaphoreWaitNodeParams* params_out)

Returns external semaphore wait node params.

Parameters

[in] hNode - Node from the graph from which graphExec was instantiated.

[out] params_out - Pointer to params.

Returns

#hipSuccess, #hipErrorInvalidValue

20.47 hipGraphExternalSemaphoresWaitNodeSetParams*

__host__cudaError_t cudaGraphExternalSemaphoresWaitNodeSetParams (cudaGraphNode_t hNode, const cudaExternalSemaphoreWaitNodeParams* nodeParams)

---

hipError_t hipGraphExternalSemaphoresWaitNodeSetParams(hipGraphNode_t hNode, const hipExternalSemaphoreWaitNodeParams* nodeParams)

Updates node parameters in the external semaphore wait node.

Parameters

[in] hNode - Node from the graph from which graphExec was instantiated.

[in] nodeParams - Pointer to the params to be set.

Returns

#hipSuccess, #hipErrorInvalidValue

20.48 hipGraphGetEdges

__host__cudaError_t cudaGraphGetEdges (cudaGraph_t graph, cudaGraphNode_t *from, cudaGraphNode_t *to, size_t *numEdges)

---

hipError_t hipGraphGetEdges(hipGraph_t graph, hipGraphNode_t* from, hipGraphNode_t* to,size_t* numEdges)

Returns a graph's dependency edges.

Parameters

[in] graph	Instance of the graph to get the edges from.
[out] from	Pointer to the graph nodes to return edge endpoints.
[out] to	Pointer to the graph nodes to return edge endpoints.
[out] from	Returns number of edges.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.49 hipGraphGetNodes

__host__cudaError_t cudaGraphGetNodes (cudaGraph_t graph, cudaGraphNode_t *nodes, size_t *numNodes)

---

hipError_t hipGraphGetNodes(hipGraph_t graph, hipGraphNode_t* nodes, size_t* numNodes)

Returns graph nodes.

Parameters

[in] graph	Instance of the graph to get the nodes.
[out] nodes	Pointer to the graph nodes.

[out] numNodes The number of graph nodes.

Returns

#hipSuccess, #hipErrorInvalidValue

20.50 hipGraphGetRootNodes

__host__cudaError_t cudaGraphGetRootNodes (cudaGraph_t graph, cudaGraphNode_t *pRootNodes, size_t *pNumRootNodes)

---

hipError_t hipGraphGetRootNodes(hipGraph_t graph, hipGraphNode_t* pRootNodes,size_t* pNumRootNodes)

Returns graph's root nodes.

Parameters

[in] graph Instance of the graph to get the nodes.

[out] pRootNodes Pointer to the parameters.

[out] pNumRootNodes The number of graph's root nodes.

Returns

#hipSuccess, #hipErrorInvalidValue

20.51 hipGraphHostNodeGetParams

__host__cudaError_t cudaGraphHostNodeGetParams (cudaGraphNode_t node, cudaHostNodeParams *pNodeParams)

---

hipError_t hipGraphHostNodeGetParams(hipGraphNode_t node, hipHostNodeParams* pNodeParams)

Returns a host node's parameters.

Parameters

[in] node Instane of the node to get parameters from.

[out] pNodeParams Pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open

to changes and may have outstanding issues.

20.52 hipGraphHostNodeSetParams

__host__cudaError_t cudaGraphHostNodeSetParams (cudaGraphNode_t node, const cudaHostNodeParams *pNodeParams)

---

hipError_t hipGraphHostNodeSetParams(hipGraphNode_t node, const hipHostNodeParams* pNodeParams)

Sets a host node's parameters.

Parameters

[in] node Instance of the node to set parameters to.

[in] pNodeParams Pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

Warning

This API is marked as beta, meaning, while this is feature complete, it is still open to changes and may have outstanding issues.

20.53 hipGraphInstantiate

__host__cudaError_t cudaGraphInstantiate (cudaGraphExec_t *pGraphExec, cudaGraph_t graph, cudaGraphNode_t *pErrorNode, char *pLogBuffer, size_t bufferSize)

---

hipError_t hipGraphInstantiate(hipGraphExec_t* pGraphExec, hipGraph_t graph,hipGraphNode_t* pErrorNode, char* pLogBuffer, size_t bufferSize)

Creates an executable graph from a graph.

Parameters

[out] pGraphExec Pointer to instantiated executable graph to create.

[in] graph Instance of graph to instantiate.

[out] pErrorNode Pointer to error node in case error occured in graph instantiation, it could modify the correponding node.

[out] pLogBuffer Pointer to log buffer.

[in] bufferSize The size of log buffer.

Returns

#hipSuccess, #hipErrorOutOfMemory

20.54 hipGraphInstantiateWithFlags*

__host__cudaError_t cudaGraphInstantiateWithFlags (cudaGraphExec_t* pGraphExec, cudaGraph_t graph, unsigned long long flags)

No hip

20.55 hipGraphKernelNodeCopyAttributes*

__host__cudaError_t cudaGraphKernelNodeCopyAttributes (cudaGraphNode_t hSrc, cudaGraphNode_t hDst)

---

hipError_t hipGraphKernelNodeCopyAttributes(hipGraphNode_t hSrc, hipGraphNode_t hDst)

Copies attributes from source node to destination node. Both node must have the same context.

Parameters

[out] hDst - Destination node.

[in] hSrc - Source node.

Returns

#hipSuccess, #hipErrorInvalidContext

20.56 hipGraphKernelNodeGetAttribute*

__host__cudaError_t cudaGraphKernelNodeGetAttribute (cudaGraphNode_t hNode, cudaKernelNodeAttrID attr, cudaKernelNodeAttrValue* value_out)

---

hipError_t hipGraphKernelNodeGetAttribute(hipGraphNode_t hNode, hipKernelNodeAttrID attr, hipKernelNodeAttrValue* value)

Gets a node attribute

Parameters

[in] hNode - instance of the node to set parameters to.

[in] attr - the attribute node is set to.

[in] value - const pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.57 hipGraphKernelNodeGetParams

__host__cudaError_t cudaGraphKernelNodeGetParams (cudaGraphNode_t node, cudaKernelNodeParams *pNodeParams)

---

hipError_t hipGraphKernelNodeGetParams(hipGraphNode_t node, hipKernelNodeParams* pNodeParams)

Gets kernel node's parameters.

Parameters

[in] node Instance of the node to get parameters from.

[out] pNodeParams Pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.58 hipGraphKernelNodeSetAttribute*

__host__cudaError_t cudaGraphKernelNodeSetAttribute (cudaGraphNode_t hNode, cudaKernelNodeAttrID attr, const cudaKernelNodeAttrValue* value)

---

hipError_t hipGraphKernelNodeSetAttribute(hipGraphNode_t hNode, hipKernelNodeAttrID attr, const hipKernelNodeAttrValue* value)

Sets a node attribute.

Parameters

[in] hNode - instance of the node to set parameters to.

[in] attr - the attribute node is set to.

[in] value - const pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

开发者社区

20.59 hipGraphKernelNodeSetParams

__host__cudaError_t cudaGraphKernelNodeSetParams (cudaGraphNode_t node, const cudaKernelNodeParams *pNodeParams)

---

hipError_t hipGraphKernelNodeSetParams(hipGraphNode_t node, const hipKernelNodeParams* pNodeParams)

Sets a kernel node's parameters.

Parameters

[in] node Instance of the node to set parameters to.

[in] pNodeParams Const pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.60 hipGraphLaunch

__host__cudaError_t cudaGraphLaunch (cudaGraphExec_t graphExec, cudaStream_t stream)

---

hipError_t hipGraphLaunch(hipGraphExec_t graphExec, hipStream_t stream)

launches an executable graph in a stream.

Parameters

[in] graphExec Instance of executable graph to launch.

[in] stream Instance of stream in which to launch executable graph.

Returns

#hipSuccess, #hipErrorOutOfMemory, #hipErrorInvalidHandle, #hipErrorInvalidValue

20.61 hipGraphMemAllocNodeGetParams*

__host__cudaError_t cudaGraphMemAllocNodeGetParams (cudaGraphNode_t node, cudaMemAllocNodeParams* params_out)

---

hipError_t hipGraphMemAllocNodeGetParams(hipGraphNode_t node, hipMemAllocNodeParams* pNodeParams)

Returns parameters for memory allocation node.

Parameters

[in] node - Memory allocation node for a query

[out] pNodeParams - Parameters for the specified memory allocation node

Returns

#hipSuccess, #hipErrorInvalidValue

20.62 hipGraphMemFreeNodeGetParams*

__host__cudaError_t cudaGraphMemFreeNodeGetParams (cudaGraphNode_t node, void* dptr_out)

---

hipError_t hipGraphMemFreeNodeGetParams(hipGraphNode_t node, void* dev_ptr)

Returns parameters for memory free node.

Parameters

[in] node - Memory free node for a query

[out] dev_ptr - Device pointer for the specified memory free node

Returns

#hipSuccess, #hipErrorInvalidValue

20.63 hipGraphMemcpyNodeGetParams

__host__cudaError_t cudaGraphMemcpyNodeGetParams (cudaGraphNode_t node, cudaMemcpy3DParms *pNodeParams)

---

hipError_t hipGraphMemcpyNodeGetParams(hipGraphNode_t node, hipMemcpy3DParms* pNodeParams)

Gets a memcpy node's parameters.

Parameters

[in] node Instance of the node to get parameters from.

[in] pNodeParams Pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.64 hipGraphMemcpyNodeSetParams

__host__cudaError_t cudaGraphMemcpyNodeSetParams (cudaGraphNode_t node, const cudaMemcpy3DParms *pNodeParams)

---

hipError_t hipGraphMemcpyNodeSetParams(hipGraphNode_t node, const hipMemcpy3DParms* pNodeParams)

Sets a memcpy node's parameters.

Parameters

[in] node Instance of the node to set parameters to.

[out] pNodeParams Const pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.65 hipGraphMemcpyNodeSetParams1D*

__host__cudaError_t cudaGraphMemcpyNodeSetParams1D (cudaGraphNode_t node, void* dst, const void* src, size_t count, cudaMemcpyKind kind)

---

hipError_t hipGraphMemcpyNodeSetParams1D(hipGraphNode_t node, void* dst, const void* src, size_t count, hipMemcpyKind kind)

Sets a memcpy node's parameters to perform a 1-dimensional copy.

Parameters

[in] node - instance of the node to set parameters to.

[in] dst - pointer to memory address to the destination.

[in] src - pointer to memory address to the source.

[in] count - the size of the memory to copy.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

20.66 hipGraphMemcpyNodeSetParamsFromSymbol*

__host__cudaError_t cudaGraphMemcpyNodeSetParamsFromSymbol (cudaGraphNode_t node, void* dst, const void* symbol, size_t count, size_t offset,

cudaMemcpyKind kind)

hipError_t hipGraphMemcpyNodeSetParamsFromSymbol(hipGraphNode_t node, void* dst, const void* symbol, size_t count, size_t offset, hipMemcpyKind kind)

Sets a memcpy node's parameters to copy from a symbol on the device.

Parameters

[in] node - instance of the node to set parameters to.

[in] dst - pointer to memory address to the destination.

[in] symbol - Device symbol address.

[in] count - the size of the memory to copy.

[in] offset - Offset from start of symbol in bytes.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

20.67 hipGraphMemcpyNodeSetParamsToSymbol*

__host__cudaError_t cudaGraphMemcpyNodeSetParamsToSymbol (cudaGraphNode_t node, const void* symbol, const void* src, size_t count, size_t offset, cudaMemcpyKind kind)

hipError_t hipGraphMemcpyNodeSetParamsToSymbol(hipGraphNode_t node, const void* symbol, const void* src, size_t count, size_t offset, hipMemcpyKind kind)

Sets a memcpy node's parameters to copy to a symbol on the device.

Parameters

[in] node - instance of the node to set parameters to.

[in] symbol - Device symbol address.

[in] src - pointer to memory address of the src.

[in] count - the size of the memory to copy.

[in] offset - Offset from start of symbol in bytes.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

20.68 hipGraphMemsetNodeGetParams

__host__cudaError_t cudaGraphMemsetNodeGetParams (cudaGraphNode_t node, cudaMemsetParams *pNodeParams)

---

hipError_t hipGraphMemsetNodeGetParams(hipGraphNode_t node, hipMemsetParams* pNodeParams)

Gets a memset node's parameters.

Parameters

[in] node Instane of the node to get parameters from.

[out] pNodeParams Pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.69 hipGraphMemsetNodeSetParams

__host__cudaError_t cudaGraphMemsetNodeSetParams (cudaGraphNode_t node, const cudaMemsetParams *pNodeParams)

---

hipError_t hipGraphMemsetNodeSetParams(hipGraphNode_t node, const hipMemsetParams* pNodeParams)

Sets a memset node's parameters.

Parameters

[in] node Instance of the node to set parameters to.

[out] node Pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue

20.70 hipGraphNodeFindInClone*

__host__cudaError_t cudaGraphNodeFindInClone (cudaGraphNode_t *pNode, cudaGraphNode_t originalNode, cudaGraph_t clonedGraph)

---

hipError_t hipGraphNodeFindInClone(hipGraphNode_t* pNode, hipGraphNode_t originalNode, hipGraph_t clonedGraph)

Finds a cloned version of a node.

Parameters

[out] pNode - Returns the cloned node.

[in] originalNode - original node handle.

[in] clonedGraph - Cloned graph to query.

Returns

#hipSuccess, #hipErrorInvalidValue

20.71 hipGraphNodeGetDependencies*

__host__cudaError_t cudaGraphNodeGetDependencies (cudaGraphNode_t node, cudaGraphNode_t *pDependencies, size_t *pNumDependencies)

---

hipError_t hipGraphNodeGetDependencies(hipGraphNode_t node, hipGraphNode_t* pDependencies, size_t* pNumDependencies)

Returns a node's dependencies.

Parameters

[in] node - graph node to get the dependencies from.

[out] pDependencies - pointer to to return the dependencies.

[out] pNumDependencies - returns the number of graph node dependencies.

Returns

#hipSuccess, #hipErrorInvalidValue

20.72 hipGraphNodeGetDependentNodes*

__host__cudaError_t cudaGraphNodeGetDependentNodes (cudaGraphNode_t node, cudaGraphNode_t *pDependentNodes, size_t *pNumDependentNodes)

---

hipError_t hipGraphNodeGetDependentNodes(hipGraphNode_t node, hipGraphNode_t* pDependentNodes, size_t* pNumDependentNodes)

Returns a node's dependent nodes.

Parameters

[in] node - graph node to get the Dependent nodes from.

[out] pDependentNodes - pointer to return the graph dependent nodes.

[out] pNumDependentNodes - returns the number of graph node dependent nodes.

Returns

#hipSuccess, #hipErrorInvalidValue

20.73 hipGraphNodeGetEnabled*

__host__cudaError_t cudaGraphNodeGetEnabled (cudaGraphExec_t hGraphExec, cudaGraphNode_t hNode, unsigned int* isEnabled)

---

hipError_t hipGraphNodeGetEnabled(hipGraphExec_t hGraphExec, hipGraphNode_t hNode, unsigned int* isEnabled)

Query whether a node in the given graphExec is enabled

Parameters

[in] hGraphExec - The executable graph in which to set the specified node.

[in] hNode - Node from the graph from which graphExec was instantiated.

[out] isEnabled - Location to return the enabled status of the node.

Returns

#hipSuccess, #hipErrorInvalidValue

20.74 hipGraphNodeGetType*

__host__cudaError_t cudaGraphNodeGetType (cudaGraphNode_t node, cudaGraphNodeType *pType)

---

hipError_t hipGraphNodeGetType(hipGraphNode_t node, hipGraphNodeType* pType)

Returns a node's type.

Parameters

[in] node - instance of the graph to add dependencies.

[out] pType - pointer to the return the type

Returns

#hipSuccess, #hipErrorInvalidValue

20.75 hipGraphNodeSetEnabled*

__host__cudaError_t cudaGraphNodeSetEnabled (cudaGraphExec_t hGraphExec, cudaGraphNode_t hNode, unsigned int isEnabled)

---

hipError_t hipGraphNodeSetEnabled(hipGraphExec_t hGraphExec, hipGraphNode_t hNode, unsigned int isEnabled)

Enables or disables the specified node in the given graphExec

Parameters

[in] hGraphExec - The executable graph in which to set the specified node.

[in] hNode - Node from the graph from which graphExec was instantiated.

[in] isEnabled - Node is enabled if != 0, otherwise the node is disabled.

Returns

#hipSuccess, #hipErrorInvalidValue

20.76 hipGraphReleaseUserObject*

__host__cudaError_t cudaGraphReleaseUserObject (cudaGraph_t graph, cudaUserObject_t object, unsigned int count = 1)

---

hipError_t hipGraphReleaseUserObject(hipGraph_t graph, hipUserObject_t object, unsigned int count __dparm(1))

Release user object from graphs.

Parameters

[in] graph - pointer to graph to retain the user object for.

[in] object - pointer to instace of userobj.

[in] count - reference to resource to be retained.

Returns

#hipSuccess, #hipErrorInvalidValue

20.77 hipGraphRemoveDependencies*

__host__cudaError_t cudaGraphRemoveDependencies (cudaGraph_t graph, cudaGraphNode_t *from, cudaGraphNode_t *to, size_t numDependencies)

---

hipError_t hipGraphRemoveDependencies(hipGraph_t graph, const hipGraphNode_t* from, const hipGraphNode_t* to, size_t numDependencies)

Removes dependency edges from a graph.

Parameters

[in] graph - instance of the graph to remove dependencies.

[in] from - Array of nodes that provide the dependencies.

[in] to - Array of dependent nodes.

[in] numDependencies - the number of dependencies to remove.

Returns

#hipSuccess, #hipErrorInvalidValue

20.78 hipGraphRetainUserObject*

__host__cudaError_t cudaGraphRetainUserObject (cudaGraph_t graph, cudaUserObject_t object, unsigned int count = 1, unsigned int flags = 0)

---

hipError_t hipGraphRetainUserObject(hipGraph_t graph, hipUserObject_t object, unsigned int count __dparm(1), unsigned int flags __dparm(0))

Retain user object for graphs

Parameters

[in] graph - pointer to graph to retain the user object for.

[in] object - pointer to instace of userobj.

[in] count - reference to resource to be retained.

[in] flags - flags passed to API.

Returns

#hipSuccess, #hipErrorInvalidValue

20.79 hipGraphUpload*

__host__cudaError_t cudaGraphUpload (cudaGraphExec_t graphExec, cudaStream_t stream)

---

hipError_t hipGraphUpload(hipGraphExec_t graphExec, hipStream_t stream)

uploads an executable graph in a stream

Parameters

[in] graphExec - instance of executable graph to launch.

[in] stream - instance of stream in which to launch executable graph.

Returns

#hipSuccess, #hipErrorInvalidValue

20.80 hipUserObjectCreate*

__host__cudaError_t cudaUserObjectCreate (cudaUserObject_t* object_out, void* ptr, cudaHostFn_t destroy, unsigned int initialRefcount, unsigned int flags)

---

hipError_t hipUserObjectCreate(hipUserObject_t* object_out, void* ptr, hipHostFn_t destroy, unsigned int initialRefcount, unsigned int flags)

Create an instance of userObject to manage lifetime of a resource.

Parameters

[out] object_out - pointer to instace of userobj.

[in] ptr - pointer to pass to destroy function.

[in] destroy - destroy callback to remove resource.

[in] initialRefcount - reference to resource.

[in] flags - flags passed to API.

Returns

#hipSuccess, #hipErrorInvalidValue

20.81 hipUserObjectRelease*

__host__cudaError_t cudaUserObjectRelease (cudaUserObject_t object, unsigned int count = 1)

---

hipError_t hipUserObjectRelease(hipUserObject_t object, unsigned int count __dparm(1))

Release number of references to resource.

Parameters

[in] object - pointer to instace of userobj.

[in] count - reference to resource to be retained.

Returns

#hipSuccess, #hipErrorInvalidValue

20.82 hipUserObjectRetain*

__host__cudaError_t cudaUserObjectRetain (cudaUserObject_t object, unsigned

int count = 1)

---

hipError_t hipUserObjectRetain(hipUserObject_t object, unsigned int count __dparm(1))

Retain number of references to resource.

Parameters

[in] object - pointer to instace of userobj.

[in] count - reference to resource to be retained.

Returns

#hipSuccess, #hipErrorInvalidValue

20.83 hipGraphAddMemAllocNode

__host__cudaError_t cudaGraphAddMemAllocNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, cudaMemAllocNodeParams* nodeParams)

---

hipError_t hipGraphAddMemAllocNode(hipGraphNode_t* pGraphNode, hipGraph_t graph,const hipGraphNode_t* pDependencies, size_t numDependencies, hipMemAllocNodeParams* pNodeParams);

Creates a memory allocation node and adds it to a graph

Parameters

[out] pGraphNode Pointer to the graph node to create and add to the graph

[in] graph Instane of the graph the node to be added

[in] pDependencies Const pointer to the node dependenties

[in] numDependencies The number of dependencies

[in] pNodeParams Node parameters for memory allocation

Returns

#hipSuccess, #hipErrorInvalidValue

20.84 hipGraphAddMemFreeNode

__host__cudaError_t cudaGraphAddMemFreeNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t

numDependencies, void* dptr)

---

hipError_t hipGraphAddMemFreeNode(hipGraphNode_t* pGraphNode, hipGraph_t graph, const hipGraphNode_t* pDependencies, size_t numDependencies, void* dev_ptr);

Creates a memory free node and adds it to a graph.

Parameters

[out] pGraphNode	Pointer to the graph node to create and add to the graph
[in] graph	Instane of the graph the node to be added
[in] pDependencies	Const pointer to the node dependenties
[in] numDependencies	The number of dependencies
[in] dev_ptr	Pointer to the memory to be freed

Returns

#hipSuccess, #hipErrorInvalidValue

20.85 hipGraphDebugDotPrint

__host__cudaError_t cudaGraphDebugDotPrint (cudaGraph_t graph, const char* path, unsigned int flags)

---

hipError_t hipGraphDebugDotPrint(hipGraph_t graph, const char* path, unsigned int flags);

Write a DOT file describing graph structure.

Parameters

[in] graph	graph object for which DOT file has to be generated
[in] path	path to write the DOT file.
[in] flags	Flags from hipGraphDebugDotFlags to get additional node information.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorOperatingSystem

20.86 hipGraphKernelNodeCopyAttributes

__host__cudaError_t cudaGraphKernelNodeCopyAttributes (cudaGraphNode_t

hSrc, cudaGraphNode_t hDst)

hipError_t hipGraphKernelNodeCopyAttributes(hipGraphNode_t hSrc, hipGraphNode_t hDst);

Copies attributes from source node to destination node.

Parameters

[out] hDst Destination node.

[in] hSrc Source node.

Returns

#hipSuccess, #hipErrorInvalidContext

20.87 hipGraphMemAllocNodeGetParams

__host__cudaError_t cudaGraphMemAllocNodeGetParams (cudaGraphNode_t node, cudaMemAllocNodeParams* params_out)

hipError_t hipGraphMemAllocNodeGetParams(hipGraphNode_t node, hipMemAllocNodeParams* pNodeParams);

Returns parameters for memory allocation node

Parameters

[in] node Memory allocation node for a query

[out] pNodeParams Parameters for the specified memory allocation node

Returns

#hipSuccess, #hipErrorInvalidValue

20.88 hipGraphMemFreeNodeGetParams

__host__cudaError_t cudaGraphMemFreeNodeGetParams (cudaGraphNode_t node, void *dptr_out)

hipError_t hipGraphMemFreeNodeGetParams(hipGraphNode_t node, void* dev_ptr);

Returns parameters for memory free node

Parameters

[in] node Memory free node for a query

[out] dev_ptr Device pointer for the specified memory free node

Returns

#hipSuccess, #hipErrorInvalidValue

20.89 hipGraphNodeGetEnabled

__host__cudaError_ cudaGraphNodeGetEnabled (cudaGraphExec_t hGraphExec, cudaGraphNode_t hNode, unsigned int *isEnabled)

---

hipError_t hipGraphNodeGetEnabled(hipGraphExec_t hGraphExec, hipGraphNode_t hNode,unsigned int* isEnabled);

Query whether a node in the given graphExec is enabled

Parameters

[in] hGraphExec The executable graph in which to set the specified node

[in] hNode Node from the graph from which graphExec was instantiated

[in] isEnabled Location to return the enabled status of the node.

Returns

#hipSuccess, #hipErrorInvalidValue

20.90 hipGraphNodeSetEnabled

__host__cudaError_ cudaGraphNodeSetEnabled (cudaGraphExec_t hGraphExec, cudaGraphNode_t hNode, unsigned int isEnabled)

---

hipError_t hipGraphNodeSetEnabled(hipGraphExec_t hGraphExec, hipGraphNode_t hNode, unsigned int isEnabled);

Enables or disables the specified node in the given graphExec.

Parameters

[in] hGraphExec The executable graph in which to set the specified node

[in] hNode Node from the graph from which graphExec was instantiated

[in] isEnabled Node is enabled if != 0, otherwise the node is disabled

Returns

#hipSuccess, #hipErrorInvalidValue

20.91 hipGraphAddNode【dtk-25.04.1 新增】

__host__cudaError_t cudaGraphAddNode (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, cudaGraphNodeParams* nodeParams)

---

hipError_t hipGraphAddNode(hipGraphNode_t *pGraphNode, hipGraph_t graph, const hipGraphNode_t *pDependencies, size_t numDependencies, hipGraphNodeParams *nodeParams)

Creates a kernel execution node and adds it to a graph.

Parameters

[out] pGraphNode - pointer to graph node to create.

[in] graph - instance of graph to add the created node.

[in] pDependencies - pointer to the dependencies on the kernel execution node.

[in] numDependencies - the number of the dependencies.

[in] nodeParams - pointer to the parameters for the node.

Returns

#hipSuccess, #hipErrorInvalidValue.

```
hipGraph_t graph;
HIP_CHECK(hipGraphCreate(&graph, 0));
hipGraphNode_t Node;
int *d_a, *d_b, *d_c;
HIP_CHECK(hipMalloc(&d_a, sizeof(int)));
HIP_CHECK(hipMalloc(&d_b, sizeof(int)));
HIP_CHECK(hipMalloc(&d_c, sizeof(int)));
void *args[] = { &d_a, &d_b, &d_c };
hipGraphNodeParams Params = {};
Params.kernel.func = reinterpret_cast<void *>(HipTest::vectorADD<int>);
Params.kernel.gridDim.x = Params.kernel.gridDim.y = Params.kernel.gridDim.z
= 1;
Params.kernel.blockDim.x = Params.kernel.blockDim.y =
```

```
  Params.kernel.blockDim.z = 1;
  Params.kernel.kernelParams = args;
  hipKernelNodeParams kernelParams = {};
  kernelParams.func = reinterpret_cast<void *>(HipTest::vectorADD<int>);
  kernelParams.gridDim = dim3(1, 1, 1);
  kernelParams.blockDim = dim3(1, 1, 1);
  kernelParams.kernelParams = args;
  HIP_CHECK(hipGraphAddKernelNode(&Node, graph, nullptr, 0, &kernelParams));
  REQUIRE(hipSuccess == hipGraphAddNode(&Node, graph, NULL, 0, &Params));
```

20.92 hipGraphExecGetFlags【dtk-25.04.1 新增】

__host__cudaError_t cudaGraphExecGetFlags (cudaGraphExec_t graphExec, unsigned long long* flags)

---

hipError_t hipGraphExecGetFlags(hipGraphExec_t graphExec, unsigned long long* flags)

Return the flags on executable graph.

Parameters

[in] graphExec - Executable graph to get the flags.

[out] flags - Flags used to instantiate this executable graph.

Returns

#hipSuccess, #hipErrorInvalidValue.

```
  hipGraph_t graph;
  hipGraphExec_t graphExec;
  unsigned long long flags;
  HIP_CHECK(hipGraphCreate(&graph, 0));
  HIP_CHECK(hipGraphInstantiateWithFlags(&graphExec, graph, hipGraphInstantiateFlagAutoFreeOnLaunch));
  REQUIRE(graphExec != nullptr);
  REQUIRE(hipSuccess == hipGraphExecGetFlags(graphExec, &flags));
  REQUIRE(flags == hipGraphInstantiateFlagAutoFreeOnLaunch);
```

20.93 hipGraphExecNodeSetParams【dtk-25.04.1 新增】

__host__cudaError_t cudaGraphExecNodeSetParams (cudaGraphExec_t graphExec, cudaGraphNode_t node, cudaGraphNodeParams* nodeParams)

---

hipError_t hipGraphExecNodeSetParams(hipGraphExec_t graphExec, hipGraphNode_t node, hipGraphNodeParams* nodeParams)

Updates parameters of a created node on executable graph.

Parameters

[in] graphExec - instance of executable graph.

[in] node - instance of the node to set parameters to.

[in] nodeParams - pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDeviceFunction, #hipErrorNotSupported.

```
hipGraph_t graph;
HIP_CHECK(hipGraphCreate(&graph, 0));
hipGraphNode_t Node;
hipGraphExec_t graphExec;
int *d_a, *d_b, *d_c;
HIP_CHECK(hipMalloc(&d_a, sizeof(int)));
HIP_CHECK(hipMalloc(&d_b, sizeof(int)));
HIP_CHECK(hipMalloc(&d_c, sizeof(int)));
void *args[] = { &d_a, &d_b, &d_c };
hipGraphNodeParams Params = {};
Params.kernel.func = reinterpret_cast<void *>(HipTest::vectorADD<int>);
Params.kernel.gridDim.x = Params.kernel.gridDim.y = Params.kernel.gridDim.z = 1;
Params.kernel.blockDim.x = Params.kernel.blockDim.y = Params.kernel.blockDim.z = 1;
Params.kernel.kernelParams = args;
```

开发者社区

```
hipKernelNodeParams kernelParams = {};
kernelParams.func = reinterpret_cast<void *>(HipTest::vectorADD<int>);
kernelParams.gridDim = dim3(1, 1, 1);
kernelParams.blockDim = dim3(1, 1, 1);
kernelParams.kernelParams = args;
HIP_CHECK(hipGraphAddKernelNode(&Node, graph, nullptr, 0, &kernelParams));
HIP_CHECK(hipGraphInstantiate(&graphExec, graph, nullptr, nullptr, 0));
REQUIRE(hipSuccess == hipGraphExecNodeSetParams(graphExec, Node, &Params));
HIP_CHECK(hipGraphExecDestroy(graphExec));
```

20.94 hipGraphInstantiateWithParams【dtk-25.04.1 新增】

__host__cudaError_t cudaGraphInstantiateWithParams (cudaGraphExec_t* pGraphExec, cudaGraph_t graph, cudaGraphInstantiateParams* instantiateParams)

---

hipError_t hipGraphInstantiateWithParams(hipGraphExec_t* pGraphExec, hipGraph_t graph, hipGraphInstantiateParams *instantiateParams)

Creates an executable graph from a graph.

Parameters

[out] pGraphExec - pointer to instantiated executable graph that is created.

[in] graph - instance of graph to instantiate.

[in] instantiateParams - Graph Instantiate Params

Returns

#hipSuccess, #hipErrorInvalidValue

20.95 hipGraphNodeSetParams【dtk-25.04.1 新增】

__host__cudaError_t cudaGraphNodeSetParams (cudaGraphNode_t node, cudaGraphNodeParams* nodeParams)

---

hipError_t hipGraphNodeSetParams(hipGraphNode_t node, hipGraphNodeParams *nodeParams)

Updates parameters of a created node.

Parameters

[in] node - instance of the node to set parameters to.

[in] nodeParams - pointer to the parameters.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDeviceFunction, #hipErrorNotSupported.

```
hipGraph_t graph;
HIP_CHECK(hipGraphCreate(&graph, 0));
hipGraphNode_t Node;
int *d_a, *d_b, *d_c;
HIP_CHECK(hipMalloc(&d_a, sizeof(int)));
HIP_CHECK(hipMalloc(&d_b, sizeof(int)));
HIP_CHECK(hipMalloc(&d_c, sizeof(int)));
void *args[] = { &d_a, &d_b, &d_c };
hipGraphNodeParams Params = {};
Params.kernel.func = reinterpret_cast<void *>(HipTest::vectorADD<int>);
Params.kernel.gridDim.x = Params.kernel.gridDim.y = Params.kernel.gridDim.z = 1;
Params.kernel.blockDim.x = Params.kernel.blockDim.y = Params.kernel.blockDim.z = 1;
Params.kernel.kernelParams = args;

hipKernelNodeParams kernelParams = {};
kernelParams.func = reinterpret_cast<void *>(HipTest::vectorADD<int>);
kernelParams.gridDim = dim3(1, 1, 1);
kernelParams.blockDim = dim3(1, 1, 1);
kernelParams.kernelParams = args;
HIP_CHECK(hipGraphAddKernelNode(&Node, graph, nullptr, 0, &kernelParams));
HIP_CHECK(hipGraphNodeSetParams(Node, &Params));
```

开发者社区

20.96 hipGraphConditionalHandleCreate【dtk-25.04.1 新增】

__host__cudaError_t cudaGraphConditionalHandleCreate (cudaGraphConditionalHandle* pHandle_out, cudaGraph_t graph, unsigned int defaultLaunchValue = 0, unsigned int flags = 0)

---

hipError_t hipGraphConditionalHandleCreate(hipGraphConditionalHandle *pHandle_out, hipGraph_t graph, unsigned int defaultLaunchValue __dparm(0), unsigned int flags __dparm(0))

Create a conditional handle. Creates a conditional handle associated with \p hGraph. The conditional handle must be associated with a conditional node in this graph or one of its children. Handles not associated with a conditional node may cause graph instantiation to fail.

Parameters

[out] pHandle_out - Pointer used to return the handle to the caller.

[in] hGraph - Graph which will contain the conditional node using this handle.

[in] defaultLaunchValue - Optional initial value for the conditional variable.

[in] flags - Currently must be hipGraphCondAssignDefault or 0.

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorNotSupported

21 Driver Entry Point Access

21.1 hipGetProcAddress

__host__cudaError_t cudaGetDriverEntryPoint (const char* symbol, void** funcPtr, unsigned long long flags, cudaDriverEntryPointQueryResult* driverStatus = NULL)

---

hipError_t hipGetProcAddress(const char* symbol, void** pfn, int hipVersion, uint64_t flags, hipDriverProcAddressQueryResult* symbolStatus)

Gets the pointer of requested HIP driver function.

Parameters

[in] symbol The Symbol name of the driver function to request.

[out] pfn Output pointer to the requested driver function.

[in] hipVersion The HIP version for the requested driver function symbol.

[in] flags Currently only default flag is suppported.

[out] symbolStatus Optional enumeration for returned status of searching for symbol driver function based on the input hipVersion.

Returns

#hipSuccess, #hipErrorInvalidValue.

22 C++ API Routines

22.1 hipBindSurfaceToArray*

__host__cudaError_t cudaBindSurfaceToArray (const surfaceReference *surfref, cudaArray_const_t array, const cudaChannelFormatDesc *desc)

---

hipError_t hipBindSurfaceToArray(const surfaceReference *surf, hipArray_const_t array, const hipChannelFormatDesc *desc)

Binds a memory area to a surface.

Parameters

[in] surf Pointer of surface reference.

[in] array Array to bind.

[in] desc Pointer of channel format descriptor.

Returns

#hipSuccess, #hipErrorInvalidValue

22.2 hipBindTexture

__host__cudaError_t cudaBindTexture (size_t *offset, const textureReference *texref, const void *devPtr, const cudaChannelFormatDesc *desc, size_t size)

---

hipError_t hipBindTexture(size_t* offset, textureReference* tex, const void* devPtr,const hipChannelFormatDesc* desc, size_t size = UINT_MAX);

```
texture<float, 1, hipReadModeElementType> tex;
float *texBuf;
```

```
size_t offset = 0;
hipChannelFormatDesc chanDesc =
hipCreateChannelDesc(32, 0, 0, 0, hipChannelFormatKindFloat);
HIPCHECK(hipMalloc(&texBuf, N * sizeof(float)));
HIPCHECK(hipMalloc(&devBuf, N * sizeof(float)));
HIPCHECK(hipMemcpy(texBuf, val, N * sizeof(float), hipMemcpyHostToDevice));
HIPCHECK(hipBindTexture(&offset, tex, (void *)texBuf, chanDesc, N * sizeof(float)));
```

22.3 hipBindTexture2D

__host__cudaError_t cudaBindTexture2D (size_t* offset, const textureReference* texref, const void* devPtr, const cudaChannelFormatDesc* desc, size_t width, size_t height, size_t pitch)

---

hipError_t hipBindTexture2D(size_t* offset,const textureReference* tex,const void* devPtr,const hipChannelFormatDesc* desc,size_t width,size_t height,size_t pitch)

22.4 hipBindTextureToArray

__host__cudaError_t cudaBindTextureToArray (const textureReference *texref, cudaArray_const_t array, const cudaChannelFormatDesc *desc)

---

hipError_t hipBindTextureToArray(struct texture<T, dim, readMode>& tex, hipArray_const_t array)

```
texture<float, 2, hipReadModeElementType> tex;
hipChannelFormatDesc channelDesc = hipCreateChannelDesc(32, 0, 0, 0, hipChannelFormatKindFloat);
hipArray* hipArray;
hipMallocArray(&hipArray, &channelDesc, width, height);
hipMemcpyToArray(hipArray, 0, 0, hData, size, hipMemcpyHostToDevice);
tex.addressMode[0] = hipAddressModeWrap;
tex.addressMode[1] = hipAddressModeWrap;
tex.filterMode = hipFilterModePoint;
```

```
tex.normalized = 0;
hipBindTextureToArray(tex, hipArray, channelDesc);
```

22.5 hipBindTextureToMipmappedArray

__host__cudaError_t cudaBindTextureToMipmappedArray (const textureReference *texref, cudaMipmappedArray_const_t mipmappedArray, const cudaChannelFormatDesc *desc)

---

hipError_t hipBindTextureToMipmappedArray(const textureReference* tex, hipMipmappedArray_const_t mipmappedArray,const hipChannelFormatDesc* desc);

22.6 hipCreateChannelDesc*

__host__cudaChannelFormatDesc cudaCreateChannelDesc (int x, int y, int z, int w, cudaChannelFormatKind f)

---

hipChannelFormatDesc hipCreateChannelDesc(int x, int y, int z, int w, hipChannelFormatKind f)

22.7 hipEventCreate

__host__cudaError_t cudaEventCreate (cudaEvent_t *event)

---

hipError_t hipEventCreate(hipEvent_t* event);

Create an event

Parameters

[in,out] event Returns the newly created event.

Returns

#hipSuccess, #hipErrorInitializationError, #hipErrorInvalidValue, #hipErrorLaunchFailure, #hipErrorMemoryAllocation

See also

hipEventCreateWithFlags, hipEventRecord, hipEventQuery, hipEventSynchronize, hipEventDestroy, hipEventElapsedTime

```
hipEvent_t start, stop;
```

```
HIPCHECK(hipEventCreate(&start));
HIPCHECK(hipEventSynchronize(start));
```

22.8 hipFuncGetAttributes

__host__ __device__

cudaError_t cudaFuncGetAttributes (cudaFuncAttributes* attr, const void* func)

---

hipError_t hipFuncGetAttributes(hipFuncAttributes* attr, const void* func);

Find out attributes for a given function.

Parameters

[out] attr

[in] func

Returns

#hipSuccess, #hipErrorInvalidDeviceFunction

```
__global__void fn(float* px, float* py){}
int main() {
hipInit(0);
hipFuncAttributes attr{};
auto r = hipFuncGetAttributes(&attr, reinterpret_cast<const void*>(&fn));
if (r != hipSuccess || attr.maxThreadsPerBlock == 0) {
failed("Failed to read attributes.");
}
passed();
}
```

22.9 hipFuncSetAttribute

__host__cudaError_t cudaFuncSetAttribute (const void* func, cudaFuncAttribute attr, int value)

---

hipError_t hipFuncSetAttribute(const void* func, hipFuncAttribute attr, int value)

Set Cache configuration for a specific function.

Parameters

[in] config

开发者社区

Returns

#hipSuccess, #hipErrorNotInitialized

22.10 hipFuncSetCacheConfig

__host__cudaError_t cudaFuncSetCacheConfig (const void *func, cudaFuncCache cacheConfig)

---

hipError_t hipFuncSetCacheConfig(const void* func, hipFuncCache_t config);

Set Cache configuration for a specific function.

Parameters

[in] config;

Returns

#hipSuccess, #hipErrorInitializationError

```
hipFuncCache_t cacheConfig;
void* func;
hipFuncSetCacheConfig(func, cacheConfig);
passed();
```

22.11 hipGetSymbolAddress

__host__cudaError_t cudaGetSymbolAddress (void **devPtr, const void *symbol)

---

hipError_t hipGetSymbolAddress(void** devPtr, const void* symbolName)

Copies the memory address of symbol symbolName to devPtr.

Parameters

[in] symbolName Symbol on device

[out] devPtr Pointer to a pointer to the memory referred to by the symbol

Returns

#hipSuccess, #hipErrorNotInitialized, #hipErrorNotFound

See also

hipGetSymbolSize, hipMemcpyToSymbol, hipMemcpyFromSymbol, hipMemcpyToSymbolAsync, hipMemcpyFromSymbolAsync

开发者社区

```
    __device__ __constant__ int globalConst[NUM];
    int *symbolAddress;
    hipGetSymbolSize(&symbolSize, HIP_SYMBOL(globalConst));
    hipGetSymbolAddress((void**) &symbolAddress,
HIP_SYMBOL(globalConst));
```

22.12 hipGetSymbolSize

__host__cudaError_t cudaGetSymbolSize (size_t *size, const void *symbol)

---

hipError_t hipGetSymbolSize(size_t* size, const void* symbolName)

Copies the size of symbol symbolName to size.

Parameters

[in] symbolName	Symbol on device
[out] size	Pointer to the size of the symbol

Returns

#hipSuccess, #hipErrorNotInitialized, #hipErrorNotFound

See also

hipGetSymbolSize, hipMemcpyToSymbol, hipMemcpyFromSymbol, hipMemcpyToSymbolAsync, hipMemcpyFromSymbolAsync

```
    __device__ __constant__ int globalConst[NUM];
    size_t symbolSize = 0;
    hipGetSymbolSize(&symbolSize, HIP_SYMBOL(globalConst));
```

22.13 hipGetTextureAlignmentOffset

__host__cudaError_t cudaGetTextureAlignmentOffset (size_t *offset, const textureReference *texref)

---

hipError_t hipGetTextureAlignmentOffset(size_t* offset, const textureReference* texref);

```
    int runTest() {
    int testResult = 1;
```

```
    float *texBuf;
    float val[N], output[N];
    size_t offset = 0;
    float *devBuf;
    for (int i = 0; i < N; i++) {
    val[i] = (float)i;
    output[i] = 0.0;
    }
    hipChannelFormatDesc chanDesc =
    hipCreateChannelDesc(32, 0, 0, 0, hipChannelFormatKindFloat);
    HIPCHECK(hipMalloc(&texBuf, N * sizeof(float)));
    HIPCHECK(hipMalloc(&devBuf, N * sizeof(float)));
    HIPCHECK(hipMemcpy(texBuf, val, N * sizeof(float),
hipMemcpyHostToDevice));
    tex.addressMode[0] = hipAddressModeClamp;
    tex.addressMode[1] = hipAddressModeClamp;
    tex.filterMode = hipFilterModePoint;
    tex.normalized = 0;
    HIPCHECK(hipBindTexture(&offset, tex, (void *)texBuf, chanDesc, N *
sizeof(float)));
    HIPCHECK(hipGetTextureAlignmentOffset(&offset,&tex));
    dim3 dimBlock(64, 1, 1);
    dim3 dimGrid(N / dimBlock.x, 1, 1);
    hipLaunchKernelGGL(kernel, dim3(dimGrid), dim3(dimBlock), 0, 0, devBuf);
    HIPCHECK(hipDeviceSynchronize());
    HIPCHECK(hipMemcpy(output, devBuf, N * sizeof(float),
hipMemcpyDeviceToHost));
    for (int i = 0; i < N; i++) {
    if (output[i] != val[i]) {
    testResult = 0;
    break;
    }
    }
```

开发者社区

```
HIPCHECK(hipUnbindTexture(&tex));
HIPCHECK(hipFree(texBuf));
HIPCHECK(hipFree(devBuf));
return testResult;
}
```

22.14 hipGraphAddMemcpyNodeFromSymbol*

__host__cudaError_t cudaGraphAddMemcpyNodeFromSymbol (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, void* dst, const void* symbol, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipGraphAddMemcpyNodeFromSymbol(hipGraphNode_t* pGraphNode, hipGraph_t graph, const hipGraphNode_t* pDependencies, size_t numDependencies, void* dst, const void* symbol, size_t count, size_t offset, hipMemcpyKind kind)

Creates a memcpy node to copy from a symbol on the device and adds it to a graph.

Parameters

[out] pGraphNode - pointer to graph node to create.

[in] graph - instance of graph to add the created node.

[in] pDependencies - const pointer to the dependencies on the memcpy execution node.

[in] numDependencies - the number of the dependencies.

[in] dst - pointer to memory address to the destination.

[in] symbol - Device symbol address.

[in] count - the size of the memory to copy.

[in] offset - Offset from start of symbol in bytes.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

22.15 hipGraphAddMemcpyNodeToSymbol*

__host__cudaError_t cudaGraphAddMemcpyNodeToSymbol (cudaGraphNode_t* pGraphNode, cudaGraph_t graph, const cudaGraphNode_t* pDependencies, size_t numDependencies, const void* symbol, const void* src, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipGraphAddMemcpyNodeToSymbol(hipGraphNode_t* pGraphNode, hipGraph_t graph, const hipGraphNode_t* pDependencies, size_t numDependencies, const void* symbol, const void* src, size_t count, size_t offset, hipMemcpyKind kind)

Creates a memcpy node to copy to a symbol on the device and adds it to a graph.

Parameters

[out] pGraphNode - pointer to graph node to create.

[in] graph - instance of graph to add the created node.

[in] pDependencies - const pointer to the dependencies on the memcpy execution node.

[in] numDependencies - the number of the dependencies.

[in] symbol - Device symbol address.

[in] src - pointer to memory address of the src.

[in] count - the size of the memory to copy.

[in] offset - Offset from start of symbol in bytes.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

22.16 hipGraphExecMemcpyNodeSetParamsFromSymbol*

__host__cudaError_t cudaGraphExecMemcpyNodeSetParamsFromSymbol (cudaGraphExec_t hGraphExec, cudaGraphNode_t node, void* dst, const void* symbol, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipGraphExecMemcpyNodeSetParamsFromSymbol(hipGraphExec_t hGraphExec, hipGraphNode_t node, void* dst, const void* symbol, size_t count, size_t

offset, hipMemcpyKind kind)

Sets the parameters for a memcpy node in the given graphExec to copy from a symbol on the device.

Parameters

[in] hGraphExec - instance of the executable graph with the node.

[in] node - instance of the node to set parameters to.

[in] dst - pointer to memory address to the destination.

[in] symbol - Device symbol address.

[in] count - the size of the memory to copy.

[in] offset - Offset from start of symbol in bytes.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

22.17 hipGraphExecMemcpyNodeSetParamsToSymbol*

__host__cudaError_t cudaGraphExecMemcpyNodeSetParamsToSymbol (cudaGraphExec_t hGraphExec, cudaGraphNode_t node, const void* symbol, const void* src, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipGraphExecMemcpyNodeSetParamsToSymbol(hipGraphExec_t hGraphExec, hipGraphNode_t node, const void* symbol, const void* src, size_t count, size_t offset, hipMemcpyKind kind)

Sets the parameters for a memcpy node in the given graphExec to copy to a symbol on the device.

Parameters

[in] hGraphExec - instance of the executable graph with the node.

[in] node - instance of the node to set parameters to.

[in] symbol - Device symbol address.

[in] src - pointer to memory address of the src.

[in] count - the size of the memory to copy.

[in] offset - Offset from start of symbol in bytes.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

22.18 hipGraphMemcpyNodeSetParamsFromSymbol*

__host__cudaError_t cudaGraphMemcpyNodeSetParamsFromSymbol (cudaGraphNode_t node, void* dst, const void* symbol, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipGraphMemcpyNodeSetParamsFromSymbol(hipGraphNode_t node, void* dst, const void* symbol, size_t count, size_t offset, hipMemcpyKind kind)

Sets a memcpy node's parameters to copy from a symbol on the device.

Parameters

[in] node - instance of the node to set parameters to.

[in] dst - pointer to memory address to the destination.

[in] symbol - Device symbol address.

[in] count - the size of the memory to copy.

[in] offset - Offset from start of symbol in bytes.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

22.19 hipGraphMemcpyNodeSetParamsToSymbol*

__host__cudaError_t cudaGraphMemcpyNodeSetParamsToSymbol (cudaGraphNode_t node, const void* symbol, const void* src, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipGraphMemcpyNodeSetParamsToSymbol(hipGraphNode_t node, const void* symbol, const void* src, size_t count, size_t offset, hipMemcpyKind kind)

Sets a memcpy node's parameters to copy to a symbol on the device.

Parameters

[in] node - instance of the node to set parameters to.

[in] symbol - Device symbol address.

[in] src - pointer to memory address of the src.

[in] count - the size of the memory to copy.

[in] offset - Offset from start of symbol in bytes.

[in] kind - the type of memory copy.

Returns

#hipSuccess, #hipErrorInvalidValue

22.20 hipLaunchCooperativeKernel

__host__cudaError_t cudaLaunchCooperativeKernel (const void *func, dim3 gridDim, dim3 blockDim, void **args, size_t sharedMem, cudaStream_t stream)

---

hipError_t hipLaunchCooperativeKernel(const void* f, dim3 gridDim, dim3 blockDimX, void** kernelParams, unsigned int sharedMemBytes, hipStream_t stream)

Launches kernel f with launch parameters and shared memory on stream with arguments passed to kernelparams or extra, where thread blocks can cooperate and synchronize as they execute.

Parameters

[in] f Kernel to launch.

[in] gridDim Grid dimensions specified as multiple of blockDim.

[in] blockDim Block dimensions specified in work-items.

[in] kernelParams A list of kernel arguments.

[in] sharedMemBytes Amount of dynamic shared memory to allocate for this kernel. The HIP-Clang compiler provides support for extern shared declarations.

[in] stream Stream where the kernel should be dispatched. May be 0, in which case the default stream is used with associated synchronization rules.

Returns

#hipSuccess, hipInvalidDevice, hipErrorNotInitialized, hipErrorInvalidValue, hipErrorCooperativeLaunchTooLarge

22.21 hipLaunchKernel

__host__cudaError_t cudaLaunchKernel (const void *func, dim3 gridDim, dim3 blockDim, void **args, size_t sharedMem, cudaStream_t stream)

---

hipLaunchKernel(F kernel, const dim3& numBlocks, const dim3& dimBlocks,std::uint32_t groupMemBytes, hipStream_t stream, Args... args)

hipLaunchKernel is a variadic macro which accepts as parameters the launch configurations (grid dims, group dims, stream, dynamic shared size) followed by a variable number of kernel arguments. This sequence is then expanded into the appropriate kernel launch syntax depending on the platform.

While this can be a convenient single-line kernel launch syntax, the macro implementation can cause issues when nested inside other macros.

```
int blocks = N / p_blockSize;
printf("running with N=%zu p_blockSize=%d blocks=%d\n", N, p_blockSize, blocks);
HIPCHECK(hipMemcpy(A_d, A_h, Nbytes, hipMemcpyHostToDevice));
HIPCHECK(hipGetLastError());
hipLaunchKernel(myKern, dim3(blocks), dim3(p_blockSize), 0, 0, C_d, A_d, N, 0);
```

22.22 hipMallocAsync*

__host__cudaError_t cudaMallocAsync (void** devPtr, size_t size, cudaStream_t hStream)

---

hipError_t hipMallocAsync(void** dev_ptr, size_t size, hipStream_t stream)

Allocates memory with stream ordered semantics

Parameters

[out] dev_ptr Returned device pointer of memory allocation

[in] size Number of bytes to allocate

[in] stream The stream establishing the stream ordering contract and the memory pool to allocate from

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorNotSupported, #hipErrorOutOfMemory

22.23 hipMallocHost

__host__cudaError_t cudaMallocHost (void **ptr, size_t size)

---

hipError_t hipMallocHost(void** ptr, size_t size);

Allocate pinned host memory [Deprecated].

Parameters

[out] ptr Pointer to the allocated host pinned memory

[in] size Requested memory size

If size is 0, no memory is allocated, *ptr returns nullptr, and hipSuccess is returned.

Returns

#hipSuccess, #hipErrorMemoryAllocation

Deprecated:

use hipHostMalloc() instead

22.24 hipMallocManaged

__host__cudaError_t cudaMallocManaged (void **devPtr, size_t size, unsigned int flags)

---

hipError_t hipMallocManaged(void** dev_ptr,size_t size,unsigned int flags __dparm(hipMemAttachGlobal))

Allocates memory that will be automatically managed by DTK HMM.

Parameters

[out] dev_ptr Pointer to allocated device memory

[in] size Requested allocation size in bytes

[in] flags must be either hipMemAttachGlobal or hipMemAttachHost(defaults to hipMemAttachGlobal)

Returns

#hipSuccess , #hipErrorMemoryAllocation, #hipErrorNotSupported, #hipErrorInvalidValue

22.25 hipMemcpyFromSymbol

__host__cudaError_t cudaMemcpyFromSymbol (void *dst, const void *symbol, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipMemcpyFromSymbol(void* dst, const void* symbolName,size_t sizeBytes, size_t offset __dparm(0),hipMemcpyKind kind __dparm(hipMemcpyDeviceToHost));

```
hipMemcpyToSymbol(HIP_SYMBOL(globalIn), A, SIZE, 0, hipMemcpyHostToDevice);
hipLaunchKernelGGL(Assign, dim3(1, 1, 1), dim3(NUM, 1, 1), 0, 0, Ad);
hipMemcpy(B, Ad, SIZE, hipMemcpyDeviceToHost);
hipMemcpyFromSymbol(C, HIP_SYMBOL(globalOut), SIZE, 0, hipMemcpyDeviceToHost);
```

22.26 hipMemcpyFromSymbolAsync

__host__cudaError_t cudaMemcpyFromSymbolAsync (void *dst, const void *symbol, size_t count, size_t offset, cudaMemcpyKind kind, cudaStream_t stream)

---

hipError_t hipMemcpyFromSymbolAsync(void* dst, const void* symbolName,size_t sizeBytes, size_t offset,hipMemcpyKind kind,hipStream_t stream __dparm(0));

```
hipStreamCreate(&stream);
hipMemcpyToSymbolAsync(HIP_SYMBOL(globalIn), Am, SIZE, 0, hipMemcpyHostToDevice, stream);
hipStreamSynchronize(stream);
hipLaunchKernelGGL(Assign, dim3(1, 1, 1), dim3(NUM, 1, 1), 0, 0, Ad);
hipMemcpy(B, Ad, SIZE, hipMemcpyDeviceToHost);
hipMemcpyFromSymbolAsync(Cm, HIP_SYMBOL(globalOut), SIZE, 0, hipMemcpyDeviceToHost, stream);
hipStreamSynchronize(stream);
```

开发者社区

22.27 hipMemcpyToSymbol

__host__cudaError_t cudaMemcpyToSymbol (const void *symbol, const void *src, size_t count, size_t offset, cudaMemcpyKind kind)

---

hipError_t hipMemcpyToSymbol(const void* symbolName, const void* src,size_t sizeBytes, size_t offset __dparm(0),hipMemcpyKind kind __dparm(hipMemcpyHostToDevice));

Copies sizeBytes bytes from the memory area pointed to by src to the memory area pointed to by offset bytes from the start of symbol symbol.

The memory areas may not overlap. Symbol can either be a variable that resides in global or constant memory space, or it can be a character string, naming a variable that resides in global or constant memory space. Kind can be either hipMemcpyHostToDevice or hipMemcpyDeviceToDevice TODO: cudaErrorInvalidSymbol and cudaErrorInvalidMemcpyDirection is not supported, use hipErrorUnknown for now.

Parameters

[in] symbolName Symbol destination on device

[in] src Data being copy from

[in] sizeBytes Data size in bytes

[in] offset Offset from start of symbol in bytes

[in] kind Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryFree, #hipErrorUnknown

See also

hipMemcpy, hipMemcpy2D, hipMemcpyToArray, hipMemcpy2DToArray, hipMemcpyFromArray, hipMemcpy2DFromArray, hipMemcpyArrayToArray, hipMemcpy2DArrayToArray, hipMemcpyFromSymbol, hipMemcpyAsync, hipMemcpy2DAsync, hipMemcpyToArrayAsync, hipMemcpy2DToArrayAsync, hipMemcpyFromArrayAsync, hipMemcpy2DFromArrayAsync, hipMemcpyToSymbolAsync, hipMemcpyFromSymbolAsync

```
    int *A, *Am, *B, *Ad, *C, *Cm;
    A = new int[NUM];
    B = new int[NUM];
    C = new int[NUM];
    for (int i = 0; i < NUM; i++) {
    A[i] = -1 * i;
    B[i] = 0;
    C[i] = 0;
    }
    hipMalloc((void**)&Ad, SIZE);
    hipHostMalloc((void**)&Am, SIZE);
    hipHostMalloc((void**)&Cm, SIZE);
    hipMemcpyToSymbol(HIP_SYMBOL(globalIn), A, SIZE, 0,
hipMemcpyHostToDevice);
    hipLaunchKernelGGL(Assign, dim3(1, 1, 1), dim3(NUM, 1, 1), 0, 0, Ad);
    hipMemcpy(B, Ad, SIZE, hipMemcpyDeviceToHost);
    hipMemcpyFromSymbol(C, HIP_SYMBOL(globalOut), SIZE, 0,
hipMemcpyDeviceToHost);
```

22.28 hipMemcpyToSymbolAsync

__host__cudaError_t cudaMemcpyToSymbolAsync (const void *symbol, const void *src, size_t count, size_t offset, cudaMemcpyKind kind, cudaStream_t stream)

---

hipError_t hipMemcpyToSymbolAsync(const void* symbolName, const void* src,size_t sizeBytes, size_t offset,hipMemcpyKind kind, hipStream_t stream __dparm(0));

Copies sizeBytes bytes from the memory area pointed to by src to the memory area pointed to by offset bytes from the start of symbol symbol.

The memory areas may not overlap. Symbol can either be a variable that resides in global or constant memory space, or it can be a character string, naming a variable that resides in global or constant memory space. Kind can be either hipMemcpyHostToDevice or hipMemcpyDeviceToDevice

hipMemcpyToSymbolAsync() is asynchronous with respect to the host, so the call may return before copy is complete. TODO: cudaErrorInvalidSymbol and cudaErrorInvalidMemcpyDirection is not supported, use hipErrorUnknown for now.

Parameters

[in] symbolName Symbol destination on device

[in] src Data being copy from

[in] sizeBytes Data size in bytes

[in] offset Offset from start of symbol in bytes

[in] kind Type of transfer

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorMemoryFree, #hipErrorUnknown

See also

hipMemcpy, hipMemcpy2D, hipMemcpyToArray, hipMemcpy2DToArray, hipMemcpyFromArray, hipMemcpy2DFromArray, hipMemcpyArrayToArray, hipMemcpy2DArrayToArray, hipMemcpyFromSymbol, hipMemcpyAsync, hipMemcpy2DAsync, hipMemcpyToArrayAsync, hipMemcpy2DToArrayAsync, hipMemcpyFromArrayAsync, hipMemcpy2DFromArrayAsync, hipMemcpyToSymbolAsync, hipMemcpyFromSymbolAsync

```
int *A, *Am, *B, *Ad, *C, *Cm;
A = new int[NUM];
B = new int[NUM];
C = new int[NUM];
for (int i = 0; i < NUM; i++) {
A[i] = -1 * i;
B[i] = 0;
C[i] = 0;
}
hipMalloc((void**)&Ad, SIZE);
hipHostMalloc((void**)&Am, SIZE);
```

```
    hipHostMalloc((void**)&Cm, SIZE);
    hipStream_t stream;
    hipStreamCreate(&stream);
    hipMemcpyToSymbolAsync(HIP_SYMBOL(globalIn), Am, SIZE, 0,
hipMemcpyHostToDevice, stream);
    hipStreamSynchronize(stream);
    hipLaunchKernelGGL(Assign, dim3(1, 1, 1), dim3(NUM, 1, 1), 0, 0, Ad);
    hipMemcpy(B, Ad, SIZE, hipMemcpyDeviceToHost);
    hipMemcpyFromSymbolAsync(Cm, HIP_SYMBOL(globalOut), SIZE, 0,
hipMemcpyDeviceToHost, stream);
    hipStreamSynchronize(stream);
```

22.29 hipOccupancyAvailableDynamicSMemPerBlock*

__host__cudaError_t cudaOccupancyAvailableDynamicSMemPerBlock(size_t* dynamicSmemSize, const void* func, int numBlocks, int blockSize)

No hip

22.30 hipOccupancyMaxActiveBlocksPerMultiprocessor

__host____device__cudaError_t cudaOccupancyMaxActiveBlocksPerMultiprocessor (int *numBlocks, const void *func, int blockSize, size_t dynamicSMemSize)

---

hipError_t hipOccupancyMaxActiveBlocksPerMultiprocessor(int* numBlocks, const void* f, int blockSize, size_t dynSharedMemPerBlk)

Returns occupancy for a device function.

Parameters

[out] numBlocks Returned occupancy

[in] func Kernel function for which occupancy is calculated

[in] blockSize Block size the kernel is intended to be launched with

[in] dynSharedMemPerBlk Dynamic shared memory usage (in bytes)

intended for each block

22.31 hipOccupancyMaxActiveBlocksPerMultiprocessorWithFlags

__host__cudaError_t cudaOccupancyMaxActiveBlocksPerMultiprocessorWithFlags (int *numBlocks, const void *func, int blockSize, size_t dynamicSMemSize, unsigned int flags)

---

hipError_t hipOccupancyMaxActiveBlocksPerMultiprocessorWithFlags(int* numBlocks, const void* f, int blockSize, size_t dynSharedMemPerBlk, unsigned int flags __dparm(hipOccupancyDefault))

Returns occupancy for a device function.

Parameters

[out] numBlocks Returned occupancy

[out] f Kernel function for which occupancy is calulated

[in] blockSize Block size the kernel is intended to be launched with

[in] dynSharedMemPerBlk Dynamic shared memory usage (in bytes) intended for each block

[in] flags Extra flags for occupancy calculation (currently ignored)

22.32 hipOccupancyMaxPotentialBlockSize*

__host__cudaError_t cudaOccupancyMaxPotentialBlockSize (int* minGridSize, int* blockSize, T func, size_t dynamicSMemSize = 0, int blockSizeLimit = 0)

---

hipError_t hipOccupancyMaxPotentialBlockSize(int* gridSize, int* blockSize, const void* f, size_t dynSharedMemPerBlk, int blockSizeLimit)

Parameters

[out] gridSize minimum grid size for maximum potential occupancy

[out] blockSize block size for maximum potential occupancy

[in] f kernel function for which occupancy is calulated

[in] dynSharedMemPerBlk dynamic shared memory usage (in bytes) intended for each block

[in] blockSizeLimit the maximum block size for the kernel, use 0 for no limit

Return

#hipSuccess, #hipInvalidDevice, #hipErrorInvalidValue

22.33 hipOccupancyMaxPotentialBlockSizeVariableSMem*

__host__cudaError_t cudaOccupancyMaxPotentialBlockSizeVariableSMem (int *minGridSize, int *blockSize, T func, UnaryFunction blockSizeToDynamicSMemSize, int blockSizeLimit)

Static hipError_t __host__ inline hipOccupancyMaxPotentialBlockSizeVariableSMem(int* min_grid_size, int* block_size, T func, UnaryFunction block_size_to_dynamic_smem_size, int block_size_limit = 0)

Returns grid and block size that achieves maximum potential occupancy for a device function

Parameters

[out] min_grid_size minimum grid size needed to achieve the best potential occupancy

[out] block_size block size required for the best potential occupancy

[in] func device function symbol

[in] block_size_to_dynamic_smem_size - a unary function/functor that takes block size, and returns the size, in bytes, of dynamic shared memory needed for a block

[in] block_size_limit the maximum block size \p func is designed to work with. 0 means no limit.

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidDeviceFunction, #hipErrorInvalidValue

22.34 hipOccupancyMaxPotentialBlockSizeVariableSMem WithFlags*

__host__cudaError_t cudaOccupancyMaxPotentialBlockSizeVariableSMemWithFlags (int *minGridSize, int *blockSize, T func, UnaryFunction blockSizeToDynamicSMemSize, int blockSizeLimit, unsigned int flags)

---

static hipError_t __host__ inline hipOccupancyMaxPotentialBlockSizeVariableSMemWithFlags(int* min_grid_size, int* block_size, T func, UnaryFunction block_size_to_dynamic_smem_size, int block_size_limit = 0, unsigned int flags = 0)

Returns grid and block size that achieves maximum potential occupancy for a device function

Parameters

[out] min_grid_size minimum grid size needed to achieve the best potential occupancy

[out] block_size block size required for the best potential occupancy

[in] func device function symbol

[in] block_size_to_dynamic_smem_size - a unary function/functor that takes block size, and returns the size, in bytes, of dynamic shared memory needed for a block

[in] block_size_limit the maximum block size \p func is designed to work with. 0 means no limit.

[in] flags reserved

Returns

#hipSuccess, #hipErrorInvalidDevice, #hipErrorInvalidDeviceFunction, #hipErrorInvalidValue

22.35 hipOccupancyMaxPotentialBlockSizeWithFlags*

__host__cudaError_t cudaOccupancyMaxPotentialBlockSizeWithFlags (int *minGridSize, int *blockSize, T func, size_t dynamicSMemSize, int blockSizeLimit,

unsigned int flags)

---

static hipError_t __host__ inline hipOccupancyMaxPotentialBlockSizeWithFlags(int* gridSize, int* blockSize,T f, size_t dynSharedMemPerBlk = 0, int blockSizeLimit = 0, unsigned int flags = 0)

22.36 hipStreamAttachMemAsync

__host__cudaError_t cudaStreamAttachMemAsync (cudaStream_t stream, void *devPtr, size_t length, unsigned int flags)

---

hipError_t hipStreamAttachMemAsync(hipStream_t stream,void* dev_ptr,size_t length __dparm(0),unsigned int flags __dparm(hipMemAttachSingle));

Attach memory to a stream asynchronously in DTK HMM.

Parameters

[in] stream Stream in which to enqueue the attach operation

[in] dev_ptr Pointer to memory (must be a pointer to managed memory or to a valid host-accessible region of system-allocated memory)

[in] length Length of memory (defaults to zero)

[in] flags Must be one of cudaMemAttachGlobal, cudaMemAttachHost or cudaMemAttachSingle (defaults to cudaMemAttachSingle)

Returns

#hipSuccess, #hipErrorOutOfMemory

22.37 hipUnbindTexture

__host__cudaError_t cudaUnbindTexture (const textureReference *texref)

---

hipError_t hipUnbindTexture(const textureReference* tex)

```
texture<float, 1, hipReadModeElementType> tex;
HIPCHECK(hipMalloc(&texBuf, N * sizeof(float)));
HIPCHECK(hipMalloc(&devBuf, N * sizeof(float)));
HIPCHECK(hipMemcpy(texBuf, val, N * sizeof(float),
hipMemcpyHostToDevice));

tex.addressMode[0] = hipAddressModeClamp;
```

```
    tex.addressMode[1] = hipAddressModeClamp;
    tex.filterMode = hipFilterModePoint;
    tex.normalized = 0;
    HIPCHECK(hipBindTexture(&offset, tex, (void *)texBuf, chanDesc, N *
sizeof(float)));
    HIPCHECK(hipGetTextureAlignmentOffset(&offset,&tex));
    dim3 dimBlock(64, 1, 1);
    dim3 dimGrid(N / dimBlock.x, 1, 1);
    hipLaunchKernelGGL(kernel, dim3(dimGrid), dim3(dimBlock), 0, 0, devBuf);
    HIPCHECK(hipDeviceSynchronize());
    HIPCHECK(hipMemcpy(output, devBuf, N * sizeof(float),
hipMemcpyDeviceToHost));
    for (int i = 0; i < N; i++) {
    if (output[i] != val[i]) {
    testResult = 0;
    break;
    }
    }
    HIPCHECK(hipUnbindTexture(&tex));
    HIPCHECK(hipFree(texBuf));
    HIPCHECK(hipFree(devBuf));
```

23 Interactions with the CUDA Driver API

23.1 hipGetKernel【dtk-25.04.1 新增】

__host__cudaError_t cudaGetKernel (cudaKernel_t* kernelPtr, const void* entryFuncAddr)

---

hipError_t hipGetKernel(hipKernel_t* kernelPtr, const void* entryFuncAddr)

Get pointer to device kernel that matches entry function entryFuncAddr

Parameters

[out] kernelPtr Returns the device kernel

[in] entryFuncAddr Address of device entry function to search kernel for

Returns

#hipSuccess, #hipErrorInvalidValue, #hipErrorInvalidDeviceFunction

24 Profiler Control [DEPRECATED]

24.1 hipProfilerInitialize*

__host__cudaError_t cudaProfilerInitialize (const char *configFile, const char *outputFile, cudaOutputMode_t outputMode)

No hip

25 Profiler Control

25.1 hipProfilerStart

__host__cudaError_t cudaProfilerStart (void)

---

hipError_t hipProfilerStart()

Start recording of profiling information When using this API, start the profiler with profiling disabled. (–startdisabled)

Warning

hipProfilerStart API is under development.

```
#define ITERATIONS 10
// Cmdline parms to control start and stop triggers
int startTriggerIteration = -1;
int stopTriggerIteration = -1;
for (int i = 0; i < ITERATIONS; i++) {
if (i == startTriggerIteration) {
hipProfilerStart();
}
if (i == stopTriggerIteration) {
hipProfilerStop();
}
```

25.2 hipProfilerStop

__host__cudaError_t cudaProfilerStop (void)

---

hipError_t hipProfilerStop()

Stop recording of profiling information. When using this API, start the profiler with profiling disabled. (–startdisabled)

Warning

hipProfilerStop API is under development.

```
#define ITERATIONS 10
// Cmdline parms to control start and stop triggers
int startTriggerIteration = -1;
int stopTriggerIteration = -1;
for (int i = 0; i < ITERATIONS; i++) {
if (i == startTriggerIteration) {
hipProfilerStart();
}
if (i == stopTriggerIteration) {
hipProfilerStop();
}
```

26 Extend API

26.1 hipExtGetNearstCPU

hipError_t hipExtGetNearstCPU(int * cpuId);

Return the nearst cpu id.

Parameters

[output] cpuId Returns the nearst cpu id.

Returns

#hipSuccess, #hipErrorNoDevice

26.2 hipExtGetLinkTypeAndHopCount

hipError_t hipExtGetLinkTypeAndHopCount(int device1, int device2, uint32_t* linktype, uint32_t* hopcount)

26.3 hipExtStreamCreateWithCUMask

hipError_t hipExtStreamCreateWithCUMask(hipStream_t* stream, uint32_t cuMaskSize, const uint32_t* cuMask)

Create an asynchronous stream with the specified CU mask.

Parameters

[in, out] stream Pointer to new stream

[in] cuMaskSize Size of CU mask bit array passed in.

[out] cuMask Bit-vector representing the CU mask. Each active bit represents using one CU.

The first 32 bits represent the first 32 CUs, and so on. If its size is greater than physical CU number (i.e., multiProcessorCount member of hipDeviceProp_t), the extra elements are ignored. It is user's responsibility to make sure the input is meaningful.

Returns

#hipSuccess, #hipErrorInvalidHandle, #hipErrorInvalidValue

See also

hipStreamCreate, hipStreamSynchronize, hipStreamWaitEvent, hipStreamDestroy

26.4 hipExtStreamGetCUMask

hipError_t hipExtStreamGetCUMask(hipStream_t stream, uint32_t cuMaskSize, uint32_t* cuMask)

Get CU mask associated with an asynchronous stream.

Parameters

[in] stream Stream to be queried.

[in] cuMaskSize Number of the block of memories (uint32_t *) allocated by user.

[in] cuMask Pointer to a pre-allocated block of memories (uint32_t *) in which the stream's CU mask is returned. The CU mask is returned in a chunck of 32 bits where each active bit represents one active CU.

Returns

#hipSuccess, #hipErrorInvalidHandle, #hipErrorInvalidValue

See also

hipStreamCreate, hipStreamSynchronize, hipStreamWaitEvent, hipStreamDestroy

26.5 hipExtMallocWithFlags

hipError_t hipExtMallocWithFlags(void** ptr, size_t sizeBytes, unsigned int flags)

Allocate memory on the default accelerator.

Parameters

[out] ptr Pointer to the allocated memory

[in] size Requested memory size

[in] flags Type of memory allocation

If size is 0, no memory is allocated, *ptr returns nullptr, and hipSuccess is returned.

Returns

#hipSuccess, #hipErrorOutOfMemory, #hipErrorInvalidValue (bad context, null *ptr)

See also

hipMallocPitch, hipFree, hipMallocArray, hipFreeArray, hipMalloc3D, hipMalloc3DArray, hipHostFree, hipHostMalloc

26.6 hipExtHostMalloc

hipError_t hipExtHostMalloc(void** ptr, size_t size, unsigned int flags, int cpuId)

Allocate device accessible page locked host memory according to cpu id means that the location of the memory will belong to the cpu id.

Parameters

[out] ptr Pointer to the allocated host pinned memory

[in] size Requested memory size

[in] flags Type of host memory allocation

[int] cpuId

Returns

#hipSuccess, #hipErrorMemoryAllocation, #hipErrorNotSupported,

#hipErrorInvalidValue

See also

hipSetDeviceFlags, hipHostFree

26.7 hipExtMallocManaged

hipError_t hipExtMallocManaged(void** dev_ptr,size_t size,int cpuId)

Allocates memory that will be automatically managed by DTK HMM.

Parameters

[out] dev_ptr Pointer to allocated device memory

[in] size Requested allocation size in bytes

[in] cpu id

Returns

#hipSuccess, #hipErrorMemoryAllocation, #hipErrorNotSupported, #hipErrorInvalidValue

26.8 hipExtLaunchMultiKernelMultiDevice

hipError_t hipExtLaunchMultiKernelMultiDevice(hipLaunchParams* launchParamsList,int numDevices, unsigned int flags)

Launches kernels on multiple devices and guarantees all specified kernels are dispatched on respective streams before enqueuing any other work on the specified streams from any other threads.

Parameters

[in] hipLaunchParams List of launch parameters, one per device

[out] numDevices Size of the launchParamsList array

[in] flags Flags to control launch behavior.

Returns

hipSuccess, hipInvalidDevice, hipErrorNotInitialized, hipErrorInvalidValue

26.9 hipExtLaunchKernel

hipError_t hipExtLaunchKernel(const void* function_address, dim3 numBlocks,

dim3 dimBlocks,void** args, size_t sharedMemBytes, hipStream_t stream,hipEvent_t startEvent, hipEvent_t stopEvent, int flags)

27 Virtual Memory Management

27.1 hipMemAddressFree

CUresult cuMemAddressFree (CUdeviceptr ptr, size_t size)

---

hipError_t hipMemAddressFree(void* devPtr, size_t size)

Frees an address range reservation made via hipMemAddressReserve.

Parameters

[in] devPtr starting address of the range.

[in] size size of the range.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

```
size_t granularity = 0;
constexpr int N = DATA_SIZE;
size_t buffer_size = N * sizeof(int);
int deviceId = 0;
hipDevice_t device;
CTX_CREATE();
HIP_CHECK(hipDeviceGet(&device, deviceId));
checkVMMSupported(device);
hipMemAllocationProp prop{};
prop.type = hipMemAllocationTypePinned;
prop.location.type = hipMemLocationTypeDevice;
prop.location.id = device;  // Current Devices
HIP_CHECK(
    hipMemGetAllocationGranularity(&granularity,                    &prop,
hipMemAllocationGranularityMinimum));
REQUIRE(granularity > 0);
size_t size_mem = ((granularity + buffer_size - 1) / granularity) * granularity;
// Allocate virtual address range
```

开发者社区

```
hipDeviceptr_t ptrA;
HIP_CHECK(hipMemAddressReserve(&ptrA, size_mem, 0, 0, 0));
HIP_CHECK(hipMemAddressFree(ptrA, size_mem));
```

27.2 hipMemAddressReserve

CUresult cuMemAddressReserve (CUdeviceptr* ptr, size_t size, size_t alignment, CUdeviceptr addr, unsigned long long flags)

---

hipError_t hipMemAddressReserve(void** ptr, size_t size, size_t alignment, void* addr, unsigned long long flags)

Reserves an address range

Parameters

[out] ptr starting address of the reserved range.

[in] size size of the reservation.

[in] alignment alignment of the address.

[in] addr requested starting address of the range.

[in] flags currently unused, must be zero.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

27.3 hipMemCreate

CUresult cuMemCreate (CUmemGenericAllocationHandle* handle, size_t size, const CUmemAllocationProp* prop, unsigned long long flags)

---

hipError_t hipMemCreate(hipMemGenericAllocationHandle_t* handle, size_t size, const hipMemAllocationProp* prop, unsigned long long flags)

Creates a memory allocation described by the properties and size.

Parameters

[out] handle value of the returned handle.

[in] size size of the allocation.

[in] prop properties of the allocation.

[in] flags currently unused, must be zero.

Returns

开发者社区

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

```
size_t granularity = 0;
int deviceId = 0;
CTX_CREATE();
hipDevice_t device;
HIP_CHECK(hipDeviceGet(&device, deviceId));
checkVMMSupported(device);
hipMemAllocationProp prop{};

SECTION("Memory Allocation Type as hipMemAllocationTypePinned") {
  prop.type = hipMemAllocationTypePinned;
}

#if HT_AMD
SECTION("Memory Allocation Type as hipMemAllocationTypeUncached") {
  prop.type = hipMemAllocationTypeUncached;
}
#endif

prop.location.type = hipMemLocationTypeDevice;
prop.location.id = device;  // Current Devices
HIP_CHECK(
    hipMemGetAllocationGranularity(&granularity,                    &prop,
hipMemAllocationGranularityMinimum));
REQUIRE(granularity > 0);
hipMemGenericAllocationHandle_t handle;
// Allocate physical memory
for (int mul = 1; mul < 64; mul++) {
  HIP_CHECK(hipMemCreate(&handle, granularity * mul, &prop, 0));
  HIP_CHECK(hipMemRelease(handle));
}

CTX_DESTROY();
```

开发者社区

27.4 hipMemExportToShareableHandle

CUresult cuMemExportToShareableHandle (void* shareableHandle, CUmemGenericAllocationHandle handle, CUmemAllocationHandleType handleType, unsigned long long flags)

---

hipError_t hipMemExportToShareableHandle(void* shareableHandle, hipMemGenericAllocationHandle_t handle, hipMemAllocationHandleType handleType, unsigned long long flags)

Exports an allocation to a requested shareable handle type.

Parameters

[out] shareableHandle value of the returned handle.

[in] handle handle to share.

[in] handleType type of the shareable handle.

[in] flags currently unused, must be zero.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

27.5 hipMemGetAccess

CUresult cuMemGetAccess (unsigned long long* flags, const CUmemLocation* location, CUdeviceptr ptr)

---

hipError_t hipMemGetAccess(unsigned long long* flags, const hipMemLocation* location, void* ptr)

Get the access flags set for the given location and ptr.

Parameters

[out] flags flags for this location.

[in] location target location.

[in] ptr address to check the access flags.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

27.6 hipMemGetAllocationGranularity

CUresult cuMemGetAllocationGranularity (size_t* granularity, const CUmemAllocationProp* prop, CUmemAllocationGranularity_flags option)

---

hipError_t hipMemGetAllocationGranularity(size_t* granularity, const hipMemAllocationProp* prop, hipMemAllocationGranularity_flags option)

Calculates either the minimal or recommended granularity.

Parameters

[out] granularity returned granularity.

[in] prop location properties.

[in] option determines which granularity to return.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

27.7 hipMemGetAllocationPropertiesFromHandle

CUresult cuMemGetAllocationPropertiesFromHandle (CUmemAllocationProp* prop, CUmemGenericAllocationHandle handle)

---

hipError_t hipMemGetAllocationPropertiesFromHandle(hipMemAllocationProp* prop, hipMemGenericAllocationHandle_t handle)

Retrieve the property structure of the given handle.

Parameters

[out] prop properties of the given handle.

[in] handle handle to perform the query on.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

```
hipDevice_t device;
CTX_CREATE();
HIP_CHECK(hipDeviceGet(&device, 0));
checkVMMSupported(device);
hipMemGenericAllocationHandle_t handle;
hipMemAllocationProp prop = {};
```

开发者社区

```
  prop.type = hipMemAllocationTypePinned;
  prop.location.type = hipMemLocationTypeDevice;
  prop.location.id = device;
  // create a temp prop structure.
  hipMemAllocationProp prop_temp = {};
  size_t granularity = 0;
  int N = DATA_SIZE;
  size_t buffer_size = N * sizeof(int);
  HIP_CHECK(
    hipMemGetAllocationGranularity(&granularity, &prop,
hipMemAllocationGranularityMinimum));
  REQUIRE(granularity > 0);
  size_t mem_size = ((granularity + buffer_size - 1) / granularity) * granularity;
  // Allocate physical memory
  HIP_CHECK(hipMemCreate(&handle, mem_size, &prop, 0));
  // verify properties has been retrived from handle
  HIP_CHECK(hipMemGetAllocationPropertiesFromHandle(&prop_temp,
handle));
  REQUIRE(prop_temp.type == prop.type);
  REQUIRE(prop_temp.location.type == prop.location.type);
  REQUIRE(prop_temp.location.id == prop.location.id);
  HIP_CHECK(hipMemRelease(handle));
```

27.8 hipMemImportFromShareableHandle

CUresult cuMemImportFromShareableHandle (CUmemGenericAllocationHandle* handle, void* osHandle, CUmemAllocationHandleType shHandleType)

---

hipError_t hipMemImportFromShareableHandle(hipMemGenericAllocationHandle_t* handle, void* osHandle, hipMemAllocationHandleType shHandleType)

Imports an allocation from a requested shareable handle type.

Parameters

[out] handle Memory handle for the memory allocation.

[in] osHandle shareable handle representing the memory allocation.

[in] shHandleType handle type.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

```
hipDevice_t device;
HIP_CHECK(hipDeviceGet(&device, 0));
checkVMMSupported(device);

hipMemAllocationProp prop = {};
prop.type = hipMemAllocationTypePinned;
prop.requestedHandleTypes = hipMemHandleTypePosixFileDescriptor;
prop.location.type = hipMemLocationTypeDevice;
prop.location.id = device;

size_t granularity;
HIP_CHECK(
   hipMemGetAllocationGranularity(&granularity,                    &prop,
hipMemAllocationGranularityMinimum));

hipMemGenericAllocationHandle_t handle;
HIP_CHECK(hipMemCreate(&handle, granularity * 2, &prop, 0));
ShareableHandle shareable_handle;
HIP_CHECK(hipMemExportToShareableHandle(&shareable_handle, handle,
                                        hipMemHandleTypePosixFileDescriptor, 0));
hipMemGenericAllocationHandle_t imported_handle;
HIP_CHECK(hipMemImportFromShareableHandle(&imported_handle,
        reinterpret_cast<void*>(static_cast<uintptr_t>(shareable_handle)),
        hipMemHandleTypePosixFileDescriptor));
HIP_CHECK(hipMemRelease(handle));
HIP_CHECK(hipMemRelease(imported_handle));
```

开发者社区

27.9 hipMemMap

CUresult cuMemMap (CUdeviceptr ptr, size_t size, size_t offset, CUmemGenericAllocationHandle handle, unsigned long long flags)

---

hipError_t hipMemMap(void* ptr, size_t size, size_t offset, hipMemGenericAllocationHandle_t handle, unsigned long long flags)

Maps an allocation handle to a reserved virtual address range.

Parameters

[in] ptr address where the memory will be mapped.

[in] size size of the mapping.

[in] offset offset into the memory, currently must be zero.

[in] handle memory allocation to be mapped.

[in] flags currently unused, must be zero.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

27.10 hipMemMapArrayAsync

CUresult cuMemMapArrayAsync (CUarrayMapInfo* mapInfoList, unsigned int count, CUstream hStream)

---

hipError_t hipMemMapArrayAsync(hipArrayMapInfo* mapInfoList, unsigned int count, hipStream_t stream)

Maps or unmaps subregions of sparse HIP arrays and sparse HIP mipmapped arrays.

Parameters

[in] mapInfoList list of hipArrayMapInfo.

[in] count number of hipArrayMapInfo in mapInfoList.

[in] stream stream identifier for the stream to use for map or unmap operations.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

开发者社区

```
    HIP_CHECK(hipSetDevice(0));
    size_t elementCount = 128;
    size_t memSizeByte = elementCount * sizeof(int);
    int *hBuffer = nullptr;
    int *hResult = nullptr;
    HIP_CHECK(hipHostMalloc(&hBuffer, memSizeByte,
hipHostMallocDefault));
    HIP_CHECK(hipHostMalloc(&hResult, memSizeByte,
hipHostMallocDefault));
    for (int i = 0; i < elementCount; ++i) { hBuffer[i] = i; }
    hipChannelFormatDesc desc;
    desc.x = 32;
    desc.y = 0;
    desc.z = 0;
    desc.w = 0;
    desc.f = hipChannelFormatKindSigned;
    hipArray_t array = nullptr;
    HIP_CHECK(hipMallocArray(&array, &desc, elementCount, 0,
hipArrayDeferredMapping));
    hipArrayMemoryRequirements memReq;
    HIP_CHECK(hipArrayGetMemoryRequirements(&memReq, array, 0));
    printf("mem:%zu\n", memReq.size);

    hipMemAllocationProp prop;
    memset(&prop, 0, sizeof(prop));
    prop.type = hipMemAllocationTypePinned;
    prop.location.id = 0;
    prop.location.type = hipMemLocationTypeDevice;
    prop.requestedHandleType = hipMemHandleTypeNone;
    prop.win32HandleMetaData = nullptr;
    prop.allocFlags.compressionType = 0;
    prop.allocFlags.gpuDirectRDMACapable = 0;
    prop.allocFlags.usage = 0;
```

开发者社区

```
    unsigned long long flags = 0;
    hipMemGenericAllocationHandle_t handle = nullptr;
    size_t granularity = 0;
    HIP_CHECK(hipMemGetAllocationGranularity(&granularity, &prop,
hipMemAllocationGranularityRecommended));
    size_t size = (memReq.size + granularity - 1) / granularity * granularity;
    HIP_CHECK(hipMemCreate(&handle, size, &prop, flags));

    hipArrayMapInfo mapInfo;
    memset(&mapInfo, 0, sizeof(mapInfo));
    mapInfo.resourceType = hipResourceTypeArray;
    mapInfo.resource.array = array;
    mapInfo.memOperationType = hipMemOperationTypeMap;
    mapInfo.memHandleType = hipMemHandleTypeGeneric;
    mapInfo.memHandle.memHandle = handle;
    mapInfo.deviceBitMask = 1;
    HIP_CHECK(hipMemMapArrayAsync(&mapInfo, 1, nullptr));
    HIP_CHECK(hipDeviceSynchronize());
    HIP_CHECK(hipMemcpyHtoA(array, 0, hBuffer, memSizeByte));
    HIP_CHECK(hipMemcpyAtoH(hResult, array, 0, memSizeByte));
    HIP_CHECK(hipDeviceSynchronize());
    for (int i = 0; i < elementCount; ++i) { REQUIRE(hBuffer[i] == hResult[i]); }
    mapInfo.memOperationType = hipMemOperationTypeUnmap;
    HIP_CHECK(hipMemMapArrayAsync(&mapInfo, 1, nullptr));
    HIP_CHECK(hipDeviceSynchronize());
    HIP_CHECK(hipMemRelease(handle));
```

27.11 hipMemRelease

CUresult cuMemRelease (CUmemGenericAllocationHandle handle)

---

hipError_t hipMemRelease(hipMemGenericAllocationHandle_t handle)

Release a memory handle representing a memory allocation which was previously allocated through hipMemCreate.

Parameters

[in] handle handle of the memory allocation.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

```
size_t granularity = 0;
int deviceId = 0;
CTX_CREATE();
hipDevice_t device;
HIP_CHECK(hipDeviceGet(&device, deviceId));
checkVMMSupported(device);
hipMemAllocationProp prop{};

SECTION("Memory Allocation Type as hipMemAllocationTypePinned") {
  prop.type = hipMemAllocationTypePinned;
}

#if HT_AMD
SECTION("Memory Allocation Type as hipMemAllocationTypeUncached") {
  prop.type = hipMemAllocationTypeUncached;
}
#endif

prop.location.type = hipMemLocationTypeDevice;
prop.location.id = device;  // Current Devices
HIP_CHECK(
    hipMemGetAllocationGranularity(&granularity,                    &prop,
hipMemAllocationGranularityMinimum));
REQUIRE(granularity > 0);
hipMemGenericAllocationHandle_t handle;
// Allocate physical memory
for (int mul = 1; mul < 64; mul++) {
  HIP_CHECK(hipMemCreate(&handle, granularity * mul, &prop, 0));
  HIP_CHECK(hipMemRelease(handle));
```

开发者社区

```
}

CTX_DESTROY();
```

27.12 hipMemRetainAllocationHandle

CUresult cuMemRetainAllocationHandle (CUmemGenericAllocationHandle* handle, void* addr)

---

hipError_t hipMemRetainAllocationHandle(hipMemGenericAllocationHandle_t* handle, void* addr)

Returns the allocation handle of the backing memory allocation given the address.

Parameters

[out] handle handle representing addr.

[in] addr address to look up.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

```
size_t granularity = 0;
constexpr int N = DATA_SIZE;
size_t buffer_size = N * sizeof(int);
int deviceId = 0;
hipDevice_t device;
HIP_CHECK(hipDeviceGet(&device, deviceId));
checkVMMSupported(device);
hipMemAllocationProp prop{};
prop.type = hipMemAllocationTypePinned;
prop.location.type = hipMemLocationTypeDevice;
prop.location.id = device;  // Current Devices
HIP_CHECK(
    hipMemGetAllocationGranularity(&granularity, &prop, hipMemAllocationGranularityMinimum));
REQUIRE(granularity > 0);
size_t size_mem = ((granularity + buffer_size - 1) / granularity) * granularity;
hipMemGenericAllocationHandle_t handle;
```

开发者社区

```
  hipDeviceptr_t ptrA;
  // Allocate physical memory
  HIP_CHECK(hipMemCreate(&handle, size_mem, &prop, 0));
  // Allocate virtual address range
  HIP_CHECK(hipMemAddressReserve(&ptrA, size_mem, 0, 0, 0));
  HIP_CHECK(hipMemMap(ptrA, size_mem, 0, handle, 0));
  // Test hipMemRetainAllocationHandle
  hipMemGenericAllocationHandle_t gethandle;
  // Check beginning of VMM ptr
  HIP_CHECK(hipMemRetainAllocationHandle(&gethandle,
reinterpret_cast<void*>(ptrA)));
  REQUIRE(gethandle == handle);
  HIP_CHECK(hipMemRelease(handle));
  HIP_CHECK(hipMemUnmap(ptrA, size_mem));
  HIP_CHECK(hipMemAddressFree(ptrA, size_mem));
```

27.13 hipMemSetAccess

CUresult cuMemSetAccess (CUdeviceptr ptr, size_t size, const CUmemAccessDesc* desc, size_t count)

---

hipError_t hipMemSetAccess(void* ptr, size_t size, const hipMemAccessDesc* desc, size_t count)

Set the access flags for each location specified in desc for the given virtual address range.

Parameters

[in] ptr starting address of the virtual address range.

[in] size size of the range.

[in] desc array of hipMemAccessDesc.

[in] count number of hipMemAccessDesc in desc.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

```
  size_t granularity = 0;
  constexpr int N = DATA_SIZE;
  size_t buffer_size = N * sizeof(int);
  int deviceId = 0;
  hipDevice_t device;
  CTX_CREATE();
  HIP_CHECK(hipDeviceGet(&device, deviceId));
  checkVMMSupported(device);
  hipMemAllocationProp prop{};
  prop.type = hipMemAllocationTypePinned;
  prop.location.type = hipMemLocationTypeDevice;
  prop.location.id = device;  // Current Devices
  HIP_CHECK(
     hipMemGetAllocationGranularity(&granularity,                    &prop,
hipMemAllocationGranularityMinimum));
  REQUIRE(granularity > 0);
  size_t size_mem = ((granularity + buffer_size - 1) / granularity) * granularity;
  hipMemGenericAllocationHandle_t handle;
  // Allocate physical memory
  HIP_CHECK(hipMemCreate(&handle, size_mem, &prop, 0));
  // Allocate virtual address range
  hipDeviceptr_t ptrA;
  HIP_CHECK(hipMemAddressReserve(&ptrA, size_mem, 0, 0, 0));
  HIP_CHECK(hipMemMap(ptrA, size_mem, 0, handle, 0));
  HIP_CHECK(hipMemRelease(handle));
  // Set access
  hipMemAccessDesc accessDesc = {};
  accessDesc.location.type = hipMemLocationTypeDevice;
  accessDesc.location.id = device;
  accessDesc.flags = hipMemAccessFlagsProtReadWrite;
  // Make the address accessible to GPU 0
  HIP_CHECK(hipMemSetAccess(ptrA, size_mem, &accessDesc, 1));
  // Validate using hipMemGetAccess()
```

开发者社区

```
hipMemLocation location;
location.type = hipMemLocationTypeDevice;
location.id = device;
unsigned long long flags = 0;  // NOLINT
HIP_CHECK(hipMemGetAccess(&flags, &location, ptrA));
REQUIRE(flags == hipMemAccessFlagsProtReadWrite);
int devicecount = 0;
HIP_CHECK(hipGetDeviceCount(&devicecount));
if (devicecount > 1) {
  flags = 0;
  HIP_CHECK(hipDeviceGet(&device, 1));
  location.type = hipMemLocationTypeDevice;
  location.id = device;
  HIP_CHECK(hipMemGetAccess(&flags, &location, ptrA));
  REQUIRE(flags == hipMemAccessFlagsProtNone);
}
HIP_CHECK(hipMemUnmap(ptrA, size_mem));
HIP_CHECK(hipMemAddressFree(ptrA, size_mem));
CTX_DESTROY();
```

27.14 hipMemUnmap

CUresult cuMemUnmap (CUdeviceptr ptr, size_t size)

---

hipError_t hipMemUnmap(void* ptr, size_t size)

Unmap memory allocation of a given address range.

Parameters

[in] ptr　　starting address of the range to unmap.

[in] size　　size of the virtual address range.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotSupported

```
size_t granularity = 0;
size_t buffer_size = N * sizeof(int);
int deviceId = 0;
```

开发者社区

```
  hipDevice_t device;

  CTX_CREATE();
  HIP_CHECK(hipDeviceGet(&device, deviceId));
  checkVMMSupported(device);

  hipMemAllocationProp prop{};
  prop.type = hipMemAllocationTypePinned;
  prop.location.type = hipMemLocationTypeDevice;
  prop.location.id = device;  // Current Devices

  HIP_CHECK(
     hipMemGetAllocationGranularity(&granularity, &prop,
hipMemAllocationGranularityMinimum));
  REQUIRE(granularity > 0);
  size_t size_mem = ((granularity + buffer_size - 1) / granularity) * granularity;

  hipMemGenericAllocationHandle_t handle;
  hipDeviceptr_t ptrA;
  // Allocate physical memory
  HIP_CHECK(hipMemCreate(&handle, size_mem, &prop, 0));
  // Allocate virtual address range
  HIP_CHECK(hipMemAddressReserve(&ptrA, size_mem, 0, 0, 0));
  HIP_CHECK(hipMemMap(ptrA, size_mem, 0, handle, 0));

  SECTION("nullptr to ptrA") {
   REQUIRE(hipMemUnmap((hipDeviceptr_t) nullptr, size_mem) ==
hipErrorInvalidValue);
  }

  SECTION("pass zero to size") { REQUIRE(hipMemUnmap(ptrA, 0) ==
hipErrorInvalidValue); }
```

```
SECTION("unmap a smaller size") {
  REQUIRE(hipMemUnmap(ptrA, (size_mem - 1)) == hipErrorInvalidValue);
}

HIP_CHECK(hipMemRelease(handle));
HIP_CHECK(hipMemUnmap(ptrA, size_mem));
HIP_CHECK(hipMemAddressFree(ptrA, size_mem));
CTX_DESTROY();
```

28 Module Management

28.1 hipModuleGetGlobal

CUresult cuModuleGetGlobal(CUdeviceptr *dptr, size_t *bytes, CUmodule hmod, const char *name)

---

hipError_t hipModuleGetGlobal(hipDeviceptr_t *dptr, size_t *bytes, hipModule_t hmod, const char *name)

Returns a global pointer from a module. Returns in *dptr and *bytes the pointer and size of the global of name name located in module hmod. If no variable of that name exists, it returns hipErrorNotFound. Both parameters dptr and bytes are optional. If one of them is NULL, it is ignored and hipSuccess is returned.

Parameters:

dptr – [out] Returns global device pointer

bytes – [out] Returns global size in bytes

hmod – [in] Module to retrieve global from

name – [in] Name of global to retrieve

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotFound, hipErrorInvalidResourceHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.2 hipModuleLoad

CUresult cuModuleLoad(CUmodule * module,const char * fname)

hipError_t hipModuleLoad(hipModule_t *module, const char *fname)

Loads code object from file into a module the currrent context.

Parameters:

fname – [in] Filename of code object to load

module – [out] Module

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorFileNotFound, hipErrorOutOfMemory, hipErrorNotInitialized, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorNotSupported, hipErrorSharedObjectSymbolNotFound, hipErrorInvalidResourceHandle, hipErrorUnknown, hipErrorInvalidDevice, hipErrorNoDevice

28.3 hipModuleLoadFatBinary【dtk-26.10 新增】

CUresult cuModuleLoadFatBinary(CUmodule * module,const void * fatCubin)

---

hipError_t hipModuleLoadFatBinary(hipModule_t *module, const void *fatbin)

Loads fatbin object.

Parameters:

fatbin – [in] fatbin to be loaded as a module

module – [out] Module

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorFileNotFound, hipErrorOutOfMemory, hipErrorNotInitialized, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorNotSupported, hipErrorSharedObjectSymbolNotFound, hipErrorInvalidResourceHandle, hipErrorUnknown, hipErrorInvalidDevice, hipErrorNoDevice

28.4 hipLibraryLoadFromFile【dtk-26.10 新增】

CUresult cuLibraryLoadFromFile(CUlibrary *library, const char *fileName, CUjit_option *jitOptions, void **jitOptionsValues, unsigned int numJitOptions, CUlibraryOption *libraryOptions, void **libraryOptionValues, unsigned int numLibraryOptions)

---

hipError_t hipLibraryLoadFromFile(hipLibrary_t *library, const char *fileName,

hipJitOption *jitOptions, void **jitOptionsValues, unsigned int numJitOptions, HIPlibraryOption *libraryOptions, void **libraryOptionValues, unsigned int numLibraryOptions)

Load a library with specified file and options.

Parameters:

library – [out] Returned library

fileName – [in] File to load from

jitOptions – [in] Options for JIT

jitOptionsValues – [in] Option values for JIT

numJitOptions – [in] Number of options

libraryOptions – [in] Options for loading

libraryOptionValues – [in] Option values for loading

numLibraryOptions – [in] Number of options for loading

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorFileNotFound, hipErrorOutOfMemory, hipErrorNotInitialized, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorNotSupported, hipErrorSharedObjectSymbolNotFound, hipErrorInvalidResourceHandle, hipErrorUnknown, hipErrorInvalidDevice, hipErrorNoDevice

28.5 hipLibraryLoadData【dtk-26.10 新增】

CUresult cuLibraryLoadData(CUlibrary *library, const void *code, CUjit_option *jitOptions, void **jitOptionsValues, unsigned int numJitOptions, CUlibraryOption *libraryOptions, void** libraryOptionValues, unsigned int numLibraryOptions)

---

hipError_t hipLibraryLoadData(hipLibrary_t *library, const void *code, hipJitOption *jitOptions, void **jitOptionsValues, unsigned int numJitOptions, HIPlibraryOption *libraryOptions, void **libraryOptionValues, unsigned int numLibraryOptions)

Load a library with specified code and options.

Parameters:

library – [out] Returned library

code – [in] Code to load

jitOptions – [in] Options for JIT

jitOptionsValues – [in] Option values for JIT

numJitOptions – [in] Number of options

libraryOptions – [in] Options for loading

libraryOptionValues – [in] Option values for loading

numLibraryOptions – [in] Number of options for loading

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorFileNotFound, hipErrorOutOfMemory, hipErrorNotInitialized, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorNotSupported, hipErrorSharedObjectSymbolNotFound, hipErrorInvalidResourceHandle, hipErrorUnknown, hipErrorInvalidDevice, hipErrorNoDevice

28.6 hipLibraryGetModule

CUresult cuLibraryGetModule(CUmodule *pMod, CUlibrary library)

---

hipError_t hipLibraryGetModule(hipModule_t *pMod, hipLibrary_t library)

Returns a module handle.

Parameters:

pMod – [out] Returned module handle

library – [in] Library to retrieve module from

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotFound, hipErrorInvalidHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.7 hipLibraryGetKernelCount【dtk-26.10 新增】

CUresult cuLibraryGetKernelCount(unsigned int *count, CUlibrary lib)

---

hipError_t hipLibraryGetKernelCount(unsigned int *count, hipLibrary_t lib)

Returns the number of kernels within a library.

Parameters:

count – [out] Number of kernels found within the library

lib – [in] Library to query

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotFound, hipErrorInvalidHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.8 hipLibraryGetKernel【dtk-26.10 新增】

CUresult cuLibraryGetKernel(CUkernel *pKernel, CUlibrary library, const char *name)

---

hipError_t hipLibraryGetKernel(hipKernel_t *pKernel, hipLibrary_t library, const char *name)

Returns a kernel handle.

Parameters:

pKernel – [out] Returned kernel handle

library – [in] Library to retrieve kernel from

name – [in] Name of kernel to retrieve

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotFound, hipErrorInvalidHandle, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.9 hipKernelGetName

CUresult cuKernelGetName(const char **name, CUkernel hfunc)

---

hipError_t hipKernelGetName(const char **name, hipKernel_t hfunc)

Returns the function name for a CUkernel handle.

Parameters:

name – [out] The returned name of the function

hfunc – [out] The function handle to retrieve the name for

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotFound, hipErrorNotInitialized, hip

开发者社区

ErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.10 hipKernelGetLibrary

CUresult cuKernelGetLibrary(CUlibrary *pLib, CUkernel kernel)

---

hipError_t hipKernelGetLibrary(hipLibrary_t *pLib, hipKernel_t kernel)

Returns a library handle.

Parameters:

pLib – [out] Returned library handle

kernel – [in] Kernel to retrieve library handle

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotFound, hipErrorInvalidHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.11 hipKernelGetParamInfo

CUresult cuKernelGetParamInfo(CUkernel kernel, size_t paramIndex, size_t *paramOffset, size_t *paramSize)

---

hipError_t hipKernelGetParamInfo(hipKernel_t kernel, size_t paramIndex, size_t *paramOffset, size_t *paramSize)

Returns the offset and size of a kernel parameter in the device-side parameter layout.

Parameters:

kernel – [in] The kernel to query

paramIndex – [in] The parameter index to query

paramOffset – [out] Returns the offset into the device-side parameter layout at which the parameter resides

paramSize – [out] Optionally returns the size of the parameter in the device-side parameter layout

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotFound, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.12 hipLibraryGetGlobal

CUresult cuLibraryGetGlobal(CUdeviceptr *dptr, size_t *bytes, CUlibrary library, const char *name)

---

hipError_t hipLibraryGetGlobal(hipDeviceptr_t *dptr, size_t *bytes, hipLibrary_t library, const char *name)

Returns a global device pointer.

Parameters:

dptr – [out] Returned global device pointer for the requested context

bytes – [out] Returned global size in bytes

library – [in] Library to retrieve global from

name – [in] Name of global to retrieve

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotFound, hipErrorInvalidHandle, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.13 hipLibraryEnumerateKernels

CUresult cuLibraryEnumerateKernels(CUkernel *kernels, unsigned int numKernels, CUlibrary lib)

---

hipError_t hipLibraryEnumerateKernels(hipKernel_t *kernels, unsigned int numKernels, hipLibrary_t lib)

Retrieve the kernel handles within a library.

Parameters:

kernels – [out] Buffer where the kernel handles are returned to

numKernels – [in] Maximum number of kernel handles may be returned to the buffer

lib – [in] Library to query from

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotFound, hipErrorInvalidHandle, hip

ErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.14 hipLibraryGetManaged

CUresult cuLibraryGetManaged(CUdeviceptr *dptr, size_t *bytes, CUlibrary library, const char *name)

---

hipError_t hipLibraryGetManaged(hipDeviceptr_t *dptr, size_t *bytes, hipLibrary_t library, const char *name)

Returns a pointer to managed memory.

Parameters:

dptr – [out] Returned pointer to the managed memory

bytes – [out] Returned memory size in bytes

library – [in] Library to retrieve managed memory from

name – [in] Name of managed memory to retrieve

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotFound, hipErrorInvalidHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.15 hipKernelGetAttribute

CUresult cuKernelGetAttribute(int *pi, CUfunction_attribute attrib, CUkernel kernel, CUdevice dev)

---

hipError_t hipKernelGetAttribute(int *pi, hipFunction_attribute attrib, hipKernel_t kernel, hipDevice_t dev)

Returns information about a kernel.

Parameters:

pi – [out] Returned attribute value

attrib – [in] Attribute requested

kernel – [in] Kernel to query attribute of

dev – [in] Device to query attribute of

开发者社区

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidDevice, hipErrorInvalidHandle, hipErrorInvalidDeviceFunction, hipErrorMissingConfiguration, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorNoDevice

28.16 hipKernelGetFunction

CUresult cuKernelGetFunction(CUfunction *pFunc, CUkernel kernel)

---

hipError_t hipKernelGetFunction(hipFunction_t *pFunc, hipKernel_t kernel)

Returns a function handle.

Parameters:

pFunc – [out] Returned function handle

kernel – [in] Kernel to retrieve function for the requested context

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.17 hipKernelSetAttribute

CUresult cuKernelSetAttribute(CUfunction_attribute attrib, int val, CUkernel kernel, CUdevice dev)

---

hipError_t hipKernelSetAttribute(hipFunction_attribute attrib, int val, hipKernel_t kernel, hipDevice_t dev)

Sets information about a kernel.

Parameters:

attrib – [in] Attribute requested

val – [in] Value to set

kernel – [in] Kernel to set attribute of

dev – [in] Device to set attribute of

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidDevice, hipErrorInvalidHandle, hipErrorInvalidDeviceFunction, hipErrorMissingConfiguration, hipErrorNotSupporte

d, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorNoDevice

28.18 hipLibraryUnload【dtk-26.10 新增】

CUresult cuLibraryUnload(CUlibrary library)

---

hipError_t hipLibraryUnload(hipLibrary_t library)

Unloads a library.

Parameters:

library – [in] Library to unload

Returns:

hipSuccess, hipErrorNotFound, hipErrorInvalidHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

hipError_t hipGetKernel(hipKernel_t *kernelPtr, const void *entryFuncAddr)

Get pointer to device kernel that matches entry function entryFuncAddr.

Parameters:

kernelPtr – [out] Returns the device kernel

entryFuncAddr – [in] Address of device entry function to search kernel for

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidDeviceFunction, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.19 hipModuleUnload

CUresult cuModuleUnload(CUmodule hmod)

---

hipError_t hipModuleUnload(hipModule_t module)

Frees the module.

The module is freed, and the code objects associated with it are destroyed.

Parameters:

module – [in] Module to free

Returns:

hipSuccess, hipErrorInvalidResourceHandle, hipErrorStreamCaptureUnsupported, hipErrorNotFound, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalid

Device, hipErrorNoDevice

28.20 hipModuleGetFunction

CUresult cuModuleGetFunction(CUfunction * hfunc,CUmodule hmod,const char * name)

---

hipError_t hipModuleGetFunction(hipFunction_t *function, hipModule_t module , const char *kname)

Function with kname will be extracted if present in module.

Parameters:

module – [in] Module to get function from

kname – [in] Pointer to the name of function

function – [out] Pointer to function handle

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotInitialized, hipErrorNotFound, hipErrorInvalidResourceHandle, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.21 hipModuleGetFunctionCount 【dtk-26.10 新增】

CUresult cuModuleGetFunctionCount(unsigned int *count, CUmodule mod)

---

hipError_t hipModuleGetFunctionCount(unsigned int *count, hipModule_t mod)

Returns the number of functions within a module.

Parameters:

count – [out] Number of functions found within the module

mod – [in] Module to query

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.22 hipModuleEnumerateFunctions

CUresult cuModuleEnumerateFunctions(CUfunction *functions, unsigned int numFunctions, CUmodule mod)

---

hipError_t hipModuleEnumerateFunctions(hipFunction_t *functions, unsigned int numFunctions, hipModule_t mod)

Returns the function handles within a module.

Parameters:

functions – [out] Buffer where the function handles are returned to

numFunctions – [in] Maximum number of function handles may be returned to the buffer

mod – [in] Module to query from

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidHandle, hipErrorNotFound, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.23 hipFuncIsLoaded

CUresult cuFuncIsLoaded(CUfunctionLoadingState *state, CUfunction function)

---

hipError_t hipFuncIsLoaded(HIPfunctionLoadingState *state, hipFunction_t function)

Returns if the function is loaded.

Parameters:

state – [out] returned loading state

function – [in] the function to check

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.24 hipFuncLoad

CUresult cuFuncLoad(CUfunction function)

---

hipError_t hipFuncLoad(hipFunction_t function)

Loads a function.

Parameters:

function – [in] the function to load

Returns:

hipSuccess, hipErrorInvalidHandle, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.25 hipFuncGetAttributes

cudaError_t cudaFuncGetAttributes(struct cudaFuncAttributes * attr,const void * func)

---

hipError_t hipFuncGetAttributes(struct hipFuncAttributes *attr, const void *func)

Find out attributes for a given function.

Parameters:

attr – [out] Attributes of funtion

func – [in] Pointer to the function handle

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidDeviceFunction, hipErrorInvalidSymbol, hipErrorInvalidKernelFile, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.26 hipFuncGetAttribute

CUresult cuFuncGetAttribute(int * pi,CUfunction_attribute attrib,CUfunction hfunc)

---

hipError_t hipFuncGetAttribute(int *value, hipFunction_attribute attrib, hipFunction_t hfunc)

Find out a specific attribute for a given function.

Parameters:

value – [out] Pointer to the value

attrib – [in] Attributes of the given funtion

hfunc – [in] Function to get attributes from

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidDeviceFunction, hipErrorInvalidHandle, hipErrorMissingConfiguration, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.27 hipFuncGetModule

CUresult cuFuncGetModule(CUmodule *hmod, CUfunction hfunc)

---

hipError_t hipFuncGetModule(hipModule_t *hmod, hipFunction_t hfunc)

Returns a module handle.

Parameters:

hmod – [out] Module

hfunc – [in] Function to get attributes from

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorNotFound, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.28 hipFuncGetName

cudaError_t cudaFuncGetName(const char **name, const void *func)

---

hipError_t hipFuncGetName(const char **name, const void *func)

Returns the function name for a device entry function pointer.

Parameters:

name – [out] The returned name of the function

func – [in] The function pointer to retrieve name for

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidDeviceFunction, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.29 hipDrvFuncGetName

CUresult cuFuncGetName(const char **name, CUfunction hfunc)

---

hipError_t hipDrvFuncGetName(const char **name, hipFunction_t hfunc)

Returns the function name for a device entry function pointer.

Parameters:

name – [out] The returned name of the function

hfunc – [in] The function pointer to retrieve name for

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.30 hipFuncGetParamInfo

cudaError_t cudaFuncGetParamInfo(const void *func, size_t paramIndex, size_t *paramOffset, size_t *paramSize)

---

hipError_t hipFuncGetParamInfo(const void *func, size_t paramIndex, size_t *paramOffset, size_t *paramSize)

Returns the offset and size of a kernel parameter in the device-side parameter layout.

Parameters:

func – [in] The function to query

paramIndex – [in] The parameter index to query

paramOffset – [out] The offset into the device-side parameter layout at which the parameter resides

paramSize – [out] The size of the parameter in the device-side parameter layout

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidDeviceFunction, hipErrorInvalidHandle, hipErrorInvalidSymbol, hipErrorInvalidKernelFile, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.31 hipDrvFuncGetParamInfo

CUresult cuFuncGetParamInfo(CUfunction func, size_t paramIndex, size_t *paramOffset, size_t *paramSize)

---

hipError_t hipDrvFuncGetParamInfo(hipFunction_t func, size_t paramIndex, size

_t *paramOffset, size_t *paramSize)

Returns the offset and size of a kernel parameter in the device-side parameter layout.

Parameters:

func – [in] The function to query

paramIndex – [in] The parameter index to query

paramOffset – [out] The offset into the device-side parameter layout at which the parameter resides

paramSize – [out] The size of the parameter in the device-side parameter layout

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.32 hipModuleGetTexRef

CUresult cuModuleGetTexRef(CUtexref * pTexRef,CUmodule hmod,const char * name)

---

hipError_t hipModuleGetTexRef(textureReference **texRef, hipModule_t hmod, const char *name)

returns the handle of the texture reference with the name from the module.

Parameters:

hmod – [in] Module

name – [in] Pointer of name of texture reference

texRef – [out] Pointer of texture reference

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorNotFound, hipErrorInvalidValue, hipErrorInvalidResourceHandle, hipErrorNotSupported, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.33 hipModuleLoadData

CUresult cuModuleLoadData(CUmodule * module,const void * image)

---

hipError_t hipModuleLoadData(hipModule_t *module, const void *image)

builds module from code object which resides in host memory. Image is pointer to that location.

Parameters:

image – [in] Pointer to the code object in host memory

module – [out] Returned module handle

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidValue, hipErrorFileNotFound, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorNotSupported, hipErrorSharedObjectSymbolNotFound, hipErrorInvalidResourceHandle, hipErrorUnknown, hipErrorInvalidDevice, hipErrorNoDevice

28.34 hipModuleLoadDataEx

CUresult cuModuleLoadDataEx(CUmodule * module,const void * image,unsigned int numOptions,CUjit_option * options,void ** optionValues)

---

hipError_t hipModuleLoadDataEx(hipModule_t *module, const void *image, unsigned int numOptions, hipJitOption *options, void **optionValues)

builds module from code object which resides in host memory. Image is pointer to that location. Options are not used. hipModuleLoadData is called.

Parameters:

image – [in] Pointer to the code object in host memory

module – [out] Returned module handle

numOptions – [in] Number of options

options – [in] Options for JIT

optionValues – [in] Option values for JIT

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidValue, hipErrorFileNotFound, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorNotSupported, hipErrorSharedObjectSymbolNotFound, hipErrorInvalidResourceHandle, hipErrorUnknown, hipErrorInvalidDevice, hipErrorNoDevice

28.35 hipLinkAddData【dtk-26.10 新增】

CUresult cuLinkAddData(CUlinkState state, CUjitInputType type, void *data, size_t size, const char *name, unsigned int numOptions, CUjit_option *options, void **optionValues)

---

hipError_t hipLinkAddData(hipLinkState_t hip_link_state, hipJitInputType input_type, void *image, size_t image_size, const char *name, unsigned int num_options, hipJitOption *options_ptr, void **option_values)

Completes the linking of the given program.

Parameters:

hip_link_state – [in] hip link state

input_type – [in] Type of the input data or bitcode

image – [in] Input data which is null terminated

image_size – [in] Size of the input data

name – [in] Optional name for this input

num_options – [in] Size of the options

options_ptr – [in] Array of options applied to this input

option_values – [in] Array of option values cast to void*

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidHandle, hipErrorInvalidImage, hipErrorInvalidConfiguration, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.36 hipLinkAddFile【dtk-26.10 新增】

CUresult cuLinkAddFile(CUlinkState state,CUjitInputType type,const char * path,unsigned int numOptions,CUjit_option * options,void ** optionValues)

---

hipError_t hipLinkAddFile(hipLinkState_t state, hipJitInputType type, const char *path, unsigned int numOptions, hipJitOption *options, void **optionValues)

Adds a file with bit code to be linked with options.

Parameters:

state – [in] hip link state

type – [in] Type of the input data or bitcode

path – [in] Path to the input file where bitcode is present

numOptions – [in] Size of the options

options – [in] Array of options applied to this input

optionValues – [in] Array of option values cast to void*

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidHandle, hipErrorInvalidConfiguration, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.37 hipLinkComplete【dtk-26.10 新增】

CUresult cuLinkComplete(CUlinkState state,void ** cubinOut,size_t * sizeOut)

---

hipError_t hipLinkComplete(hipLinkState_t state, void **hipBinOut, size_t *sizeOut)

Completes the linking of the given program.

Parameters:

state – [in] hip link state

hipBinOut – [out] Upon success, points to the output binary

sizeOut – [out] Size of the binary is stored (optional)

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidConfiguration, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.38 hipLinkCreate【dtk-26.10 新增】

CUresult cuLinkCreate(unsigned int numOptions,CUjit_option * options,void ** optionValues,CUlinkState * stateOut)

---

hipError_t hipLinkCreate(unsigned int numOptions, hipJitOption *options, void **optionValues, hipLinkState_t *stateOut)

Creates the link instance via hip APIs.

Parameters:

numOptions – [in] Number of options

options – [in] Array of options

optionValues – [in] Array of option values cast to void*

stateOut – [out] hip link state created upon success

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidConfiguration, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

See also

hipSuccess

28.39 hipLinkDestroy【dtk-26.10 新增】

CUresult cuLinkDestroy(CUlinkState state)

---

hipError_t hipLinkDestroy(hipLinkState_t state)

Deletes the link instance via hip APIs.

Parameters:

state – [in] link state instance

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

See also

hipSuccess

28.40 hipModuleLaunchKernel

CUresult cuLaunchKernel(CUfunction f,unsigned int gridDimX,unsigned int gridDimY,unsigned int gridDimZ,unsigned int blockDimX,unsigned int blockDimY,unsigned int blockDimZ,unsigned int sharedMemBytes,CUstream hStream,void ** kernelParams,void ** extra)

---

hipError_t hipModuleLaunchKernel(hipFunction_t f, unsigned int gridDimX, unsigned int gridDimY, unsigned int gridDimZ, unsigned int blockDimX, unsigned int bl

ockDimY, unsigned int blockDimZ, unsigned int sharedMemBytes, hipStream_t strea m, void **kernelParams, void **extra)

launches kernel f with launch parameters and shared memory on stream with arguments passed to kernelparams or extra

Please note, HIP does not support kernel launch with total work items defined in dimension with size gridDim x blockDim >= 2^32. So gridDim.x * blockDim.x, gridDim.y * blockDim.y and gridDim.z * blockDim.z are always less than 2^32.

Parameters:

f – [in] Kernel to launch.

gridDimX – [in] X grid dimension specified as multiple of blockDimX.

gridDimY – [in] Y grid dimension specified as multiple of blockDimY.

gridDimZ – [in] Z grid dimension specified as multiple of blockDimZ.

blockDimX – [in] X block dimensions specified in work-items

blockDimY – [in] Y grid dimension specified in work-items

blockDimZ – [in] Z grid dimension specified in work-items

sharedMemBytes – [in] Amount of dynamic shared memory to allocate for this kernel. The HIP-Clang compiler provides support for extern shared declarations.

stream – [in] Stream where the kernel should be dispatched. May be 0, in which case th default stream is used with associated synchronization rules.

kernelParams – [in] Array of pointers to kernel arguments

extra – [in] Pointer to kernel arguments. These are passed directly to the kernel and must be in the memory layout and alignment expected by the kernel. All passed arguments must be naturally aligned according to their type. The memory address of each argument should be a multiple of its size in bytes. Please refer to hip_porting_driver_api.md for sample usage.

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorInvalidValue, hipErrorContextIsDestr oyed, hipErrorStreamCaptureInvalidated, hipErrorInvalidResourceHandle, hipErrorIn validKernelFile, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorUnknown,

hipErrorInvalidConfiguration, hipErrorIllegalState, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.41 hipModuleLaunchCooperativeKernel

CUresult cuLaunchCooperativeKernel(CUfunction f,unsigned int gridDimX,unsigned int gridDimY,unsigned int gridDimZ,unsigned int blockDimX,unsigned int blockDimY,unsigned int blockDimZ,unsigned int sharedMemBytes,CUstream hStream,void ** kernelParams)

---

hipError_t hipModuleLaunchCooperativeKernel(hipFunction_t f, unsigned int gridDimX, unsigned int gridDimY, unsigned int gridDimZ, unsigned int blockDimX, unsigned int blockDimY, unsigned int blockDimZ, unsigned int sharedMemBytes, hipStream_t stream, void **kernelParams)

launches kernel f with launch parameters and shared memory on stream with arguments passed to kernelParams, where thread blocks can cooperate and synchronize as they execute

Please note, HIP does not support kernel launch with total work items defined in dimension with size gridDim x blockDim >= 2^32.

Parameters:

f – [in] Kernel to launch.

gridDimX – [in] X grid dimension specified as multiple of blockDimX.

gridDimY – [in] Y grid dimension specified as multiple of blockDimY.

gridDimZ – [in] Z grid dimension specified as multiple of blockDimZ.

blockDimX – [in] X block dimension specified in work-items.

blockDimY – [in] Y block dimension specified in work-items.

blockDimZ – [in] Z block dimension specified in work-items.

sharedMemBytes – [in] Amount of dynamic shared memory to allocate for this kernel. The HIP-Clang compiler provides support for extern shared declarations.

stream – [in] Stream where the kernel should be dispatched. May be 0, in which case the default stream is used with associated synchronization rules.

kernelParams – [in] A list of kernel arguments.

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorInvalidImage, hipErrorInvalidValue, hipErrorInvalidConfiguration, hipErrorLaunchFailure, hipErrorCooperativeLaunchTooLarge, hipErrorContextIsDestroyed, hipErrorStreamCaptureInvalidated, hipErrorInvalidResourceHandle, hipErrorInvalidKernelFile, hipErrorNoBinaryForGpu, hipErrorUnknown, hipErrorIllegalState, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.42 hipModuleLaunchCooperativeKernelMultiDevice

CUresult cuLaunchCooperativeKernelMultiDevice(CUDA_LAUNCH_PARAMS * launchParamsList,unsigned int numDevices,unsigned int flags)

---

hipError_t hipModuleLaunchCooperativeKernelMultiDevice(hipFunctionLaunchParams *launchParamsList, unsigned int numDevices, unsigned int flags)

Launches kernels on multiple devices where thread blocks can cooperate and synchronize as they execute.

Parameters:

launchParamsList – [in] List of launch parameters, one per device.

numDevices – [in] Size of the launchParamsList array.

flags – [in] Flags to control launch behavior.

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorInvalidImage, hipErrorInvalidValue, hipErrorInvalidConfiguration, hipErrorInvalidResourceHandle, hipErrorLaunchFailure, hipErrorCooperativeLaunchTooLarge, hipErrorInvalidDevice, hipErrorInvalidKernelFile, hipErrorNoBinaryForGpu, hipErrorUnknown, hipErrorIllegalState, hipErrorOutOfMemory, hipErrorNoDevice

28.43 hipLaunchCooperativeKernel

CUresult cuLaunchCooperativeKernel(CUfunction f,unsigned int gridDimX,unsigned int gridDimY,unsigned int gridDimZ,unsigned int

blockDimX,unsigned int blockDimY,unsigned int blockDimZ,unsigned int sharedMemBytes,CUstream hStream,void ** kernelParams)

---

hipError_t hipLaunchCooperativeKernel(const void *f, dim3 gridDim, dim3 blockDimX, void **kernelParams, unsigned int sharedMemBytes, hipStream_t stream)

launches kernel f with launch parameters and shared memory on stream with arguments passed to kernelparams or extra, where thread blocks can cooperate and synchronize as they execute

Please note, HIP does not support kernel launch with total work items defined in dimension with size gridDim x blockDim >= 2^32.

Parameters:

f – [in] - Kernel to launch.

gridDim – [in] - Grid dimensions specified as multiple of blockDim.

blockDimX – [in] - Block dimensions specified in work-items

kernelParams – [in] - Pointer of arguments passed to the kernel. If the kernel has multiple parameters, ‘kernelParams’ should be array of pointers, each points the corresponding argument.

sharedMemBytes – [in] - Amount of dynamic shared memory to allocate for this kernel. The HIP-Clang compiler provides support for extern shared declarations.

stream – [in] - Stream where the kernel should be dispatched. May be 0, in which case th default stream is used with associated synchronization rules.

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorInvalidValue, hipErrorCooperativeLaunchTooLarge, hipErrorContextIsDestroyed, hipErrorStreamCaptureInvalidated, hipErrorInvalidHandle, hipErrorInvalidDeviceFunction, hipErrorInvalidConfiguration, hipErrorInvalidResourceHandle, hipErrorInvalidKernelFile, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorUnknown, hipErrorLaunchFailure, hipErrorIllegalState, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.44 hipLaunchCooperativeKernelMultiDevice

CUresult

cuLaunchCooperativeKernelMultiDevice(CUDA_LAUNCH_PARAMS * launchParamsList,unsigned int numDevices,unsigned int flags)

hipError_t hipLaunchCooperativeKernelMultiDevice(hipLaunchParams *launchParamsList, int numDevices, unsigned int flags)

Launches kernels on multiple devices where thread blocks can cooperate and synchronize as they execute.

Parameters:

launchParamsList – [in] List of launch parameters, one per device.

numDevices – [in] Size of the launchParamsList array

flags – [in] Flags to control launch behavior.

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorInvalidValue, hipErrorCooperativeLaunchTooLarge, hipErrorInvalidDevice, hipErrorInvalidResourceHandle, hipErrorStreamCaptureUnsupported, hipErrorStreamCaptureInvalidated, hipErrorInvalidDeviceFunction, hipErrorInvalidSymbol, hipErrorInvalidKernelFile, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorInvalidConfiguration, hipErrorUnknown, hipErrorLaunchFailure, hipErrorIllegalState, hipErrorOutOfMemory, hipErrorNoDevice

28.45 hipParamSeti

CUresult cuParamSeti(CUfunction hfunc,int offset,unsigned int value)

hipError_t hipParamSeti(hipFunction_t hfunc, int offset, unsigned int value)

Set an integer parameter to the function’s argument list.

Parameters:

hfunc – [in] Kernel to set parameter to

offset – [in] Offset to set parameter to argument list

value – [in] Value of parameter

Returns:

hipSuccess, hipErrorInvalidHandle, hipErrorInvalidDeviceFunction, hipErrorInvalidValue, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.46 hipParamSetf

CUresult cuParamSetf(CUfunction hfunc,int offset,float value)

---

hipError_t hipParamSetf(hipFunction_t hfunc, int offset, float value)

Set a floating-point parameter to the function’s argument list.

Parameters:

hfunc – [in] Kernel to set parameter to

offset – [in] Offset to set parameter to argument list

value – [in] Value of parameter

Returns:

hipSuccess, hipErrorInvalidHandle, hipErrorInvalidDeviceFunction, hipErrorInvalidValue, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.47 hipParamSetv

CUresult cuParamSetv(CUfunction hfunc,int offset,void * ptr,unsigned int numbytes)

---

hipError_t hipParamSetv(hipFunction_t hfunc, int offset, void *ptr, unsigned int numbytes)

Set arbitrary data to the function’s argument list.

Parameters:

hfunc – [in] Kernel to set parameter to

offset – [in] Offset to set parameter to argument list

ptr – [in] Pointer to arbitrary data

numbytes – [in] Size of data to copy in bytes

Returns:

hipSuccess, hipErrorInvalidHandle, hipErrorInvalidDeviceFunction, hipErrorInvalidValue, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

开发者社区

28.48 hipFuncSetBlockShape

CUresult cuFuncSetBlockShape(CUfunction hfunc,int x,int y,int z)

---

hipError_t hipFuncSetBlockShape(hipFunction_t hfunc, int x, int y, int z)

Sets the block-dimensions for the function.

Parameters:

hfunc – [in] Kernel to specify dimensions of

x – [in] X dimension

y – [in] Y dimension

z – [in] Z dimension

Returns:

hipSuccess, hipErrorInvalidHandle, hipErrorInvalidDeviceFunction, hipErrorMissingConfiguration, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.49 hipFuncSetSharedSize

CUresult cuFuncSetSharedSize(CUfunction hfunc,unsigned int bytes)

---

hipError_t hipFuncSetSharedSize(hipFunction_t hfunc, unsigned int bytes)

Sets the dynamic shared-memory size for the function.

Parameters:

hfunc – [in] Kernel to specify dynamic shared-memory size for

bytes – [in] Dynamic shared-memory size per thread in bytes

Returns:

hipSuccess, hipErrorInvalidHandle, hipErrorInvalidDeviceFunction, hipErrorMissingConfiguration, hipErrorInvalidValue, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.50 hipLaunch

CUresult CUDAAPI cuLaunch(CUfunction f)

---

hipError_t hipLaunch(hipFunction_t hfunc)

Launches a HIP function.

Parameters:

hfunc – [in] Kernel to launch

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorInvalidHandle, hipErrorInvalidImage, hipErrorInvalidValue, hipErrorInvalidConfiguration, hipErrorLaunchOutOfResources, hipErrorInvalidDeviceFunction, hipErrorMissingConfiguration, hipErrorInvalidResourceHandle, hipErrorInvalidKernelFile, hipErrorNoBinaryForGpu, hipErrorUnknown, hipErrorIllegalState, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.51 hipLaunchGrid

CUresult cuLaunchGrid(CUfunction f,int grid_width,int grid_height)

---

hipError_t hipLaunchGrid(hipFunction_t hfunc, int grid_width, int grid_height)

Launches a HIP function.

Parameters:

hfunc – [in] Kernel to launch

grid_width – [in] Width of grid in blocks

grid_height – [in] Height of grid in blocks

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorInvalidHandle, hipErrorInvalidImage, hipErrorInvalidValue, hipErrorInvalidConfiguration, hipErrorLaunchOutOfResources, hipErrorInvalidDeviceFunction, hipErrorMissingConfiguration, hipErrorInvalidResourceHandle, hipErrorInvalidKernelFile, hipErrorNoBinaryForGpu, hipErrorUnknown, hipErrorIllegalState, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.52 hipLaunchGridAsync

CUresult cuLaunchGridAsync(CUfunction f,int grid_width,int grid_height,CUstream hStream)

---

hipError_t hipLaunchGridAsync(hipFunction_t hfunc, int grid_width, int grid_he

ight, hipStream_t hstream)

Launches a HIP function.

Parameters:

hfunc – [in] Kernel to launch

grid_width – [in] Width of grid in blocks

grid_height – [in] Height of grid in blocks

hstream – [in] Stream on which the kernel is launched

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorInvalidHandle, hipErrorInvalidImage, hipErrorInvalidValue, hipErrorInvalidConfiguration, hipErrorLaunchOutOfResources, hipErrorInvalidDeviceFunction, hipErrorMissingConfiguration, hipErrorInvalidResourceHandle, hipErrorInvalidKernelFile, hipErrorNoBinaryForGpu, hipErrorUnknown, hipErrorIllegalState, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.53 hipModuleGetLoadingMode

CUresult cuModuleGetLoadingMode(CUmoduleLoadingMode *mode)

---

hipError_t hipModuleGetLoadingMode(hipModuleLoadingMode *mode)

return loading mode.

Parameters:

mode – [out] return the loading mode

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.54 hipDrvFuncSetAttribute

CUresult cuFuncSetAttribute(CUfunction hfunc,CUfunction_attribute attrib,int value)

---

hipError_t hipDrvFuncSetAttribute(hipFunction_t hfunc, hipFunction_attribute attrib, int value)

Set attribute for a specific function.

Parameters:

hfunc – [in] Pointer of the function

attrib – [in] Attribute to set

value – [in] Value to set

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorInvalidDeviceFunction, hipErrorInvalidHandle, hipErrorMissingConfiguration, hipErrorNotSupported, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.55 hipGetFuncBySymbol

cudaError_t cudaGetFuncBySymbol(cudaFunction_t* functionPtr, const void* symbolPtr)

---

hipError_t hipGetFuncBySymbol(hipFunction_t *functionPtr, const void *symbolPtr)

Get pointer to device entry function that matches entry function symbolPtr.

Parameters:

functionPtr – [in] Device entry function

symbolPtr – [in] Pointer to device entry function to search for

Returns:

hipSuccess, hipErrorInvalidDeviceFunction, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.56 hipGetDriverEntryPoint【dtk-26.10 新增】

cudaError_t cudaGetDriverEntryPoint(const char *symbol, void **funcPtr, unsigned long long flags)

---

hipError_t hipGetDriverEntryPoint(const char *symbol, void **funcPtr, unsigned long long flags, hipDriverEntryPointQueryResult *driverStatus)

Gets function pointer of a requested HIP API.

Parameters:

symbol – [in] The API base name

funcPtr – [out] Pointer to the requested function

flags – [in] Flags for the search

driverStatus – [out] Optional returned status of the search

Returns:

hipSuccess, hipErrorInvalidValue, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

28.57 hipParamSetSize

CUresult cuParamSetSize(CUfunction hfunc,unsigned int numbytes)

---

hipError_t hipParamSetSize(hipFunction_t hfunc, unsigned int numbytes)

Sets the parameter size for the function.

Parameters:

hfunc – [in] Kernel to set parameter size for

numbytes – [in] Size of parameter list in bytes

Returns:

hipSuccess, hipErrorInvalidHandle, hipErrorInvalidDeviceFunction, hipErrorInvalidValue, hipErrorNotInitialized, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

29 定制化 API

29.1 duDeviceGetHwQueueNum

hipError_t duDeviceGetHwQueueNum(uint32_t* defaultValue, uint32_t* usedValue)

returns hareware queue number of current device.

Parameters

[out] defaultValue Pointer of current device default hardware queue number.

[out] usedValue Pointer of current device real used hardware queue number.

Returns

hipSuccess, hipErrorInvalidValue

```
uint32_t queueDNum = 0;
uint32_t queueNum = 0;
HIP_CHECK(duDeviceGetHwQueueNum(&queueDNum,&queueNum));
REQUIRE(queueNum != 0);
```

29.2 duStreamGetHwQueue

hipError_t duStreamGetHwQueue(hipStream_t stream, uint64_t *value)

return the queue id of stream. if stream is created but not used it will return hipErrorNotInitialized

Parameters

[in] stream which Stream want to query.

[out] value Pointer of stream queue id.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorContextIsDestroyed, hipErrorNotInitialized, hipErrorInvalidResourceHandle

```
hipStream_t stream;
HIP_CHECK(hipStreamCreate(&stream));
HIP_CHECK_ERROR(duStreamGetHwQueue(stream,
&hwQueue),hipErrorNotInitialized); // create but not used
```

29.3 duListStreamQueueIds

hipError_t duListStreamQueueIds(int deviceId, char* buffer, size_t* size)

returns stream hareware queue of specific deviceId device.

will return queue id stream list into buffer. if buffer is not enough, will just copy to buffer with size, return hipErrorUnsupportedLimit and set size with needed size.

Parameters

[in] deviceId which device want to list hardware queue.

[out] buffer which buffer to store hardware queue id list.

[in/out] size buffer size.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorUnsupportedLimit

29.4 duDeviceFlushCache

hipError_t duDeviceFlushCache(uint32_t flushFlag, hipStream_t stream __dparm(NULL))

Flush Device Cache.

Parameters

[in] flushFlag Flush control flag. value should be duFlushScope.

[in] stream use which stream to flush cache. May be 0, in which case the default stream is used with associated synchronization rules.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorContextIsDestroyed

```
HIP_CHECK(duDeviceFlushCache(duFlushScope::duFlushScopeAgent));// use null stream
hipStream_t stream;
HIP_CHECK(hipStreamCreate(&stream));
HIP_CHECK(duDeviceFlushCache(duFlushScope::duFlushScopeSystem, stream));
HIP_CHECK_ERROR(duDeviceFlushCache(7788, stream), hipErrorInvalidValue);
```

29.5 duLaunchKernel

hipError_t duLaunchKernel(const void* hostFunction,

dim3 gridDim,

dim3 blockDim,

void** args,

size_t sharedMemBytes __dparm(0),

hipStream_t stream __dparm(0),

duLaunchConfig_t* config __dparm(NULL))

C compliant kernel launch API

Parameters

[in] hostFunction Kernel stub function pointer.

[in] gridDim Number of blocks.

[in] blockDim Dimension of a block.

[in] args Pointer of arguments passed to the kernel. If the kernel has multiple parameters, 'args' should be array of pointers, each points the corresponding argument.

[in] sharedMemBytes Amount of dynamic shared memory to allocate for this kernel. The HIP-Clang compiler provides support for extern shared declarations.

[in] stream Stream where the kernel should be dispatched. May be 0, in which case the default stream is used with associated synchronization rules.

[in] config Launch configuration.

Returns

hipSuccess, hipErrorInvalidValue, hipInvalidDevice

```
duLaunchConfig_t config;
config.beginCache = 1;
config.endCache = 1;
HIP_CHECK(duLaunchKernel((const void *) emptyKernel, dim3(1, 1, 1), dim3(1, 1, 1), NULL, 0, nullptr, &config));
HIP_CHECK(hipStreamSynchronize(nullptr));
```

29.6 duModuleLaunchKernel【dtk-26.10 新增】

hipError_t duModuleLaunchKernel(hipFunction_t f, unsigned int gridDimX, unsigned int gridDimY, unsigned int gridDimZ, unsigned int blockDimX, unsigned int blockDimY, unsigned int blockDimZ, unsigned int sharedMemBytes, hipStream_t stream, void** kernelParams, void** extra, duLaunchConfig_t* config __dparm(NULL))

launches kernel f with launch parameters and shared memory on stream with arguments passed to kernelparams or extra.

Please note, HIP does not support kernel launch with total work items defined in

dimension with size gridDim x blockDim >= 2^32. So gridDim.x * blockDim.x, gridDim.y * blockDim.y * and gridDim.z * blockDim.z are always less than 2^32.

Parameters

[in] f Kernel to launch.

[in] gridDimX X grid dimension specified as multiple of blockDimX.

[in] gridDimY Y grid dimension specified as multiple of blockDimY.

[in] gridDimZ Z grid dimension specified as multiple of blockDimZ.

[in] blockDimX X block dimensions specified in work-items.

[in] blockDimY Y grid dimension specified in work-items.

[in] blockDimZ Z grid dimension specified in work-items.

[in] sharedMemBytes Amount of dynamic shared memory to allocate for this kernel. The HIP-Clang compiler provides support for extern shared declarations.

[in] stream Stream where the kernel should be dispatched. May be 0, in which case the default stream is used with associated synchronization rules.

[in] kernelParams

[in] extra Pointer to kernel arguments. These are passed directly to the kernel and must be in the memory layout and alignment expected by the kernel. All passed arguments must be naturally aligned according to their type. The memory address of each argument should be a multiple of its size in bytes. Please refer to hip_porting_driver_api.md for sample usage.

[in] config Launch configuration.

```
float *A, *B;
void *Ad, *Bd;
A = new float[LEN];
B = new float[LEN];

for (uint32_t i = 0; i < LEN; i++) {
   A[i] = i * 1.0f;
   B[i] = 0.0f;
}
```

```
    HIP_CHECK(hipMalloc((void **) &Ad, SIZE));
    HIP_CHECK(hipMalloc((void **) &Bd, SIZE));

    HIP_CHECK(hipMemcpyHtoD(Ad, A, SIZE));
    HIP_CHECK(hipMemcpyHtoD(Bd, B, SIZE));

    hipModule_t Module;
    hipFunction_t Function;
    HIP_CHECK(hipModuleLoad(&Module, fileName));
    HIP_CHECK(hipModuleGetFunction(&Function, Module, kernel_name));

    void *args[2] = {&Ad, &Bd};
    duLaunchConfig_t config;
    config.beginCache = 2;
    config.endCache = 2;
    HIP_CHECK(duModuleLaunchKernel(Function, 1, 1, 1, LEN, 1, 1, 0, 0, args,
nullptr, &config));

    HIP_CHECK(hipMemcpyDtoH(B, Bd, SIZE));
    int mismatchCount = 0;
    for (uint32_t i = 0; i < LEN; i++) {
        if (A[i] != B[i]) {
            mismatchCount++;
            std::cout << "error: mismatch " << A[i] << " != " << B[i] << std::endl;
        }
    }
    REQUIRE(mismatchCount == 0);

    HIP_CHECK(hipFree(Ad));
    HIP_CHECK(hipFree(Bd));
    delete[] A;
    delete[] B;
```

开发者社区

29.7 duAqlPacketCreate

hipError_t duAqlPacketCreate(duAqlPacket_t* aqlPacket)

Create a AQL packet handler, which can be used to launch kernels.

Parameters

[out] aqlPacket AQL packet handler.

Returns

hipSuccess, hipErrorInvalidValue

29.8 duAqlPacketDestroy

hipError_t duAqlPacketDestroy(duAqlPacket_t AqlPacket)

Destroy the AQL packet handler created by hipAqlPacketCreate. It's a sync API, if the AQL packet is still in use, it will wait until the AQL packet is finished in DCU.

Parameters

[in] aqlPacket AQL packet handler.

Returns

hipSuccess, hipErrorInvalidResourceHandle

29.9 duKernelGetAql

hipError_t duKernelGetAql(const void *hostFunction, dim3 gridDim, dim3 blockDim, void **args, size_t sharedMemBytes, hipStream_t stream, duAqlPacket_t aql, duLaunchConfig_t *config __dparm(NULL))

Get Launched AQL packet, there will no sync

Parameters

[in] hostFunction Kernel stub function pointer.

[in] gridDim Number of blocks.

[in] blockDim Dimension of a block

[in] args Pointer of arguments passed to the kernel. If the kernel has multiple parameters, 'args' should be array of pointers, each points the corresponding argument.

[in] sharedMemBytes Amount of dynamic shared memory to allocate for this

kernel. The HIP-Clang compiler provides support for extern shared declarations.

[in] stream Stream where the kernel should be dispatched.

[out] aql the aql packet handler to store.

[in] config Launch configuration.

Returns

hipSuccess, hipErrorInvalidValue, hipInvalidDevice

29.10 duKernelAqlLaunch

hipError_t duKernelAqlLaunch(const duAqlPacket_t aql, hipStream_t stream __dparm(0), int count __dparm(1))

launch the Kernel get by duKernelGetAql. the duKernelGetAql and duKernelAqlLaunch stream should in same device, otherwise will return hipErrorInvalidResourceHandle.

Parameters

[in] aql AQL packet handler.

[in] stream Stream where the kernel should be dispatched. May be 0, in which case the default stream is used with associated synchronization rules.

[in] count repeat times.

Returns

hipSuccess, hipErrorInvalidResourceHandle

```
float *A, *B;
void *Ad, *Bd;
A = new float[LEN];
B = new float[LEN];

for (uint32_t i = 0; i < LEN; i++) {
  A[i] = i * 1.0f;
  B[i] = 0.0f;
}

HIP_CHECK(hipMalloc((void **) &Ad, SIZE));
```

开发者社区

```
    HIP_CHECK(hipMalloc((void **) &Bd, SIZE));

    HIP_CHECK(hipMemcpyHtoD(Ad, A, SIZE));
    HIP_CHECK(hipMemcpyHtoD(Bd, B, SIZE));

    void *args[2] = {&Ad, &Bd};
    duAqlPacket_t aql1;
    HIP_CHECK(duAqlPacketCreate(&aql1));
    duAqlPacket_t aql2;
    HIP_CHECK(duAqlPacketCreate(&aql2));
    duLaunchConfig_t config;
    config.beginCache = 2;
    config.endCache = 2;

    HIP_CHECK(
        duKernelGetAql((const void *) emptyKernel, dim3(1, 1, 1), dim3(1, 1, 1),
NULL, 0, nullptr, aql1, &config));

    HIP_CHECK(duKernelAqlLaunch(aql1, nullptr, 20));

    HIP_CHECK(duKernelGetAql((const void *) assignFunc, dim3(1, 1, 1), dim3(1,
1, 1), args, 0, nullptr, aql2, &config));
    HIP_CHECK(duKernelAqlLaunch(aql2, nullptr, 20));

    std::vector<duAqlPacket_t> aqls;
    aqls.push_back(aql1);
    aqls.push_back(aql2);
    aqls.push_back(aql1);
    aqls.push_back(aql2);
    aqls.push_back(aql1);
    aqls.push_back(aql2);
    aqls.push_back(aql1);
    aqls.push_back(aql2);
```

开发者社区

```
aqls.push_back(aql1);
aqls.push_back(aql2);
HIP_CHECK(duKernelAqlBatchLaunch(aqls.data(), aqls.size(), nullptr))
HIP_CHECK(duAqlPacketDestroy(aql1));
HIP_CHECK(duAqlPacketDestroy(aql2));

HIP_CHECK(hipStreamSynchronize(nullptr));
HIP_CHECK(hipFree(Ad));
HIP_CHECK(hipFree(Bd));
delete[] A;
delete[] B;
```

29.11 duKernelAqlBatchLaunch

hipError_t duKernelAqlBatchLaunch(const duAqlPacket_t* aqls, size_t size, hipStream_t stream __dparm(0))

launch Kernels get by duKernelGetAql. the getted aql should be same device with duKernelAqlBatchLaunch stream, otherwise will return hipErrorInvalidResourceHandle.

Parameters

[in] aqls AQL packets pointer.

[in] size the size of aqls.

[in] stream Stream where the kernel should be dispatched. May be 0, in which case the default stream is used with associated synchronization rules.

Returns

hipSuccess, hipErrorInvalidResourceHandle

29.12 duDeviceFlushHDP

hipError_t duDeviceFlushHDP()

Flush HDP cache in current device.

Parameters

Returns

hipSuccess

29.13 duGraphGetNodeByLabel

hipError_t duGraphGetNodeByLabel(hipGraph_t graph, const char* label, hipGraphNode_t* node)

Get node form graph which added by capture mode with duLaunchKernel/duModuleLaunchKernel API param with duLaunchConfig_t config.label

Parameters

[in] graph the graph which want to search from.

[in] label the unique label of node.

[out] node the found node to store.

Returns

hipSuccess, hipErrorNotFound, hipErrorInvalidValue, hipErrorIllegalState

```
float *A, *B, *C;
void *Ad, *Bd, *Cd;
A = new float[LEN];
B = new float[LEN];
C = new float[LEN];

for (uint32_t i = 0; i < LEN; i++) {
  A[i] = i * 1.0f;
  B[i] = 0.0f;
  C[i] = i * 3.0f;
}

HIP_CHECK(hipMalloc((void **) &Ad, SIZE));
HIP_CHECK(hipMalloc((void **) &Bd, SIZE));
HIP_CHECK(hipMalloc((void **) &Cd, SIZE));

HIP_CHECK(hipMemcpyHtoD(Ad, A, SIZE));
HIP_CHECK(hipMemcpyHtoD(Bd, B, SIZE));
HIP_CHECK(hipMemcpyHtoD(Cd, C, SIZE));
```

开发者社区

```
    hipModule_t Module;
    hipFunction_t Function;
    HIP_CHECK(hipModuleLoad(&Module, fileName));
    HIP_CHECK(hipModuleGetFunction(&Function, Module, kernel_name));

    hipGraph_t graph;
    hipStream_t stream;
    HIP_CHECK(hipStreamCreate(&stream));
    HIP_CHECK(hipStreamBeginCapture(stream, hipStreamCaptureModeGlobal));
    void *args1[2] = {&Ad, &Bd};
    duLaunchConfig_t config;
    config.beginCache = 1;
    config.endCache = 1;
    config.label = "add1";
    //b = a + 1
    HIP_CHECK(duLaunchKernel((const void *) add1, dim3(1, 1, 1), dim3(LEN, 1,
1), args1, 0, stream, &config));

    void *args2[2] = {&Bd, &Ad};
    config.beginCache = 2;
    config.endCache = 2;
    config.label = "hello_world";
    //a = b;
    HIP_CHECK(duModuleLaunchKernel(Function, 1, 1, 1, LEN, 1, 1, 0, stream,
args2, nullptr, &config));
    HIP_CHECK(hipStreamEndCapture(stream, &graph));
    hipGraphExec_t execGraph;
    HIP_CHECK(hipGraphInstantiate(&execGraph, graph, nullptr, nullptr, 0));

     HIP_CHECK(hipGraphLaunch(execGraph, stream));
     HIP_CHECK(hipStreamSynchronize(stream));
     HIP_CHECK(hipMemcpyDtoH(A, Ad, SIZE));
```

开发者社区

```
HIP_CHECK(hipMemcpyDtoH(B, Bd, SIZE));
//a = i;
//execGraph and graph all same
//node1: b = a + 1
//node2: a = b
//==> a = i + 1
//==> b = i + 1
int mismatchCount = 0;
for (uint32_t i = 0; i < LEN; i++) {
   if (A[i] != (i + 1)) {
      mismatchCount++;
      std::cout << "error: mismatch " << A[i] << " != " << (i + 1) << std::endl;
   }
}
REQUIRE(mismatchCount == 0);
hipGraphNode_t node;
HIP_CHECK(duGraphGetNodeByLabel(graph, "hello_world", &node));
hipKernelNodeParams kNodeParams;
void *kernelArgs[] = {&Bd, &Cd};

kNodeParams.func = reinterpret_cast<void *>(add2);
kNodeParams.gridDim = dim3(1, 1, 1);
kNodeParams.blockDim = dim3(LEN, 1, 1);
kNodeParams.sharedMemBytes = 0;
kNodeParams.kernelParams = reinterpret_cast<void **>(kernelArgs);
kNodeParams.extra = nullptr;
HIP_CHECK(hipGraphKernelNodeSetParams(node, &kNodeParams));

hipGraphExec_t execGraph2;
HIP_CHECK(hipGraphInstantiate(&execGraph2, graph, nullptr, nullptr, 0));

HIP_CHECK(hipGraphLaunch(execGraph2, stream));
HIP_CHECK(hipStreamSynchronize(stream));
```

开发者社区

```
HIP_CHECK(hipMemcpyDtoH(C, Cd, SIZE));
HIP_CHECK(hipMemcpyDtoH(B, Bd, SIZE));
//==> a = i + 1
//==> b = i + 1
//execGraph
//node1: b = a + 1
//node2: a = b
//graph and execGraph2
//node1: b = a + 1  ==> b = i + 2
//node2: c = b + 2  ==> c = i + 4
//c = i + 4
// ==> a = i + 1
// ==> b = i + 2
// ==> c = i + 4
mismatchCount = 0;
for (uint32_t i = 0; i < LEN; i++) {
    if (C[i] != (i + 4.0f)) {
        mismatchCount++;
        std::cout << "error: mismatch " << C[i] << " != " << (i + 4) << std::endl;
    }
}
REQUIRE(mismatchCount == 0);

HIP_CHECK(hipGraphExecDestroy(execGraph));
HIP_CHECK(hipGraphExecDestroy(execGraph2));
HIP_CHECK(hipStreamDestroy(stream));
HIP_CHECK(hipFree(Ad));
HIP_CHECK(hipFree(Bd));
HIP_CHECK(hipFree(Cd));
delete[] A;
delete[] B;
delete[] C;
```

开发者社区

29.14 duGraphExecKernelNodeGetParamsByLabel

hipError_t duGraphExecKernelNodeGetParamsByLabel(hipGraphExec_t hGraphExec, const char* label, hipKernelNodeParams* pNodeParams)

Get kernel node params from graph exec with unique label.

Parameters

[in] hGraphExec the graph exec which want to search from.

[in] label the unique label of node.

[out] pNodeParams the kernel node params to store.

Returns

hipSuccess, hipErrorNotFound, hipErrorInvalidValue

29.15 duGraphExecKernelNodeSetParamsByLabel

hipError_t duGraphExecKernelNodeSetParamsByLabel(hipGraphExec_t hGraphExec, const char* label, const hipKernelNodeParams* pNodeParams)

Set kernel node params to graph exec with unique label.

Parameters

[in] hGraphExec the graph exec which want to search from.

[in] label the unique label of node.

[in] pNodeParams the kernel node params to store.

Returns

hipSuccess, hipErrorNotFound, hipErrorInvalidValue

29.16 duStreamWaitEvent

hipError_t duStreamWaitEvent(hipStream_t stream, hipEvent_t event, unsigned int flags, uint64_t delayUs)

Make the specified compute stream wait for an event.

Wait on the stream until the event occurs, and then continue to wait for delayUs microseconds.

Parameters

[in] stream stream to make wait.

[in] event event to wait on.

[in] flags Flags to control event behavior. Valid values are hipEventWaitDefault, hipEventWaitExternal

* #hipEventWaitDefault : Default flag. Default event wait flag.

* #hipEventWaitExternal : Event wait is captured in the graph as an EventWait node when performing stream capture.

[in] delayUs the time in us want to wait

Returns

hipSuccess, hipErrorInvalidHandle

29.17 duEventRecord

hipError_t duEventRecord(hipEvent_t event, hipStream_t stream, uint64_t delayUs)

Record an event in the specified stream.

After the Event on the stream is executed, continue to wait for delayUs microseconds.

Parameters

[in] event event to record.

[in] stream stream in which to record event.

[in] delayUs the time in us want to wait.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorNotInitialized, hipErrorInvalidHandle, hipErrorLaunchFailure

29.18 duMemcpyAsync

hipError_t duMemcpyAsync(void* dst, const void* src, size_t sizeBytes, hipMemcpyKind kind, hipStream_t stream)

Copy data from src to dst asynchronously.

warning If host or dest are not pinned, the memory copy will be performed synchronously. For best performance, use hipHostMalloc to allocate host memory that

开发者社区

is transferred asynchronously. warning on HCC hipMemcpyAsync does not support overlapped H2D and D2H copies. For hipMemcpy, the copy is always performed by the device associated with the specified stream. For multi-gpu or peer-to-peer configurations, it is recommended to use a stream which is a attached to the device where the src data is physically located. For optimal peer-to-peer copies, the copy device must be able to access the src and dst pointers (by calling hipDeviceEnablePeerAccess with copy agent as the current device and src/dest as the peerDevice argument. if this is not done, the hipMemcpy will still work, but will perform the copy using a staging buffer on the host.

Parameters

[out] dst Data being copy to.

[in] src Data being copy from.

[in] sizeBytes Data size in bytes.

[in] kind type of memory transfer.

[in] stream stream identifier.

Returns

hipSuccess, hipErrorInvalidValue, hipErrorMemoryFree, hipErrorUnknown

29.19 duModuleLaunchKernel

hipError_t duModuleLaunchKernel(hipFunction_t f, unsigned int gridDimX, unsigned int gridDimY, unsigned int gridDimZ, unsigned int blockDimX, unsigned int blockDimY, unsigned int blockDimZ, unsigned int sharedMemBytes, hipStream_t stream, void **kernelParams, void **extra, duLaunchConfig_t *config)

launches kernel f with launch parameters and shared memory on stream with arguments passed to kernelparams or extra.Please note, HIP does not support kernel launch with total work items defined in dimension with size gridDim x blockDim >= 2^32. So gridDim.x * blockDim.x, gridDim.y * blockDim.y and gridDim.z * blockDim.z are always less than 2^32.

Parameters:

f – [in] Kernel to launch.

gridDimX – [in] X grid dimension specified as multiple of blockDimX.

gridDimY – [in] Y grid dimension specified as multiple of blockDimY.

gridDimZ – [in] Z grid dimension specified as multiple of blockDimZ.

blockDimX – [in] X block dimensions specified in work-items

blockDimY – [in] Y grid dimension specified in work-items

blockDimZ – [in] Z grid dimension specified in work-items

sharedMemBytes – [in] Amount of dynamic shared memory to allocate for this kernel. The HIP-Clang compiler provides support for extern shared declarations.

stream – [in] Stream where the kernel should be dispatched. May be 0, in which case th default stream is used with associated synchronization rules.

kernelParams – [in] Array of pointers to kernel arguments

extra – [in] Pointer to kernel arguments. These are passed directly to the kernel and must be in the memory layout and alignment expected by the kernel. All passed arguments must be naturally aligned according to their type. The memory address of each argument should be a multiple of its size in bytes. Please refer to hip_porting_driver_api.md for sample usage.

config – [in] - Launch configuration

Returns:

hipSuccess, hipErrorNotInitialized, hipErrorInvalidValue, hipErrorContextIsDestroyed, hipErrorStreamCaptureInvalidated, hipErrorInvalidResourceHandle, hipErrorInvalidKernelFile, hipErrorInvalidImage, hipErrorNoBinaryForGpu, hipErrorUnknown, hipErrorInvalidConfiguration, hipErrorIllegalState, hipErrorOutOfMemory, hipErrorInvalidDevice, hipErrorNoDevice

开发者社区

保密声明

在此声明，该文档展示的全部技术信息及其相关内容，版权皆属于开发者社区（https://developer.sourcefind.cn/）所有。未经允许，严禁截屏、大规模传播及转发。另外，对使用该技术文档而导致任何侵犯第三方专利或其他权利的行为，开发者社区不承担任何责任。

感谢您的理解与支持。

开发者社区